

Here's a structured **SQL topic list**, from beginner to advanced:

1. SQL Basics

- What is SQL?
- Database, Table, Row, Column
- DBMS vs RDBMS
- SQL Data Types
- SQL Syntax

2. Database Operations (DDL)

- CREATE DATABASE
- CREATE TABLE
- ALTER TABLE
- DROP TABLE
- TRUNCATE TABLE
- RENAME TABLE

3. Data Manipulation (DML)

- INSERT
- UPDATE
- DELETE
- MERGE (where supported)

4. Data Querying (DQL)

- SELECT
- DISTINCT
- WHERE
- ORDER BY
- LIMIT / TOP
- OFFSET

5. Filtering Data

- Comparison Operators (=, >, <, !=)
- Logical Operators (AND, OR, NOT)
- BETWEEN
- IN
- LIKE
- IS NULL

6. Aggregate Functions

- COUNT()
- SUM()
- AVG()

- `MIN()`
- `MAX()`

7. Grouping Data

- `GROUP BY`
- `HAVING`

8. Joins

- Inner Join
- Left Join
- Right Join
- Full Outer Join
- Cross Join
- Self Join

9. SQL Constraints

- `PRIMARY KEY`
- `FOREIGN KEY`
- `UNIQUE`
- `NOT NULL`
- `CHECK`
- `DEFAULT`

10. Keys

- Primary Key
- Foreign Key
- Candidate Key
- Composite Key
- Super Key
- Alternate Key

11. Subqueries

- Single-row Subquery
- Multiple-row Subquery
- Correlated Subquery
- Nested Subquery

12. Set Operators

- `UNION`
- `UNION ALL`
- `INTERSECT`
- `EXCEPT / MINUS`

13. Views

- Create View
- Update View
- Drop View

14. Indexes

- Create Index
- Unique Index
- Clustered Index
- Non-Clustered Index

15. SQL Functions

- String Functions
- Numeric Functions
- Date Functions
- Conversion Functions
- NULL Functions

16. Window Functions

- `ROW_NUMBER()`
- `RANK()`
- `DENSE_RANK()`
- `LEAD()`
- `LAG()`
- `NTILE()`
- Aggregate Window Functions

17. Common Table Expressions (CTEs)

- Simple CTE
- Recursive CTE

18. Stored Procedures & Functions

- Create Procedure
- Execute Procedure
- User-Defined Functions (UDFs)

19. Triggers

- BEFORE Trigger
- AFTER Trigger
- INSTEAD OF Trigger

20. Transactions (TCL)

- `BEGIN`
- `COMMIT`

- `ROLLBACK`
- `SAVEPOINT`

21. SQL Security (DCL)

- `GRANT`
- `REVOKE`

22. Normalization

- 1NF
- 2NF
- 3NF
- BCNF
- Denormalization

23. Performance Optimization

- Query Optimization
- Execution Plans
- Index Optimization
- Avoiding Full Table Scans

24. Advanced SQL

- Recursive Queries
- Pivot & Unpivot
- Dynamic SQL
- Temporary Tables
- Materialized Views (where supported)

1. SQL Standards
2. SQL Statements & Keywords
3. SQL Comments
4. `DROP DATABASE`
5. `CREATE SCHEMA`
6. `ALTER SCHEMA`
7. `DROP SCHEMA`
8. Add/Modify/Drop Columns
9. Insert Multiple Rows
10. Insert from `SELECT`
11. Column Aliases
12. Table Aliases
13. `NOT IN`
14. `NOT LIKE`
15. Wildcards (`%`, `_`)
16. `COUNT(DISTINCT)`
17. Multiple-Column `GROUP BY`
18. Equi Join

19. Non-Equi Join
20. Semi Join
21. Anti Join
22. Multiple Joins
23. Cascading Constraints
24. Natural Key
25. Surrogate Key
26. Arithmetic Operators
27. **ANY**
28. **ALL**
29. **EXISTS**
30. **NOT EXISTS**
31. Multiple-Column Subqueries
32. **CASE** Expression
33. Materialized Views
34. Composite Index
35. Covering Index
36. Filtered/Partial Index
37. Included Columns
38. Index Seek
39. Index Scan
40. Index Fragmentation
41. **OVER()**
42. Window Frames
43. **FIRST_VALUE()**
44. **LAST_VALUE()**
45. Moving Average
46. Multiple CTEs
47. Temporary Tables
48. Table Variables
49. Stored Procedure Parameters
50. Output Parameters
51. Scalar UDF
52. Table-Valued Functions
53. DML Triggers
54. DDL Triggers
55. ACID Properties
56. Transaction Isolation Levels
57. Dirty Read
58. Non-Repeatable Read
59. Phantom Read
60. Locks
61. Blocking
62. Deadlocks
63. Optimistic Concurrency
64. Pessimistic Concurrency

- 65. Functional Dependency
- 66. 4NF
- 67. 5NF
- 68. ER Model
- 69. ER Diagram
- 70. One-to-One Relationship
- 71. One-to-Many Relationship
- 72. Many-to-Many Relationship
- 73. Junction/Bridge Tables
- 74. Roles & Permissions
- 75. Execution Plans
- 76. Table Scan
- 77. Key Lookup
- 78. Nested Loop Join
- 79. Hash Join
- 80. Merge Join
- 81. SARGability
- 82. Query Statistics
- 83. Parameter Sniffing
- 84. N+1 Query Problem
- 85. Pagination Optimization
- 86. Dynamic SQL
- 87. PIVOT
- 88. UNPIVOT
- 89. Hierarchical Queries
- 90. JSON Queries
- 91. XML Queries
- 92. Full-Text Search
- 93. Temporal Tables
- 94. Sequences
- 95. Identity/Auto Increment
- 96. Generated Columns
- 97. Variables
- 98. Cursors
- 99. Error Handling
- 100. Bulk Insert
- 101. Data Migration
- 102. ETL Basics
- 103. Backup & Restore
- 104. Point-in-Time Recovery
- 105. Replication
- 106. Read Replicas
- 107. Sharding
- 108. Partitioning
- 109. High Availability
- 110. Failover

- 111. Distributed Transactions
 - 112. ADO.NET
 - 113. Parameterized Queries
 - 114. Connection Pooling
 - 115. SQL Injection Prevention
 - 116. Row-Level Security
 - 117. Data Masking
 - 118. Auditing
 - 119. Encryption
 - 120. SQL Interview Problem Patterns
-

1. SQL Basics

- What is SQL?

SQL stands for **Structured Query Language**. It is a standard programming language used to **store, retrieve, update, and manage data** in relational databases.

Think of a database as a collection of tables, similar to spreadsheets. SQL lets you work with the data in those tables.

For example, imagine a table called **Employees**:

ID	Name	Department
1	Alice	Sales
2	Bob	IT
3	Carol	HR

Here are some common SQL commands:

- **Retrieve data**

```
SELECT * FROM Employees;
```

This displays all rows in the **Employees** table.

- **Add new data**

```
INSERT INTO Employees (Name, Department)
VALUES ('David', 'Finance');
```

- **Update existing data**

```
UPDATE Employees
SET Department = 'Marketing'
```

```
WHERE Name = 'Alice';
```

- **Delete data**

```
DELETE FROM Employees  
WHERE Name = 'Bob';
```

Why is SQL important?

- It is widely used in web applications, banking, healthcare, and e-commerce.
- It works with popular database systems such as:
 - MySQL
 - PostgreSQL
 - Microsoft SQL Server
 - Oracle Database
 - SQLite

If you're interested in learning databases or becoming a software developer, data analyst, or data scientist, SQL is one of the most valuable skills to learn.

- Database, Table, Row, Column

Database, Table, Row, and Column

These are the **basic building blocks of a relational database**.

1. Database

A **database** is a collection of organized data.

For example, an application might have a database called **CompanyDB** containing:

```
CompanyDB  
|  
├── Employees  
├── Departments  
├── Projects  
└── Salaries
```

A database can contain many **tables**.

2. Table

A **table** stores data in a structured format using **rows and columns**.

Example: **Employees** table

ID	Name	Department	Salary
1	Rahul	IT	50000
2	Amit	HR	45000
3	Priya	Finance	60000

Here, **Employees** is the **table**.

You can think of a table like an Excel sheet.

3. Column

A **column** represents a particular **attribute/property** of the data.

In the **Employees** table:

```
ID
Name
Department
Salary
```

are columns.

For example:

```
Name
----
Rahul
Amit
Priya
```

The **Name** column contains the names of employees.

Column = What type of information are we storing?

4. Row

A **row** represents one complete **record**.

For example:

ID	Name	Department	Salary
1	Rahul	IT	50000

This entire line is **one row**.

It represents one employee.

Row = One complete record

Easy way to remember

```
Database
↓
Collection of Tables
↓
Table
↓
Rows + Columns
```

For example:

```
CompanyDB
├── Employees Table
│   ├── Columns
│   │   ├── ID
│   │   ├── Name
│   │   ├── Department
│   │   └── Salary
│   └── Rows
│       ├── 1, Rahul, IT, 50000
│       ├── 2, Amit, HR, 45000
│       └── 3, Priya, Finance, 60000
```

In SQL

```
CREATE DATABASE CompanyDB;

CREATE TABLE Employees
(
    ID INT,
    Name VARCHAR(100),
    Department VARCHAR(50),
    Salary DECIMAL(10,2)
);
```

Then:

```
INSERT INTO Employees
VALUES (1, 'Rahul', 'IT', 50000);
```

Here:

- **CompanyDB** → **Database**
- **Employees** → **Table**
- **ID, Name, Department, Salary** → **Columns**
- **1, Rahul, IT, 50000** → **Row/Record**

Interview definition:

A database stores organized data, a table organizes related data into rows and columns, a row represents a single record, and a column represents an attribute of that record.

- DBMS vs RDBMS

DBMS vs RDBMS

DBMS = Database Management System **RDBMS** = Relational Database Management System

Both are software systems used to **store, manage, retrieve, and manipulate data**, but an RDBMS specifically follows the **relational model**, where data is organized into related tables.

Key Differences

Feature	DBMS	RDBMS
Full form	Database Management System	Relational Database Management System
Data storage	Can use different structures	Primarily tables
Relationships	May not support relationships	Supports relationships between tables
Primary Key	Not necessarily required	Used to uniquely identify rows
Foreign Key	Generally not a core requirement	Supports foreign keys
Normalization	May not be required	Commonly used
Data integrity	Limited depending on system	Strong integrity constraints
Transactions	May have limited support	Strong transaction support
SQL	May or may not use SQL	Typically uses SQL
Examples	File-based DBMS, older hierarchical/network systems	SQL Server, MySQL, PostgreSQL, Oracle

Example

Suppose we have:

Customers

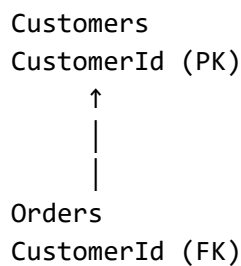
CustomerId	Name
1	Rahul

CustomerId	Name
2	Priya

Orders

OrderId	CustomerId	Amount
101	1	5000
102	2	3000

In an **RDBMS**, `Orders.CustomerId` can reference `Customers.CustomerId`.



This relationship is one of the major characteristics of an **RDBMS**.

Simple way to remember

DBMS:

Manage data.

RDBMS:

Manage data + organize it into related tables + enforce relationships and integrity.

Important interview point

Every RDBMS is a DBMS, but not every DBMS is an RDBMS.

For a .NET backend developer, you'll commonly work with **SQL Server, PostgreSQL, or MySQL**, all of which are RDBMSs.

- SQL Data Types

SQL Data Types

SQL data types define **what kind of data a column can store**.

For example, an employee table might have:

```
CREATE TABLE Employees  
(
```



```
Id INT,  
Name VARCHAR(100),  
Salary DECIMAL(10,2),  
JoiningDate DATE,  
IsActive BIT  
);
```

Here `INT`, `VARCHAR`, `DECIMAL`, `DATE`, and `BIT` are **data types**.

1. Numeric Data Types

Used to store numbers.

Data Type	Purpose	Example
INT	Whole numbers	100
BIGINT	Very large whole numbers	90000000000
SMALLINT	Smaller whole numbers	1000
TINYINT	Very small whole numbers	255
DECIMAL(p,s)	Exact decimal values	12345.67
NUMERIC(p,s)	Same concept as DECIMAL	12345.67
FLOAT	Approximate decimal values	12.3456
REAL	Approximate decimal values	12.34

DECIMAL example

```
Salary DECIMAL(10,2)
```

- `10` = maximum total digits
- `2` = digits after decimal

So:

```
12345678.90
```

is valid.

For **money, salary, prices, financial calculations**, prefer `DECIMAL` rather than `FLOAT`.

2. Character/String Data Types

Used to store text.

VARCHAR

Variable-length string.

```
Name VARCHAR(100)
```

Can store up to 100 characters.

```
'Rahul'
```

CHAR

Fixed-length string.

```
CountryCode CHAR(2)
```

Example:

```
IN  
US  
UK
```

CHAR is useful when the value has a fixed length.

NVARCHAR

Variable-length Unicode string.

```
Name NVARCHAR(100)
```

Useful when storing multilingual text.

For example:

```
Name = N'राहुल'
```

NCHAR

Fixed-length Unicode string.

```
Code NCHAR(5)
```

3. Date and Time Data Types

Used for dates and times.

Data Type	Example
DATE	2026-08-14
TIME	22:30:00
DATETIME	2026-08-14 22:30:00
DATETIME2	2026-08-14 22:30:00.1234567
DATETIMEOFFSET	Date/time with timezone offset

Example:

```
BirthDate DATE,  
CreatedAt DATETIME2
```

For modern SQL Server applications, **DATETIME2** is generally preferred over the older **DATETIME** when you need date + time.

4. Boolean Data Type

In SQL Server, **BIT** is commonly used for Boolean-like values.

```
IsActive BIT
```

Values:

```
0 = False  
1 = True
```

Example:

```
INSERT INTO Employees  
VALUES (1, 'Rahul', 50000, 1);
```

5. Binary Data Types

Used to store binary data.

Data Type	Purpose
BINARY	Fixed-length binary data
VARBINARY	Variable-length binary data
VARBINARY(MAX)	Large binary data

Example:

```
FileData VARBINARY(MAX)
```

Can be used for binary content such as files, although storing large files directly in a database isn't always the best architecture.

6. Large Text Data

SQL Server provides:

```
VARCHAR(MAX)  
NVARCHAR(MAX)
```

Example:

```
Description NVARCHAR(MAX)
```

Useful for large amounts of text.

7. Unique Identifier

SQL Server provides **UNIQUEIDENTIFIER** for GUID values.

```
UserId UNIQUEIDENTIFIER
```

Example:

```
550e8400-e29b-41d4-a716-446655440000
```

In C#, this commonly maps to:

Guid

8. JSON and XML

SQL Server can store JSON in **VARCHAR**/**NVARCHAR** columns and provides functions for working with it.

Example:

Metadata NVARCHAR(MAX)

XML has a dedicated:

XML

data type.

Example:

EmployeeData XML

Common SQL Server Data Types

For a **.NET + SQL Server backend developer**, these are especially important:

INT
BIGINT
DECIMAL
FLOAT
VARCHAR
NVARCHAR
CHAR
DATE
TIME
DATETIME2
DATETIMEOFFSET
BIT
VARBINARY
UNIQUEIDENTIFIER
XML

Quick mapping to C#

SQL Server	C#
INT	int
BIGINT	long
SMALLINT	short
TINYINT	byte
DECIMAL	decimal
FLOAT	double
VARCHAR	string
NVARCHAR	string
DATE	DateOnly / DateTime
DATETIME2	DateTime
DATETIMEOFFSET	DateTimeOffset
BIT	bool
UNIQUEIDENTIFIER	Guid
VARBINARY	byte[]

Interview tip

If asked "**Which data type should you use for salary?**", a good answer is:

DECIMAL, because salary is an exact monetary value and **DECIMAL** avoids the rounding/precision issues associated with approximate types such as **FLOAT**.

- SQL Syntax

SQL Syntax

SQL syntax means the rules and structure used to write SQL statements.

A typical SQL statement looks like:

```
SELECT column_name
FROM table_name
WHERE condition;
```

For example:

```
SELECT Name, Salary
FROM Employees
```

```
WHERE Salary > 50000;
```

This means: **Get the Name and Salary of employees whose salary is greater than 50,000.**

1. CREATE DATABASE

Creates a new database.

```
CREATE DATABASE CompanyDB;
```

2. CREATE TABLE

Creates a table.

```
CREATE TABLE Employees
(
    Id INT,
    Name VARCHAR(100),
    Salary DECIMAL(10,2),
    IsActive BIT
);
```

3. INSERT

Adds records to a table.

```
INSERT INTO Employees (Id, Name, Salary, IsActive)
VALUES (1, 'Rahul', 50000, 1);
```

Multiple rows:

```
INSERT INTO Employees (Id, Name, Salary, IsActive)
VALUES
    (1, 'Rahul', 50000, 1),
    (2, 'Priya', 60000, 1),
    (3, 'Amit', 45000, 0);
```

4. SELECT

Retrieves data.

```
SELECT *  
FROM Employees;
```

Select specific columns:

```
SELECT Id, Name, Salary  
FROM Employees;
```

5. WHERE

Filters records.

```
SELECT *  
FROM Employees  
WHERE Salary > 50000;
```

Multiple conditions:

```
SELECT *  
FROM Employees  
WHERE Salary > 50000  
    AND IsActive = 1;
```

6. UPDATE

Modifies existing data.

```
UPDATE Employees  
SET Salary = 55000  
WHERE Id = 1;
```

⚠ Always be careful with **UPDATE** without a **WHERE** clause:

```
UPDATE Employees  
SET Salary = 55000;
```

This updates **every employee**.

7. DELETE

Deletes records.

```
DELETE FROM Employees  
WHERE Id = 3;
```

Without **WHERE**:

```
DELETE FROM Employees;
```

This deletes **all rows** from the table.

8. ORDER BY

Sorts results.

Ascending:

```
SELECT *  
FROM Employees  
ORDER BY Salary ASC;
```

Descending:

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC;
```

ASC is the default.

9. DISTINCT

Removes duplicate values.

```
SELECT DISTINCT Department  
FROM Employees;
```

10. TOP

Gets a limited number of rows in SQL Server.

```
SELECT TOP 5 *  
FROM Employees;
```

Top 5 highest salaries:

```
SELECT TOP 5 *  
FROM Employees  
ORDER BY Salary DESC;
```

11. LIKE

Used for pattern matching.

Names starting with **R**:

```
SELECT *  
FROM Employees  
WHERE Name LIKE 'R%';
```

Names ending with **a**:

```
SELECT *  
FROM Employees  
WHERE Name LIKE '%a';
```

Names containing **ah**:

```
SELECT *  
FROM Employees  
WHERE Name LIKE '%ah%';
```

% means **zero or more characters**.

_ means **exactly one character**.

12. IN

Checks whether a value exists in a list.

```
SELECT *  
FROM Employees  
WHERE Department IN ('IT', 'HR', 'Finance');
```

13. BETWEEN

Checks whether a value falls within a range.

```
SELECT *  
FROM Employees  
WHERE Salary BETWEEN 40000 AND 60000;
```

14. NULL

Checks for missing values.

Correct:

```
SELECT *  
FROM Employees  
WHERE Salary IS NULL;
```

Not:

```
WHERE Salary = NULL
```

For non-null:

```
SELECT *  
FROM Employees  
WHERE Salary IS NOT NULL;
```

15. GROUP BY

Groups rows for aggregate calculations.

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department;
```

Example result:

Department	EmployeeCount
IT	10
HR	5
Finance	8

16. HAVING

Filters grouped results.

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 5;
```

WHERE vs HAVING

```
WHERE → filters rows before grouping
HAVING → filters groups after GROUP BY
```

17. Aggregate Functions

Common aggregate functions:

```
COUNT()
SUM()
AVG()
MIN()
MAX()
```

Example:

```
SELECT
    COUNT(*) AS TotalEmployees,
    AVG(Salary) AS AverageSalary,
```

```
MAX(Salary) AS HighestSalary,  
MIN(Salary) AS LowestSalary  
FROM Employees;
```

18. JOIN

Used to retrieve data from multiple related tables.

Example:

```
SELECT  
    e.Name,  
    d.DepartmentName  
FROM Employees e  
INNER JOIN Departments d  
    ON e.DepartmentId = d.Id;
```

Common joins:

```
INNER JOIN  
LEFT JOIN  
RIGHT JOIN  
FULL OUTER JOIN  
CROSS JOIN
```

19. Aliases

Give temporary names to tables or columns.

Column alias:

```
SELECT  
    Name AS EmployeeName,  
    Salary AS EmployeeSalary  
FROM Employees;
```

Table alias:

```
SELECT e.Name  
FROM Employees AS e;
```

20. Comments

Single-line comment:

```
-- Get all employees
SELECT *
FROM Employees;
```

Multi-line comment:

```
/*
    Get active employees
    with salary above 50000
*/
SELECT *
FROM Employees
WHERE Salary > 50000
AND IsActive = 1;
```

Basic SQL Statement Structure

A very important structure to remember:

```
SELECT column1, column2
FROM table
WHERE condition
GROUP BY column1
HAVING condition
ORDER BY column1;
```

For example:

```
SELECT
    Department,
    COUNT(*) AS EmployeeCount,
    AVG(Salary) AS AverageSalary
FROM Employees
WHERE IsActive = 1
GROUP BY Department
HAVING COUNT(*) > 2
ORDER BY AverageSalary DESC;
```

Logical SQL execution order

Although we **write** SQL like:

```
SELECT
FROM
WHERE
GROUP BY
HAVING
ORDER BY
```

SQL logically processes it approximately as:

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

This execution order becomes **very important when you start learning SQL queries, JOINS, subqueries, CTEs, and query optimization.**

2. Database Operations (DDL)

- **CREATE DATABASE**

CREATE DATABASE

CREATE DATABASE is a **DDL (Data Definition Language)** command used to create a new database.

Syntax

```
CREATE DATABASE DatabaseName;
```

Example

```
CREATE DATABASE EmployeeDB;
```

This creates a database named **EmployeeDB**.

Select the Database

After creating it, you can select it using:

```
USE EmployeeDB;
```

Then you can create tables inside it:

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

Check Existing Databases

MySQL:

```
SHOW DATABASES;
```

SQL Server:

```
SELECT name
FROM sys.databases;
```

Create Only If It Doesn't Exist

MySQL:

```
CREATE DATABASE IF NOT EXISTS EmployeeDB;
```

SQL Server:

```
IF NOT EXISTS (
    SELECT name
    FROM sys.databases
    WHERE name = 'EmployeeDB'
)
BEGIN
    CREATE DATABASE EmployeeDB;
END;
```

Interview Point

CREATE DATABASE creates the **database container**. It does not automatically create tables or application data.

```
CREATE DATABASE
↓
EmployeeDB
```



```
↓  
CREATE TABLE  
↓  
Employees  
↓  
INSERT DATA
```

- CREATE TABLE

CREATE TABLE

CREATE TABLE is a **DDL command** used to create a new table inside a database.

A table contains **columns** that define the structure and **rows** that store the actual data.

Syntax

```
CREATE TABLE TableName  
(  
    Column1 DataType,  
    Column2 DataType,  
    Column3 DataType  
);
```

Example

First select the database:

```
USE EmployeeDB;
```

Then create a table:

```
CREATE TABLE Employees  
(  
    EmployeeId INT,  
    Name VARCHAR(100),  
    Salary DECIMAL(10,2),  
    Department VARCHAR(50)  
);
```

This creates:

Column	Data Type
EmployeeId	INT
Name	VARCHAR(100)

Column	Data Type
Salary	DECIMAL(10,2)
Department	VARCHAR(50)

With Primary Key

Usually, tables should have a primary key:

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    Salary DECIMAL(10,2),
    Department VARCHAR(50)
);
```

`EmployeeId` must now be **unique** and **not NULL**.

With Multiple Constraints

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(150) UNIQUE,
    Salary DECIMAL(10,2) CHECK (Salary > 0),
    Department VARCHAR(50) DEFAULT 'IT'
);
```

Here:

- **PRIMARY KEY** → uniquely identifies each employee
 - **NOT NULL** → value is required
 - **UNIQUE** → prevents duplicate emails
 - **CHECK** → validates a condition
 - **DEFAULT** → provides a default value
-

Example with Foreign Key

```
CREATE TABLE Departments
(
    DepartmentId INT PRIMARY KEY,
    DepartmentName VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Salary DECIMAL(10,2),
    DepartmentId INT,

    FOREIGN KEY (DepartmentId)
        REFERENCES Departments(DepartmentId)
);
```

Here, `Employees.DepartmentId` references `Departments.DepartmentId`.

Important Interview Point

`CREATE TABLE` creates the **structure of a table**. It does not insert data.

```
Database
  ↓
Table
  ↓
Columns
  ↓
Rows (data)
```

For example:

```
CREATE TABLE Students
(
    Id INT PRIMARY KEY,
    Name VARCHAR(100),
    Age INT
);
```

The table exists, but it initially contains **zero rows**.

- `ALTER TABLE`

ALTER TABLE

`ALTER TABLE` is a **DDL command** used to **modify the structure of an existing table**.

You can use it to:

- Add a column
- Remove a column
- Modify a column

- Add a constraint
- Remove a constraint

1. Add a Column

Syntax

```
ALTER TABLE TableName
ADD ColumnName DataType;
```

Example

```
ALTER TABLE Employees
ADD Email VARCHAR(150);
```

Before:

```
Employees
-----
EmployeeId
Name
Salary
```

After:

```
Employees
-----
EmployeeId
Name
Salary
Email
```

2. Add Multiple Columns

Some databases allow:

```
ALTER TABLE Employees
ADD Phone VARCHAR(20),
    Address VARCHAR(200);
```

3. Drop a Column

Removes a column from the table.

Syntax

```
ALTER TABLE TableName  
DROP COLUMN ColumnName;
```

Example

```
ALTER TABLE Employees  
DROP COLUMN Email;
```

⚠ The column and its existing data are removed.

4. Modify a Column

The syntax depends on the database.

SQL Server

```
ALTER TABLE Employees  
ALTER COLUMN Name VARCHAR(200);
```

This changes the size of **Name** from, for example, **VARCHAR(100)** to **VARCHAR(200)**.

MySQL

```
ALTER TABLE Employees  
MODIFY Name VARCHAR(200);
```

5. Add a Primary Key

```
ALTER TABLE Employees  
ADD CONSTRAINT PK_Employees  
PRIMARY KEY (EmployeeId);
```

6. Add a UNIQUE Constraint

```
ALTER TABLE Employees  
ADD CONSTRAINT UQ_Employees_Email  
UNIQUE (Email);
```

This prevents duplicate email addresses.

7. Add a Foreign Key

Suppose we have:

```
CREATE TABLE Departments
(
    DepartmentId INT PRIMARY KEY,
    DepartmentName VARCHAR(100)
);
```

We can add a foreign key to **Employees**:

```
ALTER TABLE Employees
ADD CONSTRAINT FK_Employees_Departments
FOREIGN KEY (DepartmentId)
REFERENCES Departments(DepartmentId);
```

8. Drop a Constraint

In SQL Server:

```
ALTER TABLE Employees
DROP CONSTRAINT UQ_Employees_Email;
```

Common **ALTER TABLE** Operations

Operation	Example
Add column	<code>ADD Email VARCHAR(150)</code>
Drop column	<code>DROP COLUMN Email</code>
Modify column	<code>ALTER COLUMN Name VARCHAR(200)</code>
Add PK	<code>ADD CONSTRAINT ... PRIMARY KEY</code>
Add FK	<code>ADD CONSTRAINT ... FOREIGN KEY</code>
Add UNIQUE	<code>ADD CONSTRAINT ... UNIQUE</code>
Drop constraint	<code>DROP CONSTRAINT ...</code>

Easy way to remember

ALTER TABLE



Change existing table structure



ADD	→ Add column/constraint
DROP	→ Remove column/constraint
ALTER/MODIFY	→ Change column definition

Interview definition:

ALTER TABLE is a DDL command used to modify the structure of an existing database table without necessarily deleting the entire table.

- **DROP TABLE**

DROP TABLE

DROP TABLE is a **DDL command** used to **completely remove a table from a database**.

It removes:

- The table structure
- All rows/data
- Columns
- Constraints
- Indexes associated with the table

Syntax

```
DROP TABLE TableName;
```

Example

```
DROP TABLE Employees;
```

After executing this command, the **Employees** table **no longer exists**.

```
SELECT * FROM Employees;
```

✗ This will produce an error because the table has been deleted.

Example

Suppose we have:

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

And it contains:

EmployeeId	Name	Salary
1	John	50000
2	Alice	60000

Now execute:

```
DROP TABLE Employees;
```

The result is:

```
Employees table
  ↓
Completely
deleted
```

Both the **data and table structure** are gone.

DROP TABLE IF EXISTS

In SQL Server and MySQL, you can safely check whether the table exists.

SQL Server

```
DROP TABLE IF EXISTS Employees;
```

MySQL

```
DROP TABLE IF EXISTS Employees;
```

This avoids an error if the table doesn't exist.

DROP vs TRUNCATE vs DELETE

This is a **very important interview question**.

Feature	DELETE	TRUNCATE	DROP
Removes rows	✓	✓	✓
Removes table	✗	✗	✓
Removes structure	✗	✗	✓
WHERE allowed	✓	✗	✗
Table remains	✓	✓	✗
Type	DML	DDL	DDL

Example

```
DELETE FROM Employees
WHERE EmployeeId = 10;
```

→ Deletes selected rows.

```
TRUNCATE TABLE Employees;
```

→ Deletes **all rows**, table remains.

```
DROP TABLE Employees;
```

→ Deletes **the entire table**.

Easy way to remember

```
DELETE    → Delete rows
TRUNCATE  → Delete all rows
DROP      → Delete the table itself
```

Interview definition:

DROP TABLE is a DDL command that permanently removes a table, including its data and structure, from the database.

- **TRUNCATE TABLE**

TRUNCATE TABLE

TRUNCATE TABLE is a **DDL command** used to **remove all rows from a table while keeping the table structure**.

It is commonly used when you want to empty a table completely.

Syntax

```
TRUNCATE TABLE TableName;
```

Example

```
TRUNCATE TABLE Employees;
```

Suppose the table contains:

EmployeeId	Name	Salary
1	John	50000
2	Alice	60000
3	Bob	55000

After:

```
TRUNCATE TABLE Employees;
```

The result is:

EmployeeId	Name	Salary
(no rows)		

The **table still exists**.

```
SELECT * FROM Employees;
```

✓ Query works, but returns zero rows.

What Does **TRUNCATE** Remove?

```
TRUNCATE TABLE Employees
```



Remove all rows



Keep table structure



Keep columns



Keep constraints

TRUNCATE vs DELETE

This is an important SQL interview topic.

DELETE

```
DELETE FROM Employees;
```

Removes all rows using **DELETE**.

You can also delete selected rows:

```
DELETE FROM Employees  
WHERE EmployeeId = 10;
```

TRUNCATE

```
TRUNCATE TABLE Employees;
```

Removes **all rows**.

You cannot use:

```
TRUNCATE TABLE Employees  
WHERE EmployeeId = 10; -- ❌ Invalid
```

TRUNCATE vs DELETE vs DROP

Feature	DELETE	TRUNCATE	DROP
Remove selected rows	✓	❌	❌

Feature	DELETE	TRUNCATE	DROP
Remove all rows	✓	✓	✓
Table remains	✓	✓	✗
Structure remains	✓	✓	✗
WHERE clause	✓	✗	✗
Command category	DML	DDL*	DDL

*In SQL Server, **TRUNCATE TABLE** is classified as DDL.

Important Point: Identity/Auto-Increment

In **SQL Server**, **TRUNCATE TABLE** resets an identity column's counter.

Example:

```
CREATE TABLE Employees
(
    EmployeeId INT IDENTITY(1,1),
    Name VARCHAR(100)
);
```

After inserting:

```
EmployeeId
-----
1
2
3
```

Then:

```
TRUNCATE TABLE Employees;
```

The next inserted row will normally receive:

```
EmployeeId = 1
```

Whereas:

```
DELETE FROM Employees;
```

normally does **not** reset the identity counter in SQL Server.

When to Use TRUNCATE

Use **TRUNCATE** when:

- You want to remove **all records**
- You want to keep the table
- You don't need a **WHERE** condition
- You want a faster way to empty a large table in appropriate database systems

Easy way to remember

DELETE	→ Remove rows
TRUNCATE	→ Empty the table
DROP	→ Remove the table

Interview definition:

TRUNCATE TABLE is a DDL command that removes all rows from a table while preserving the table's structure.

- **RENAME TABLE**

RENAME TABLE

RENAME TABLE is used to **change the name of an existing table** without deleting its data.

The exact syntax depends on the database system.

1. MySQL

Syntax

```
RENAME TABLE OldTableName TO NewTableName;
```

Example

```
RENAME TABLE Employees TO Staff;
```

Before:

```
Employees
```

After:

```
Staff
```

The data remains unchanged.

```
SELECT * FROM Staff;
```

2. SQL Server

SQL Server commonly uses `sp_rename`:

```
EXEC sp_rename 'Employees', 'Staff';
```

Now the table is called:

```
Staff
```

3. PostgreSQL

PostgreSQL uses:

```
ALTER TABLE Employees  
RENAME TO Staff;
```

What Happens When You Rename?

Suppose we have:

```
CREATE TABLE Employees  
(  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Salary DECIMAL(10,2)  
);
```

Data:

EmployeeId	Name	Salary
1	John	50000
2	Alice	60000

Rename it:

```
ALTER TABLE Employees  
RENAME TO Staff;
```

The result is:

Employees ❌
↓
Staff ✅

The data is still there:

```
SELECT * FROM Staff;
```

Does RENAME Delete Data?

No.

```
RENAME TABLE  
↓  
Change table name  
↓  
Data remains  
↓  
Columns remain  
↓  
Table structure remains
```

Important Interview Point

`RENAME TABLE` is **database-specific**.

Database	Rename Syntax
MySQL	<code>RENAME TABLE Employees TO Staff;</code>

Database	Rename Syntax
SQL Server	<code>EXEC sp_rename 'Employees', 'Staff';</code>
PostgreSQL	<code>ALTER TABLE Employees RENAME TO Staff;</code>
Oracle	<code>RENAME Employees TO Staff;</code>

Easy way to remember

RENAME → Change the table name

Interview definition:

Renaming a table changes its name while preserving its data and structure. The syntax varies between database systems.

3. Data Manipulation (DML)

- `INSERT`

INSERT in SQL

`INSERT` is a **DML (Data Manipulation Language)** command used to **add new rows/records** to a table.

Basic Syntax

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, value2, value3);
```

1. Insert a single row

Suppose we have:

```
CREATE TABLE Employees (
  Id INT,
  Name VARCHAR(100),
  Department VARCHAR(50),
  Salary DECIMAL(10,2)
);
```

Insert a record:

```
INSERT INTO Employees (Id, Name, Department, Salary)
VALUES (1, 'Rahul', 'IT', 50000);
```


Result:

Id	Name	Department	Salary
1	Rahul	IT	50000

2. Insert multiple rows

You can insert multiple records in one statement:

```
INSERT INTO Employees (Id, Name, Department, Salary)
VALUES
  (2, 'Amit', 'HR', 45000),
  (3, 'Priya', 'IT', 60000),
  (4, 'Neha', 'Finance', 55000);
```

3. Insert without specifying column names

```
INSERT INTO Employees
VALUES (5, 'Raj', 'IT', 52000);
```

This works only when the values are provided in the **same order as the table columns**.

It is generally better to explicitly specify columns:

```
INSERT INTO Employees (Id, Name, Department, Salary)
VALUES (5, 'Raj', 'IT', 52000);
```

4. Insert only selected columns

If other columns allow **NULL** or have default values:

```
INSERT INTO Employees (Id, Name)
VALUES (6, 'Karan');
```

For example:

Id	Name	Department	Salary
6	Karan	NULL	NULL

5. Insert using **DEFAULT**

If a column has a default value:

```
CREATE TABLE Employees (  
    Id INT,  
    Name VARCHAR(100),  
    Department VARCHAR(50) DEFAULT 'IT'  
);
```

Then:

```
INSERT INTO Employees (Id, Name)  
VALUES (7, 'Ravi');
```

Department automatically becomes:

```
IT
```

You can also explicitly use:

```
INSERT INTO Employees (Id, Name, Department)  
VALUES (8, 'Anil', DEFAULT);
```

6. Insert data from another table

You can use **INSERT ... SELECT**:

```
INSERT INTO Employees (Id, Name, Department, Salary)  
SELECT Id, Name, Department, Salary  
FROM OldEmployees;
```

This copies records from **OldEmployees** into **Employees**.

You can also filter them:

```
INSERT INTO Employees (Id, Name, Department, Salary)  
SELECT Id, Name, Department, Salary  
FROM OldEmployees  
WHERE Department = 'IT';
```

7. Insert with an identity/auto-increment column

If **Id** is automatically generated:

```
CREATE TABLE Employees (  
    Id INT IDENTITY(1,1),  
    Name VARCHAR(100),  
    Department VARCHAR(50),  
    Salary DECIMAL(10,2)  
);
```

You don't provide **Id**:

```
INSERT INTO Employees (Name, Department, Salary)  
VALUES ('Rahul', 'IT', 50000);
```

The database generates the **Id**.

For example:

Id	Name
1	Rahul

8. INSERT with NULL

If a column accepts **NULL**:

```
INSERT INTO Employees (Id, Name, Department, Salary)  
VALUES (10, 'Vijay', NULL, 40000);
```

Here **Department** is **NULL**.

9. INSERT and constraints

An **INSERT** can fail because of constraints.

Primary Key

```
INSERT INTO Employees (Id, Name)  
VALUES (1, 'John');
```

If **Id = 1** already exists and **Id** is a primary key, the insert fails.

NOT NULL

```
Name VARCHAR(100) NOT NULL
```

Then this fails:

```
INSERT INTO Employees (Id)
VALUES (11);
```

because **Name** is required.

UNIQUE

If **Email** is unique:

```
INSERT INTO Employees (Id, Name, Email)
VALUES (12, 'John', 'john@example.com');
```

Trying to insert the same email again can fail.

10. Verify inserted data

After inserting:

```
SELECT *
FROM Employees;
```

Important points

Feature	Example
Single row	INSERT INTO ... VALUES (...)
Multiple rows	VALUES (...), (...), (...)
Copy from another table	INSERT ... SELECT
Automatic value	DEFAULT
Auto-generated ID	Identity/auto-increment
Add NULL	VALUES (... , NULL)

Most important syntax to remember

```
INSERT INTO Employees (Id, Name, Department, Salary)
VALUES (1, 'Rahul', 'IT', 50000);
```

Remember: **INSERT** adds **new rows**; it does not modify existing rows.

- UPDATE

UPDATE in SQL

UPDATE is a **DML (Data Manipulation Language)** command used to **modify existing records** in a table.

Basic Syntax

```
UPDATE table_name
SET column1 = value1,
    column2 = value2
WHERE condition;
```

⚠ Always be careful with the **WHERE** clause. Without it, **all rows** can be updated.

1. Update a single row

Suppose we have:

Employees

Id	Name	Department	Salary
1	Rahul	IT	50000
2	Amit	HR	45000
3	Priya	IT	60000

Update Rahul's salary:

```
UPDATE Employees
SET Salary = 55000
WHERE Id = 1;
```

Result:

Id	Name	Department	Salary
1	Rahul	IT	55000
2	Amit	HR	45000
3	Priya	IT	60000

2. Update multiple columns

You can update multiple columns in one statement:

```
UPDATE Employees
SET Department = 'Management',
    Salary = 70000
WHERE Id = 1;
```

3. Update multiple rows

Update the salary of all IT employees:

```
UPDATE Employees
SET Salary = Salary + 5000
WHERE Department = 'IT';
```

If there are three IT employees, all three will receive the increase.

4. Update using a calculation

You can use expressions in **SET**:

```
UPDATE Employees
SET Salary = Salary * 1.10
WHERE Department = 'IT';
```

This increases IT employees' salary by **10%**.

5. Update a string value

```
UPDATE Employees
SET Name = 'Rahul Sharma'
WHERE Id = 1;
```

6. Update using multiple conditions

```
UPDATE Employees
SET Salary = 65000
WHERE Department = 'IT'
    AND Salary < 60000;
```

Only IT employees whose salary is below 60000 are updated.

7. Update using **IN**

```
UPDATE Employees
SET Salary = Salary + 3000
WHERE Department IN ('IT', 'Finance');
```

Employees from either IT or Finance receive the increase.

8. Update using **LIKE**

```
UPDATE Employees
SET Department = 'Information Technology'
WHERE Department LIKE 'IT%';
```

Useful when matching text patterns.

9. Update using **NULL**

Set a value to **NULL**:

```
UPDATE Employees
SET Department = NULL
WHERE Id = 3;
```

This works only if the column allows **NULL**.

To find NULL values:

```
SELECT *
FROM Employees
WHERE Department IS NULL;
```

10. Update all rows

You can intentionally update every row:

```
UPDATE Employees
SET Salary = Salary + 1000;
```

There is **no WHERE clause**, so every employee gets a 1000 increase.

Dangerous example

```
UPDATE Employees
SET Salary = 0;
```

This sets **everyone's salary to 0**.

Always verify the query before executing it in production.

11. Update using another table

You can update one table using data from another table.

For example:

```
UPDATE Employees
SET Salary = (
    SELECT Salary
    FROM EmployeeUpdates
    WHERE EmployeeUpdates.Id = Employees.Id
)
WHERE EXISTS (
    SELECT 1
    FROM EmployeeUpdates
    WHERE EmployeeUpdates.Id = Employees.Id
);
```

The exact syntax can vary by database system.

12. Check before updating

A good practice is to first run a **SELECT** using the same condition:

```
SELECT *
FROM Employees
WHERE Department = 'IT'
AND Salary < 60000;
```

If the returned rows are correct, execute:

```
UPDATE Employees
SET Salary = Salary + 5000
```



```
WHERE Department = 'IT'  
AND Salary < 60000;
```

This is especially useful in production environments.

13. UPDATE with transactions

For important operations, use a transaction:

```
BEGIN TRANSACTION;  
  
UPDATE Employees  
SET Salary = Salary + 5000  
WHERE Department = 'IT';  
  
-- Check the result  
  
COMMIT;
```

If something goes wrong:

```
ROLLBACK;
```

UPDATE VS INSERT VS DELETE

Command	Purpose
INSERT	Adds new rows
UPDATE	Modifies existing rows
DELETE	Removes existing rows
SELECT	Retrieves rows

Most important syntax

```
UPDATE Employees  
SET Salary = 60000  
WHERE Id = 1;
```

Remember: UPDATE = change existing data.

- DELETE

DELETE in SQL

DELETE is a **DML (Data Manipulation Language)** command used to **remove existing rows from a table**.

Basic Syntax

```
DELETE FROM table_name
WHERE condition;
```

⚠ The **WHERE** clause is extremely important. Without it, **all rows in the table are deleted**.

1. Delete a single row

Suppose we have:

Id	Name	Department	Salary
1	Rahul	IT	50000
2	Amit	HR	45000
3	Priya	IT	60000

Delete Rahul:

```
DELETE FROM Employees
WHERE Id = 1;
```

Result:

Id	Name	Department	Salary
2	Amit	HR	45000
3	Priya	IT	60000

2. Delete multiple rows

Delete all employees from HR:

```
DELETE FROM Employees
WHERE Department = 'HR';
```

Every row satisfying the condition is deleted.

3. Delete using multiple conditions

```
DELETE FROM Employees
WHERE Department = 'IT'
AND Salary < 50000;
```

Only IT employees with salary below 50000 are deleted.

4. Delete using **IN**

```
DELETE FROM Employees
WHERE Department IN ('HR', 'Finance');
```

This deletes employees belonging to either HR or Finance.

5. Delete using **LIKE**

```
DELETE FROM Employees
WHERE Name LIKE 'Rahul%';
```

This deletes employees whose names start with **Rahul**.

6. Delete rows with **NULL**

Use **IS NULL**:

```
DELETE FROM Employees
WHERE Department IS NULL;
```

Do **not** use:

```
DELETE FROM Employees
WHERE Department = NULL;
```

NULL must be checked using **IS NULL** or **IS NOT NULL**.

7. Delete all rows

```
DELETE FROM Employees;
```

Because there is no **WHERE** clause, **all rows are deleted**.

However, the **table itself remains**.

You can still run:

```
SELECT *  
FROM Employees;
```

The result will contain zero rows.

8. DELETE vs TRUNCATE

This is an important SQL interview question.

Feature	DELETE	TRUNCATE
Removes rows	Yes	Yes
WHERE supported	Yes	No
Can remove selected rows	Yes	No
Table remains	Yes	Yes
Usually faster for all rows	Slower	Faster
DML/DDDL classification	DML	Commonly classified as DDL
Identity reset	DB-specific	Often resets/reseeds identity in systems such as SQL Server

Example:

```
DELETE FROM Employees  
WHERE Id = 5;
```

removes **specific rows**.

```
TRUNCATE TABLE Employees;
```

removes **all rows**.

9. DELETE vs DROP

Another important distinction:

DELETE

```
DELETE FROM Employees;
```

Removes the **data**, but keeps the table.

DROP

```
DROP TABLE Employees;
```

Removes the **entire table structure and its data**.

Think:

```
DELETE    → Remove rows
TRUNCATE  → Remove all rows quickly
DROP      → Remove the table
```

10. Using a transaction

For important deletes, you can use a transaction:

```
BEGIN TRANSACTION;

DELETE FROM Employees
WHERE Department = 'IT';

-- Verify the result

COMMIT;
```

If you discover a mistake before committing:

```
ROLLBACK;
```

This can undo the transaction where supported and depending on the database/transaction state.

11. Best practice before DELETE

Before running:

```
DELETE FROM Employees
WHERE Department = 'IT'
```

```
AND Salary < 50000;
```

First run:

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
AND Salary < 50000;
```

Check that the returned rows are exactly the rows you want to remove.

Then execute the **DELETE**.

Quick comparison

Command	Example	Result
INSERT	INSERT INTO Employees ...	Adds rows
UPDATE	UPDATE Employees SET ...	Changes rows
DELETE	DELETE FROM Employees WHERE ...	Removes selected rows
TRUNCATE	TRUNCATE TABLE Employees	Removes all rows
DROP	DROP TABLE Employees	Removes table

Most important syntax

```
DELETE FROM Employees  
WHERE Id = 10;
```

Remember: **DELETE** = remove existing rows from a table.

- **MERGE** (where supported)

MERGE in SQL

MERGE is a SQL command used to **synchronize data between a source and target table**.

It can perform different operations depending on whether a matching record exists:

- **Match found** → **UPDATE**
- **No match found** → **INSERT**
- In supported databases, additional conditions can also perform **DELETE**

This makes **MERGE** useful for **upsert** operations.

⚠ **MERGE** syntax and support vary significantly between database systems such as SQL Server, Oracle, PostgreSQL, and others.

1. Basic Concept

Suppose we have a target table:

Employees

Id	Name	Department	Salary
1	Rahul	IT	50000
2	Amit	HR	45000
3	Priya	IT	60000

And a source table:

EmployeeUpdates

Id	Name	Department	Salary
1	Rahul	IT	55000
4	Neha	Finance	50000

We want:

Id = 1 → Already exists → UPDATE
Id = 4 → Doesn't exist → INSERT

2. SQL Server **MERGE** Example

A typical SQL Server syntax is:

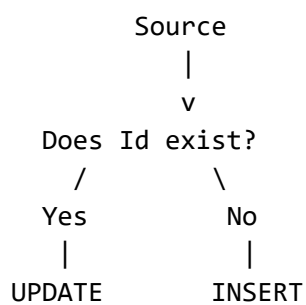
```
MERGE Employees AS Target
USING EmployeeUpdates AS Source
ON Target.Id = Source.Id

WHEN MATCHED THEN
    UPDATE SET
        Target.Name = Source.Name,
        Target.Department = Source.Department,
        Target.Salary = Source.Salary
```

```
WHEN NOT MATCHED BY TARGET THEN
  INSERT (Id, Name, Department, Salary)
  VALUES (
    Source.Id,
    Source.Name,
    Source.Department,
    Source.Salary
  );
```

What happens?

For each source record:



So:

```
Id = 1 → UPDATE Rahul's salary to 55000
Id = 4 → INSERT Neha
```

3. WHEN MATCHED

WHEN MATCHED executes when the source row matches an existing target row.

```
WHEN MATCHED THEN
  UPDATE SET
    Target.Salary = Source.Salary;
```

For example:

```
Target:
1 | Rahul | IT | 50000

Source:
1 | Rahul | IT | 55000
```

Because **Id = 1** matches:


```
50000 → 55000
```

4. WHEN NOT MATCHED

WHEN NOT MATCHED BY TARGET is used when a source row **doesn't exist in the target**.

```
WHEN NOT MATCHED BY TARGET THEN
  INSERT (Id, Name, Department, Salary)
  VALUES (
    Source.Id,
    Source.Name,
    Source.Department,
    Source.Salary
  );
```

For example:

```
Source:
4 | Neha | Finance | 50000
```

```
Target:
No Id = 4
```

Therefore:

```
INSERT Id = 4
```

5. WHEN NOT MATCHED BY SOURCE

Some implementations, such as SQL Server, also support:

```
WHEN NOT MATCHED BY SOURCE THEN
  DELETE;
```

This means:

If a target row has no corresponding row in the source, delete the target row.

For example:

Target:
1 Rahul
2 Amit
3 Priya

Source:
1 Rahul
2 Amit

Id = 3 exists in the target but not in the source.

Therefore:

Id = 3 → DELETE

A complete example:

```
MERGE Employees AS Target
USING EmployeeUpdates AS Source
ON Target.Id = Source.Id

WHEN MATCHED THEN
  UPDATE SET
    Target.Name = Source.Name,
    Target.Department = Source.Department,
    Target.Salary = Source.Salary

WHEN NOT MATCHED BY TARGET THEN
  INSERT (Id, Name, Department, Salary)
  VALUES (
    Source.Id,
    Source.Name,
    Source.Department,
    Source.Salary
  )

WHEN NOT MATCHED BY SOURCE THEN
  DELETE;
```

⚠ Be very careful with `WHEN NOT MATCHED BY SOURCE THEN DELETE`, because it can delete target records that aren't present in the source.

6. Conditional UPDATE

You can add conditions to `WHEN MATCHED`.

For example, update only when the source salary is different:

```
MERGE Employees AS Target
USING EmployeeUpdates AS Source
ON Target.Id = Source.Id

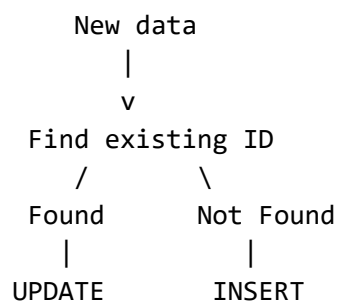
WHEN MATCHED
    AND Target.Salary <> Source.Salary
THEN
    UPDATE SET
        Target.Salary = Source.Salary;
```

7. MERGE as an UPSERT

A common reason to use **MERGE** is an **upsert**.

UPSERT = UPDATE + INSERT

Conceptually:



Example:

```
MERGE Employees AS Target
USING EmployeeUpdates AS Source
ON Target.Id = Source.Id

WHEN MATCHED THEN
    UPDATE SET
        Target.Salary = Source.Salary

WHEN NOT MATCHED BY TARGET THEN
    INSERT (Id, Name, Department, Salary)
    VALUES (
        Source.Id,
        Source.Name,
        Source.Department,
        Source.Salary
    );
```

8. MERGE with a query as the source

The source doesn't necessarily have to be a table. It can be a query.

For example:

```
MERGE Employees AS Target
USING (
    SELECT Id, Name, Department, Salary
    FROM EmployeeUpdates
    WHERE Department = 'IT'
) AS Source
ON Target.Id = Source.Id

WHEN MATCHED THEN
    UPDATE SET
        Target.Salary = Source.Salary

WHEN NOT MATCHED BY TARGET THEN
    INSERT (Id, Name, Department, Salary)
    VALUES (
        Source.Id,
        Source.Name,
        Source.Department,
        Source.Salary
    );
```

9. MERGE vs INSERT vs UPDATE

Command	Purpose
INSERT	Add new records
UPDATE	Modify existing records
DELETE	Remove records
MERGE	Synchronize source and target data

For example:

```
-- INSERT
INSERT INTO Employees (...)
VALUES (...);
```

Only inserts.

```
-- UPDATE
UPDATE Employees
SET Salary = 60000
WHERE Id = 1;
```

Only updates matching records.

```
-- MERGE
MERGE Employees AS Target
USING EmployeeUpdates AS Source
ON Target.Id = Source.Id
...
```

Can decide whether to **insert or update** based on the match.

10. Important **MERGE** caution

Although **MERGE** looks convenient, you should **not automatically use it everywhere**.

Different database engines implement it differently, and some have documented edge cases involving concurrency, duplicate source matches, triggers, and other behavior.

For example, in SQL Server, a source containing multiple rows matching the same target row can cause problems with a **WHEN MATCHED** update.

Therefore, make sure the source data has a suitable unique key:

```
Source
Id
1
2
3
```

rather than:

```
Source
Id
1
1    ← duplicate match
2
```

For many applications, explicit **UPDATE** + **INSERT** logic or a database-specific upsert feature can be easier to reason about.

11. Interview Question

What is **MERGE**?

Answer:

MERGE is a SQL command used to synchronize data between a source and target table. It compares rows using a matching condition and can perform **UPDATE** when a row exists and **INSERT** when it doesn't. Some database systems also support conditional **DELETE**.

Easy way to remember

```
MERGE
|
+-- MATCHED
|    └─ UPDATE
|
+-- NOT MATCHED
|    └─ INSERT
|
|-- NOT MATCHED BY SOURCE
|    └─ DELETE (where supported)
```

Key idea: **MERGE** = **compare source with target and take the appropriate data operation.**

4. Data Querying (DQL)

- **SELECT**

SQL **SELECT**

SELECT is the **most important SQL command for retrieving data** from a database table.

It is mainly used in **DQL (Data Query Language)**.

1. Basic Syntax

```
SELECT column1, column2
FROM table_name;
```

Example:

```
SELECT Name, Salary
FROM Employees;
```

This returns only the **Name** and **Salary** columns.

2. Select All Columns

Use ***** to retrieve every column:

```
SELECT *  
FROM Employees;
```

Example result:

Id	Name	Department	Salary
1	Rahul	IT	60000
2	Amit	HR	50000
3	Priya	IT	70000

3. Select Specific Columns

```
SELECT Name, Department  
FROM Employees;
```

Result:

Name	Department
Rahul	IT
Amit	HR
Priya	IT

Best practice: Prefer selecting specific columns instead of **SELECT *** when you don't need every column.

4. **SELECT** with **WHERE**

You can filter the records:

```
SELECT Name, Salary  
FROM Employees  
WHERE Salary > 60000;
```

Result:

Name	Salary
Priya	70000

5. **SELECT** with **ORDER BY**

Sort the result:

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary DESC;
```

Result:

Name	Salary
Priya	70000
Rahul	60000
Amit	50000

6. **SELECT DISTINCT**

Remove duplicate values:

```
SELECT DISTINCT Department
FROM Employees;
```

Result:

Department
IT
HR

7. **SELECT** with an Expression

You can perform calculations:

```
SELECT Name, Salary, Salary * 12 AS AnnualSalary
FROM Employees;
```

Result:

Name	Salary	AnnualSalary
Rahul	60000	720000
Amit	50000	600000
Priya	70000	840000

AS creates an **alias** for the calculated column.

8. SELECT with TOP — SQL Server

```
SELECT TOP 2 *  
FROM Employees;
```

Returns only 2 rows.

For the highest salaries:

```
SELECT TOP 2 Name, Salary  
FROM Employees  
ORDER BY Salary DESC;
```

9. SELECT with LIMIT — MySQL/PostgreSQL

```
SELECT *  
FROM Employees  
LIMIT 2;
```

Returns 2 rows.

10. SELECT with OFFSET

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 2 OFFSET 2;
```

Meaning:

```
OFFSET 2 → Skip 2 rows  
LIMIT 2 → Return next 2 rows
```

11. SELECT with Multiple Conditions

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
AND Salary > 60000;
```

12. SELECT with IN

```
SELECT *  
FROM Employees  
WHERE Department IN ('IT', 'HR');
```

13. SELECT with LIKE

Find employees whose names start with R:

```
SELECT *  
FROM Employees  
WHERE Name LIKE 'R%';
```

14. SELECT with NULL

To find records where salary is missing:

```
SELECT *  
FROM Employees  
WHERE Salary IS NULL;
```

Not:

```
WHERE Salary = NULL
```

Use IS NULL / IS NOT NULL.

Complete Example

```
SELECT Name, Department, Salary  
FROM Employees
```

```
WHERE Salary >= 50000
ORDER BY Salary DESC
LIMIT 5;
```

This means:

```
SELECT    → Which columns?
FROM      → Which table?
WHERE     → Which rows?
ORDER BY  → What order?
LIMIT     → How many rows?
```

SQL Server version

```
SELECT TOP 5 Name, Department, Salary
FROM Employees
WHERE Salary >= 50000
ORDER BY Salary DESC;
```

Important **SELECT** patterns to learn

```
SELECT *
SELECT column1, column2
SELECT DISTINCT column
SELECT ... WHERE ...
SELECT ... ORDER BY ...
SELECT ... GROUP BY ...
SELECT ... HAVING ...
SELECT ... JOIN ...
SELECT ... LIMIT ...
SELECT TOP ...
SELECT ... OFFSET ...
SELECT ... subquery
SELECT ... window function
```

SELECT becomes much more powerful when combined with **WHERE**, **JOIN**, **GROUP BY**, **HAVING**, **subqueries**, **CTEs**, and **window functions**.

- **DISTINCT**

SQL **DISTINCT**

DISTINCT is used with **SELECT** to **remove duplicate rows from the result**.

It returns only **unique values/combinations**.

1. Basic Syntax

```
SELECT DISTINCT column_name  
FROM table_name;
```

Example table: **Employees**

Id	Name	Department	Salary
1	Rahul	IT	60000
2	Amit	HR	50000
3	Priya	IT	70000
4	John	HR	55000

Without **DISTINCT**:

```
SELECT Department  
FROM Employees;
```

Result:

```
IT  
HR  
IT  
HR
```

With **DISTINCT**:

```
SELECT DISTINCT Department  
FROM Employees;
```

Result:

```
IT  
HR
```

2. **DISTINCT** with Multiple Columns

You can use **DISTINCT** with multiple columns:

```
SELECT DISTINCT Department, Salary
```

```
FROM Employees;
```

Here, SQL considers the **combination of Department and Salary**.

For example:

Department	Salary
IT	60000
IT	60000
IT	70000
HR	50000

Result:

Department	Salary
IT	60000
IT	70000
HR	50000

Only completely duplicate combinations are removed.

3. **DISTINCT** Does Not Mean Distinct for Each Column Separately

Consider:

```
SELECT DISTINCT Department, Salary
FROM Employees;
```

It does **not** mean:

```
unique departments
+
unique salaries
```

Instead, it means:

```
unique (Department, Salary) combinations
```

4. **DISTINCT** with **WHERE**

You can combine **DISTINCT** and **WHERE**:

```
SELECT DISTINCT Department
FROM Employees
WHERE Salary > 55000;
```

SQL first considers the matching rows and returns unique departments.

5. **DISTINCT** with **ORDER BY**

```
SELECT DISTINCT Department
FROM Employees
ORDER BY Department;
```

Result:

```
HR
IT
```

Descending:

```
SELECT DISTINCT Department
FROM Employees
ORDER BY Department DESC;
```

6. **DISTINCT** with **COUNT**

A very common use is counting unique values.

Count all employees

```
SELECT COUNT(*)
FROM Employees;
```

Count unique departments

```
SELECT COUNT(DISTINCT Department)
FROM Employees;
```

If there are:

```
IT
IT
```

```
HR
HR
Finance
```

Then:

```
SELECT COUNT(DISTINCT Department)
FROM Employees;
```

returns:

```
3
```

7. **DISTINCT** and **NULL**

DISTINCT treats multiple **NULL** values as one distinct result.

Example:

Department
IT
HR
NULL
IT
NULL

```
SELECT DISTINCT Department
FROM Employees;
```

Result will contain:

```
IT
HR
NULL
```

8. **DISTINCT** vs **GROUP BY**

Both can sometimes produce unique values.

Using **DISTINCT**

```
SELECT DISTINCT Department
FROM Employees;
```

Using GROUP BY

```
SELECT Department
FROM Employees
GROUP BY Department;
```

For simply getting unique values, **DISTINCT** is usually clearer.

GROUP BY becomes more useful when performing aggregate calculations:

```
SELECT Department, COUNT(*)
FROM Employees
GROUP BY Department;
```

Result:

Department	COUNT
IT	2
HR	2

Real-World Examples

Get unique customer cities

```
SELECT DISTINCT City
FROM Customers;
```

Get unique product categories

```
SELECT DISTINCT Category
FROM Products;
```

Get unique job titles

```
SELECT DISTINCT JobTitle
FROM Employees;
```


Count unique customers

```
SELECT COUNT(DISTINCT CustomerId)
FROM Orders;
```

Get unique combinations

```
SELECT DISTINCT Country, City
FROM Customers;
```

Important Point

DISTINCT applies to the **entire selected row**.

```
SELECT DISTINCT Department, Salary
FROM Employees;
```

Think of it as:

```
DISTINCT(Department + Salary)
```

not:

```
DISTINCT(Department)
DISTINCT(Salary)
```

Quick Reference

Query	Purpose
SELECT DISTINCT Department	Unique departments
SELECT DISTINCT City	Unique cities
SELECT DISTINCT A, B	Unique A+B combinations
COUNT(DISTINCT Id)	Count unique IDs
DISTINCT + WHERE	Unique values after filtering
DISTINCT + ORDER BY	Unique values sorted

Next: **WHERE** — used to filter the rows returned by **SELECT**.

- **WHERE**

SQL WHERE

WHERE is used to **filter rows** based on a condition.

It is one of the most important SQL clauses because it lets you retrieve **only the records you need**.

1. Basic Syntax

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example:

```
SELECT *
FROM Employees
WHERE Department = 'IT';
```

This returns only employees whose department is **IT**.

2. Example Table

Suppose **Employees** contains:

Id	Name	Department	Salary	City
1	Rahul	IT	60000	Mumbai
2	Amit	HR	50000	Pune
3	Priya	IT	70000	Mumbai
4	John	Finance	55000	Delhi
5	Neha	IT	65000	Pune

3. Using =

Find employees from IT:

```
SELECT *
FROM Employees
WHERE Department = 'IT';
```

Result:

Id	Name	Department	Salary
1	Rahul	IT	60000
3	Priya	IT	70000
5	Neha	IT	65000

4. Comparison Operators

WHERE supports comparison operators:

Operator	Meaning
=	Equal
<>	Not equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Greater than

```
SELECT *  
FROM Employees  
WHERE Salary > 60000;
```

Returns employees earning more than 60000.

Greater than or equal

```
SELECT *  
FROM Employees  
WHERE Salary >= 60000;
```

Includes employees earning exactly 60000.

Not equal

```
SELECT *  
FROM Employees  
WHERE Department <> 'IT';
```

Returns employees who are not in IT.

5. WHERE with AND

AND means **all conditions must be true**.

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
AND Salary > 60000;
```

This means:

```
Department must be IT  
    AND  
Salary must be greater than 60000
```

Result:

Name	Department	Salary
Priya	IT	70000
Neha	IT	65000

6. WHERE with OR

OR means **at least one condition must be true**.

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
OR Department = 'HR';
```

This returns employees from either IT or HR.

7. AND + OR

You can combine conditions.

```
SELECT *
FROM Employees
WHERE Department = 'IT'
AND (Salary > 60000 OR City = 'Mumbai');
```

Important

Use parentheses when combining **AND** and **OR** to make the intended logic clear.

8. IN

IN checks whether a value exists in a list.

Instead of:

```
SELECT *
FROM Employees
WHERE Department = 'IT'
      OR Department = 'HR'
      OR Department = 'Finance';
```

You can write:

```
SELECT *
FROM Employees
WHERE Department IN ('IT', 'HR', 'Finance');
```

Much cleaner.

9. NOT IN

Find employees whose department isn't IT or HR:

```
SELECT *
FROM Employees
WHERE Department NOT IN ('IT', 'HR');
```

10. BETWEEN

Used to find values within a range.

```
SELECT *  
FROM Employees  
WHERE Salary BETWEEN 50000 AND 60000;
```

BETWEEN is generally **inclusive of both boundaries**.

So this includes:

```
50000  
55000  
60000
```

11. NOT BETWEEN

```
SELECT *  
FROM Employees  
WHERE Salary NOT BETWEEN 50000 AND 60000;
```

Returns salaries outside that range.

12. LIKE

LIKE is used for **pattern matching**.

Starts with **R**

```
SELECT *  
FROM Employees  
WHERE Name LIKE 'R%';
```

% means zero or more characters.

Examples matching **R%**:

```
Rahul  
Raj  
Rohit
```

Ends with **a**

```
SELECT *  
FROM Employees  
WHERE Name LIKE '%a';
```

Contains **ri**

```
SELECT *  
FROM Employees  
WHERE Name LIKE '%ri%';
```

13. **_** Wildcard

_ represents **exactly one character**.

```
SELECT *  
FROM Employees  
WHERE Name LIKE 'R_hu1';
```

For example:

```
Rahu1
```

can match because **_** represents one character.

14. **IS NULL**

To find missing values:

```
SELECT *  
FROM Employees  
WHERE City IS NULL;
```

Important

Don't write:

```
WHERE City = NULL
```

Use:

```
WHERE City IS NULL
```

For non-null values:

```
SELECT *  
FROM Employees  
WHERE City IS NOT NULL;
```

15. WHERE with Dates

Suppose you have:

```
HireDate
```

You can filter by date:

```
SELECT *  
FROM Employees  
WHERE HireDate >= '2025-01-01';
```

For date ranges:

```
SELECT *  
FROM Employees  
WHERE HireDate BETWEEN '2025-01-01' AND '2025-12-31';
```

For production SQL, be careful with **DATETIME** columns because an upper-bound date can exclude rows later that same day. A half-open range is often safer:

```
SELECT *  
FROM Employees  
WHERE HireDate >= '2025-01-01'  
      AND HireDate < '2026-01-01';
```

16. WHERE with SELECT

You normally use **WHERE** with **SELECT**:


```
SELECT Name, Salary
FROM Employees
WHERE Salary > 60000;
```

17. WHERE with UPDATE

WHERE is extremely important with UPDATE.

```
UPDATE Employees
SET Salary = 75000
WHERE Id = 3;
```

Only employee `Id = 3` is updated.

⚠ Without WHERE:

```
UPDATE Employees
SET Salary = 75000;
```

Every employee's salary is updated.

18. WHERE with DELETE

Similarly:

```
DELETE FROM Employees
WHERE Id = 3;
```

Only employee 3 is deleted.

⚠ Without WHERE:

```
DELETE FROM Employees;
```

All rows are deleted.

19. WHERE vs HAVING

This is an important interview question.

WHERE

Filters **rows before grouping**:

```
SELECT *  
FROM Employees  
WHERE Salary > 60000;
```

HAVING

Filters **groups after GROUP BY**:

```
SELECT Department, COUNT(*) AS EmployeeCount  
FROM Employees  
GROUP BY Department  
HAVING COUNT(*) > 2;
```

Think:

```
WHERE → filter rows  
HAVING → filter groups
```

20. Combining Everything

Example:

```
SELECT Name, Department, Salary  
FROM Employees  
WHERE Department IN ('IT', 'Finance')  
AND Salary >= 60000  
ORDER BY Salary DESC;
```

SQL is essentially saying:

```
SELECT      → Give me Name, Department, Salary  
FROM        → from Employees  
WHERE       → only IT or Finance employees  
AND         → whose salary is at least 60000  
ORDER BY    → highest salary first
```

Common WHERE Patterns

```
-- Equality
WHERE Salary = 60000

-- Greater than
WHERE Salary > 60000

-- Multiple conditions
WHERE Department = 'IT' AND Salary > 60000

-- Either condition
WHERE Department = 'IT' OR Department = 'HR'

-- List
WHERE Department IN ('IT', 'HR')

-- Range
WHERE Salary BETWEEN 50000 AND 70000

-- Pattern
WHERE Name LIKE 'R%'

-- NULL
WHERE City IS NULL

-- NOT NULL
WHERE City IS NOT NULL
```

Interview Tip

The most important things to understand about **WHERE** are:

=, <>, >, <, >=, <=, AND, OR, IN, NOT IN, BETWEEN, LIKE, IS NULL, and IS NOT NULL.

- **ORDER BY**

SQL ORDER BY

ORDER BY is used to **sort the rows returned by a query.**

You can sort data in:

- **Ascending order** → **ASC**
- **Descending order** → **DESC**

1. Basic Syntax

```
SELECT column1, column2
FROM table_name
```

```
ORDER BY column_name;
```

By default, **ORDER BY** uses **ascending order (ASC)**.

2. Example Table

Suppose we have:

Id	Name	Department	Salary
1	Rahul	IT	60000
2	Amit	HR	50000
3	Priya	IT	70000
4	John	Finance	55000
5	Neha	IT	65000

3. Ascending Order — ASC

Sort salary from lowest to highest:

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary ASC;
```

Result:

Name	Salary
Amit	50000
John	55000
Rahul	60000
Neha	65000
Priya	70000

ASC means:

```
Small → Large
A → Z
Old → New
```

4. Descending Order — DESC

Sort salary from highest to lowest:

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary DESC;
```

Result:

Name	Salary
Priya	70000
Neha	65000
Rahul	60000
John	55000
Amit	50000

DESC means:

```
Large → Small
Z → A
New → Old
```

5. ORDER BY Without ASC

This:

```
SELECT *
FROM Employees
ORDER BY Salary;
```

is equivalent to:

```
SELECT *
FROM Employees
ORDER BY Salary ASC;
```

ASC is the default.

6. Sorting Text

You can sort text columns.

```
SELECT *  
FROM Employees  
ORDER BY Name ASC;
```

Result:

```
Amit  
John  
Neha  
Priya  
Rahul
```

Descending:

```
SELECT *  
FROM Employees  
ORDER BY Name DESC;
```

Result:

```
Rahul  
Priya  
Neha  
John  
Amit
```

7. Sorting by Multiple Columns

This is very important.

```
SELECT *  
FROM Employees  
ORDER BY Department ASC, Salary DESC;
```

SQL first sorts by:

```
Department → ASC
```

Then, employees within the same department are sorted by:

```
Salary → DESC
```

For example:

Name	Department	Salary
John	Finance	55000
Amit	HR	50000
Priya	IT	70000
Neha	IT	65000
Rahul	IT	60000

Notice that IT employees are ordered by salary from highest to lowest.

8. ORDER BY with WHERE

You can filter first and then sort the result.

```
SELECT Name, Department, Salary
FROM Employees
WHERE Department = 'IT'
ORDER BY Salary DESC;
```

Result:

Name	Department	Salary
Priya	IT	70000
Neha	IT	65000
Rahul	IT	60000

9. ORDER BY with DISTINCT

```
SELECT DISTINCT Department
FROM Employees
ORDER BY Department ASC;
```

Result:

Finance
HR
IT

10. ORDER BY with TOP

In SQL Server, you can get the highest-paid employees:

```
SELECT TOP 3 Name, Salary
FROM Employees
ORDER BY Salary DESC;
```

Result:

Name	Salary
Priya	70000
Neha	65000
Rahul	60000

⚠ The **ORDER BY** is important. Without it, **TOP 3** does **not** mean the three highest-paid employees.

11. ORDER BY with LIMIT

MySQL/PostgreSQL:

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 3;
```

This means:

```
ORDER BY Salary DESC → highest salary first
LIMIT 3                → return only 3 rows
```

12. ORDER BY with OFFSET

For pagination:


```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 10 OFFSET 20;
```

Meaning:

```
ORDER BY → establish a predictable order  
OFFSET 20 → skip 20 rows  
LIMIT 10 → return the next 10 rows
```

13. ORDER BY Using Column Position

Some databases allow:

```
SELECT Name, Department, Salary  
FROM Employees  
ORDER BY 3 DESC;
```

Here:

```
1 → Name  
2 → Department  
3 → Salary
```

So it sorts by **Salary DESC**.

However, **prefer column names** because they are clearer:

```
ORDER BY Salary DESC;
```

14. ORDER BY Using an Alias

You can sort using a calculated column alias:

```
SELECT Name,  
       Salary,  
       Salary * 12 AS AnnualSalary  
FROM Employees  
ORDER BY AnnualSalary DESC;
```

Result:

Name	Salary	AnnualSalary
Priya	70000	840000
Neha	65000	780000
Rahul	60000	720000
John	55000	660000
Amit	50000	600000

15. ORDER BY and NULL

How **NULL** values are sorted can **vary between database systems**.

For example:

```
SELECT *  
FROM Employees  
ORDER BY Salary ASC;
```

If some salaries are **NULL**, their position depends on the database's null-ordering behavior.

Some databases also support explicit control such as:

```
ORDER BY Salary ASC NULLS LAST;
```

16. Important SQL Clause Order

A typical query is written as:

```
SELECT column1, column2  
FROM table_name  
WHERE condition  
ORDER BY column_name DESC  
LIMIT 10;
```

For example:

```
SELECT Name, Salary  
FROM Employees  
WHERE Salary >= 55000
```

```
ORDER BY Salary DESC  
LIMIT 3;
```

Conceptually:

```
FROM  
↓  
WHERE  
↓  
SELECT  
↓  
ORDER BY  
↓  
LIMIT
```

17. Real-World Examples

Highest-paid employee

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC  
LIMIT 1;
```

Lowest-paid employee

```
SELECT *  
FROM Employees  
ORDER BY Salary ASC  
LIMIT 1;
```

Newest employees

```
SELECT *  
FROM Employees  
ORDER BY HireDate DESC;
```

Alphabetical customer list

```
SELECT *  
FROM Customers  
ORDER BY Name ASC;
```

Highest salary in each department

For this requirement, `ORDER BY` alone isn't enough; you'd typically use a **window function** such as `ROW_NUMBER()` or `RANK()`.

ASC vs DESC

Syntax	Meaning
<code>ORDER BY Salary ASC</code>	Lowest → Highest
<code>ORDER BY Salary DESC</code>	Highest → Lowest
<code>ORDER BY Name ASC</code>	A → Z
<code>ORDER BY Name DESC</code>	Z → A

Key Point

`ORDER BY` **does not filter rows**. It only controls their **order**.

```
WHERE      → Which rows?
ORDER BY   → In what order?
LIMIT      → How many rows?
```

For example:

```
SELECT Name, Salary
FROM Employees
WHERE Department = 'IT'
ORDER BY Salary DESC
LIMIT 3;
```

This means: **find IT employees** → **sort by highest salary** → **return the top 3**.

- `LIMIT` / `TOP`

SQL `LIMIT` / `TOP`

`LIMIT` and `TOP` are used to **restrict the number of rows returned by a query**.

The syntax depends on the database:

Database	Common syntax
MySQL	<code>LIMIT</code>
PostgreSQL	<code>LIMIT</code>

Database	Common syntax
SQLite	<code>LIMIT</code>
SQL Server	<code>TOP</code>
Oracle	<code>FETCH FIRST</code>

1. `LIMIT`

`LIMIT` is commonly used in **MySQL**, **PostgreSQL**, and **SQLite**.

Basic Syntax

```
SELECT *  
FROM Employees  
LIMIT 5;
```

This returns a maximum of **5 rows**.

2. `LIMIT` with `ORDER BY`

Suppose:

Id	Name	Salary
1	Rahul	60000
2	Amit	50000
3	Priya	70000
4	John	55000
5	Neha	65000

To get the **3 highest-paid employees**:

```
SELECT Name, Salary  
FROM Employees  
ORDER BY Salary DESC  
LIMIT 3;
```

Result:

Name	Salary
Priya	70000

Name	Salary
Neha	65000
Rahul	60000

Important

Don't write:

```
SELECT *  
FROM Employees  
LIMIT 3;
```

if you mean **"top 3 highest-paid"**.

Without **ORDER BY**, the database does not guarantee that the returned 3 rows are the highest-paid employees.

3. LIMIT with OFFSET

OFFSET tells the database how many rows to skip.

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 2 OFFSET 2;
```

Meaning:

```
OFFSET 2 → Skip 2 rows  
LIMIT 2 → Return the next 2 rows
```

For:

```
1 Rahul  
2 Amit  
3 Priya  
4 John  
5 Neha
```

Result:

```
3 Priya
```

4 John

4. Pagination with **LIMIT**

Suppose each page contains **10 records**.

Page 1

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 10 OFFSET 0;
```

Page 2

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 10 OFFSET 10;
```

Page 3

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 10 OFFSET 20;
```

Formula:

```
OFFSET = (PageNumber - 1) × PageSize
```

5. Alternative **LIMIT** Syntax

Some databases support:

```
SELECT *  
FROM Employees  
LIMIT 10, 20;
```

This means:

```
10 → skip 10 rows  
20 → return 20 rows
```

The clearer and more portable form is:

```
SELECT *  
FROM Employees  
LIMIT 20 OFFSET 10;
```

6. TOP

TOP is primarily used by **SQL Server**.

Basic Syntax

```
SELECT TOP 5 *  
FROM Employees;
```

Returns up to 5 rows.

7. TOP with ORDER BY

Get the 3 highest-paid employees:

```
SELECT TOP 3 Name, Salary  
FROM Employees  
ORDER BY Salary DESC;
```

Result:

Name	Salary
Priya	70000
Neha	65000
Rahul	60000

8. TOP with Percentage

SQL Server also supports **PERCENT**.


```
SELECT TOP 10 PERCENT *  
FROM Employees;
```

This returns approximately the top **10% of rows**.

For meaningful "top" results, combine it with **ORDER BY**:

```
SELECT TOP 10 PERCENT *  
FROM Employees  
ORDER BY Salary DESC;
```

9. TOP with WITH TIES

SQL Server supports **WITH TIES**.

Suppose:

Name	Salary
Priya	70000
Neha	65000
Rahul	60000
John	60000
Amit	50000

Normally:

```
SELECT TOP 3 Name, Salary  
FROM Employees  
ORDER BY Salary DESC;
```

could return:

```
Priya 70000  
Neha 65000  
Rahul 60000
```

But:

```
SELECT TOP 3 WITH TIES Name, Salary  
FROM Employees
```

```
ORDER BY Salary DESC;
```

can include **John**, because John has the same salary as the third-ranked employee.

Result:

```
Priya 70000
Neha 65000
Rahul 60000
John 60000
```

10. TOP vs LIMIT

MySQL / PostgreSQL

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 3;
```

SQL Server

```
SELECT TOP 3 Name, Salary
FROM Employees
ORDER BY Salary DESC;
```

Both are commonly used to get the top 3 results.

11. Oracle Equivalent

Modern Oracle commonly uses:

```
SELECT Name, Salary
FROM Employees
ORDER BY Salary DESC
FETCH FIRST 3 ROWS ONLY;
```

This is another SQL-standard-style approach.

12. LIMIT / TOP with WHERE

You can filter first and then limit the result.

MySQL/PostgreSQL

```
SELECT Name, Salary
FROM Employees
WHERE Department = 'IT'
ORDER BY Salary DESC
LIMIT 3;
```

SQL Server

```
SELECT TOP 3 Name, Salary
FROM Employees
WHERE Department = 'IT'
ORDER BY Salary DESC;
```

Meaning:

1. Find IT employees
2. Sort by salary descending
3. Return 3 employees

13. LIMIT / TOP vs WHERE

These solve different problems.

```
WHERE
↓
Filters rows
```

```
ORDER BY
↓
Sorts rows
```

```
LIMIT / TOP
↓
Restricts number of rows
```

Example:

```
SELECT Name, Salary
FROM Employees
WHERE Salary >= 50000
ORDER BY Salary DESC
LIMIT 5;
```

Means:

Find employees earning at least 50,000, sort them from highest to lowest salary, and return the first 5.

14. Important Interview Question

How do you find the highest salary?

```
SELECT MAX(Salary)
FROM Employees;
```

How do you find the employee with the highest salary?

MySQL/PostgreSQL:

```
SELECT *
FROM Employees
ORDER BY Salary DESC
LIMIT 1;
```

SQL Server:

```
SELECT TOP 1 *
FROM Employees
ORDER BY Salary DESC;
```

How do you find the second-highest salary?

One approach in MySQL/PostgreSQL:

```
SELECT DISTINCT Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 1 OFFSET 1;
```

This is a useful interview pattern, though for more complex ranking requirements, window functions such as `DENSE_RANK()` are often better.

Quick Reference

Requirement	MySQL/PostgreSQL	SQL Server
First 5 rows	<code>LIMIT 5</code>	<code>TOP 5</code>
Top 5 salaries	<code>ORDER BY Salary DESC LIMIT 5</code>	<code>TOP 5 ... ORDER BY Salary DESC</code>
Skip 10, get 5	<code>LIMIT 5 OFFSET 10</code>	<code>OFFSET 10 ROWS FETCH NEXT 5 ROWS ONLY</code>
Top 10%	—	<code>TOP 10 PERCENT</code>
Include ties	—	<code>TOP ... WITH TIES</code>

Remember

LIMIT = MySQL/PostgreSQL/SQLite

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC  
LIMIT 5;
```

TOP = SQL Server

```
SELECT TOP 5 *  
FROM Employees  
ORDER BY Salary DESC;
```

For **pagination**, learn **LIMIT + OFFSET** and SQL Server's **OFFSET + FETCH** next.

- **OFFSET**

SQL OFFSET

OFFSET is used to **skip a specified number of rows** before returning the remaining results.

It is commonly used with **LIMIT** for **pagination**.

1. Basic Concept

Suppose the table contains:

Id	Name
1	Rahul
2	Amit

Id	Name
3	Priya
4	John
5	Neha
6	Raj

If you use:

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 2 OFFSET 2;
```

SQL does:

```
OFFSET 2 → Skip rows 1 and 2  
LIMIT 2 → Return rows 3 and 4
```

Result:

Id	Name
3	Priya
4	John

2. Basic Syntax

For MySQL/PostgreSQL:

```
SELECT columns  
FROM table_name  
ORDER BY column_name  
LIMIT number_of_rows  
OFFSET number_of_rows_to_skip;
```

Example:

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 10  
OFFSET 20;
```

Meaning:

```
Skip 20 rows
↓
Return the next 10 rows
```

3. OFFSET with LIMIT

OFFSET is usually used together with LIMIT.

```
SELECT *
FROM Employees
ORDER BY Id
LIMIT 5 OFFSET 10;
```

Meaning:

```
LIMIT 5
  ↓
Return 5 rows

OFFSET 10
  ↓
Skip first 10 rows
```

So rows **11–15** are returned, assuming **ORDER BY Id** gives a stable sequence.

4. Pagination

The most common use of **OFFSET** is pagination.

Suppose an API displays **10 employees per page**.

Page 1

```
SELECT *
FROM Employees
ORDER BY Id
LIMIT 10 OFFSET 0;
```

Rows:

1 - 10

Page 2

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 10 OFFSET 10;
```

Rows:

11 - 20

Page 3

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 10 OFFSET 20;
```

Rows:

21 - 30

5. Offset Formula

For pagination:

$$\text{OFFSET} = (\text{PageNumber} - 1) \times \text{PageSize}$$

Example:

Page Number = 4
Page Size = 10

Therefore:

$$\begin{aligned}\text{OFFSET} &= (4 - 1) \times 10 \\ &= 30\end{aligned}$$

Query:

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 10 OFFSET 30;
```

This returns approximately rows **31–40**.

6. **OFFSET** with **ORDER BY**

You should generally use **ORDER BY** when paginating.

Good:

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 10 OFFSET 20;
```

Avoid relying on:

```
SELECT *  
FROM Employees  
LIMIT 10 OFFSET 20;
```

Without an explicit ordering, the database does not guarantee a stable row order.

7. SQL Server **OFFSET**

SQL Server uses **OFFSET** differently.

Example:

```
SELECT *  
FROM Employees  
ORDER BY Id  
OFFSET 20 ROWS  
FETCH NEXT 10 ROWS ONLY;
```

Meaning:

```
OFFSET 20 ROWS      → Skip 20 rows  
FETCH NEXT 10 ROWS → Return next 10 rows
```

SQL Server pagination

Page 1:

```
SELECT *  
FROM Employees  
ORDER BY Id  
OFFSET 0 ROWS  
FETCH NEXT 10 ROWS ONLY;
```

Page 2:

```
SELECT *  
FROM Employees  
ORDER BY Id  
OFFSET 10 ROWS  
FETCH NEXT 10 ROWS ONLY;
```

Page 3:

```
SELECT *  
FROM Employees  
ORDER BY Id  
OFFSET 20 ROWS  
FETCH NEXT 10 ROWS ONLY;
```

8. OFFSET with WHERE

You can filter before pagination.

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
ORDER BY Id  
LIMIT 10 OFFSET 20;
```

Conceptually:

```
Employees  
↓
```

```
WHERE Department = 'IT'  
  ↓  
ORDER BY Id  
  ↓  
OFFSET 20  
  ↓  
LIMIT 10
```

This returns the third page of IT employees, assuming 10 employees per page.

9. **OFFSET** for Second-Highest Salary

You can use **OFFSET** to skip the highest salary.

```
SELECT DISTINCT Salary  
FROM Employees  
ORDER BY Salary DESC  
LIMIT 1 OFFSET 1;
```

Conceptually:

Salary ranking:

```
1 → 100000 ← OFFSET skips this  
2 → 90000  ← LIMIT returns this
```

Result:

```
90000
```

Using **DISTINCT** is important if multiple employees have the same salary and you specifically want the **second-highest distinct salary**.

10. **OFFSET** vs **LIMIT**

They have different jobs:

Keyword	Purpose
OFFSET	Skip rows
LIMIT	Return maximum number of rows

Example:

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 10 OFFSET 20;
```

Think:

```
OFFSET 20 → "Skip these"  
LIMIT 10 → "Give me these"
```

11. OFFSET vs TOP

TOP is primarily associated with SQL Server:

```
SELECT TOP 10 *  
FROM Employees;
```

It says:

Give me the first 10 rows.

OFFSET says:

Skip some rows before returning results.

SQL Server combines them with **FETCH**:

```
SELECT *  
FROM Employees  
ORDER BY Id  
OFFSET 20 ROWS  
FETCH NEXT 10 ROWS ONLY;
```

12. Real-World API Pagination

Suppose an API receives:

```
page = 3  
pageSize = 20
```

Calculate:

```
OFFSET = (3 - 1) × 20  
        = 40
```

SQL:

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 20 OFFSET 40;
```

The API gets records:

```
41 - 60
```

13. Important Performance Consideration

Large offsets can become inefficient.

For example:

```
SELECT *  
FROM Employees  
ORDER BY Id  
LIMIT 20 OFFSET 1000000;
```

The database may need to process/skip a very large number of rows before finding the required records.

For large datasets, **keyset/cursor pagination** can be more efficient:

```
SELECT *  
FROM Employees  
WHERE Id > 1000000  
ORDER BY Id  
LIMIT 20;
```

This approach is often preferable for high-volume APIs.

Quick Reference

MySQL / PostgreSQL

```
SELECT *
FROM Employees
ORDER BY Id
LIMIT 10 OFFSET 20;
```

Skip 20 → return 10

SQL Server

```
SELECT *
FROM Employees
ORDER BY Id
OFFSET 20 ROWS
FETCH NEXT 10 ROWS ONLY;
```

Skip 20 → return next 10

Pagination formula

$$\text{OFFSET} = (\text{PageNumber} - 1) \times \text{PageSize}$$

Key idea: **OFFSET** is primarily a **row-skipping mechanism**, especially useful for implementing pagination.

5. Filtering Data

- Comparison Operators (**=**, **>**, **<**, **!=**)

SQL Comparison Operators

Comparison operators are used in SQL to **compare two values**. They are commonly used with the **WHERE** clause to filter rows.

Assume we have this table:

Employees					
Id	Name	Age	Salary	Department	
1	Amit	25	40000	IT	
2	Priya	30	60000	HR	
3	Rahul	35	75000	IT	
4	Sneha	28	50000	Sales	

1. = — Equal To

Checks whether two values are equal.

```
SELECT *  
FROM Employees  
WHERE Department = 'IT';
```

Result: Amit and Rahul.

Another example:

```
SELECT *  
FROM Employees  
WHERE Salary = 60000;
```

Returns Priya.

2. > — Greater Than

Checks whether the left value is greater than the right value.

```
SELECT *  
FROM Employees  
WHERE Salary > 50000;
```

Returns:

```
Priya    60000  
Rahul    75000
```

Another example:

```
SELECT *  
FROM Employees  
WHERE Age > 30;
```

Returns Rahul.

3. < — Less Than

Checks whether the left value is less than the right value.

```
SELECT *  
FROM Employees
```

```
WHERE Salary < 60000;
```

Returns:

```
Amit    40000
Sneha   50000
```

Another example:

```
SELECT *
FROM Employees
WHERE Age < 30;
```

Returns Amit and Sneha.

4. != — Not Equal To

Checks whether two values are different.

```
SELECT *
FROM Employees
WHERE Department != 'IT';
```

Returns:

```
Priya   HR
Sneha   Sales
```

<> is also commonly supported as **not equal**:

```
SELECT *
FROM Employees
WHERE Department <> 'IT';
```

Quick Reference

Operator	Meaning	Example
=	Equal to	Age = 30
>	Greater than	Age > 30
<	Less than	Age < 30

Operator	Meaning	Example
<code>!=</code>	Not equal to	<code>Age != 30</code>

Combined example

```
SELECT Name, Age, Salary
FROM Employees
WHERE Age > 25
      AND Salary < 70000;
```

This returns employees whose **age is greater than 25 AND salary is less than 70,000**.

Important: SQL also supports `>=` (greater than or equal) and `<=` (less than or equal), which you'll frequently use with these operators.

- Logical Operators (`AND`, `OR`, `NOT`)

SQL Logical Operators

Logical operators are used to **combine or reverse multiple conditions** in SQL. They are mainly used with the `WHERE` clause.

The three important logical operators are:

- `AND`
- `OR`
- `NOT`

Assume this table:

Id	Name	Age	Salary	Department
1	Amit	25	40000	IT
2	Priya	30	60000	HR
3	Rahul	35	75000	IT
4	Sneha	28	50000	Sales
5	John	40	90000	IT

1. `AND`

`AND` requires **all conditions to be TRUE**.

Example

Find employees who are in IT **AND** have salary greater than 50,000:

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
      AND Salary > 50000;
```

Result:

```
Rahul  
John
```

Why?

Employee	IT?	Salary > 50000?	Result
Amit	Yes	No	✗
Priya	No	Yes	✗
Rahul	Yes	Yes	✓
Sneha	No	No	✗
John	Yes	Yes	✓

Multiple **AND**

You can have more than two conditions:

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
      AND Salary > 50000  
      AND Age < 40;
```

Only Rahul satisfies all three conditions.

2. **OR**

OR requires **at least one condition to be TRUE**.

Example

Find employees who are in IT **OR** HR:

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
       OR Department = 'HR';
```

Result:

```
Amit  
Priya  
Rahul  
John
```

Because an employee only needs to satisfy **one** of the conditions.

Another example

Find employees whose age is less than 25 **OR** salary is greater than 80,000:

```
SELECT *  
FROM Employees  
WHERE Age < 25  
       OR Salary > 80000;
```

John is returned because his salary is greater than 80,000.

3. NOT

NOT reverses a condition.

Example

Find employees who are **not** in IT:

```
SELECT *  
FROM Employees  
WHERE NOT Department = 'IT';
```

Equivalent to:

```
SELECT *  
FROM Employees  
WHERE Department != 'IT';
```

Result:

Priya
Sneha

NOT with another condition

Find employees whose salary is **not greater than 50,000**:

```
SELECT *  
FROM Employees  
WHERE NOT Salary > 50000;
```

Combining AND, OR, and NOT

You can combine logical operators.

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
    AND (Salary > 50000 OR Age > 35);
```

The parentheses are important.

The query means:

Employee must be in **IT**, and must have either **salary > 50,000** or **age > 35**.

Operator Truth Table

AND

Condition A	Condition B	A AND B
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

OR

Condition A	Condition B	A OR B
TRUE	TRUE	TRUE

Condition A	Condition B	A OR B
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

NOT

Condition	NOT Condition
TRUE	FALSE
FALSE	TRUE

Important: AND vs OR

AND

```
WHERE Age > 25  
AND Salary > 50000
```

Means:

Both must be true.

OR

```
WHERE Age > 25  
OR Salary > 50000
```

Means:

At least one must be true.

Common Interview Example

Suppose you need:

Get IT employees whose salary is between 50,000 and 80,000 OR HR employees whose age is greater than 28.

Use parentheses:

```
SELECT *  
FROM Employees
```

```
WHERE (Department = 'IT'
      AND Salary BETWEEN 50000 AND 80000)
OR (Department = 'HR'
    AND Age > 28);
```

Key point

When mixing **AND** and **OR**, **use parentheses** to clearly define the intended logic and avoid unexpected results.

- **BETWEEN**

SQL BETWEEN

BETWEEN is used to **filter values within a specified range**.

It is commonly used with numbers, dates, and sometimes text.

Syntax

```
SELECT column1, column2
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Important: **BETWEEN** is **inclusive**, meaning both the starting and ending values are included.

1. BETWEEN with Numbers

Assume:

```
Employees
+---+-----+-----+-----+
| Id | Name  | Age | Salary |
+---+-----+-----+-----+
| 1  | Amit  | 25  | 40000  |
| 2  | Priya | 30  | 60000  |
| 3  | Rahul | 35  | 75000  |
| 4  | Sneha | 28  | 50000  |
| 5  | John  | 40  | 90000  |
+---+-----+-----+-----+
```

Find employees whose salary is between 50,000 and 75,000:

```
SELECT *
FROM Employees
WHERE Salary BETWEEN 50000 AND 75000;
```

Result:

```
Priya    60000
Rahul    75000
Sneha    50000
```

Both 50000 and 75000 are included.

2. BETWEEN with Age

Find employees aged between 25 and 35:

```
SELECT *
FROM Employees
WHERE Age BETWEEN 25 AND 35;
```

This is equivalent to:

```
SELECT *
FROM Employees
WHERE Age >= 25
      AND Age <= 35;
```

So:

```
BETWEEN 25 AND 35
```

means:

```
25 <= Age <= 35
```

3. NOT BETWEEN

NOT BETWEEN finds values **outside the specified range**.

```
SELECT *
FROM Employees
WHERE Salary NOT BETWEEN 50000 AND 75000;
```

Result:

Amit	40000
John	90000

It is equivalent to:

```
SELECT *  
FROM Employees  
WHERE Salary < 50000  
       OR Salary > 75000;
```

4. BETWEEN with Dates

BETWEEN is frequently used with dates.

For example:

```
SELECT *  
FROM Orders  
WHERE OrderDate BETWEEN '2026-01-01' AND '2026-01-31';
```

This searches for orders between January 1 and January 31, inclusive.

Date/time caution: if **OrderDate** contains a time component, a half-open range is often safer for an entire month:

```
SELECT *  
FROM Orders  
WHERE OrderDate >= '2026-01-01'  
       AND OrderDate < '2026-02-01';
```

This includes all times on January 31.

5. BETWEEN with AND

You can combine **BETWEEN** with other conditions:

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
       AND Salary BETWEEN 50000 AND 80000;
```

Meaning:

Get employees from IT whose salary is between 50,000 and 80,000.

6. BETWEEN with NOT

Usually, simply use:

```
WHERE Salary NOT BETWEEN 50000 AND 75000
```

rather than:

```
WHERE NOT Salary BETWEEN 50000 AND 75000
```

Both can express the same logic, but the first is clearer.

Quick Reference

SQL	Meaning
Salary BETWEEN 50000 AND 75000	50,000 through 75,000
Salary NOT BETWEEN 50000 AND 75000	Below 50,000 or above 75,000
Age BETWEEN 20 AND 30	Age 20 through 30
Date BETWEEN '2026-01-01' AND '2026-01-31'	Dates in that range

Key Point

```
WHERE Salary BETWEEN 50000 AND 75000
```

is equivalent to:

```
WHERE Salary >= 50000  
AND Salary <= 75000
```

Remember: BETWEEN includes both boundary values.

- IN

SQL IN

IN is used to check whether a value **matches any value in a specified list**.

It is very useful when you want to filter a column against multiple possible values.

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name IN (value1, value2, value3);
```

1. Basic IN

Assume:

```
Employees  
+---+-----+-----+-----+  
| Id | Name  | Department | Salary |  
+---+-----+-----+-----+  
| 1  | Amit  | IT         | 40000  |  
| 2  | Priya | HR         | 60000  |  
| 3  | Rahul | IT         | 75000  |  
| 4  | Sneha | Sales      | 50000  |  
| 5  | John  | Finance    | 90000  |  
+---+-----+-----+-----+
```

Find employees from **IT, HR, or Sales**:

```
SELECT *  
FROM Employees  
WHERE Department IN ('IT', 'HR', 'Sales');
```

Result:

```
Amit  
Priya  
Rahul  
Sneha
```

2. IN vs OR

Without **IN**, you could write:

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
       OR Department = 'HR'  
       OR Department = 'Sales';
```

Using **IN**:

```
SELECT *  
FROM Employees  
WHERE Department IN ('IT', 'HR', 'Sales');
```

IN makes the query **shorter and easier to read**.

Conceptually:

```
Department IN ('IT', 'HR', 'Sales')
```

means:

```
Department = 'IT'  
OR  
Department = 'HR'  
OR  
Department = 'Sales'
```

3. **IN** with Numbers

IN is not limited to text.

Find employees with specific salaries:

```
SELECT *  
FROM Employees  
WHERE Salary IN (40000, 60000, 90000);
```

Result:

```
Amit  
Priya  
John
```

You can also use it with IDs:

```
SELECT *  
FROM Employees  
WHERE Id IN (1, 3, 5);
```

4. NOT IN

NOT IN finds rows whose value **doesn't match any value in the list**.

For example, find employees who are not in IT or HR:

```
SELECT *  
FROM Employees  
WHERE Department NOT IN ('IT', 'HR');
```

Result:

```
Sneha  
John
```

Conceptually:

```
Department != 'IT'  
AND  
Department != 'HR'
```

5. IN with WHERE and AND

You can combine **IN** with other conditions.

Find IT or HR employees earning more than 50,000:

```
SELECT *  
FROM Employees  
WHERE Department IN ('IT', 'HR')  
AND Salary > 50000;
```

Result:

```
Priya  
Rahul
```

6. IN with a Subquery

IN can also use the result of another query.

For example:

```
SELECT *
FROM Employees
WHERE DepartmentId IN (
    SELECT Id
    FROM Departments
    WHERE Location = 'Mumbai'
);
```

Here, SQL first finds department IDs located in Mumbai and then returns employees belonging to those departments.

This is very common in real-world SQL.

7. NOT IN and NULL — Important

Be careful when using NOT IN if the list or subquery can contain NULL.

For example:

```
WHERE Department NOT IN ('IT', 'HR', NULL)
```

can produce unexpected results because SQL uses **three-valued logic** (TRUE, FALSE, UNKNOWN).

When dealing with subqueries, NOT EXISTS is often safer when NULL is possible.

Quick Reference

Syntax	Meaning
Department IN ('IT', 'HR')	IT or HR
Department NOT IN ('IT', 'HR')	Anything except IT and HR
Salary IN (40000, 60000)	Salary is 40,000 or 60,000
Id IN (1, 3, 5)	ID is 1, 3, or 5

Key Point

Instead of writing:

```
WHERE Department = 'IT'
OR Department = 'HR'
```

```
OR Department = 'Sales'
```

prefer:

```
WHERE Department IN ('IT', 'HR', 'Sales')
```

IN = "Does this value belong to this list?"

- **LIKE**

SQL LIKE

LIKE is used for **pattern matching** in SQL. It is mainly used with text/string columns to find values that match a particular pattern.

Syntax

```
SELECT *  
FROM table_name  
WHERE column_name LIKE 'pattern';
```

The two main wildcards are:

Wildcard	Meaning
%	Zero or more characters
—	Exactly one character

1. Starts With — %

Find employees whose name starts with **A**:

```
SELECT *  
FROM Employees  
WHERE Name LIKE 'A%';
```

Matches:

```
Amit  
Anil  
Ajay
```

A% means:

A + anything after A

2. Ends With — %

Find names ending with a:

```
SELECT *  
FROM Employees  
WHERE Name LIKE '%a';
```

Matches:

Priya
Sneha
Anita

%a means:

anything + a

3. Contains — %

Find names containing it:

```
SELECT *  
FROM Employees  
WHERE Name LIKE '%it%';
```

Matches names where it appears anywhere.

Amit
Smith

%it% means:

anything + it + anything

4. Exactly One Character — _

The `_` wildcard represents **exactly one character**.

Find names where the first character is `A` and the second character is `m`:

```
SELECT *  
FROM Employees  
WHERE Name LIKE 'Am%';
```

For exactly one character after `A`:

```
SELECT *  
FROM Employees  
WHERE Name LIKE 'A_';
```

This matches a two-character value such as:

```
An  
Al
```

but not:

```
Amit
```

5. Multiple `_`

```
SELECT *  
FROM Employees  
WHERE Name LIKE 'A___';
```

This means:

```
A + 3 characters
```

So it matches names with **exactly 4 characters starting with A**.

For example:

```
Amit  
Ajay
```

6. LIKE with Numbers

Although **LIKE** is mainly used with strings, you can sometimes use it with numeric values after conversion, depending on the database.

For example, with a string employee ID:

```
SELECT *  
FROM Employees  
WHERE EmployeeCode LIKE 'EMP%';
```

Matches:

```
EMP001  
EMP002  
EMP100
```

7. NOT LIKE

NOT LIKE finds values that **don't match a pattern**.

Find employees whose names don't start with **A**:

```
SELECT *  
FROM Employees  
WHERE Name NOT LIKE 'A%';
```

8. Combining LIKE with AND

Find IT employees whose names start with **R**:

```
SELECT *  
FROM Employees  
WHERE Department = 'IT'  
AND Name LIKE 'R%';
```

9. Combining LIKE with OR

Find employees whose names start with either **A** or **P**:

```
SELECT *
FROM Employees
WHERE Name LIKE 'A%'
      OR Name LIKE 'P%';
```

10. LIKE with IN

You **cannot** directly do this:

```
-- Incorrect
WHERE Name LIKE IN ('A%', 'P%')
```

Instead:

```
SELECT *
FROM Employees
WHERE Name LIKE 'A%'
      OR Name LIKE 'P%';
```

Common Patterns

Pattern	Meaning	Example
'A%'	Starts with A	Amit
'%a'	Ends with a	Priya
'%it%'	Contains it	Amit
'A_'	A + exactly 1 character	An
'A__'	A + exactly 2 characters	Amit ❌, Ann ✅
'_a%'	Second character is a	Rahul
'___'	Exactly 3 characters	Ram

Real-World Example

Suppose you have:

```
Customers
+-----+
| Id | Name |
+-----+
```

```
+----+-----+
| 1 | Amit Sharma |
| 2 | Rahul Patil |
| 3 | Priya Shah |
| 4 | Anil Kumar |
+----+-----+
```

Find customers whose name contains **Sh**:

```
SELECT *
FROM Customers
WHERE Name LIKE '%Sh%';
```

Result:

```
Amit Sharma
Priya Shah
```

Key Point

Think of **LIKE** as:

```
LIKE 'A%'      → starts with A
LIKE '%A'      → ends with A
LIKE '%A%'     → contains A
LIKE 'A_'      → A + exactly one character
```

% = any number of characters

_ = exactly one character

- **IS NULL**

SQL IS NULL

IS NULL is used to find rows where a column contains a **NULL value**.

NULL means the value is **missing, unknown, or not provided**.

1. Basic IS NULL

Suppose we have:

```
Employees
+----+-----+-----+-----+
| Id | Name | Department | Salary |
```

1	Amit	IT	40000
2	Priya	HR	60000
3	Rahul	NULL	75000
4	Sneha	Sales	50000

Find employees whose department is missing:

```
SELECT *  
FROM Employees  
WHERE Department IS NULL;
```

Result:

Rahul

2. IS NOT NULL

To find values that **are not NULL**, use **IS NOT NULL**.

```
SELECT *  
FROM Employees  
WHERE Department IS NOT NULL;
```

Result:

Amit
Priya
Sneha

3. ❌ Don't Use = NULL

This is incorrect:

```
SELECT *  
FROM Employees  
WHERE Department = NULL;
```

Why?

Because **NULL** is not a normal value that can be compared using `=`.

Use:

```
WHERE Department IS NULL
```

instead.

4. `!= NULL` Is Also Incorrect

Don't write:

```
WHERE Department != NULL;
```

Use:

```
WHERE Department IS NOT NULL;
```

5. **NULL** Is Not the Same as Empty String

These are different:

```
NULL  
' '
```

NULL means:

No value / unknown value

`' '` means:

An empty string

For example:

```
SELECT *  
FROM Employees  
WHERE Department = ' ';
```

finds an empty string, **not NULL**.

To find NULL:

```
SELECT *  
FROM Employees  
WHERE Department IS NULL;
```

6. Combining **IS NULL** with Other Conditions

You can combine it with **AND**:

```
SELECT *  
FROM Employees  
WHERE Department IS NULL  
      AND Salary > 50000;
```

This finds employees whose department is missing **and** salary is greater than 50,000.

Or with **OR**:

```
SELECT *  
FROM Employees  
WHERE Department IS NULL  
      OR Salary IS NULL;
```

This finds employees where either department or salary is missing.

7. **NULL** in **INSERT**

You can explicitly insert **NULL**:

```
INSERT INTO Employees  
  (Id, Name, Department, Salary)  
VALUES  
  (5, 'John', NULL, 70000);
```

Then:

```
SELECT *  
FROM Employees  
WHERE Department IS NULL;
```

will return John.

8. NULL in UPDATE

You can set a column to **NULL**:

```
UPDATE Employees
SET Department = NULL
WHERE Id = 5;
```

Then:

```
SELECT *
FROM Employees
WHERE Department IS NULL;
```

will find John.

9. NULL and Calculations

NULL can affect calculations.

For example:

```
SELECT Salary + Bonus
FROM Employees;
```

If **Bonus** is **NULL**, the result may be **NULL**.

To replace NULL with a value, functions such as **COALESCE** are commonly used:

```
SELECT Salary + COALESCE(Bonus, 0)
FROM Employees;
```

This treats a NULL bonus as **0**.

Quick Reference

SQL	Meaning
<code>column IS NULL</code>	Column has no value
<code>column IS NOT NULL</code>	Column has a value

SQL	Meaning
<code>column = NULL</code>	✗ Incorrect
<code>column != NULL</code>	✗ Incorrect
<code>column = ''</code>	Checks for empty string, not NULL

Most important rule

```
-- Find NULL
WHERE Department IS NULL

-- Find non-NULL
WHERE Department IS NOT NULL
```

Remember: **NULL** represents the absence/unknown value, so use **IS NULL** and **IS NOT NULL** rather than **=** or **!=**.

6. Aggregate Functions

- `COUNT()`

COUNT() in SQL

`COUNT()` is an **aggregate function** used to count rows or non-**NULL** values.

1. COUNT(*)

Counts **all rows** in a table.

```
SELECT COUNT(*) AS TotalEmployees
FROM Employees;
```

If the table has 5 employees:

```
TotalEmployees
-----
5
```

`COUNT(*)` counts rows even if some columns contain **NULL**.

2. COUNT(column)

Counts the number of **non-NULL values** in a specific column.


```
SELECT COUNT(Salary) AS EmployeesWithSalary
FROM Employees;
```

Suppose:

Id	Name	Salary
1	John	50000
2	Alice	60000
3	Bob	NULL
4	David	55000

Result:

```
EmployeesWithSalary
-----
3
```

Bob is not counted because **Salary** is **NULL**.

3. COUNT(DISTINCT column)

Counts unique, non-**NULL** values.

```
SELECT COUNT(DISTINCT Department) AS DepartmentCount
FROM Employees;
```

If departments are:

```
IT
IT
HR
HR
Finance
```

Result:

```
DepartmentCount
-----
3
```

4. COUNT() with WHERE

You can count rows that satisfy a condition.

```
SELECT COUNT(*) AS ITEmployees
FROM Employees
WHERE Department = 'IT';
```

For example, if 3 employees belong to IT:

```
ITEmployees
-----
3
```

5. COUNT() with GROUP BY

Count employees in each department:

```
SELECT
    Department,
    COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department;
```

Result:

Department	EmployeeCount
IT	3
HR	2
Finance	4

This is one of the **most important uses of COUNT()**.

6. COUNT() with HAVING

Find departments having more than 5 employees:

```
SELECT
    Department,
    COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 5;
```

Remember:

- **WHERE** → filters **rows before grouping**
- **HAVING** → filters **groups after aggregation**

7. **COUNT(*)** vs **COUNT(column)**

This is very important for interviews.

Suppose:

Id	Name	Email
1	John	john@test.com
2	Alice	alice@test.com
3	Bob	NULL
4	David	david@test.com

```
SELECT COUNT(*) FROM Employees;
```

Result:

4

But:

```
SELECT COUNT(Email) FROM Employees;
```

Result:

3

Because **COUNT(Email)** ignores **NULL**.

Key rule

```
COUNT(*)           → counts rows
COUNT(column)     → counts non-NULL values
COUNT(DISTINCT column) → counts unique non-NULL values
```

Common interview question

Q: Does **COUNT(column) count **NULL** values?**

Answer: No. **COUNT(column)** ignores **NULL** values. If you want to count every row, use **COUNT(*)**.

- `SUM()`

SUM() in SQL

`SUM()` is an **aggregate function** used to calculate the **total of numeric values** in a column.

1. Basic SUM()

Suppose we have:

Id	Product	Amount
1	Laptop	50000
2	Mouse	2000
3	Keyboard	3000
4	Monitor	15000

Calculate the total amount:

```
SELECT SUM(Amount) AS TotalAmount
FROM Sales;
```

Result:

```
TotalAmount
-----
70000
```

Because:

```
50000 + 2000 + 3000 + 15000 = 70000
```

2. SUM() with WHERE

Calculate total sales for the IT department:

```
SELECT SUM(Salary) AS TotalSalary
FROM Employees
WHERE Department = 'IT';
```

Example:

Name	Department	Salary
John	IT	50000
Alice	IT	60000
Bob	HR	45000
David	IT	70000

Result:

```
TotalSalary
-----
180000
```

3. `SUM()` with `GROUP BY`

Calculate total salary for each department:

```
SELECT
    Department,
    SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department;
```

Result:

Department	TotalSalary
IT	180000
HR	45000

This is one of the most common uses of `SUM()`.

4. `SUM()` ignores `NULL`

Suppose:

Id	Amount
1	1000
2	2000
3	NULL
4	3000

```
SELECT SUM(Amount) AS TotalAmount
FROM Payments;
```

Result:

```
TotalAmount
-----
6000
```

NULL is ignored:

```
1000 + 2000 + 3000 = 6000
```

5. SUM() with DISTINCT

You can calculate the sum of unique values:

```
SELECT SUM(DISTINCT Amount) AS TotalAmount
FROM Payments;
```

For:

Amount
1000
1000
2000
3000

Normal SUM():

```
1000 + 1000 + 2000 + 3000 = 7000
```

SUM(DISTINCT Amount):

```
1000 + 2000 + 3000 = 6000
```

6. SUM() with HAVING

Find departments whose total salary is greater than 100000:

```
SELECT
    Department,
    SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department
HAVING SUM(Salary) > 100000;
```

HAVING is used because we're filtering based on an **aggregate result**.

7. **SUM()** with a condition

For example, calculate only successful transactions:

```
SELECT SUM(Amount) AS SuccessfulAmount
FROM Transactions
WHERE Status = 'SUCCESS';
```

This is common in payment/fintech systems.

You can also use conditional aggregation:

```
SELECT
    SUM(CASE
        WHEN Status = 'SUCCESS' THEN Amount
        ELSE 0
    END) AS SuccessfulAmount
FROM Transactions;
```

SUM() vs **COUNT()**

Function	Purpose
COUNT()	Counts rows/values
SUM()	Adds numeric values

Example:

```
SELECT
    COUNT(*) AS TransactionCount,
    SUM(Amount) AS TotalAmount
FROM Transactions;
```

Result might be:

TransactionCount	TotalAmount
100	500000

Meaning:

- **100 transactions**
- **₹500,000 total transaction amount**

Important interview point

`SUM()` works with **numeric data** such as `INT`, `DECIMAL`, `NUMERIC`, etc. It ignores `NULL` values.

- `AVG()`

AVG() in SQL

`AVG()` is an **aggregate function** used to calculate the **average (mean) value** of a numeric column.

Formula

```
Average = SUM(values) / COUNT(values)
```

1. Basic `AVG()`

Suppose we have:

Id	Name	Salary
1	John	50000
2	Alice	60000
3	Bob	40000
4	David	70000

```
SELECT AVG(Salary) AS AverageSalary
FROM Employees;
```

Calculation:

```
(50000 + 60000 + 40000 + 70000) / 4
= 220000 / 4
= 55000
```

Result:

AverageSalary

55000

2. AVG() with WHERE

Find the average salary of IT employees:

```
SELECT AVG(Salary) AS AverageSalary
FROM Employees
WHERE Department = 'IT';
```

Example:

Name	Department	Salary
John	IT	50000
Alice	IT	60000
Bob	HR	40000
David	IT	70000

Calculation:

```
(50000 + 60000 + 70000) / 3
= 60000
```

3. AVG() with GROUP BY

Find the average salary for each department:

```
SELECT
    Department,
    AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department;
```

Result:

Department	AverageSalary
IT	60000
HR	40000

This is a very common use of **AVG()**.

4. `AVG()` ignores `NULL`

Suppose:

Id	Salary
1	50000
2	60000
3	NULL
4	70000

```
SELECT AVG(Salary) AS AverageSalary
FROM Employees;
```

The `NULL` value is ignored.

Calculation:

```
(50000 + 60000 + 70000) / 3
= 60000
```

So the result is:

```
60000
```

It is **not**:

```
(50000 + 60000 + 70000) / 4
```

5. `AVG()` with `HAVING`

Find departments where the average salary is greater than `50000`:

```
SELECT
    Department,
    AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

`HAVING` filters the grouped/aggregated result.

6. `AVG()` with `DISTINCT`

You can calculate the average of only unique values:

```
SELECT AVG(DISTINCT Salary) AS AverageSalary
FROM Employees;
```

Suppose salaries are:

```
50000
50000
60000
70000
```

Normal `AVG()`:

```
(50000 + 50000 + 60000 + 70000) / 4
= 57500
```

`AVG(DISTINCT Salary)`:

```
(50000 + 60000 + 70000) / 3
= 60000
```

7. `AVG()` for transaction amounts

In a fintech application, you might calculate the average successful transaction:

```
SELECT AVG(Amount) AS AverageTransaction
FROM Transactions
WHERE Status = 'SUCCESS';
```

For example:

Transaction	Amount
T001	1000
T002	2000
T003	5000

Result:

```
AverageTransaction
```

```
-----
```

```
2666.67
```

AVG() vs SUM() vs COUNT()

Function	Purpose	Example
COUNT()	Number of values/rows	COUNT(*)
SUM()	Total	SUM(Salary)
AVG()	Average	AVG(Salary)

Interview point

AVG() ignores NULL values.

```
SELECT AVG(Salary)
FROM Employees;
```

Conceptually, it behaves like:

```
SUM(Salary) / COUNT(Salary)
```

not:

```
SUM(Salary) / COUNT(*)
```

when Salary contains NULL values.

- MIN()

MIN() in SQL

MIN() is an **aggregate function** that returns the **smallest value** from a column.

It can be used with numbers, dates, and strings, depending on the database.

1. Basic MIN()

Suppose we have:

Id	Name	Salary
1	John	50000
2	Alice	60000
3	Bob	40000
4	David	70000

Find the lowest salary:

```
SELECT MIN(Salary) AS MinimumSalary
FROM Employees;
```

Result:

MinimumSalary
40000

2. `MIN()` with `WHERE`

Find the minimum salary among IT employees:

```
SELECT MIN(Salary) AS MinimumSalary
FROM Employees
WHERE Department = 'IT';
```

If IT salaries are:

```
50000
60000
70000
```

Result:

```
50000
```

3. `MIN()` with `GROUP BY`

Find the minimum salary for each department:

```
SELECT
    Department,
    MIN(Salary) AS MinimumSalary
```

```
FROM Employees
GROUP BY Department;
```

Result:

Department	MinimumSalary
IT	50000
HR	40000

This is one of the most common uses of `MIN()`.

4. `MIN()` ignores `NULL`

Suppose:

Id	Salary
1	50000
2	NULL
3	40000
4	70000

```
SELECT MIN(Salary) AS MinimumSalary
FROM Employees;
```

Result:

```
40000
```

`NULL` is ignored when calculating `MIN()`.

5. `MIN()` with Dates

`MIN()` can find the **earliest date**.

```
SELECT MIN(HireDate) AS EarliestHireDate
FROM Employees;
```

For example:

Name	HireDate
John	2022-05-10
Alice	2021-03-15
Bob	2023-01-20

Result:

```
2021-03-15
```

So **MIN()** with dates means **earliest date**.

6. **MIN()** with **HAVING**

Find departments where the minimum salary is greater than **40000**:

```
SELECT
    Department,
    MIN(Salary) AS MinimumSalary
FROM Employees
GROUP BY Department
HAVING MIN(Salary) > 40000;
```

HAVING filters the groups based on the aggregate result.

7. **MIN()** with **DISTINCT**

You can use **DISTINCT**, although it usually doesn't change the minimum:

```
SELECT MIN(DISTINCT Salary) AS MinimumSalary
FROM Employees;
```

For:

```
50000
50000
60000
70000
```

Both:

```
MIN(Salary)
```

and:

```
MIN(DISTINCT Salary)
```

return:

```
50000
```

Because duplicate values don't affect the smallest value.

Real-World Example

For a payment system, find the smallest successful transaction:

```
SELECT MIN(Amount) AS SmallestTransaction
FROM Transactions
WHERE Status = 'SUCCESS';
```

For example:

Transaction	Amount	Status
T001	5000	SUCCESS
T002	1200	SUCCESS
T003	3000	FAILED
T004	2500	SUCCESS

Result:

```
SmallestTransaction
-----
1200
```

MIN() Summary

Query	Meaning
MIN(Salary)	Lowest salary
MIN(Price)	Lowest price
MIN(Amount)	Smallest amount
MIN(HireDate)	Earliest hire date

Query	Meaning
<code>MIN(OrderDate)</code>	Earliest order date

Key point

`MIN()` → Finds the smallest value

It **ignores NULL values** and is frequently combined with `WHERE` and `GROUP BY`.

- `MAX()`

`MAX()` in SQL

`MAX()` is an **aggregate function** that returns the **largest value** from a column.

It is commonly used with numbers, dates, and strings.

1. Basic `MAX()`

Suppose we have an `Employees` table:

Id	Name	Department	Salary
1	John	IT	50000
2	Alice	IT	60000
3	Bob	HR	40000
4	David	IT	70000

Find the highest salary:

```
SELECT MAX(Salary) AS MaximumSalary
FROM Employees;
```

Result:

MaximumSalary
70000

2. `MAX()` with `WHERE`

Find the highest salary among IT employees:

```
SELECT MAX(Salary) AS MaximumSalary
FROM Employees
WHERE Department = 'IT';
```

IT salaries are:

```
50000
60000
70000
```

Result:

```
70000
```

3. `MAX()` with `GROUP BY`

Find the highest salary in each department:

```
SELECT
    Department,
    MAX(Salary) AS MaximumSalary
FROM Employees
GROUP BY Department;
```

Result:

Department	MaximumSalary
IT	70000
HR	40000

This is a very common use of `MAX()`.

4. `MAX()` ignores `NULL`

Suppose:

Id	Salary
1	50000
2	NULL
3	60000

Id	Salary
4	70000

```
SELECT MAX(Salary) AS MaximumSalary
FROM Employees;
```

Result:

```
70000
```

NULL values are ignored.

5. **MAX()** with Dates

MAX() can find the **latest date**.

```
SELECT MAX(HireDate) AS LatestHireDate
FROM Employees;
```

Suppose:

Name	HireDate
John	2022-05-10
Alice	2021-03-15
Bob	2023-01-20

Result:

```
2023-01-20
```

So:

```
MIN(Date) → Earliest date
MAX(Date) → Latest date
```

6. **MAX()** with **HAVING**

Find departments where the highest salary is greater than **60000**:

```
SELECT
    Department,
    MAX(Salary) AS MaximumSalary
FROM Employees
GROUP BY Department
HAVING MAX(Salary) > 60000;
```

Result:

Department	MaximumSalary
IT	70000

7. MAX() with DISTINCT

You can use:

```
SELECT MAX(DISTINCT Salary) AS MaximumSalary
FROM Employees;
```

However, **DISTINCT** normally doesn't make a difference for **MAX()** because duplicate values don't affect the largest value.

For:

```
50000
50000
60000
70000
```

Both return:

```
70000
```

8. Real-World Transaction Example

Find the largest successful transaction:

```
SELECT MAX(Amount) AS LargestTransaction
FROM Transactions
WHERE Status = 'SUCCESS';
```

Suppose:

Transaction	Amount	Status
T001	5000	SUCCESS
T002	1200	SUCCESS
T003	9000	FAILED
T004	7500	SUCCESS

Result:

```
LargestTransaction
-----
7500
```

The failed transaction is excluded because of the **WHERE** condition.

MIN() vs MAX()

Function	Purpose	Example
MIN()	Smallest value	MIN(Salary)
MAX()	Largest value	MAX(Salary)

For dates:

```
MIN(OrderDate) → First/earliest order
MAX(OrderDate) → Last/latest order
```

Key point

```
MAX() → Finds the largest value
```

It **ignores NULL values** and is frequently used with **WHERE**, **GROUP BY**, and **HAVING**.

7. Grouping Data

- **GROUP BY**

GROUP BY in SQL

GROUP BY is used to **combine rows having the same values** into groups. It is commonly used with aggregate functions such as **COUNT()**, **SUM()**, **AVG()**, **MIN()**, and **MAX()**.

Syntax

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

Example Table

Suppose we have an **Employees** table:

Id	Name	Department	Salary
1	John	IT	50000
2	Alice	HR	40000
3	Bob	IT	60000
4	David	HR	45000
5	Mike	Sales	55000

Count employees by department

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department;
```

Result

Department	EmployeeCount
IT	2
HR	2
Sales	1

SQL groups the rows like this:

```
IT
├── John
└── Bob

HR
├── Alice
└── David
```

```
Sales
└─ Mike
```

Then `COUNT(*)` is calculated for each group.

GROUP BY with SUM()

Find the total salary for each department:

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department;
```

Result:

Department	TotalSalary
IT	110000
HR	85000
Sales	55000

GROUP BY with AVG()

Find the average salary of each department:

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department;
```

Result:

Department	AverageSalary
IT	55000
HR	42500
Sales	55000

GROUP BY with MIN() and MAX()

```
SELECT
    Department,
    MIN(Salary) AS MinimumSalary,
    MAX(Salary) AS MaximumSalary
```

```
FROM Employees
GROUP BY Department;
```

Result:

Department	MinimumSalary	MaximumSalary
IT	50000	60000
HR	40000	45000
Sales	55000	55000

Grouping by Multiple Columns

You can group by more than one column.

```
SELECT Department, JobTitle, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department, JobTitle;
```

Here, SQL creates a group for each unique **Department + JobTitle combination**.

GROUP BY with WHERE

WHERE filters rows **before** grouping.

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
WHERE Salary > 45000
GROUP BY Department;
```

Process:

```
Employees
  ↓
WHERE Salary > 45000
  ↓
GROUP BY Department
  ↓
COUNT(*)
```

GROUP BY vs DISTINCT

They can sometimes produce similar results:


```
SELECT DISTINCT Department
FROM Employees;
```

and

```
SELECT Department
FROM Employees
GROUP BY Department;
```

Both can return:

```
IT
HR
Sales
```

But **GROUP BY** is primarily used when you want to **perform calculations on each group**.

For example:

```
SELECT Department, COUNT(*)
FROM Employees
GROUP BY Department;
```

Important Rule

If you use a normal column in **SELECT** along with an aggregate function, that column generally needs to appear in **GROUP BY**.

✅ Correct:

```
SELECT Department, COUNT(*)
FROM Employees
GROUP BY Department;
```

❌ Incorrect:

```
SELECT Department, Name, COUNT(*)
FROM Employees
GROUP BY Department;
```

Name is neither grouped nor aggregated.

Key Point

GROUP BY divides rows into groups so that aggregate functions can calculate a result for each group.

Common pattern:

```
SELECT column, COUNT(*)  
FROM table  
GROUP BY column;
```

- **HAVING**

HAVING in SQL

HAVING is used to **filter groups created by GROUP BY**.

The main difference is:

- **WHERE** → filters **individual rows**
- **HAVING** → filters **groups**

Syntax

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)  
FROM table_name  
GROUP BY column_name  
HAVING condition;
```

Example Table

Suppose we have:

Id	Name	Department	Salary
1	John	IT	50000
2	Alice	HR	40000
3	Bob	IT	60000
4	David	HR	45000
5	Mike	Sales	55000

Find departments having more than 1 employee

```
SELECT Department, COUNT(*) AS EmployeeCount  
FROM Employees
```

```
GROUP BY Department
HAVING COUNT(*) > 1;
```

Result

Department	EmployeeCount
IT	2
HR	2

Sales is excluded because it contains only one employee.

HAVING with SUM()

Find departments where the **total salary is greater than 90,000**:

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department
HAVING SUM(Salary) > 90000;
```

Result:

Department	TotalSalary
IT	110000

HAVING with AVG()

Find departments where the **average salary is greater than 45,000**:

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 45000;
```

Result:

Department	AverageSalary
IT	55000
Sales	55000

WHERE VS HAVING

This is one of the most important SQL concepts.

WHERE

Filters rows **before** grouping:

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
WHERE Salary > 45000
GROUP BY Department;
```

HAVING

Filters groups **after** grouping:

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 1;
```

Think:

```
WHERE
  ↓
Filter individual rows
  ↓
GROUP BY
  ↓
Create groups
  ↓
HAVING
  ↓
Filter groups
```

Using WHERE and HAVING Together

You can use both:

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
WHERE Salary > 40000
```

```
GROUP BY Department
HAVING COUNT(*) >= 2;
```

Meaning:

1. Select employees with salary greater than 40000.
2. Group them by department.
3. Keep only departments having at least 2 employees.

Multiple Conditions in HAVING

You can use AND and OR:

```
SELECT Department, AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 40000
      AND COUNT(*) >= 2;
```

This returns departments where:

- Average salary > 40000
- Employee count >= 2

HAVING Without GROUP BY

Some SQL databases allow HAVING without an explicit GROUP BY, treating the result as one group.

For example:

```
SELECT COUNT(*) AS EmployeeCount
FROM Employees
HAVING COUNT(*) > 3;
```

If there are more than 3 employees, the result is returned.

However, in normal SQL queries, HAVING is most commonly used with GROUP BY.

SQL Execution Order

A simplified logical order is:

```
FROM
↓
```

```
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
ORDER BY
```

So:

WHERE filters rows, while **HAVING** filters groups.

Most common pattern

```
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 1;
```

8. Joins

- Inner Join

INNER JOIN in SQL

An **INNER JOIN** combines rows from two tables **only when there is a matching value in both tables**.

Basic Syntax

```
SELECT columns
FROM TableA
INNER JOIN TableB
ON TableA.CommonColumn = TableB.CommonColumn;
```

Example

Suppose we have two tables.

Employees

EmployeeId	Name	DepartmentId
1	Amit	10
2	Rahul	20

EmployeeId	Name	DepartmentId
3	Priya	30
4	Neha	40

Departments

DepartmentId	DepartmentName
10	IT
20	HR
30	Finance
50	Sales

Now join them:

```
SELECT
  e.EmployeeId,
  e.Name,
  d.DepartmentName
FROM Employees e
INNER JOIN Departments d
ON e.DepartmentId = d.DepartmentId;
```

Result

EmployeeId	Name	DepartmentName
1	Amit	IT
2	Rahul	HR
3	Priya	Finance

Neha is excluded because **DepartmentId = 40** doesn't exist in **Departments**.

Sales is excluded because **DepartmentId = 50** doesn't exist in **Employees**.

Visual idea

Employees		Departments
-----		-----
10 Amit	→	10 IT ✓
20 Rahul	→	20 HR ✓
30 Priya	→	30 Finance ✓
40 Neha	—X	50 Sales —X

Only matching records are returned.

INNER JOIN with WHERE

You can also filter the joined result:

```
SELECT
    e.Name,
    d.DepartmentName
FROM Employees e
INNER JOIN Departments d
    ON e.DepartmentId = d.DepartmentId
WHERE d.DepartmentName = 'IT';
```

Result:

Name	DepartmentName
Amit	IT

Multiple INNER JOINS

You can join more than two tables.

```
SELECT
    e.Name,
    d.DepartmentName,
    p.ProjectName
FROM Employees e
INNER JOIN Departments d
    ON e.DepartmentId = d.DepartmentId
INNER JOIN Projects p
    ON e.EmployeeId = p.EmployeeId;
```

This returns employees who have:

1. A matching department
 2. A matching project
-

JOIN vs INNER JOIN

These are equivalent:

```
SELECT *
FROM Employees e
```



```
JOIN Departments d
ON e.DepartmentId = d.DepartmentId;
```

and:

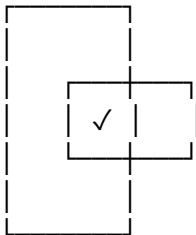
```
SELECT *
FROM Employees e
INNER JOIN Departments d
ON e.DepartmentId = d.DepartmentId;
```

INNER is optional because **JOIN** defaults to **INNER JOIN**.

Important Interview Point

INNER JOIN returns only the intersection of two tables based on the join condition.

Table A Table B



↑
Matching
records

Real-world example

If you have:

Customers
Orders

and want customers who **actually placed an order**:

```
SELECT
    c.CustomerId,
    c.Name,
    o.OrderId
FROM Customers c
INNER JOIN Orders o
ON c.CustomerId = o.CustomerId;
```

Customers without orders will **not** appear.

Rule to remember:

INNER JOIN = return only matching records from both tables.

- Left Join

LEFT JOIN in SQL

A **LEFT JOIN** returns **all rows from the left table** and only the matching rows from the right table.

If there is no match, the columns from the right table contain **NULL**.

Basic Syntax

```
SELECT columns
FROM TableA
LEFT JOIN TableB
  ON TableA.CommonColumn = TableB.CommonColumn;
```

Example

Employees

EmployeeId	Name	DepartmentId
1	Amit	10
2	Rahul	20
3	Priya	30
4	Neha	40

Departments

DepartmentId	DepartmentName
10	IT
20	HR
30	Finance
50	Sales

Now:

```
SELECT
    e.EmployeeId,
    e.Name,
    d.DepartmentName
FROM Employees e
LEFT JOIN Departments d
    ON e.DepartmentId = d.DepartmentId;
```

Result

EmployeeId	Name	DepartmentName
1	Amit	IT
2	Rahul	HR
3	Priya	Finance
4	Neha	NULL

Why is Neha included?

Because **Employees** is the **left table**.

Neha has **DepartmentId = 40**, but there is no department **40** in the **Departments** table.

Therefore:

Employee		Department
-----		-----
Amit	→	IT
Rahul	→	HR
Priya	→	Finance
Neha	→	NULL

The LEFT JOIN says:

Keep every row from the left table, even when there is no match.

INNER JOIN vs LEFT JOIN

INNER JOIN

```
SELECT e.Name, d.DepartmentName
FROM Employees e
INNER JOIN Departments d
    ON e.DepartmentId = d.DepartmentId;
```

Result:

Name	DepartmentName
Amit	IT
Rahul	HR
Priya	Finance

Neha is removed.

LEFT JOIN

```
SELECT e.Name, d.DepartmentName
FROM Employees e
LEFT JOIN Departments d
ON e.DepartmentId = d.DepartmentId;
```

Result:

Name	DepartmentName
Amit	IT
Rahul	HR
Priya	Finance
Neha	NULL

Neha remains.

Very Common Use Case

Find **all customers, including customers who have never placed an order.**

Customers

CustomerId	Name
-----	-----
1	Amit
2	Rahul
3	Priya

Orders

OrderId	CustomerId
101	1
102	1
103	2

Query:

```
SELECT
  c.CustomerId,
  c.Name,
  o.OrderId
FROM Customers c
LEFT JOIN Orders o
  ON c.CustomerId = o.CustomerId;
```

Result:

CustomerId	Name	OrderId
1	Amit	101
1	Amit	102
2	Rahul	103
3	Priya	NULL

Priya appears because **Customers** is the left table.

Find Records With No Match

This is one of the **most important uses of LEFT JOIN**.

For example, find customers who have **never placed an order**:

```
SELECT
  c.CustomerId,
  c.Name
FROM Customers c
LEFT JOIN Orders o
  ON c.CustomerId = o.CustomerId
WHERE o.OrderId IS NULL;
```

Result:

CustomerId	Name
3	Priya

The pattern is:

```
LEFT JOIN ...  
WHERE RightTable.PrimaryKey IS NULL
```

Meaning:

Find rows in the left table that don't have a matching row in the right table.

LEFT JOIN with Multiple Tables

You can chain multiple joins:

```
SELECT  
    c.Name,  
    o.OrderId,  
    p.ProductName  
FROM Customers c  
LEFT JOIN Orders o  
    ON c.CustomerId = o.CustomerId  
LEFT JOIN Products p  
    ON o.ProductId = p.ProductId;
```

This keeps all customers even if they don't have an order or product.

Important: ON vs WHERE

This is a common SQL interview topic.

Condition in ON

```
SELECT c.Name, o.OrderId  
FROM Customers c  
LEFT JOIN Orders o  
    ON c.CustomerId = o.CustomerId  
    AND o.Status = 'Completed';
```

All customers remain, but only completed orders are joined.

Condition in WHERE

```
SELECT c.Name, o.OrderId
FROM Customers c
LEFT JOIN Orders o
    ON c.CustomerId = o.CustomerId
WHERE o.Status = 'Completed';
```

This can eliminate customers with no matching order, making the result behave more like an **INNER JOIN**.

LEFT JOIN Mental Model

Remember:

LEFT JOIN

↓

Keep EVERYTHING from LEFT

+

Matching rows from RIGHT

+

NULL when no match

Quick comparison

JOIN	Left table	Right table
INNER JOIN	Matching only	Matching only
LEFT JOIN	Everything	Matching + NULL
RIGHT JOIN	Matching + NULL	Everything
FULL JOIN	Everything	Everything

Interview definition:

LEFT JOIN returns all rows from the left table and the matching rows from the right table. If no matching row exists, the right-side columns contain **NULL**.

- Right Join

RIGHT JOIN in SQL

A **RIGHT JOIN** returns **all rows from the right table** and only the matching rows from the left table.

If there is no match, the columns from the left table contain **NULL**.

Basic Syntax

```
SELECT columns
FROM TableA
RIGHT JOIN TableB
    ON TableA.CommonColumn = TableB.CommonColumn;
```

Here, **TableB is the right table**, so all rows from **TableB** are preserved.

Example

Employees

EmployeeId	Name	DepartmentId
1	Amit	10
2	Rahul	20
3	Priya	30
4	Neha	40

Departments

DepartmentId	DepartmentName
10	IT
20	HR
30	Finance
50	Sales

Query:

```
SELECT
    e.EmployeeId,
    e.Name,
    d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d
    ON e.DepartmentId = d.DepartmentId;
```

Result

EmployeeId	Name	DepartmentName
1	Amit	IT
2	Rahul	HR

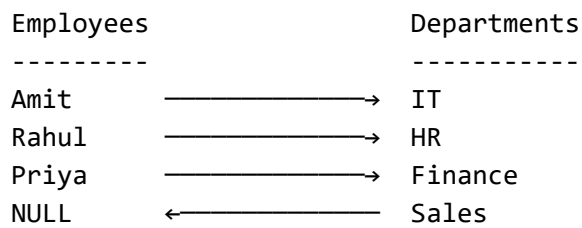
EmployeeId	Name	DepartmentName
3	Priya	Finance
NULL	NULL	Sales

Why is Sales included?

Because **Departments** is the **right table**.

Sales has **DepartmentId = 50**, but there is no employee with **DepartmentId = 50**.

Therefore:



The RIGHT JOIN says:

Keep every row from the right table, even when there is no match in the left table.

RIGHT JOIN vs INNER JOIN

INNER JOIN

```
SELECT
    e.Name,
    d.DepartmentName
FROM Employees e
INNER JOIN Departments d
    ON e.DepartmentId = d.DepartmentId;
```

Result:

Name	DepartmentName
Amit	IT
Rahul	HR
Priya	Finance

Sales is excluded because it has no employee.

RIGHT JOIN

```
SELECT
    e.Name,
    d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d
    ON e.DepartmentId = d.DepartmentId;
```

Result:

Name	DepartmentName
Amit	IT
Rahul	HR
Priya	Finance
NULL	Sales

Sales is included because **Departments** is the right table.

Finding Unmatched Rows

A useful pattern is finding departments that have **no employees**:

```
SELECT
    d.DepartmentId,
    d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d
    ON e.DepartmentId = d.DepartmentId
WHERE e.EmployeeId IS NULL;
```

Result:

DepartmentId	DepartmentName
50	Sales

This means:

Find departments that don't have any matching employees.

RIGHT JOIN Can Be Rewritten as LEFT JOIN

This is very important in practice.

These two queries are equivalent:

```
SELECT
    e.Name,
    d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d
    ON e.DepartmentId = d.DepartmentId;
```

and:

```
SELECT
    e.Name,
    d.DepartmentName
FROM Departments d
LEFT JOIN Employees e
    ON d.DepartmentId = e.DepartmentId;
```

The second version is often preferred because **LEFT JOIN is more commonly used and easier to read.**

RIGHT JOIN Mental Model

Remember:

```
RIGHT JOIN
    ↓
Keep EVERYTHING from RIGHT
    +
Matching rows from LEFT
    +
NULL when no match
```

Quick comparison

JOIN	Left Table	Right Table
INNER JOIN	Matching	Matching
LEFT JOIN	Everything	Matching + NULL
RIGHT JOIN	Matching + NULL	Everything
FULL OUTER JOIN	Everything	Everything

Interview Definition

RIGHT JOIN returns all rows from the right table and matching rows from the left table. If there is no matching row in the left table, the left-side columns contain **NULL**.

- Full Outer Join

FULL OUTER JOIN in SQL

A **FULL OUTER JOIN** returns:

- **All matching rows** from both tables
- **All unmatched rows** from the left table
- **All unmatched rows** from the right table
- **NULL** where there is no matching row

Basic Syntax

```
SELECT columns
FROM TableA
FULL OUTER JOIN TableB
    ON TableA.CommonColumn = TableB.CommonColumn;
```

Example

Employees

EmployeeId	Name	DepartmentId
1	Amit	10
2	Rahul	20
3	Priya	30
4	Neha	40

Departments

DepartmentId	DepartmentName
10	IT
20	HR
30	Finance
50	Sales

Query:

```
SELECT
    e.EmployeeId,
    e.Name,
    d.DepartmentId,
    d.DepartmentName
FROM Employees e
FULL OUTER JOIN Departments d
    ON e.DepartmentId = d.DepartmentId;
```

Result

EmployeeId	Name	DepartmentId	DepartmentName
1	Amit	10	IT
2	Rahul	20	HR
3	Priya	30	Finance
4	Neha	40	NULL
NULL	NULL	50	Sales

What happened?

1. Matching rows

These exist in both tables:

```
10 → IT
20 → HR
30 → Finance
```

So they are combined.

2. Unmatched left row

Neha has:

```
DepartmentId = 40
```

But department 40 doesn't exist.

So:

```
Neha → NULL
```

3. Unmatched right row

Sales has:

```
DepartmentId = 50
```

But no employee belongs to department 50.

So:

```
NULL → Sales
```

Visual Understanding



It keeps **everything from both sides**.

INNER vs LEFT vs RIGHT vs FULL

JOIN	What it returns
INNER JOIN	Matching rows only
LEFT JOIN	All left + matching right
RIGHT JOIN	Matching left + all right
FULL OUTER JOIN	Everything from both tables

Think of it like this:

```
INNER JOIN
[ MATCH ]

LEFT JOIN
[ LEFT + MATCH ]
```

```
RIGHT JOIN
[ MATCH + RIGHT ]

FULL OUTER JOIN
[ LEFT + MATCH + RIGHT ]
```

Finding Unmatched Records

A very useful use case is finding records that exist in **only one of the two tables**.

```
SELECT
    e.EmployeeId,
    e.Name,
    d.DepartmentName
FROM Employees e
FULL OUTER JOIN Departments d
    ON e.DepartmentId = d.DepartmentId
WHERE e.EmployeeId IS NULL
    OR d.DepartmentId IS NULL;
```

This returns:

EmployeeId	Name	DepartmentName
4	Neha	NULL
NULL	NULL	Sales

So you can identify:

- Employees without departments
- Departments without employees

FULL OUTER JOIN in MySQL

Some databases support **FULL OUTER JOIN** directly, such as SQL Server and PostgreSQL.

MySQL does not support **FULL OUTER JOIN directly.**

It can be simulated using **LEFT JOIN** + **RIGHT JOIN** with **UNION**:

```
SELECT
    e.EmployeeId,
    e.Name,
    d.DepartmentName
FROM Employees e
```

```
LEFT JOIN Departments d
  ON e.DepartmentId = d.DepartmentId

UNION

SELECT
  e.EmployeeId,
  e.Name,
  d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d
  ON e.DepartmentId = d.DepartmentId;
```

Interview Definition

FULL OUTER JOIN returns all rows from both tables. Matching rows are combined, while unmatched rows from either table contain **NULL** for the columns of the other table.

Easy memory trick

FULL = Everything from both sides.

- Cross Join

CROSS JOIN in SQL

A **CROSS JOIN** returns the **Cartesian product** of two tables.

That means:

Every row from the first table is combined with every row from the second table.

Unlike **INNER JOIN**, **LEFT JOIN**, or **RIGHT JOIN**, a **CROSS JOIN** normally **does not use an ON condition**.

Basic Syntax

```
SELECT columns
FROM TableA
CROSS JOIN TableB;
```

Example

Employees

EmployeeId	Name
1	Amit
2	Rahul
3	Priya

Departments

DepartmentId	DepartmentName
10	IT
20	HR

Query:

```
SELECT
    e.Name,
    d.DepartmentName
FROM Employees e
CROSS JOIN Departments d;
```

Result

Name	DepartmentName
Amit	IT
Amit	HR
Rahul	IT
Rahul	HR
Priya	IT
Priya	HR

Every employee is combined with every department.

How Many Rows?

This is the most important concept.

If:

```
Employees = 3 rows
Departments = 2 rows
```

Then:

$3 \times 2 = 6$ rows

If:

Table A = 100 rows

Table B = 50 rows

Then:

$100 \times 50 = 5,000$ rows

So a **CROSS JOIN** can produce a **very large result set**.

Visual Explanation

Suppose:

Employees

Amit

Rahul

Priya

and:

Departments

IT

HR

CROSS JOIN produces:

Amit → IT

Amit → HR

Rahul → IT

Rahul → HR

Priya → IT

Priya → HR

Think:

Every A × Every B

CROSS JOIN vs INNER JOIN

INNER JOIN

```
SELECT
    e.Name,
    d.DepartmentName
FROM Employees e
INNER JOIN Departments d
    ON e.DepartmentId = d.DepartmentId;
```

Only matching departments are returned.

CROSS JOIN

```
SELECT
    e.Name,
    d.DepartmentName
FROM Employees e
CROSS JOIN Departments d;
```

Every employee is combined with every department.

INNER JOIN	CROSS JOIN
Requires matching condition	No matching condition
Returns matching rows	Returns every combination
Usually fewer rows	Can produce many rows
Based on relationship	Cartesian product

Real-World Example

Suppose an e-commerce application has:

Products

```
T-Shirt  
Shoes  
Jeans
```

Sizes

```
S  
M  
L  
XL
```

You want to generate **every possible product-size combination**:

```
SELECT  
    p.ProductName,  
    s.Size  
FROM Products p  
CROSS JOIN Sizes s;
```

Result:

```
T-Shirt → S  
T-Shirt → M  
T-Shirt → L  
T-Shirt → XL  
  
Shoes   → S  
Shoes   → M  
Shoes   → L  
Shoes   → XL  
  
Jeans   → S  
Jeans   → M  
Jeans   → L  
Jeans   → XL
```

That's a useful case for **CROSS JOIN**.

Another Example: Dates × Employees

Suppose you need a report containing **every employee for every day**:

```
SELECT  
    e.Name,  
    c.CalendarDate
```

```
FROM Employees e  
CROSS JOIN Calendar c;
```

If there are:

```
100 employees  
30 days
```

you get:

```
100 × 30 = 3,000 rows
```

This can be useful for reporting and scheduling.

Important Warning

Avoid accidentally creating a Cartesian product when you actually wanted an **INNER JOIN**.

For example:

```
SELECT *  
FROM Employees e  
CROSS JOIN Departments d;
```

If you have:

```
10,000 Employees  
×  
1,000 Departments
```

you could generate:

```
10,000,000 rows
```

This can cause serious performance problems.

Interview Definition

CROSS JOIN returns the Cartesian product of two tables, meaning every row from the first table is combined with every row from the second table.

Easy memory trick

CROSS JOIN
↓
Every row × Every row
↓
Cartesian Product

Formula:

Result rows = Rows in Table A × Rows in Table B

- Self Join

SELF JOIN in SQL

A **SELF JOIN** is when a table is joined **with itself**.

It is useful when rows in the same table have a relationship with other rows in that same table.

SQL doesn't have a special **SELF JOIN** keyword. You use **JOIN** with **different aliases** for the same table.

Basic Syntax

```
SELECT
    a.Column1,
    b.Column2
FROM TableName a
JOIN TableName b
    ON a.RelatedColumn = b.KeyColumn;
```

Here:

a → first reference to the table
b → second reference to the same table

Example: Employee and Manager

Suppose we have an **Employees** table:

EmployeeId	Name	ManagerId
1	Amit	NULL

EmployeeId	Name	ManagerId
2	Rahul	1
3	Priya	1
4	Neha	2
5	Arjun	2

The important relationship is:

ManagerId → EmployeeId

For example:

Rahul.ManagerId = 1

Employee **1** is Amit, so **Amit is Rahul's manager**.

SELF JOIN Query

```
SELECT
  e.Name AS Employee,
  m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m
  ON e.ManagerId = m.EmployeeId;
```

Notice that **Employees** appears twice:

```
FROM Employees e
LEFT JOIN Employees m
```

They are two references to the **same table**.

Result

Employee	Manager
Amit	NULL
Rahul	Amit
Priya	Amit
Neha	Rahul

Employee	Manager
Arjun	Rahul

How it works

For Rahul:

```
Rahul.ManagerId = 1
```

Then SQL finds:

```
Employees.EmployeeId = 1
```

which is Amit.

So:

```
Rahul → Amit
```

For Neha:

```
Neha.ManagerId = 2
```

Employee 2 is Rahul.

Therefore:

```
Neha → Rahul
```

Why Use Different Aliases?

Without aliases, SQL wouldn't know which reference you're talking about.

```
Employees e  
Employees m
```

We use:

```
e = Employee  
m = Manager
```


So:

```
e.Name
```

means the employee's name.

And:

```
m.Name
```

means the manager's name.

INNER SELF JOIN

If you only want employees who **have a manager**:

```
SELECT
    e.Name AS Employee,
    m.Name AS Manager
FROM Employees e
INNER JOIN Employees m
    ON e.ManagerId = m.EmployeeId;
```

Result:

Employee	Manager
Rahul	Amit
Priya	Amit
Neha	Rahul
Arjun	Rahul

Amit is excluded because his `ManagerId` is `NULL`.

LEFT SELF JOIN

If you want **all employees**, including employees who don't have a manager:

```
SELECT
    e.Name AS Employee,
    m.Name AS Manager
FROM Employees e
```

```
LEFT JOIN Employees m
ON e.ManagerId = m.EmployeeId;
```

Result:

Employee	Manager
Amit	NULL
Rahul	Amit
Priya	Amit
Neha	Rahul
Arjun	Rahul

This is generally the more useful version for employee hierarchies.

Another Example: Product Categories

A self join isn't limited to employees.

Suppose:

Categories

CategoryId	CategoryName	ParentCategoryId
1	Electronics	NULL
2	Mobile Phones	1
3	Laptops	1
4	Android Phones	2
5	iPhones	2

Here, a category can have another category as its parent.

Query:

```
SELECT
  c.CategoryName AS Category,
  p.CategoryName AS ParentCategory
FROM Categories c
LEFT JOIN Categories p
ON c.ParentCategoryId = p.CategoryId;
```

Result:

Category	ParentCategory
Electronics	NULL
Mobile Phones	Electronics
Laptops	Electronics
Android Phones	Mobile Phones
iPhones	Mobile Phones

Common Uses of SELF JOIN

1. Employee → Manager

Employee → Manager

2. Category → Parent Category

Category → Parent Category

3. Employee → Mentor

Employee → Mentor

4. Organization hierarchy

```
CEO
├── Manager
│   ├── Employee
│   └── Employee
└── Manager
    └── Employee
```

5. Comparing rows within the same table

For example, finding employees who belong to the same department:

```
SELECT
    e1.Name AS Employee1,
    e2.Name AS Employee2
FROM Employees e1
JOIN Employees e2
```

```
ON e1.DepartmentId = e2.DepartmentId
AND e1.EmployeeId < e2.EmployeeId;
```

The condition:

```
e1.EmployeeId < e2.EmployeeId
```

prevents getting duplicate pairs such as:

```
Amit - Rahul
Rahul - Amit
```

SELF JOIN vs Other JOINS

JOIN	Purpose
INNER JOIN	Matching rows between tables
LEFT JOIN	All left + matching right
RIGHT JOIN	All right + matching left
FULL OUTER JOIN	Everything from both
CROSS JOIN	Every combination
SELF JOIN	A table joined with itself

A **SELF JOIN** can also be an **INNER JOIN** or **LEFT JOIN**. "Self" describes **what you're joining**, while **INNER/LEFT** describes **how the matching rows are handled**.

Interview Definition

A SELF JOIN is a join where a table is joined to itself using different aliases. It is commonly used to represent hierarchical or row-to-row relationships, such as employees and their managers.

Easy memory trick

```
SELF JOIN
  ↓
Same Table
  ↓
Two Aliases
  ↓
Compare/Relate Rows
```

Example:

```
FROM Employees e
JOIN Employees m
  ON e.ManagerId = m.EmployeeId
```

Same table + different aliases = SELF JOIN.

9. SQL Constraints

- PRIMARY KEY

PRIMARY KEY in SQL

A **PRIMARY KEY** is a constraint used to **uniquely identify each row** in a table.

Example

```
CREATE TABLE Employees (
  EmployeeId INT PRIMARY KEY,
  Name VARCHAR(100),
  Salary DECIMAL(10,2)
);
```

Here:

EmployeeId → PRIMARY KEY

Every employee must have a unique **EmployeeId**.

Valid data

```
INSERT INTO Employees VALUES (1, 'Rahul', 50000);
INSERT INTO Employees VALUES (2, 'Amit', 60000);
INSERT INTO Employees VALUES (3, 'Priya', 55000);
```

EmployeeId	Name	Salary
1	Rahul	50000
2	Amit	60000
3	Priya	55000

Duplicate value is not allowed

```
INSERT INTO Employees VALUES (1, 'John', 45000);
```

✗ Error because `EmployeeId = 1` already exists.

`NULL` is not allowed

```
INSERT INTO Employees VALUES (NULL, 'John', 45000);
```

✗ Error because a primary key cannot contain `NULL`.

Important Properties

A primary key:

- Must be **unique**
- Cannot contain `NULL`
- Identifies each row
- A table can have **only one primary key constraint**
- Can contain **one or multiple columns**

Composite Primary Key

A primary key can use multiple columns.

```
CREATE TABLE OrderItems (  
    OrderId INT,  
    ProductId INT,  
    Quantity INT,  
  
    PRIMARY KEY (OrderId, ProductId)  
);
```

Here, the combination of:

```
OrderId + ProductId
```

must be unique.

For example:

OrderId	ProductId	Quantity
100	1	2
100	2	5

OrderId	ProductId	Quantity
101	1	3

This is allowed because each **combination** is unique.

But:

```
100, 1
100, 1
```

✗ would violate the primary key.

Primary Key vs Unique

Feature	PRIMARY KEY	UNIQUE
Duplicate values	✗ No	✗ No
NULL	✗ No	DB-dependent
Purpose	Identify row	Prevent duplicates
Per table	One primary key	Multiple unique constraints possible

Simple rule

PRIMARY KEY = unique identifier for a row.

- FOREIGN KEY

FOREIGN KEY in SQL

A **FOREIGN KEY** is a constraint used to **create a relationship between two tables**.

It ensures that a value in one table refers to an existing value in another table.

1. Basic Example

Suppose we have two tables:

Parent Table — **Departments**

```
CREATE TABLE Departments (  
    DepartmentId INT PRIMARY KEY,  
    DepartmentName VARCHAR(100)  
);
```

Data:

DepartmentId	DepartmentName
1	IT
2	HR
3	Finance

Now create an **Employees** table:

```
CREATE TABLE Employees (  
  EmployeeId INT PRIMARY KEY,  
  Name VARCHAR(100),  
  DepartmentId INT,  
  
  FOREIGN KEY (DepartmentId)  
    REFERENCES Departments(DepartmentId)  
);
```

Here:

```
Departments  
  DepartmentId ← PRIMARY KEY  
    ↑  
    |  
    |  
Employees  
  DepartmentId ← FOREIGN KEY
```

Employees.DepartmentId references **Departments.DepartmentId**.

2. Valid Insert

Because department **1** exists:

```
INSERT INTO Employees  
VALUES (101, 'Rahul', 1);
```

✓ Valid.

Result:

EmployeeId	Name	DepartmentId
101	Rahul	1

3. Invalid Insert

Suppose we try:

```
INSERT INTO Employees
VALUES (102, 'Amit', 99);
```

✗ Invalid because department 99 does not exist in Departments.

The foreign key protects the database from having an employee assigned to a nonexistent department.

4. Why Use a Foreign Key?

The main purpose is **referential integrity**.

Without a foreign key, you could have:

```
Employees

EmployeeId | Name   | DepartmentId
-----|-----|-----
101        | Rahul | 1
102        | Amit  | 99
```

But there is no department 99.

With a foreign key, the database prevents this invalid relationship.

5. Foreign Key with CREATE TABLE

You can define it inline:

```
CREATE TABLE Employees (
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    DepartmentId INT REFERENCES Departments(DepartmentId)
);
```

Or explicitly:

```
CREATE TABLE Employees (
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    DepartmentId INT,
```

```
FOREIGN KEY (DepartmentId)
REFERENCES Departments(DepartmentId)
);
```

The second style is especially useful when you want to give the constraint a name.

6. Named Foreign Key

```
CREATE TABLE Employees (
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    DepartmentId INT,

    CONSTRAINT FK_Employees_Departments
        FOREIGN KEY (DepartmentId)
        REFERENCES Departments(DepartmentId)
);
```

Now the foreign key constraint is named:

```
FK_Employees_Departments
```

This makes it easier to modify or remove later.

7. Foreign Key with ALTER TABLE

You can also add a foreign key after creating the table:

```
ALTER TABLE Employees
ADD CONSTRAINT FK_Employees_Departments
FOREIGN KEY (DepartmentId)
REFERENCES Departments(DepartmentId);
```

8. ON DELETE

You can specify what happens to child records when the referenced parent record is deleted.

CASCADE

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100),  
    DepartmentId INT,  
  
    FOREIGN KEY (DepartmentId)  
        REFERENCES Departments(DepartmentId)  
        ON DELETE CASCADE  
);
```

If department 1 is deleted:

```
DELETE FROM Departments  
WHERE DepartmentId = 1;
```

Employees belonging to department 1 will also be deleted.

```
Department 1 deleted  
    ↓  
Employees with DepartmentId = 1  
    ↓  
Also deleted
```

⚠ Use **CASCADE** carefully because it can delete many related records.

9. ON DELETE SET NULL

You can instead set the foreign key to **NULL**.

```
FOREIGN KEY (DepartmentId)  
REFERENCES Departments(DepartmentId)  
ON DELETE SET NULL
```

If department 1 is deleted:

Before:

EmployeeId	Name	DepartmentId
101	Rahul	1

After:

EmployeeId	Name	DepartmentId
101	Rahul	NULL

The employee remains, but no longer belongs to a department.

The `DepartmentId` column must allow `NULL` for this approach.

10. ON DELETE RESTRICT / NO ACTION

This prevents deleting a parent record when child records reference it, depending on the database's supported behavior.

For example:

```
Departments
1 | IT
  |
  |
Employees
101 | Rahul | 1
```

Trying to delete department 1:

```
DELETE FROM Departments
WHERE DepartmentId = 1;
```

✗ The database rejects the deletion because employee 101 still references it.

11. Foreign Key and Primary Key Relationship

A very common relationship is:

```
Parent Table
-----
Departments
DepartmentId (PK)
DepartmentName

1
|
|
|
∞
|
↓

Child Table
```

```
-----
Employees
EmployeeId (PK)
Name
DepartmentId (FK)
```

This represents a **one-to-many relationship**:

One department can have many employees.

12. Multiple Foreign Keys

A table can have multiple foreign keys.

```
CREATE TABLE Orders (
  OrderId INT PRIMARY KEY,
  CustomerId INT,
  ProductId INT,

  FOREIGN KEY (CustomerId)
    REFERENCES Customers(CustomerId),

  FOREIGN KEY (ProductId)
    REFERENCES Products(ProductId)
);
```

Here:

```
Orders.CustomerId → Customers.CustomerId

Orders.ProductId → Products.ProductId
```

PRIMARY KEY vs FOREIGN KEY

PRIMARY KEY	FOREIGN KEY
Identifies a row	Creates a relationship
Must be unique	Can contain duplicates
Cannot be NULL	Can be NULL unless NOT NULL
One primary key constraint per table	Multiple foreign keys possible
Usually exists in parent table	Usually references parent table

Simple rule

Primary Key = "Who am I?" Foreign Key = "Who do I belong to?"

- UNIQUE

UNIQUE Constraint in SQL

The **UNIQUE** constraint ensures that **duplicate values are not allowed** in a column or combination of columns.

It is commonly used for values such as:

- Email address
- Username
- Phone number
- Employee code
- Aadhaar/customer number, where appropriate

1. Basic Example

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Email VARCHAR(150) UNIQUE  
);
```

Here:

Email → UNIQUE

means every non-NULL email value must be unique.

Valid

```
INSERT INTO Employees  
VALUES (1, 'Rahul', 'rahul@gmail.com');  
  
INSERT INTO Employees  
VALUES (2, 'Amit', 'amit@gmail.com');
```

✓ Both are valid.

Duplicate value

```
INSERT INTO Employees
```

```
VALUES (3, 'Priya', 'rahul@gmail.com');
```

✗ Error because `rahul@gmail.com` already exists.

2. Multiple **UNIQUE** Columns

A table can have multiple **UNIQUE** constraints.

```
CREATE TABLE Users (  
    UserId INT PRIMARY KEY,  
    Username VARCHAR(50) UNIQUE,  
    Email VARCHAR(150) UNIQUE,  
    Phone VARCHAR(20) UNIQUE  
);
```

Now all three must be unique:

```
Username → unique  
Email    → unique  
Phone    → unique
```

3. Named **UNIQUE** Constraint

You can explicitly name the constraint:

```
CREATE TABLE Users (  
    UserId INT PRIMARY KEY,  
    Username VARCHAR(50),  
    Email VARCHAR(150),  
  
    CONSTRAINT UQ_Users_Username UNIQUE (Username),  
    CONSTRAINT UQ_Users_Email UNIQUE (Email)  
);
```

The names are:

```
UQ_Users_Username  
UQ_Users_Email
```

Naming constraints is useful when managing a database later.

4. Composite **UNIQUE**

You can apply **UNIQUE** to multiple columns together.

```
CREATE TABLE StudentCourses (  
    StudentId INT,  
    CourseId INT,  
  
    CONSTRAINT UQ_Student_Course  
        UNIQUE (StudentId, CourseId)  
);
```

This means the **combination** of **StudentId** + **CourseId** must be unique.

Valid

StudentId	CourseId
1	101
1	102
2	101

Invalid

```
StudentId = 1  
CourseId = 101
```

again:

```
StudentId = 1  
CourseId = 101
```

✗ Duplicate combination.

Notice that **StudentId** itself can repeat and **CourseId** itself can repeat. Only their **combination** must be unique.

5. **UNIQUE** and **NULL**

The behavior of **NULL** with **UNIQUE** constraints is **database-dependent**.

For example, many databases allow **NULL** in a unique column because **NULL** represents an unknown/missing value.

Therefore:

```
CREATE TABLE Users (  
  UserId INT PRIMARY KEY,  
  Email VARCHAR(150) UNIQUE  
);
```

You should not assume that **UNIQUE** automatically means **NOT NULL**.

If the value is required **and** must be unique, use both:

```
Email VARCHAR(150) NOT NULL UNIQUE
```

6. UNIQUE vs PRIMARY KEY

This is an important interview question.

Feature	PRIMARY KEY	UNIQUE
Duplicate values	✗ Not allowed	✗ Not allowed
NULL	✗ Not allowed	DB-dependent
Main purpose	Identify each row	Prevent duplicates
Number per table	One primary key constraint	Multiple unique constraints
Can be composite	✓ Yes	✓ Yes

Example:

```
CREATE TABLE Employees (  
  EmployeeId INT PRIMARY KEY,  
  Email VARCHAR(150) UNIQUE,  
  Username VARCHAR(50) UNIQUE  
);
```

Here:

```
EmployeeId → identifies the employee  
Email      → must be unique  
Username   → must be unique
```

7. Adding UNIQUE Using ALTER TABLE

If the table already exists:

```
ALTER TABLE Employees
ADD CONSTRAINT UQ_Employees_Email
UNIQUE (Email);
```

8. Removing a **UNIQUE** Constraint

If the constraint has a name:

```
ALTER TABLE Employees
DROP CONSTRAINT UQ_Employees_Email;
```

The exact syntax can vary between database systems such as SQL Server, MySQL, PostgreSQL, and Oracle.

Simple Rule

UNIQUE = "This value must not be duplicated."

For example:

```
Email      → UNIQUE
Username   → UNIQUE
PhoneNumber → UNIQUE
```

- **NOT NULL**

NOT NULL Constraint in SQL

The **NOT NULL** constraint ensures that a column **must always have a value**.

In other words:

NOT NULL means **this column cannot contain NULL**.

1. Basic Example

```
CREATE TABLE Employees (
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Salary DECIMAL(10,2)
);
```

Here:

```
Name → NOT NULL
```

means every employee must have a name.

Valid

```
INSERT INTO Employees  
VALUES (1, 'Rahul', 50000);
```

✓ Valid.

Invalid

```
INSERT INTO Employees  
VALUES (2, NULL, 60000);
```

✗ Error because Name is NOT NULL.

2. Why Use NOT NULL?

Use NOT NULL when a value is **required** for every row.

For example:

```
CREATE TABLE Customers (  
    CustomerId INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(150) NOT NULL,  
    Phone VARCHAR(20)  
);
```

Here:

```
CustomerId → required  
Name       → required  
Email      → required  
Phone      → optional
```

3. NULL vs Empty String

This is important.

NULL means:

No value / unknown value

An empty string:

```
''
```

means:

A value exists, but it contains zero characters.

For example:

```
Name = NULL
```

and

```
Name = ''
```

are not the same.

A **NOT NULL** column can still potentially contain an empty string:

```
INSERT INTO Employees  
VALUES (1, '', 50000);
```

Whether this is accepted depends on the database, but '' is not the same as SQL **NULL**.

4. **NOT NULL** with **DEFAULT**

You can combine **NOT NULL** and **DEFAULT**.

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Status VARCHAR(20) NOT NULL DEFAULT 'Active'  
);
```

Now:

```
INSERT INTO Employees (EmployeeId, Name)  
VALUES (1, 'Rahul');
```

The database automatically uses:

```
Status = 'Active'
```

Result:

EmployeeId	Name	Status
1	Rahul	Active

5. Adding NOT NULL to an Existing Column

Suppose you have:

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100)  
);
```

You can change Name to NOT NULL.

SQL Server

```
ALTER TABLE Employees  
ALTER COLUMN Name VARCHAR(100) NOT NULL;
```

PostgreSQL

```
ALTER TABLE Employees  
ALTER COLUMN Name SET NOT NULL;
```

The syntax varies between database systems.

⚠ If existing rows contain NULL, you generally need to fix those rows before adding the constraint.

6. NOT NULL vs UNIQUE

Feature	NOT NULL	UNIQUE
Allows NULL	✗	DB-dependent
Allows duplicates	✓	✗
Main purpose	Require a value	Prevent duplicates

You can combine them:

```
Email VARCHAR(150) NOT NULL UNIQUE
```

This means:

Email is **required** and **must be unique**.

7. Common Examples

```
CREATE TABLE Users (  
    UserId INT PRIMARY KEY,  
    Username VARCHAR(50) NOT NULL UNIQUE,  
    Email VARCHAR(150) NOT NULL UNIQUE,  
    PasswordHash VARCHAR(255) NOT NULL,  
    Phone VARCHAR(20),  
    Status VARCHAR(20) NOT NULL DEFAULT 'Active'  
);
```

Here:

UserId	→ PRIMARY KEY
Username	→ NOT NULL + UNIQUE
Email	→ NOT NULL + UNIQUE
PasswordHash	→ NOT NULL
Phone	→ optional
Status	→ NOT NULL + DEFAULT

Simple Rule

NOT NULL = "A value is required in this column."

- CHECK

CHECK Constraint in SQL

The **CHECK** constraint is used to **restrict the values that can be stored in a column** based on a condition.

CHECK = "The value must satisfy this condition."

1. Basic Example

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100),
```

```
Age INT CHECK (Age >= 18)
);
```

Here:

```
Age → CHECK (Age >= 18)
```

means the employee's age must be **18 or greater**.

Valid

```
INSERT INTO Employees
VALUES (1, 'Rahul', 25);
```

✓ Valid.

Invalid

```
INSERT INTO Employees
VALUES (2, 'Amit', 15);
```

✗ Error because:

```
15 >= 18
```

is false.

2. CHECK with Salary

```
CREATE TABLE Employees (
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
    CHECK (Salary > 0)
);
```

Valid:

```
INSERT INTO Employees
VALUES (1, 'Rahul', 50000);
```

✗ Invalid:

```
INSERT INTO Employees  
VALUES (2, 'Amit', -5000);
```

because salary cannot be less than or equal to zero.

3. CHECK with a Range

You can use **BETWEEN**:

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT CHECK (Age BETWEEN 18 AND 65)  
);
```

Allowed:

```
18  
25  
40  
65
```

Not allowed:

```
17  
66
```

4. CHECK with Multiple Values

You can use **IN**:

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Department VARCHAR(20)  
    CHECK (Department IN ('IT', 'HR', 'Finance'))  
);
```

Valid:


```
INSERT INTO Employees  
VALUES (1, 'Rahul', 'IT');
```

Valid:

```
INSERT INTO Employees  
VALUES (2, 'Amit', 'HR');
```

Invalid:

```
INSERT INTO Employees  
VALUES (3, 'Priya', 'Sales');
```

✗ **Sales** isn't one of the permitted values.

5. CHECK with Multiple Conditions

You can use **AND** and **OR**.

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Age INT,  
    Salary DECIMAL(10,2),  
  
    CHECK (Age >= 18 AND Salary > 0)  
);
```

Both conditions must be satisfied:

```
Age >= 18  
AND  
Salary > 0
```

6. Named CHECK Constraint

You can give the constraint a name:

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Age INT,
```

```
CONSTRAINT CK_Employees_Age  
CHECK (Age >= 18)  
);
```

The constraint name is:

```
CK_Employees_Age
```

Naming constraints is useful when you need to modify or remove them later.

7. Adding CHECK with ALTER TABLE

You can add a constraint to an existing table:

```
ALTER TABLE Employees  
ADD CONSTRAINT CK_Employees_Salary  
CHECK (Salary > 0);
```

8. Removing a CHECK Constraint

If the constraint has a name:

```
ALTER TABLE Employees  
DROP CONSTRAINT CK_Employees_Salary;
```

The exact syntax can vary between database systems.

9. CHECK vs NOT NULL

These constraints have different purposes.

Constraint	Purpose
NOT NULL	Value must be provided
CHECK	Value must satisfy a condition

Example:

```
Age INT NOT NULL CHECK (Age >= 18)
```

This means:

1. Age cannot be **NULL**.
2. Age must be at least **18**.

10. Real-World Example

```
CREATE TABLE Products (  
    ProductId INT PRIMARY KEY,  
    ProductName VARCHAR(100) NOT NULL,  
    Price DECIMAL(10,2) CHECK (Price > 0),  
    Quantity INT CHECK (Quantity >= 0),  
    Status VARCHAR(20)  
        CHECK (Status IN ('Active', 'Inactive'))  
);
```

Here:

```
ProductId → PRIMARY KEY  
ProductName → NOT NULL  
Price → must be > 0  
Quantity → must be >= 0  
Status → must be Active or Inactive
```

Simple Rule

CHECK = "Only allow values that satisfy a specified condition."

- **DEFAULT**

DEFAULT Constraint in SQL

The **DEFAULT** constraint automatically provides a value for a column when an **INSERT** statement **does not provide a value for that column**.

DEFAULT = "Use this value if no value is supplied."

1. Basic Example

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Status VARCHAR(20) DEFAULT 'Active'  
);
```

Now insert an employee without specifying **Status**:

```
INSERT INTO Employees (EmployeeId, Name)
VALUES (1, 'Rahul');
```

The database automatically uses:

```
Status = 'Active'
```

Result:

EmployeeId	Name	Status
1	Rahul	Active

2. DEFAULT with Numbers

```
CREATE TABLE Products (
    ProductId INT PRIMARY KEY,
    ProductName VARCHAR(100),
    Quantity INT DEFAULT 0
);
```

Insert:

```
INSERT INTO Products (ProductId, ProductName)
VALUES (101, 'Laptop');
```

Result:

ProductId	ProductName	Quantity
101	Laptop	0

3. DEFAULT with Dates

You can use a database-supported date/time function.

For example:

```
CREATE TABLE Orders (
    OrderId INT PRIMARY KEY,
    OrderDate DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Then:

```
INSERT INTO Orders (OrderId)
VALUES (1001);
```

The database automatically sets `OrderDate` to the current date/time.

4. Explicit Value Overrides `DEFAULT`

Suppose:

```
CREATE TABLE Employees (
  EmployeeId INT PRIMARY KEY,
  Name VARCHAR(100),
  Status VARCHAR(20) DEFAULT 'Active'
);
```

You can still explicitly provide another value:

```
INSERT INTO Employees (EmployeeId, Name, Status)
VALUES (1, 'Rahul', 'Inactive');
```

Result:

EmployeeId	Name	Status
1	Rahul	Inactive

The `DEFAULT` value is only used when the column is omitted from the `INSERT`.

5. `DEFAULT` vs `NULL`

This distinction is important.

```
INSERT INTO Employees (EmployeeId, Name)
VALUES (1, 'Rahul');
```

If `Status` has:

```
DEFAULT 'Active'
```

then:

```
Status = 'Active'
```

But if you explicitly insert **NULL**:

```
INSERT INTO Employees (EmployeeId, Name, Status)
VALUES (2, 'Amit', NULL);
```

the result is generally:

```
Status = NULL
```

The **DEFAULT** does **not** normally replace an explicitly supplied **NULL**.

If you want to prevent **NULL**, combine the constraints:

```
Status VARCHAR(20) NOT NULL DEFAULT 'Active'
```

6. DEFAULT with NOT NULL

A common pattern is:

```
CREATE TABLE Users (
  UserId INT PRIMARY KEY,
  Username VARCHAR(50) NOT NULL,
  Status VARCHAR(20) NOT NULL DEFAULT 'Active'
);
```

Now:

```
INSERT INTO Users (UserId, Username)
VALUES (1, 'Rahul');
```

automatically produces:

```
UserId    = 1
Username  = Rahul
Status    = Active
```

And because of **NOT NULL**, this is not allowed:

```
INSERT INTO Users (UserId, Username, Status)
```

```
VALUES (2, 'Amit', NULL);
```

7. Named **DEFAULT** Constraint

Some databases, especially SQL Server, allow you to explicitly name the constraint:

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100),  
  
    CONSTRAINT DF_Employees_Status  
        DEFAULT 'Active' FOR Status  
);
```

The exact syntax for named defaults varies between database systems.

8. Adding a **DEFAULT** to an Existing Table

For example, in SQL Server:

```
ALTER TABLE Employees  
ADD CONSTRAINT DF_Employees_Status  
DEFAULT 'Active' FOR Status;
```

Database-specific syntax can differ.

Real-World Example

```
CREATE TABLE Orders (  
    OrderId INT PRIMARY KEY,  
    CustomerName VARCHAR(100) NOT NULL,  
    Quantity INT NOT NULL DEFAULT 1,  
    Status VARCHAR(20) NOT NULL DEFAULT 'Pending',  
    CreatedDate DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

If you run:

```
INSERT INTO Orders (OrderId, CustomerName)  
VALUES (1001, 'Rahul');
```

The database can automatically populate:

```
OrderId      = 1001
CustomerName = Rahul
Quantity     = 1
Status       = Pending
CreatedDate  = current date/time
```

Quick Rule

DEFAULT = automatically use a predefined value when the column is omitted from **INSERT**.

10. Keys

- Primary Key

Primary Key in SQL

A **Primary Key** is a column, or combination of columns, that **uniquely identifies each row in a table**.

Example

```
CREATE TABLE Employees (
  EmployeeId INT PRIMARY KEY,
  Name VARCHAR(100),
  Salary DECIMAL(10,2)
);
```

Here, **EmployeeId** is the Primary Key.

EmployeeId	Name	Salary
101	Rahul	50000
102	Amit	60000
103	Priya	55000

EmployeeId must be different for every employee.

Rules of Primary Key

1. Must be unique

```
EmployeeId
101
102
103
```


This is valid.

But:

```
101
101  ✗
```

is not allowed.

2. Cannot contain NULL

```
EmployeeId
101
102
NULL ✗
```

A Primary Key must always have a value.

3. One Primary Key per table

A table can have only **one Primary Key constraint**.

However, that Primary Key can contain multiple columns.

Composite Primary Key

A Primary Key can consist of multiple columns.

```
CREATE TABLE StudentCourses (
    StudentId INT,
    CourseId INT,

    PRIMARY KEY (StudentId, CourseId)
);
```

Here:

```
StudentId + CourseId
      ↓
Composite Primary Key
```

Example:

StudentId	CourseId
101	1
101	2
102	1
102	2

StudentId alone isn't unique, and **CourseId** alone isn't unique, but their **combination** is unique.

Adding Primary Key After Table Creation

You can also add it using **ALTER TABLE**:

```
ALTER TABLE Employees
ADD CONSTRAINT PK_Employees
PRIMARY KEY (EmployeeId);
```

Primary Key with Identity/Auto Increment

In SQL Server:

```
CREATE TABLE Employees (
    EmployeeId INT IDENTITY(1,1) PRIMARY KEY,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

IDENTITY(1,1) automatically generates:

```
1
2
3
4
5
...
```

So you don't need to manually provide the **EmployeeId**.

Primary Key vs UNIQUE

Feature	Primary Key	UNIQUE
Uniqueness	✓ Yes	✓ Yes

Feature	Primary Key	UNIQUE
NULL	✗ Not allowed	DB-specific behavior
Per table	One constraint	Multiple allowed
Main row identifier	✓ Yes	✗ Not necessarily
Foreign Key can reference it	✓ Yes	✓ Often, if uniquely constrained

Easy way to remember

Primary Key = the main identity of a row.

For example:

```
EmployeeId → Primary Key
101        → Rahul
102        → Amit
103        → Priya
```

Each employee can be uniquely identified using `EmployeeId`.

- Foreign Key

Foreign Key in SQL

A **Foreign Key** is a column or set of columns that **references a key in another table**. It is mainly used to establish a **relationship between tables** and maintain **referential integrity**.

Example

Suppose we have two tables:

1. Departments

```
CREATE TABLE Departments (  
    DepartmentId INT PRIMARY KEY,  
    DepartmentName VARCHAR(100)  
);
```

2. Employees

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100),  
    DepartmentId INT,  
  
    FOREIGN KEY (DepartmentId)
```

```
REFERENCES Departments(DepartmentId)
);
```

Here:



Employees.DepartmentId is the **Foreign Key**.

Example Data

Departments

DepartmentId	DepartmentName
1	IT
2	HR
3	Finance

Employees

EmployeeId	Name	DepartmentId
101	Rahul	1
102	Amit	1
103	Priya	2

Here:

```
Employee 101 → Department 1 → IT
Employee 102 → Department 1 → IT
Employee 103 → Department 2 → HR
```

Multiple employees can have the same Foreign Key value.

Why Use a Foreign Key?

The main purpose is to **prevent invalid relationships**.

Suppose the **Departments** table contains only:

```
1 → IT
2 → HR
```

This would be invalid:

```
INSERT INTO Employees
VALUES (104, 'John', 99);
```

because Department 99 doesn't exist.

The Foreign Key prevents this, helping maintain **referential integrity**.

Foreign Key Syntax

While creating a table

```
CREATE TABLE Employees (
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    DepartmentId INT,

    FOREIGN KEY (DepartmentId)
        REFERENCES Departments(DepartmentId)
);
```

Named constraint

```
CREATE TABLE Employees (
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    DepartmentId INT,

    CONSTRAINT FK_Employees_Departments
        FOREIGN KEY (DepartmentId)
        REFERENCES Departments(DepartmentId)
);
```

Giving the constraint a name is useful when you later need to modify or remove it.

Add Foreign Key Later

```
ALTER TABLE Employees
ADD CONSTRAINT FK_Employees_Departments
FOREIGN KEY (DepartmentId)
REFERENCES Departments(DepartmentId);
```

Foreign Key Can Contain NULL

Unless you specify **NOT NULL**, a Foreign Key can normally contain **NULL**.

```
CREATE TABLE Employees (
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    DepartmentId INT NULL,

    FOREIGN KEY (DepartmentId)
        REFERENCES Departments(DepartmentId)
);
```

Example:

EmployeeId	Name	DepartmentId
101	Rahul	1
102	Amit	2
103	John	NULL

John currently doesn't have a department assigned.

If every employee **must** belong to a department:

```
DepartmentId INT NOT NULL
```

Foreign Key with ON DELETE

You can specify what happens when the referenced parent row is deleted.

CASCADE

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100),  
    DepartmentId INT,  
  
    FOREIGN KEY (DepartmentId)  
        REFERENCES Departments(DepartmentId)  
        ON DELETE CASCADE  
);
```

If Department 1 is deleted, its related employees are also deleted.



SET NULL

```
FOREIGN KEY (DepartmentId)  
REFERENCES Departments(DepartmentId)  
ON DELETE SET NULL
```

The employee remains, but DepartmentId becomes NULL.

RESTRICT / NO ACTION

Prevents deleting the parent if related child rows exist. Exact syntax/behavior varies by database system.

Foreign Key vs Primary Key

Primary Key	Foreign Key
Uniquely identifies a row	References a key in another table
Must be unique	Doesn't have to be unique
Cannot be NULL	Can be NULL unless NOT NULL
Usually one per table	A table can have multiple Foreign Keys
Defines the identity of a row	Defines a relationship

Easy way to remember



Foreign Key



"Which other row/table am I related to?"

For example:

```
Departments
DepartmentId = 1
      ↑
      | Foreign Key
      |
Employees
DepartmentId = 1
```

Primary Key identifies. Foreign Key connects.

- Candidate Key

Candidate Key in SQL

A **Candidate Key** is a column, or combination of columns, that can **uniquely identify each row** in a table and is **eligible to become the Primary Key**.

A table can have **multiple Candidate Keys**, but only **one is normally selected as the Primary Key**.

Example

Consider an **Employees** table:

```
CREATE TABLE Employees (
  EmployeeId INT,
  Email VARCHAR(100),
  AadhaarNumber VARCHAR(20),
  Name VARCHAR(100)
);
```

Suppose:

- **EmployeeId** is unique.
- **Email** is unique.
- **AadhaarNumber** is unique.

Then all three can uniquely identify an employee:


```
EmployeeId    → Candidate Key
Email         → Candidate Key
AadhaarNumber → Candidate Key
```

We might choose:

```
EmployeeId → Primary Key
```

The remaining Candidate Keys become **Alternate Keys**:

```
Email        → Alternate Key
AadhaarNumber → Alternate Key
```

Properties of a Candidate Key

A Candidate Key should be:

1. Unique

Two rows cannot have the same Candidate Key value.

```
EmployeeId
101
102
103
```

Valid.

```
EmployeeId
101
101 ✖
```

Not valid.

2. Minimal

It should contain **only the columns necessary for uniqueness**.

For example, if **Email** is already unique:

```
Email
```

is a Candidate Key.

You don't need:

Email + Name

to uniquely identify the employee.

Therefore:

Email → Candidate Key
Email + Name → Super Key, but not minimal

Candidate Key vs Primary Key

Suppose:

EmployeeId
Email
PhoneNumber

are all unique.

Then:

Candidate Keys

- |
- ├── EmployeeId
- ├── Email
- └── PhoneNumber

We select one:

EmployeeId → Primary Key

The others:

Email
PhoneNumber

are **Alternate Keys**.

So:

Primary Key is a Candidate Key selected as the main identifier.

Candidate Key vs Super Key

This is important for interviews.

Super Key

Any set of columns that can uniquely identify a row.

Example:

```
EmployeeId  
EmployeeId + Name  
EmployeeId + Email  
EmployeeId + Name + Email
```

Candidate Key

A **minimal Super Key**.

```
EmployeeId
```

is enough, so it is a Candidate Key.

```
EmployeeId + Name
```

has an unnecessary column, so it is only a Super Key.

```
Super Key  
  ↓  
Remove unnecessary attributes  
  ↓  
Candidate Key
```

Composite Candidate Key

A Candidate Key can also consist of multiple columns.

For example:

```
CREATE TABLE StudentCourses (  
  StudentId INT,  
  CourseId INT,
```

```
EnrollmentDate DATE
);
```

Suppose neither **StudentId** nor **CourseId** is individually unique, but their combination is unique:

StudentId + **CourseId**

Then:

StudentId + **CourseId** → Composite Candidate Key

Easy Example

Imagine a student table:

StudentId	Email	Phone	Name
101	a@gmail.com	98765	Amit
102	b@gmail.com	87654	Rahul
103	c@gmail.com	76543	Priya

If **StudentId**, **Email**, and **Phone** are all unique:

Candidate Keys:
StudentId
Email
Phone

Choose:

StudentId → Primary Key

Then:

Email → Alternate Key
Phone → Alternate Key

Interview definition

A Candidate Key is a minimal set of one or more attributes that uniquely identifies each row and can be chosen as the Primary Key.

- Composite Key

Composite Key in SQL

A **Composite Key** is a key made up of **two or more columns** that work together to uniquely identify a row.

The important point is that **the combination is unique**, even if the individual columns are not.

Example

Consider a **StudentCourses** table:

```
CREATE TABLE StudentCourses (  
    StudentId INT,  
    CourseId INT,  
    EnrollmentDate DATE,  
  
    PRIMARY KEY (StudentId, CourseId)  
);
```

Here:

StudentId + CourseId
↓
Composite Key

Data

StudentId	CourseId	EnrollmentDate
101	10	2026-01-10
101	20	2026-01-10
102	10	2026-01-11
102	20	2026-01-11

StudentId alone is not unique:

101
101
102
102

CourseId alone is also not unique:

10
20

```
10
20
```

But:

```
101 + 10
101 + 20
102 + 10
102 + 20
```

are unique combinations.

Why Use a Composite Key?

It is useful when a row is naturally identified by **multiple values together**.

For example, in a student-course enrollment system:

```
StudentId = 101
CourseId  = 10
```

uniquely identifies the student's enrollment in that course.

This is common in **many-to-many relationships**.

```
Students
  ↓
StudentCourses
  ↑
Courses
```

Composite Primary Key

The most common form is a **Composite Primary Key**:

```
CREATE TABLE OrderItems (
  OrderId INT,
  ProductId INT,
  Quantity INT,

  PRIMARY KEY (OrderId, ProductId)
);
```

Here:

OrderId + ProductId
↓
Composite Primary Key

One order can contain many products:

OrderId	ProductId	Quantity
1001	501	2
1001	502	1
1002	501	5

OrderId is repeated and ProductId can be repeated, but the **combination** must be unique.

Composite Key vs Single-Column Key

Single-column key

PRIMARY KEY (EmployeeId)

One column identifies the row.

Composite key

PRIMARY KEY (StudentId, CourseId)

Multiple columns together identify the row.

Composite Key vs Candidate Key

A Composite Key describes a key containing **multiple columns**.

A Candidate Key describes a key that is **minimal and eligible to become the Primary Key**.

Therefore, a composite key can also be a Candidate Key.

Example:

StudentId + CourseId
↓

Composite Candidate Key

If selected as the Primary Key:

StudentId + CourseId
↓
Composite Primary Key

Adding a Composite Key Later

```
ALTER TABLE StudentCourses  
ADD CONSTRAINT PK_StudentCourses  
PRIMARY KEY (StudentId, CourseId);
```

Important Rule

This is allowed:

StudentId	CourseId
101	10
101	20
102	10

But this is **not** allowed:

StudentId	CourseId
101	10
101	10

because the combination (101, 10) already exists.

Interview definition

A Composite Key is a key consisting of two or more columns whose combined values uniquely identify a row.

Easy memory trick:

One column → Simple Key
Multiple columns → Composite Key

- Super Key

Super Key in SQL

A **Super Key** is **any column or combination of columns that can uniquely identify a row** in a table.

The key idea is:

If a set of columns can uniquely identify every row, it is a Super Key.

Example

Consider:

EmployeeId	Email	Name	Phone
101	a@x.com	Rahul	98765
102	b@x.com	Amit	87654
103	c@x.com	Priya	76543

Assume **EmployeeId**, **Email**, and **Phone** are all unique.

Then these are Super Keys:

```
EmployeeId
Email
Phone
EmployeeId + Name
EmployeeId + Email
Email + Phone
EmployeeId + Name + Email
EmployeeId + Email + Phone
```

Why?

Because each combination can uniquely identify an employee.

Super Key Can Have Extra Columns

This is the most important difference between a **Super Key** and a **Candidate Key**.

Suppose:

```
EmployeeId
```

is already unique.

Then:

EmployeeId + Name

is also unique.

Therefore:

EmployeeId
EmployeeId + Name
EmployeeId + Email
EmployeeId + Name + Email

are all Super Keys.

But the additional columns are unnecessary.

Super Key vs Candidate Key

A **Candidate Key** is a **minimal Super Key**.

Example:

EmployeeId

uniquely identifies the employee.

So:

EmployeeId → Candidate Key

But:

EmployeeId + Name

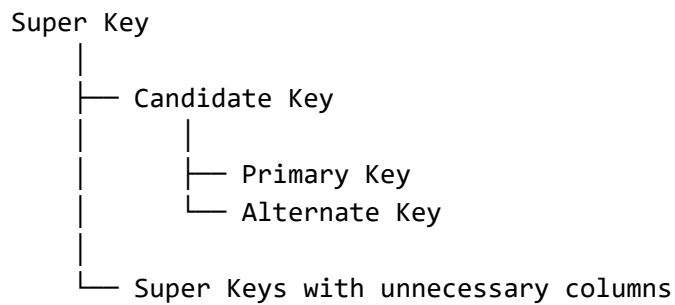
also uniquely identifies the employee, but **Name** isn't necessary.

Therefore:

EmployeeId + Name → Super Key

but **not** a Candidate Key.

Relationship



Example with Composite Super Key

Suppose:

StudentId	CourseId	Name
101	10	Rahul
101	20	Rahul
102	10	Amit

Assume **StudentId + CourseId** uniquely identifies the enrollment.

Then:

StudentId + CourseId

is a Super Key.

But:

StudentId + CourseId + Name

is also a Super Key because it still uniquely identifies the row.

The extra **Name** makes it non-minimal.

Therefore:

StudentId + CourseId
↓
Candidate Key

StudentId + CourseId + Name
↓
Super Key

Super Key vs Primary Key

Super Key	Primary Key
Any unique-identifying column(s)	Selected main identifier
Can have unnecessary columns	Should be minimal
Multiple Super Keys can exist	Only one Primary Key constraint
May contain multiple columns	Can also contain multiple columns

Simple Example

Suppose **EmployeeId** is unique.

Super Keys:

EmployeeId
EmployeeId + Name
EmployeeId + Email
EmployeeId + Name + Email

The **minimal** one is:

EmployeeId

So:

EmployeeId → Candidate Key

If we select it as the main key:

EmployeeId → Primary Key

Interview definition

A Super Key is a set of one or more attributes that uniquely identifies each row in a table. It may contain extra attributes that are not necessary for uniqueness.

Easy memory trick:

Super Key = Any unique identifier Candidate Key = Minimal unique identifier

- Alternate Key

Alternate Key in SQL

An **Alternate Key** is a **Candidate Key** that was not selected as the **Primary Key**.

In simple terms:

Candidate Keys = all possible unique identifiers Primary Key = the one selected Alternate Keys = the remaining Candidate Keys

Example

Consider an **Employees** table:

EmployeeId	Email	Phone	Name
101	rahul@example.com	9876543210	Rahul
102	amit@example.com	8765432109	Amit
103	priya@example.com	7654321098	Priya

Suppose these three columns are unique:

EmployeeId → Candidate Key
Email → Candidate Key
Phone → Candidate Key

We choose **EmployeeId** as the Primary Key:

EmployeeId → Primary Key

The remaining Candidate Keys become:

Email → Alternate Key
Phone → Alternate Key

SQL Example

```
CREATE TABLE Employees (  
    EmployeeId INT PRIMARY KEY,
```

```
Email VARCHAR(100) UNIQUE,  
Phone VARCHAR(15) UNIQUE,  
Name VARCHAR(100)  
);
```

Here:

EmployeeId → Primary Key

Email → Alternate Key

Phone → Alternate Key

The **UNIQUE** constraints ensure that **Email** and **Phone** cannot contain duplicate values.

Why Do We Need Alternate Keys?

Sometimes a table has multiple ways to uniquely identify a row.

For example, an employee can be identified by:

EmployeeId
Email
Phone

But we only need one **Primary Key**.

So:

EmployeeId → Primary Key
Email → Alternate Key
Phone → Alternate Key

The alternate keys provide other unique ways to identify the same record.

Candidate Key vs Alternate Key

This is important for interviews.

Suppose:

EmployeeId
Email
Phone

are all unique.

Initially:

```
Candidate Keys:
  EmployeeId
  Email
  Phone
```

After selecting `EmployeeId`:

```
Primary Key:
  EmployeeId

Alternate Keys:
  Email
  Phone
```

Therefore:

Every Alternate Key is a Candidate Key, but a Candidate Key becomes an Alternate Key only after another Candidate Key is selected as the Primary Key.

Alternate Key vs UNIQUE Constraint

They are closely related, but technically they are not exactly the same concept.

Alternate Key is a **relational/database-design concept**: a Candidate Key not chosen as the Primary Key.

UNIQUE constraint is a **SQL mechanism** used to enforce uniqueness.

For example:

```
Email VARCHAR(100) UNIQUE
```

can enforce the uniqueness required for an alternate key.

Example with Student

```
CREATE TABLE Students (
  StudentId INT PRIMARY KEY,
  Email VARCHAR(150) UNIQUE,
  AadhaarNumber VARCHAR(20) UNIQUE,
```

```
Name VARCHAR(100)
);
```

Conceptually:

Candidate Keys

```

|
├── StudentId
├── Email
└── AadhaarNumber

```

↓ Select StudentId

Primary Key

```

|
└── StudentId

```

Alternate Keys

```

|
├── Email
└── AadhaarNumber

```

Interview definition

An Alternate Key is a Candidate Key that is not selected as the Primary Key but can still uniquely identify a record.

Easy memory trick

Candidate Keys

↓

Choose one

↓

Primary Key

Remaining Candidate Keys

↓

Alternate Keys

11. Subqueries

- Single-row Subquery

Single-row Subquery in SQL

A **single-row subquery** is a subquery that returns **exactly one row/value** to the outer query.

It is commonly used with comparison operators:

= > < >= <= <>

Basic Syntax

```
SELECT column1, column2
FROM table
WHERE column1 = (
    SELECT column1
    FROM table
    WHERE condition
);
```

Example 1: Employees earning more than average salary

Suppose we have:

Id	Name	Department	Salary
1	Amit	IT	60000
2	Rahul	HR	50000
3	Priya	IT	80000
4	Neha	Finance	70000

Query:

```
SELECT Name, Salary
FROM Employees
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employees
);
```

Inner query

```
SELECT AVG(Salary)
FROM Employees;
```

Returns **one value**, for example:

65000

The outer query effectively becomes:

```
SELECT Name, Salary
FROM Employees
WHERE Salary > 65000;
```

Result:

Priya	80000
Neha	70000

Example 2: Find the highest-paid employee

```
SELECT Name, Salary
FROM Employees
WHERE Salary = (
    SELECT MAX(Salary)
    FROM Employees
);
```

The subquery:

```
SELECT MAX(Salary)
FROM Employees;
```

returns one value:

80000

So the outer query finds the employee whose salary is 80000.

Result:

Priya	80000
-------	-------

Example 3: Find employee earning the same as Amit

```
SELECT Name, Salary
FROM Employees
WHERE Salary = (
    SELECT Salary
    FROM Employees
```

```
WHERE Name = 'Amit'  
);
```

The inner query:

```
SELECT Salary  
FROM Employees  
WHERE Name = 'Amit';
```

returns:

```
60000
```

Then the outer query finds employees with salary 60000.

Important Rule

A single-row subquery should return **one value** when used with `=`, `>`, `<`, etc.

✗ This can cause an error if multiple rows are returned:

```
SELECT Name  
FROM Employees  
WHERE Salary = (  
    SELECT Salary  
    FROM Employees  
    WHERE Department = 'IT'  
);
```

If IT has two employees, the subquery returns:

```
60000  
80000
```

But `=` expects a single value.

For multiple values, use `IN`:

```
SELECT Name  
FROM Employees  
WHERE Salary IN (  
    SELECT Salary  
    FROM Employees  
    WHERE Department = 'IT'  
);
```

Single-row vs Multiple-row

```
Single-row:
SELECT AVG(Salary) ...
      ↓
    65000
      ↓
Salary > 65000

Multiple-row:
SELECT Salary ...
      ↓
60000
80000
      ↓
Salary IN (60000, 80000)
```

Key point: A **single-row subquery produces one value**, which the outer query can compare using operators like `=`, `>`, `<`, `>=`, `<=`, or `<>`.

- Multiple-row Subquery

Multiple-row Subquery in SQL

A **multiple-row subquery** is a subquery that returns **more than one row/value**.

Because multiple values can be returned, we commonly use:

- `IN`
- `ANY / SOME`
- `ALL`

Basic Syntax

```
SELECT column1
FROM table1
WHERE column1 IN (
    SELECT column1
    FROM table2
    WHERE condition
);
```

Example Table

Employees

Id	Name	Department	Salary
1	Amit	IT	60000
2	Rahul	HR	50000
3	Priya	IT	80000
4	Neha	Finance	70000
5	Raj	HR	55000

1. Using IN

Find employees whose salary matches any salary earned by employees in the **IT** department.

```
SELECT Name, Salary
FROM Employees
WHERE Salary IN (
    SELECT Salary
    FROM Employees
    WHERE Department = 'IT'
);
```

The subquery returns:

```
60000
80000
```

So SQL effectively checks:

```
WHERE Salary IN (60000, 80000)
```

Result:

```
Amit    60000
Priya    80000
```

2. Using ANY

ANY means the condition must be true for **at least one** value returned by the subquery.

Example:

```
SELECT Name, Salary
FROM Employees
WHERE Salary > ANY (
    SELECT Salary
    FROM Employees
    WHERE Department = 'HR'
);
```

HR salaries are:

```
50000
55000
```

So:

```
Salary > ANY (50000, 55000)
```

means salary must be greater than **at least one** of those values.

For example:

```
60000 > 50000 → TRUE
```

Therefore, employees earning more than the lower HR salary can qualify.

3. Using ALL

ALL means the condition must be true for **every value** returned by the subquery.

```
SELECT Name, Salary
FROM Employees
WHERE Salary > ALL (
    SELECT Salary
    FROM Employees
    WHERE Department = 'HR'
);
```

HR salaries:

```
50000
55000
```

The condition becomes:

```
Salary > 50000
AND
Salary > 55000
```

Therefore, the salary must be greater than **both** HR salaries.

Result:

```
Amit    60000
Priya    80000
Neha    70000
```

IN VS ANY VS ALL

Operator	Meaning
IN	Matches any exact value
ANY	Condition is true for at least one value
ALL	Condition is true for every value

Example:

```
-- Exact matching
WHERE Salary IN (50000, 55000)

-- Greater than at least one
WHERE Salary > ANY (50000, 55000)

-- Greater than every value
WHERE Salary > ALL (50000, 55000)
```

Multiple-row Subquery vs Single-row Subquery

Single-row

Returns **one value**:

```
SELECT MAX(Salary)
FROM Employees;
```

Can use:

```
WHERE Salary = (subquery)
```

Multiple-row

Returns **multiple values**:

```
SELECT Salary  
FROM Employees  
WHERE Department = 'IT';
```

Should commonly use:

```
WHERE Salary IN (subquery)
```

Easy way to remember

```
Single-row  
  ↓  
  80000  
  ↓  
Salary = 80000  
  
Multiple-row  
  ↓  
60000  
80000  
  ↓  
Salary IN (60000, 80000)
```

Interview definition: A **multiple-row subquery** is a subquery that returns multiple rows and is typically used with **IN**, **ANY**, **SOME**, or **ALL** operators.

- Correlated Subquery

Correlated Subquery in SQL

A **correlated subquery** is a subquery that **depends on the outer query**.

The inner query references a column from the outer query, so it is logically evaluated **for each row** processed by the outer query.

Basic Syntax


```
SELECT column1
FROM table1 t1
WHERE column1 > (
    SELECT AVG(column1)
    FROM table1 t2
    WHERE t2.some_column = t1.some_column
);
```

The important part is:

```
t2.some_column = t1.some_column
```

Here, **t1** belongs to the **outer query**, while **t2** belongs to the **inner query**.

Example

Consider this **Employees** table:

Id	Name	Department	Salary
1	Amit	IT	60000
2	Rahul	HR	50000
3	Priya	IT	80000
4	Neha	Finance	70000
5	Raj	HR	55000

Requirement

Find employees whose salary is **greater than the average salary of their own department**.

```
SELECT e.Name,
       e.Department,
       e.Salary
FROM Employees e
WHERE e.Salary > (
    SELECT AVG(e2.Salary)
    FROM Employees e2
    WHERE e2.Department = e.Department
);
```

How does it work?

For each employee, the inner query calculates the average salary for **that employee's department**.

For Amit

Amit:

```
Department = IT
Salary = 60000
```

Inner query calculates:

```
IT average = (60000 + 80000) / 2
            = 70000
```

Comparison:

```
60000 > 70000
FALSE
```

Amit is not returned.

For Priya

Priya:

```
Department = IT
Salary = 80000
```

IT average:

```
70000
```

Comparison:

```
80000 > 70000
TRUE
```

Priya is returned.

For Rahul

HR average:

```
(50000 + 55000) / 2  
= 52500
```

Comparison:

```
50000 > 52500  
FALSE
```

For Raj

```
55000 > 52500  
TRUE
```

So the result is:

Name	Department	Salary
Priya	IT	80000
Raj	HR	55000

Why is it called "Correlated"?

Look at this condition:

```
WHERE e2.Department = e.Department
```

`e2.Department` belongs to the **inner query**.

`e.Department` belongs to the **outer query**.

Therefore, the inner query needs information from the current outer-row.

That's the **correlation**.

Another Example

Find employees who have the **highest salary in their department**.

```
SELECT e.Name,  
       e.Department,  
       e.Salary
```

```
FROM Employees e
WHERE e.Salary = (
    SELECT MAX(e2.Salary)
    FROM Employees e2
    WHERE e2.Department = e.Department
);
```

Result:

Name	Department	Salary
Priya	IT	80000
Raj	HR	55000
Neha	Finance	70000

Notice again:

```
e2.Department = e.Department
```

The inner query depends on the outer employee's department.

Correlated vs Non-Correlated Subquery

Non-correlated

The inner query is independent:

```
SELECT Name, Salary
FROM Employees
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employees
);
```

The average salary is calculated independently of each employee.

Correlated

The inner query depends on the outer row:

```
SELECT e.Name, e.Salary
FROM Employees e
WHERE e.Salary > (
    SELECT AVG(e2.Salary)
    FROM Employees e2
```

```
WHERE e2.Department = e.Department  
);
```

The average is calculated based on the employee's department.

Important Interview Point

A correlated subquery is **not necessarily physically executed once per row** by the database engine. The optimizer may transform it into a more efficient execution plan.

So for interviews, say:

A correlated subquery is logically evaluated for each row of the outer query because the inner query references a value from the outer query.

Easy memory trick

Normal Subquery:

Outer Query → Inner Query

↓

Independent

Correlated Subquery:

Outer Query → Inner Query

↑ ↓
└──────────┘

depends on
outer row

Key point: If the subquery references a column from the outer query, it is a **correlated subquery**.

- Nested Subquery

Nested Subquery in SQL

A **nested subquery** is a subquery that contains **another subquery inside it**.

In simple terms:

Outer Query

↓

Subquery

↓

Subquery

You can have multiple levels of nested subqueries, although too many levels can make SQL difficult to read and maintain.

Basic Syntax

```
SELECT column
FROM table
WHERE column IN (
    SELECT column
    FROM table
    WHERE column IN (
        SELECT column
        FROM table
        WHERE condition
    )
);
```

Example

Suppose we have these tables:

Employees

Id	Name	DepartmentId	Salary
1	Amit	10	60000
2	Rahul	20	50000
3	Priya	10	80000
4	Neha	30	70000
5	Raj	20	55000

Departments

DepartmentId	DepartmentName
10	IT
20	HR
30	Finance

Requirement

Find employees who work in departments where the **average salary is greater than 60000**.

We can write:

```
SELECT Name, Salary
FROM Employees
WHERE DepartmentId IN (
    SELECT DepartmentId
    FROM Employees
    GROUP BY DepartmentId
    HAVING AVG(Salary) > (
        SELECT 60000
    )
);
```

Here, there is a subquery inside the **HAVING** condition.

However, this example is unnecessarily nested because **60000** is already a value.

A more meaningful nested example is:

Nested Subquery with Multiple Tables

Suppose we want:

Find employees who work in the department with the **highest average salary**.

```
SELECT Name, Salary, DepartmentId
FROM Employees
WHERE DepartmentId = (
    SELECT DepartmentId
    FROM Employees
    GROUP BY DepartmentId
    HAVING AVG(Salary) = (
        SELECT MAX(AvgSalary)
        FROM (
            SELECT DepartmentId,
                   AVG(Salary) AS AvgSalary
            FROM Employees
            GROUP BY DepartmentId
        ) AS DepartmentAverages
    )
);
```

There are multiple query levels here:

```
Outer Query
  ↓
Find employees
  ↓
```



Step-by-Step

Level 1: Calculate average salary for each department

```
SELECT DepartmentId,
        AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY DepartmentId;
```

Result:

DepartmentId	AvgSalary
10	70000
20	52500
30	70000

Level 2: Find the highest average salary

```
SELECT MAX(AvgSalary)
FROM (
    SELECT DepartmentId,
           AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentId
) AS DepartmentAverages;
```

Result:

70000

Level 3: Find departments having that average

```
SELECT DepartmentId
FROM Employees
GROUP BY DepartmentId
HAVING AVG(Salary) = 70000;
```

Result:

```
10
30
```

Level 4: Find employees in those departments

```
SELECT Name, Salary
FROM Employees
WHERE DepartmentId IN (10, 30);
```

Result:

Name	Salary
Amit	60000
Priya	80000
Neha	70000

Nested Subquery vs Correlated Subquery

These two concepts are different.

Nested Subquery

The main idea is **query inside another query**:

```
SELECT ...
FROM ...
WHERE x IN (
    SELECT ...
    WHERE y IN (
        SELECT ...
    )
);
```

Correlated Subquery

The inner query references the outer query:

```
SELECT e.Name
FROM Employees e
WHERE e.Salary > (
    SELECT AVG(e2.Salary)
    FROM Employees e2
    WHERE e2.DepartmentId = e.DepartmentId
);
```

The key difference:

Concept	Meaning
Nested Subquery	A subquery exists inside another subquery
Correlated Subquery	Inner query references the outer query
Single-row Subquery	Returns one value/row
Multiple-row Subquery	Returns multiple values/rows

Simple Example

Think of nested queries like layers:



For example:

```
SELECT *
FROM Employees
WHERE DepartmentId IN (

    -- Query 2
    SELECT DepartmentId
    FROM Employees
    WHERE Salary > (

        -- Query 3
        SELECT AVG(Salary)
        FROM Employees

    )

);
```

Here:

- **Query 1** → outer query
- **Query 2** → subquery
- **Query 3** → nested subquery

Interview Definition

A **nested subquery** is a SQL query embedded inside another subquery, creating multiple levels of query execution. It is used when the result of one query is required to build the condition or input for another query.

12. Set Operators

- **UNION**

UNION in SQL

UNION is a **set operator** used to combine the results of two or more **SELECT** queries into a **single result set**.

The most important feature of **UNION** is that it **removes duplicate rows**.

Syntax

```
SELECT column1, column2
FROM table1

UNION

SELECT column1, column2
FROM table2;
```

Example

Suppose we have two tables:

Employees

id	name	department
1	Amit	IT
2	Rahul	HR
3	Priya	IT

Managers

id	name	department
2	Rahul	HR
3	Priya	IT
4	Neha	Finance

Query:

```
SELECT name
FROM Employees

UNION

SELECT name
FROM Managers;
```

Result

name
Amit
Rahul
Priya
Neha

Rahul and **Priya** occur in both tables, but **UNION** returns them **only once**.

UNION removes duplicates

```
SELECT 10 AS number
UNION
SELECT 10 AS number;
```

Result:

```
10
```

Because duplicate values are removed.

Compare with:

```
SELECT 10 AS number
UNION ALL
```

```
SELECT 10 AS number;
```

Result:

```
10
10
```

Conditions for using UNION

The **SELECT** statements should have the **same number of columns**.

✗ Invalid:

```
SELECT id, name
FROM Employees

UNION

SELECT id
FROM Managers;
```

There are 2 columns in the first query but only 1 in the second.

Data types should also be compatible

Valid:

```
SELECT id, name
FROM Employees

UNION

SELECT id, name
FROM Managers;
```

ORDER BY with UNION

You normally place **ORDER BY** at the **end** of the combined query:

```
SELECT name
FROM Employees

UNION

SELECT name
FROM Managers
```

```
ORDER BY name;
```

Result:

name

Amit

Neha

Priya

Rahul

UNION vs UNION ALL

UNION	UNION ALL
Removes duplicates	Keeps duplicates
May require duplicate elimination/sorting	Usually faster
Use when unique results are required	Use when duplicates are meaningful

Easy way to remember

UNION = Combine + Remove duplicates

Interview definition:

UNION combines the result sets of two or more **SELECT** statements and removes duplicate rows from the final result.

- **UNION ALL**

UNION ALL in SQL

UNION ALL is a **set operator** used to combine the results of two or more **SELECT** queries.

Unlike **UNION**, **UNION ALL** **does not remove duplicate rows**.

Syntax

```
SELECT column1, column2
FROM table1

UNION ALL
```

```
SELECT column1, column2
FROM table2;
```

Example

Suppose we have:

Employees

id	name	department
1	Amit	IT
2	Rahul	HR
3	Priya	IT

Managers

id	name	department
2	Rahul	HR
3	Priya	IT
4	Neha	Finance

Query:

```
SELECT name
FROM Employees

UNION ALL

SELECT name
FROM Managers;
```

Result

name
Amit
Rahul
Priya
Rahul
Priya
Neha

Notice that **Rahul** and **Priya appear twice** because they exist in both result sets.

UNION ALL keeps duplicates

```
SELECT 10 AS number
UNION ALL
SELECT 10 AS number
UNION ALL
SELECT 20 AS number;
```

Result:

number
10
10
20

With UNION:

```
SELECT 10 AS number
UNION
SELECT 10 AS number
UNION
SELECT 20 AS number;
```

Result:

number
10
20

Requirements

Both queries should return the **same number of columns**.

Valid:

```
SELECT id, name
FROM Employees

UNION ALL
```



```
SELECT id, name
FROM Managers;
```

Invalid:

```
SELECT id, name
FROM Employees

UNION ALL

SELECT id
FROM Managers;
```

The first query returns **2 columns**, while the second returns **1 column**.

The corresponding columns should also have **compatible data types**.

UNION ALL with ORDER BY

Put **ORDER BY** at the end:

```
SELECT name
FROM Employees

UNION ALL

SELECT name
FROM Managers

ORDER BY name;
```

Why is UNION ALL usually faster?

UNION has to identify and eliminate duplicate rows.

UNION ALL simply combines the rows.

```
UNION
↓
Combine rows
↓
Find duplicates
↓
Remove duplicates
↓
Result
```



Therefore, when you **do not need duplicate removal**, `UNION ALL` is generally preferred.

Real-world example

Suppose you have sales from two different systems:

```
SELECT customer_id, amount
FROM OnlineSales

UNION ALL

SELECT customer_id, amount
FROM StoreSales;
```

This keeps **every sale**, including rows that happen to have the same customer and amount.

This is useful when combining data from multiple sources where **every record matters**.

UNION VS UNION ALL

Feature	UNION	UNION ALL
Combines result sets	✓	✓
Removes duplicates	✓	✗
Keeps duplicates	✗	✓
Usually faster	✗	✓
Use when	Unique rows needed	All rows needed

Easy way to remember

UNION = Combine + Remove duplicates

UNION ALL = Combine + Keep everything

Interview definition:

`UNION ALL` combines the result sets of two or more `SELECT` statements while preserving duplicate rows. It is generally faster than `UNION` because it does not perform duplicate elimination.

- `INTERSECT`

INTERSECT in SQL

INTERSECT is a **set operator** that returns only the rows that are **common to both SELECT query results**.

In simple terms:

```
INTERSECT = Common rows between A and B
```

Syntax

```
SELECT column1, column2
FROM table1

INTERSECT

SELECT column1, column2
FROM table2;
```

Example

Suppose we have:

Employees

id	name
1	Amit
2	Rahul
3	Priya
4	Neha

Managers

id	name
2	Rahul
3	Priya
5	Karan

Query:

```
SELECT name
FROM Employees
```

INTERSECT

```
SELECT name
FROM Managers;
```

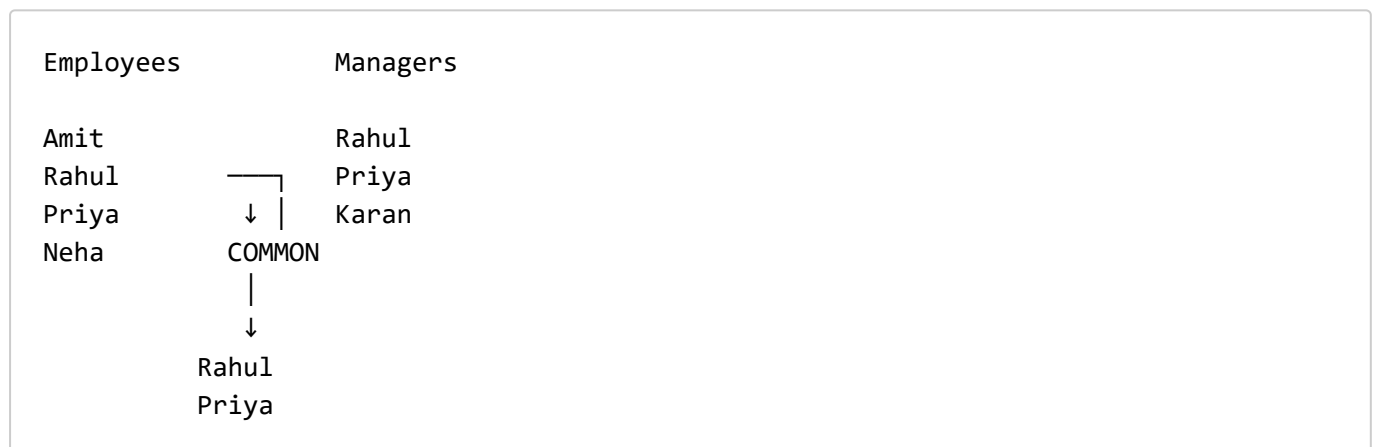
Result

name
Rahul
Priya

Only **Rahul** and **Priya** exist in **both** result sets.

Visual representation

Think of two sets:



Mathematically:

$$A \cap B$$

means the **intersection** of A and B.

INTERSECT removes duplicates

For example:

```
SELECT 10 AS number
INTERSECT
SELECT 10 AS number;
```

Result:

number

10

If the same row appears multiple times, the **INTERSECT** result contains it only once.

Multiple columns

INTERSECT compares the **complete row**.

```
SELECT id, name
FROM Employees

INTERSECT

SELECT id, name
FROM Managers;
```

Result:

id	name
2	Rahul
3	Priya

Both **id** and **name** must match.

For example:

```
Employees: 2, Rahul
Managers:  2, Raj
```

These are **not considered the same row**.

Requirements

Both queries should have:

1. Same number of columns

```
SELECT id, name
FROM Employees

INTERSECT
```

```
SELECT id, name
FROM Managers;
```

2. Compatible data types

For example:

```
INTEGER ↔ INTEGER
VARCHAR ↔ VARCHAR
DATE ↔ DATE
```

INTERSECT vs UNION

UNION	INTERSECT
Combines all rows	Returns common rows
Removes duplicates	Removes duplicates
A OR B	A AND B
$A \cup B$	$A \cap B$

Example:

```
A = {1, 2, 3}
B = {2, 3, 4}

UNION    → {1, 2, 3, 4}
INTERSECT → {2, 3}
```

Real-world example

Find customers who purchased **both online and in-store**:

```
SELECT customer_id
FROM OnlineOrders

INTERSECT

SELECT customer_id
FROM StoreOrders;
```

Result:

Customers who ordered online
AND
customers who ordered in-store

So **INTERSECT** is useful when you need to find **common records between two result sets**.

Interview definition

INTERSECT combines two **SELECT** result sets and returns only the distinct rows that exist in both result sets.

- **EXCEPT / MINUS**

EXCEPT / MINUS in SQL

EXCEPT and **MINUS** are **set operators** used to find rows that exist in the **first query but not in the second query**.

EXCEPT = First result - Second result

- **SQL Server / PostgreSQL:** **EXCEPT**
- **Oracle:** **MINUS**

1. EXCEPT

Syntax

```
SELECT column1, column2
FROM table1

EXCEPT

SELECT column1, column2
FROM table2;
```

Example

Suppose:

Employees

id	name
1	Amit
2	Rahul

id	name
3	Priya
4	Neha

Managers

id	name
2	Rahul
3	Priya
5	Karan

Query:

```
SELECT name
FROM Employees

EXCEPT

SELECT name
FROM Managers;
```

Result

name
Amit
Neha

Why?

```
Employees = {Amit, Rahul, Priya, Neha}

Managers  = {Rahul, Priya, Karan}

Employees EXCEPT Managers
      ↓
    {Amit, Neha}
```

2. MINUS

MINUS performs the equivalent operation in **Oracle**.


```
SELECT name
FROM Employees

MINUS

SELECT name
FROM Managers;
```

Result:

name
Amit
Neha

So:

```
EXCEPT ≈ MINUS
```

The exact supported operator depends on the database system.

Order Matters

This is extremely important.

Employees except Managers

```
SELECT name
FROM Employees

EXCEPT

SELECT name
FROM Managers;
```

Result:

Amit
Neha

But reversing the queries:

```
SELECT name
FROM Managers
```

```
EXCEPT
```

```
SELECT name  
FROM Employees;
```

Result:

```
Karan
```

So:

```
A EXCEPT B
```

is **not** the same as:

```
B EXCEPT A
```

EXCEPT removes duplicates

EXCEPT returns distinct rows.

For example:

```
SELECT 10 AS number  
UNION ALL  
SELECT 10 AS number  
UNION ALL  
SELECT 20 AS number  
  
EXCEPT  
  
SELECT 20 AS number;
```

Conceptually, the final result contains:

```
10
```

Duplicate 10 values don't remain as separate rows in the EXCEPT result.

Multiple Columns

EXCEPT can compare multiple columns.

```
SELECT id, name
FROM Employees

EXCEPT

SELECT id, name
FROM Managers;
```

The **complete row** is compared.

For example:

```
Employees:
1, Amit
2, Rahul

Managers:
2, Rahul
```

Result:

```
1, Amit
```

Requirements

Both queries should return:

Same number of columns

```
SELECT id, name
FROM Employees

EXCEPT

SELECT id, name
FROM Managers;
```

Compatible data types

For example:

```
INTEGER    ⇔ INTEGER
VARCHAR    ⇔ VARCHAR
```

DATE ↔ DATE

EXCEPT VS INTERSECT

These two are easy to confuse.

Suppose:

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

INTERSECT

Finds what is **common**:

A INTERSECT B
↓
{3, 4}

EXCEPT

Finds what is in **A but not B**:

A EXCEPT B
↓
{1, 2}

Operator	Meaning
INTERSECT	Common rows
EXCEPT	First query minus second
MINUS	Oracle's equivalent of EXCEPT

Real-World Example

Suppose you want to find customers who registered but **never placed an order**.

SELECT customer_id
FROM Customers

EXCEPT

```
SELECT customer_id
FROM Orders;
```

Result:

```
Customers who registered
      MINUS
Customers who placed orders
      ↓
Customers who never ordered
```

Quick Set Operator Comparison

A = {1, 2, 3}

B = {2, 3, 4}

UNION

→ {1, 2, 3, 4}

UNION ALL

→ {1, 2, 3, 2, 3, 4} -- conceptually, duplicates preserved

INTERSECT

→ {2, 3}

EXCEPT

→ {1}

Easy way to remember

UNION	→ A OR B
UNION ALL	→ A OR B + duplicates
INTERSECT	→ A AND B
EXCEPT	→ A BUT NOT B
MINUS	→ A BUT NOT B (Oracle)

Interview definition:

EXCEPT returns distinct rows from the first **SELECT** result that are not present in the second result. In Oracle, the equivalent set operator is **MINUS**.

13. Views

- Create View

CREATE VIEW in SQL

A **View** is a virtual table created from the result of a **SELECT** query. It allows you to save a query and reuse it as if it were a table.

Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example

Suppose we have an **Employees** table:

```
CREATE TABLE Employees (
    EmployeeId INT,
    Name VARCHAR(100),
    Department VARCHAR(50),
    Salary DECIMAL(10,2)
);
```

Create a view containing only IT employees:

```
CREATE VIEW ITEmployees AS
SELECT EmployeeId, Name, Salary
FROM Employees
WHERE Department = 'IT';
```

Now use the view:

```
SELECT *
FROM ITEmployees;
```

You can treat the view like a table:

```
SELECT Name, Salary
FROM ITEmployees
WHERE Salary > 50000;
```

View with Multiple Tables

Views can also contain joins:

```
CREATE VIEW EmployeeDetails AS
SELECT
    e.EmployeeId,
    e.Name,
    d.DepartmentName
FROM Employees e
INNER JOIN Departments d
    ON e.DepartmentId = d.DepartmentId;
```

Then:

```
SELECT *
FROM EmployeeDetails;
```

Why Use a View?

1. **Simplify complex queries**
2. **Improve security** by exposing only required columns
3. **Reuse queries**
4. **Hide database complexity**
5. **Provide a consistent interface** to application code

Important Point

A normal view generally **does not store a separate copy of the data**. When you query the view, the database uses its underlying query to retrieve the current data.

```
Employees Table
  ↓
SELECT Query
  ↓
ITEmployees View
  ↓
SELECT * FROM ITEmployees
```

- Update View

UPDATE VIEW in SQL

Updating a view can mean two different things:

1. **Change the definition/query of the view**
2. **Update data through the view**

1. Modify the View Definition

To change what a view returns, use **ALTER VIEW** or **CREATE OR ALTER VIEW**, depending on the database.

SQL Server

```
ALTER VIEW ITEmployees AS
SELECT EmployeeId, Name, Department, Salary
FROM Employees
WHERE Department = 'IT';
```

Here, the existing **ITEmployees** view is modified to include the **Department** column.

You can then query it:

```
SELECT *
FROM ITEmployees;
```

CREATE OR ALTER VIEW

SQL Server also supports:

```
CREATE OR ALTER VIEW ITEmployees AS
SELECT EmployeeId, Name, Department, Salary
FROM Employees
WHERE Department = 'IT';
```

This is useful because it creates the view if it doesn't exist, or modifies it if it already exists.

2. Update Data Through a View

Some views allow you to update the underlying table.

Example:

```
CREATE VIEW ITEmployees AS
SELECT EmployeeId, Name, Department, Salary
FROM Employees
WHERE Department = 'IT';
```

You can update an employee through the view:

```
UPDATE ITEmployees
SET Salary = 60000
WHERE EmployeeId = 101;
```

The corresponding row in the **Employees** table is updated.


```
UPDATE ITEmployees
    ↓
Underlying Employees table
    ↓
Salary changed to 60000
```

Important

Not every view is updatable. Simple views based on a single table are generally easier to update. Views involving **GROUP BY**, aggregate functions, **DISTINCT**, and certain joins may not be directly updatable, depending on the database system.

Quick Difference

Operation	Example	Meaning
Change view definition	ALTER VIEW	Changes the query that defines the view
Update data through view	UPDATE ViewName	Changes data in the underlying table

Interview point: **ALTER VIEW** changes the **view definition**, while **UPDATE ViewName** changes the **underlying data** when the view is updatable.

- Drop View

DROP VIEW in SQL

DROP VIEW is used to **permanently remove a view** from the database.

It removes the **view definition**, but it does **not delete the underlying table or its data**.

Syntax

```
DROP VIEW view_name;
```

Example

Suppose we have:

```
CREATE VIEW ITEmployees AS
SELECT EmployeeId, Name, Salary
FROM Employees
WHERE Department = 'IT';
```

To delete this view:

```
DROP VIEW Imployees;
```

After this, the following query will fail because the view no longer exists:

```
SELECT *  
FROM Imployees;
```

However, the original table is still available:

```
SELECT *  
FROM Employees;
```

DROP VIEW IF EXISTS

Some databases support:

```
DROP VIEW IF EXISTS Imployees;
```

This safely drops the view only if it exists.

Important Difference

```
DROP VIEW Imployees;  
    ↓  
Deletes the VIEW  
    ↓  
Employees table remains  
    ↓  
Employee data remains
```

Interview Point

DROP VIEW removes only the view, not the underlying tables or their data.

14. Indexes

- Create Index

CREATE INDEX in SQL

CREATE INDEX is used to create an **index on one or more columns** of a table to improve query performance.

Think of an index like the **index of a book**: instead of searching every page, the database can quickly locate the required data.

1. Basic Syntax

```
CREATE INDEX index_name
ON table_name(column_name);
```

Example

Suppose we have:

```
CREATE TABLE Employees
(
    EmployeeId INT,
    Name VARCHAR(100),
    Department VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

Create an index on **Name**:

```
CREATE INDEX IX_Employees_Name
ON Employees(Name);
```

Now a query like:

```
SELECT *
FROM Employees
WHERE Name = 'Rahul';
```

can potentially use the index to find matching rows faster.

2. Index on Multiple Columns

You can create a **composite index**:

```
CREATE INDEX IX_Employees_Department_Name
ON Employees(Department, Name);
```

Useful for queries such as:

```
SELECT *
FROM Employees
```

```
WHERE Department = 'IT'  
AND Name = 'Rahul';
```

Column order matters

This:

```
CREATE INDEX IX_Employees_Department_Name  
ON Employees(Department, Name);
```

is most useful when queries filter by **Department** first, such as:

```
WHERE Department = 'IT'
```

or:

```
WHERE Department = 'IT'  
AND Name = 'Rahul'
```

3. Create Index for Sorting

Indexes can also help with queries involving **ORDER BY**.

```
CREATE INDEX IX_Employees_Salary  
ON Employees(Salary);
```

Query:

```
SELECT *  
FROM Employees  
ORDER BY Salary;
```

The database may use the index to make the operation more efficient.

4. Create Unique Index

If you want the indexed values to be unique:

```
CREATE UNIQUE INDEX IX_Employees_Email  
ON Employees(Email);
```

For example:

```
rahul@gmail.com  
amit@gmail.com  
john@gmail.com
```

Allowed.

But:

```
rahul@gmail.com  
rahul@gmail.com
```

is not allowed.

5. Create Non-Clustered Index — SQL Server

In SQL Server, you can explicitly specify:

```
CREATE NONCLUSTERED INDEX IX_Employees_Name  
ON Employees(Name);
```

You can also create a clustered index:

```
CREATE CLUSTERED INDEX IX_Employees_EmployeeId  
ON Employees(EmployeeId);
```

6. Index with Included Columns — SQL Server

SQL Server supports **INCLUDE**, which can reduce the need to look up additional columns.

```
CREATE INDEX IX_Employees_Department  
ON Employees(DepartmentId)  
INCLUDE (Name, Salary);
```

Query:

```
SELECT Name, Salary  
FROM Employees  
WHERE DepartmentId = 10;
```

The index contains **DepartmentId** plus the included **Name** and **Salary** values.

7. Delete an Index

SQL Server

```
DROP INDEX IX_Employees_Name  
ON Employees;
```

MySQL

```
DROP INDEX IX_Employees_Name  
ON Employees;
```

8. When Should You Create an Index?

Good candidates are columns frequently used in:

```
WHERE  
JOIN  
ORDER BY  
GROUP BY
```

Example:

```
SELECT *  
FROM Orders  
WHERE CustomerId = 100;
```

Create:

```
CREATE INDEX IX_Orders_CustomerId  
ON Orders(CustomerId);
```

9. Important Disadvantage

Indexes improve **read performance**, but they aren't free.

When you execute:

```
INSERT  
UPDATE  
DELETE
```

the database may also need to update the indexes.

Therefore:

Don't create indexes on every column.

Use indexes on columns that are frequently searched, joined, sorted, or otherwise important to query performance.

Interview Definition

An index is a database structure created using `CREATE INDEX` to speed up data retrieval from a table. It can improve read performance but requires additional storage and can add overhead to `INSERT`, `UPDATE`, and `DELETE` operations.

- Unique Index

UNIQUE INDEX in SQL

A **Unique Index** is an index that ensures that **duplicate values are not allowed** in the indexed column or combination of columns.

It provides two benefits:

1. **Faster data retrieval**
2. **Prevents duplicate values**

1. Basic Syntax

```
CREATE UNIQUE INDEX index_name
ON table_name(column_name);
```

Example

Suppose we have:

```
CREATE TABLE Employees
(
    EmployeeId INT,
    Name VARCHAR(100),
    Email VARCHAR(100)
);
```

Create a unique index:

```
CREATE UNIQUE INDEX IX_Employees_Email
ON Employees(Email);
```

Now:

```
INSERT INTO Employees  
VALUES (1, 'Rahul', 'rahul@gmail.com');
```

is allowed.

But:

```
INSERT INTO Employees  
VALUES (2, 'Amit', 'rahul@gmail.com');
```

will fail because `rahul@gmail.com` already exists.

2. Unique Index on Multiple Columns

You can create a unique index on multiple columns.

```
CREATE UNIQUE INDEX IX_Employee_Department  
ON Employees(EmployeeId, DepartmentId);
```

Here, the **combination** of `EmployeeId` and `DepartmentId` must be unique.

For example:

EmployeeId	DepartmentId	Allowed?
101	10	✓
101	20	✓
102	10	✓
101	10	✗ Duplicate

The individual values don't necessarily have to be unique; the **combination** must be unique.

3. Unique Index vs Normal Index

Normal Index

```
CREATE INDEX IX_Employees_Email  
ON Employees(Email);
```

Duplicates are allowed:


```
rahul@gmail.com  
amit@gmail.com  
rahul@gmail.com
```

Unique Index

```
CREATE UNIQUE INDEX IX_Employees_Email  
ON Employees(Email);
```

Duplicates are not allowed:

```
rahul@gmail.com  
amit@gmail.com  
rahul@gmail.com ❌
```

4. Unique Constraint vs Unique Index

These are related but conceptually different.

Unique Constraint

```
CREATE TABLE Employees  
(  
    EmployeeId INT,  
    Email VARCHAR(100) UNIQUE  
);
```

The purpose is primarily **data integrity**: email values must be unique.

Unique Index

```
CREATE UNIQUE INDEX IX_Employees_Email  
ON Employees(Email);
```

The purpose is to create an **index structure that also enforces uniqueness**.

In many database systems, a **UNIQUE** constraint is implemented using a unique index internally.

5. Unique Index with SQL Server

In SQL Server:

```
CREATE UNIQUE NONCLUSTERED INDEX IX_Employees_Email  
ON Employees(Email);
```

You can also create a unique clustered index:

```
CREATE UNIQUE CLUSTERED INDEX IX_Employees_EmployeeId  
ON Employees(EmployeeId);
```

A table can have **only one clustered index**, but can have multiple non-clustered indexes.

6. Why Use a Unique Index?

Suppose an application frequently searches users by email:

```
SELECT *  
FROM Users  
WHERE Email = 'user@gmail.com';
```

Create:

```
CREATE UNIQUE INDEX IX_Users_Email  
ON Users(Email);
```

Now the database knows that:

- **Email** values are unique.
 - Searching by email can be optimized using the index.
 - Duplicate emails cannot be inserted.
-

7. Remove a Unique Index

In SQL Server:

```
DROP INDEX IX_Employees_Email  
ON Employees;
```

After dropping it, the uniqueness protection provided specifically by that index is removed.

Simple Interview Definition

A Unique Index is an index that improves data retrieval while also ensuring that duplicate values are not stored in the indexed column or combination of columns.

Quick Difference

```
Normal Index
  ↓
Faster search
  ↓
Duplicates allowed

Unique Index
  ↓
Faster search
  +
No duplicate values
```

- Clustered Index

Clustered Index in SQL

A **Clustered Index** determines how the rows of a table are **organized according to the index key**.

A simple way to think about it:

Clustered Index = The table data itself is organized around the index key.

1. Basic Syntax — SQL Server

```
CREATE CLUSTERED INDEX index_name
ON table_name(column_name);
```

Example

```
CREATE CLUSTERED INDEX IX_Employees_EmployeeId
ON Employees(EmployeeId);
```

The table's data is organized based on **EmployeeId**.

Conceptually:

```
EmployeeId
-----
101
102
103
104
105
```

2. Example

Suppose the table contains:

EmployeeId	Name	Salary
105	Rahul	50000
101	Amit	60000
103	John	45000
102	Priya	70000
104	Neha	55000

After creating:

```
CREATE CLUSTERED INDEX IX_Employees_EmployeeId
ON Employees(EmployeeId);
```

the database organizes the table's data based on **EmployeeId**:

```
101 → Amit
102 → Priya
103 → John
104 → Neha
105 → Rahul
```

The exact physical storage behavior is engine-specific, but this is a useful conceptual model.

3. Why Use a Clustered Index?

Consider:

```
SELECT *
FROM Employees
WHERE EmployeeId = 103;
```

If **EmployeeId** has an appropriate clustered index, the database can efficiently locate the row.

Clustered indexes are particularly useful for:

- Primary key lookups
- Range queries
- Sorting

- Sequential access

For example:

```
SELECT *  
FROM Employees  
WHERE EmployeeId BETWEEN 100 AND 200;
```

A clustered index on `EmployeeId` can be very useful for this type of range query.

4. Only One Clustered Index

A table can generally have **only one clustered index**.

Why?

Because the table's data can have only one clustered organization at a time.

For example:

```
CREATE CLUSTERED INDEX IX_Employees_EmployeeId  
ON Employees(EmployeeId);
```

You cannot then create another clustered index on `Salary` on the same table:

```
CREATE CLUSTERED INDEX IX_Employees_Salary  
ON Employees(Salary);
```

This will fail in SQL Server because the table already has a clustered index.

However, you can have multiple non-clustered indexes:

```
CREATE NONCLUSTERED INDEX IX_Employees_Name  
ON Employees(Name);  
  
CREATE NONCLUSTERED INDEX IX_Employees_Salary  
ON Employees(Salary);
```

5. Clustered Index and Primary Key

In **SQL Server**, when you create a primary key, it is clustered by default if there isn't already a clustered index.

For example:

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

SQL Server normally creates a **clustered index** for the primary key.

You can explicitly specify it:

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY CLUSTERED,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

But remember:

Primary Key ≠ Clustered Index

A primary key is a **constraint**.

A clustered index is an **index structure**.

They are often used together, but they are not the same concept.

6. Clustered vs Non-Clustered Index

Feature	Clustered	Non-Clustered
Number per table	Usually 1	Multiple
Organizes table data	Yes	No
Separate index structure	Different depending on storage engine	Yes
Good for range queries	Yes	Yes
Can have multiple	✗ Usually one	✓ Yes
Example	EmployeeId	Name, Email

Simple analogy

Imagine a library.

Clustered

Books themselves are arranged by book number:

```
Book 101
Book 102
Book 103
Book 104
Book 105
```

Non-Clustered

A separate index tells you where books are:

```
"Database" → Shelf 10
"Java"      → Shelf 15
"SQL"       → Shelf 20
```

7. Clustered Index with Range Query

Suppose:

```
CREATE CLUSTERED INDEX IX_Orders_OrderId
ON Orders(OrderId);
```

Then:

```
SELECT *
FROM Orders
WHERE OrderId BETWEEN 1000 AND 2000;
```

The ordered nature of the index can make locating and scanning that range efficient.

8. Important Disadvantage

Clustered indexes are not automatically better for every query.

If the clustered key is frequently changed, the database may have to reorganize index pages/data structures.

For example:

```
UPDATE Employees
SET EmployeeId = 500
WHERE EmployeeId = 100;
```

Changing an indexed key can be more expensive than updating a non-indexed column.

Therefore, clustered keys are commonly chosen to be:

- Stable
- Frequently searched
- Frequently used for ranges
- Relatively narrow

9. When Should You Use a Clustered Index?

Good candidates include:

```
Primary Key
  ↓
Frequently searched ID
  ↓
Frequently used range column
  ↓
Frequently sorted/sequential data
```

For example:

```
CREATE CLUSTERED INDEX IX_Orders_OrderId
ON Orders(OrderId);
```

is a common design.

Interview Definition

A clustered index is an index that determines the organization of table data based on the indexed key. A table can generally have only one clustered index, and it is especially useful for primary-key lookups, range queries, and ordered access.

Remember

```
Clustered Index
  ↓
Organizes table data
  ↓
Usually ONE per table
  ↓
Good for range searches
  ↓
Often used with Primary Key
```


- Non-Clustered Index

Non-Clustered Index in SQL

A **Non-Clustered Index** is a separate index structure that stores the indexed column values along with a reference to the corresponding table row.

Think of it like the **index at the back of a book**:

Keyword	→ Page
Database	→ 25
Index	→ 48
SQL	→ 72

The index helps the database find the data without scanning the entire table.

1. Basic Syntax

In SQL Server:

```
CREATE NONCLUSTERED INDEX index_name
ON table_name(column_name);
```

Example

```
CREATE NONCLUSTERED INDEX IX_Employees_Name
ON Employees(Name);
```

Now a query such as:

```
SELECT *
FROM Employees
WHERE Name = 'Rahu1';
```

may use the non-clustered index to find the matching employee efficiently.

2. Example

Suppose the table contains:

EmployeeId	Name	Department	Salary
101	Amit	IT	60000

EmployeeId	Name	Department	Salary
102	Rahul	HR	50000
103	John	IT	70000
104	Priya	Finance	65000

Create:

```
CREATE NONCLUSTERED INDEX IX_Employees_Name
ON Employees(Name);
```

Conceptually, the index might look like:

```
Name
↓
Amit   → Employee row
John   → Employee row
Priya  → Employee row
Rahul  → Employee row
```

The database can search the index for **Rahul** and then locate the corresponding table row.

3. Multiple Non-Clustered Indexes

Unlike a clustered index, a table can have **multiple non-clustered indexes**.

For example:

```
CREATE NONCLUSTERED INDEX IX_Employees_Name
ON Employees(Name);

CREATE NONCLUSTERED INDEX IX_Employees_Email
ON Employees(Email);

CREATE NONCLUSTERED INDEX IX_Employees_Department
ON Employees(Department);
```

Now the table has three separate indexes.

This is useful when different columns are frequently used in queries.

4. Composite Non-Clustered Index

You can create an index using multiple columns.

```
CREATE NONCLUSTERED INDEX IX_Employees_Department_Name
ON Employees(Department, Name);
```

This can help with:

```
SELECT *
FROM Employees
WHERE Department = 'IT'
AND Name = 'Rahul';
```

Important

Column order matters.

This:

```
(Department, Name)
```

is different from:

```
(Name, Department)
```

The best order depends on your query patterns and data distribution.

5. Non-Clustered Index with INCLUDE

SQL Server allows additional columns to be stored in the index using **INCLUDE**.

```
CREATE NONCLUSTERED INDEX IX_Employees_Department
ON Employees(Department)
INCLUDE (Name, Salary);
```

For:

```
SELECT Name, Salary
FROM Employees
WHERE Department = 'IT';
```

the index contains:

Department → Name → Salary

This can allow SQL Server to satisfy the query directly from the index without needing an additional lookup to the base table.

6. Unique Non-Clustered Index

A non-clustered index can also enforce uniqueness.

```
CREATE UNIQUE NONCLUSTERED INDEX IX_Employees_Email
ON Employees(Email);
```

Now duplicate emails aren't allowed:

rahul@gmail.com ✓

amit@gmail.com ✓

rahul@gmail.com ✗

7. Non-Clustered Index vs Clustered Index

Feature	Clustered Index	Non-Clustered Index
Number per table	Usually 1	Multiple
Table data organization	Determines it	Separate structure
Separate from table data	No, conceptually	Yes
Can have multiple	✗	✓
Useful for searching	✓	✓
Useful for range queries	✓	✓
Extra storage	Depends on storage engine	Usually required
Example	EmployeeId	Name, Email

8. How a Non-Clustered Index Works

Suppose:

```
CREATE NONCLUSTERED INDEX IX_Employees_Email  
ON Employees(Email);
```

Query:

```
SELECT *  
FROM Employees  
WHERE Email = 'rahul@gmail.com';
```

Conceptually:

```
Query  
↓  
Non-Clustered Index  
↓  
Find "rahul@gmail.com"  
↓  
Row reference  
↓  
Employee table  
↓  
Return row
```

If the index contains all columns required by the query, the database may be able to avoid the additional table lookup.

9. Disadvantages

Indexes are useful, but creating too many can hurt performance.

Additional storage

Indexes require additional disk/memory resources.

INSERT overhead

```
INSERT INTO Employees ...
```

The database may need to update every relevant index.

UPDATE overhead

If an indexed column changes:

```
UPDATE Employees
SET Name = 'Rahul Kumar'
WHERE EmployeeId = 101;
```

the corresponding index must also be updated.

DELETE overhead

When a row is deleted, its entries must be removed from indexes.

Therefore:

Don't create an index on every column. Create indexes based on actual query patterns and performance requirements.

10. When to Use a Non-Clustered Index

Good candidates are columns frequently used in:

```
WHERE
JOIN
ORDER BY
GROUP BY
```

For example:

```
SELECT *
FROM Orders
WHERE CustomerId = 1001;
```

Create:

```
CREATE NONCLUSTERED INDEX IX_Orders_CustomerId
ON Orders(CustomerId);
```

Interview Definition

A non-clustered index is a separate index structure that stores indexed column values and references to the corresponding table rows. A table can have multiple non-clustered indexes, and they are primarily used to improve data retrieval performance.

Easy way to remember

Clustered Index

↓

Organizes table data

↓

Usually ONE

Non-Clustered Index

↓

Separate lookup structure

↓

Multiple allowed

↓

Points to table data

15. SQL Functions

- String Functions

SQL String Functions

String functions are used to manipulate, search, format, and extract text values.

Assume this table:

```
CREATE TABLE Employees (  
  Id INT,  
  FirstName VARCHAR(50),  
  LastName VARCHAR(50),  
  Email VARCHAR(100)  
);
```

Example data:

```
1 | John | Smith | john.smith@gmail.com  
2 | Alice | Brown | alice.brown@gmail.com
```

1. LEN()

Returns the number of characters in a string.

```
SELECT LEN('Hello');
```

Result:

5

With a column:

```
SELECT FirstName, LEN(FirstName) AS NameLength
FROM Employees;
```

In SQL Server, `LEN()` does not count trailing spaces.

2. UPPER()

Converts text to uppercase.

```
SELECT UPPER('hello');
```

Result:

HELLO

Example:

```
SELECT UPPER(FirstName) AS FirstName
FROM Employees;
```

3. LOWER()

Converts text to lowercase.

```
SELECT LOWER('HELLO');
```

Result:

hello

Example:

```
SELECT LOWER(Email) AS Email
FROM Employees;
```


4. LEFT()

Returns a specified number of characters from the **left side**.

```
SELECT LEFT('Database', 4);
```

Result:

```
Data
```

Example:

```
SELECT LEFT(FirstName, 2)
FROM Employees;
```

5. RIGHT()

Returns characters from the **right side**.

```
SELECT RIGHT('Database', 4);
```

Result:

```
base
```

Example:

```
SELECT RIGHT(Email, 10)
FROM Employees;
```

6. SUBSTRING()

Extracts a portion of a string.

SQL Server syntax:

```
SUBSTRING(string, start_position, length)
```

Example:

```
SELECT SUBSTRING('Database', 1, 4);
```

Result:

Data

Another example:

```
SELECT SUBSTRING('ABCDEFGF', 3, 3);
```

Result:

CDE

7. LTRIM()

Removes spaces from the **left side**.

```
SELECT LTRIM(' Hello');
```

Result:

Hello

Useful when cleaning user input.

8. RTRIM()

Removes spaces from the **right side**.

```
SELECT RTRIM('Hello ');
```

Result:

Hello

9. TRIM()

Removes spaces from both sides.

```
SELECT TRIM('  Hello ');
```

Result:

```
Hello
```

This is generally preferable when you want to clean both leading and trailing spaces.

10. REPLACE()

Replaces one piece of text with another.

Syntax:

```
REPLACE(string, old_value, new_value)
```

Example:

```
SELECT REPLACE('Hello World', 'World', 'SQL');
```

Result:

```
Hello SQL
```

Example with a column:

```
SELECT REPLACE(Email, 'gmail.com', 'outlook.com')  
FROM Employees;
```

11. CONCAT()

Combines multiple strings.

```
SELECT CONCAT('John', ' ', 'Smith');
```

Result:

```
John Smith
```

Very useful for combining columns:

```
SELECT CONCAT(FirstName, ' ', LastName) AS FullName
FROM Employees;
```

12. CONCAT_WS()

Concatenates values using a separator.

WS means **With Separator**.

```
SELECT CONCAT_WS('-', '2026', '08', '14');
```

Result:

```
2026-08-14
```

Example:

```
SELECT CONCAT_WS(' ', FirstName, LastName) AS FullName
FROM Employees;
```

13. CHARINDEX()

Finds the position of a substring inside another string.

```
SELECT CHARINDEX('a', 'Database');
```

Result:

```
2
```

Because:

```
D a t a b a s e
1 2 3 4 5 6 7 8
```

Example:

```
SELECT CHARINDEX('@', Email)
```

```
FROM Employees;
```

This can be used to find the position of @ in an email address.

14. PATINDEX()

Finds the position of a pattern.

```
SELECT PATINDEX('%SQL%', 'I am learning SQL');
```

Result:

```
15
```

PATINDEX() supports wildcard patterns.

Example:

```
SELECT PATINDEX('%@%', Email)
FROM Employees;
```

15. REVERSE()

Reverses a string.

```
SELECT REVERSE('Hello');
```

Result:

```
olleH
```

Example:

```
SELECT REVERSE(FirstName)
FROM Employees;
```

16. SPACE()

Generates a specified number of spaces.

```
SELECT 'Hello' + SPACE(3) + 'World';
```

Result:

```
Hello   World
```

17. REPLICATE()

Repeats a string a specified number of times.

```
SELECT REPLICATE('*', 5);
```

Result:

```
*****
```

Example:

```
SELECT REPLICATE('-', 20);
```

Result:

```
-----
```

18. STUFF()

Deletes part of a string and inserts another string.

Syntax:

```
STUFF(string, start, length, replacement)
```

Example:

```
SELECT STUFF('Hello World', 7, 5, 'SQL');
```

Result:

```
Hello SQL
```

Here:

- Start = 7
- Length = 5
- Replace `World`
- With `SQL`

Practical Example

Suppose:

```
FirstName = ' john '  
LastName  = ' smith '
```

We can clean and combine them:

```
SELECT  
    CONCAT(  
        UPPER(LEFT(TRIM(FirstName), 1)),  
        LOWER(SUBSTRING(TRIM(FirstName), 2, LEN(TRIM(FirstName)))),  
        ' ',  
        UPPER(LEFT(TRIM(LastName), 1)),  
        LOWER(SUBSTRING(TRIM(LastName), 2, LEN(TRIM(LastName))))  
    ) AS FullName  
FROM Employees;
```

Result:

```
John Smith
```

Most Important String Functions for Interviews

Focus on these first:

```
LEN()  
UPPER()  
LOWER()  
LEFT()  
RIGHT()  
SUBSTRING()
```

```
TRIM()  
LTRIM()  
RTRIM()  
REPLACE()  
CONCAT()  
CHARINDEX()  
PATINDEX()  
REVERSE()  
STUFF()
```

Quick memory trick

```
LEN      → Length  
UPPER    → Uppercase  
LOWER    → Lowercase  
LEFT     → Characters from left  
RIGHT    → Characters from right  
SUBSTRING → Extract characters  
TRIM     → Remove spaces  
REPLACE  → Replace text  
CONCAT   → Join text  
CHARINDEX → Find text position  
PATINDEX → Find pattern  
REVERSE  → Reverse text  
STUFF    → Delete + insert text
```

- Numeric Functions

SQL Numeric Functions

Numeric functions are used to perform mathematical calculations and manipulate numeric values such as **INT**, **DECIMAL**, **FLOAT**, etc.

Assume:

```
CREATE TABLE Products (  
    Id INT,  
    ProductName VARCHAR(100),  
    Price DECIMAL(10,2),  
    Quantity INT  
);
```

1. **ABS()**

Returns the **absolute value** of a number.

```
SELECT ABS(-100);
```


Result:

```
100
```

Example:

```
SELECT ABS(Price - 1000) AS PriceDifference
FROM Products;
```

2. ROUND()

Rounds a number to a specified number of decimal places.

```
SELECT ROUND(123.4567, 2);
```

Result:

```
123.46
```

Another example:

```
SELECT ROUND(123.4567, 0);
```

Result:

```
123
```

3. CEILING()

Rounds a number **upward** to the next integer.

```
SELECT CEILING(10.2);
```

Result:

```
11
```

```
SELECT CEILING(10.9);
```

Result:

11

4. FLOOR()

Rounds a number **downward** to the previous integer.

```
SELECT FLOOR(10.9);
```

Result:

10

```
SELECT FLOOR(10.2);
```

Result:

10

CEILING() vs FLOOR()

```
10.2 → CEILING = 11  
10.2 → FLOOR   = 10
```

5. POWER()

Returns a number raised to a power.

```
SELECT POWER(2, 3);
```

Result:

8

Because:

```
2 × 2 × 2 = 8
```

Another example:

```
SELECT POWER(10, 2);
```

Result:

```
100
```

6. SQRT()

Returns the square root.

```
SELECT SQRT(25);
```

Result:

```
5
```

Another example:

```
SELECT SQRT(144);
```

Result:

```
12
```

7. SIGN()

Returns the sign of a number.

```
SELECT SIGN(100);
```

Result:

```
1
```

```
SELECT SIGN(-100);
```

Result:

```
-1
```

```
SELECT SIGN(0);
```

Result:

```
0
```

Summary

```
Positive → 1  
Negative → -1  
Zero     → 0
```

8. RAND()

Generates a pseudo-random number between 0 and 1.

```
SELECT RAND();
```

Example output:

```
0.7345621
```

Generate a random integer from 1 to 100:

```
SELECT FLOOR(RAND() * 100) + 1;
```

9. MOD() / %

Returns the **remainder** after division.

In SQL Server, use %.

```
SELECT 10 % 3;
```

Result:

1

Because:

$10 \div 3 = 3 \text{ remainder } 1$

Find even numbers

```
SELECT *  
FROM Products  
WHERE Id % 2 = 0;
```

Find odd numbers

```
SELECT *  
FROM Products  
WHERE Id % 2 <> 0;
```

Oracle/MySQL/PostgreSQL commonly use `MOD()` as well, e.g. `MOD(10, 3)`.

10. `EXP()`

Returns `e` raised to a specified power.

```
SELECT EXP(1);
```

Result is approximately:

2.7182818

11. `LOG()`

Returns the natural logarithm in SQL Server.

```
SELECT LOG(10);
```

You can also specify a base:

```
SELECT LOG(100, 10);
```

Result:

2

12. LOG10()

Returns the base-10 logarithm.

```
SELECT LOG10(100);
```

Result:

2

Because:

$$10^2 = 100$$

Practical Example

Suppose:

Price = 1250.756
Quantity = 3

Calculate total price:

```
SELECT
    Price,
    Quantity,
    ROUND(Price * Quantity, 2) AS TotalPrice
FROM Products;
```

Result:

Price	Quantity	TotalPrice
1250.756	3	3752.27

Numeric Functions with **SELECT**

You can combine multiple numeric functions:

```
SELECT
    Price,
    ABS(Price - 1000) AS Difference,
    ROUND(Price, 2) AS RoundedPrice,
    CEILING(Price) AS CeilingPrice,
    FLOOR(Price) AS FloorPrice
FROM Products;
```

Most Important Numeric Functions for Interviews

Focus on these first:

Function	Purpose
ABS()	Absolute value
ROUND()	Round number
CEILING()	Round upward
FLOOR()	Round downward
POWER()	Power calculation
SQRT()	Square root
SIGN()	Positive/negative/zero
% / MOD()	Remainder
RAND()	Random number
LOG()	Logarithm
LOG10()	Base-10 logarithm
EXP()	Exponential

Quick memory trick

```
ABS      → Absolute value
ROUND    → Round
CEILING  → Up
FLOOR    → Down
POWER    → x^n
SQRT     → √x
```

SIGN	→ -1 / 0 / 1
%	→ Remainder
RAND	→ Random number
LOG	→ Logarithm

- Date Functions

SQL Date Functions

Date functions are used to work with dates and times: getting the current date, adding/subtracting dates, finding differences, and extracting parts such as year, month, and day.

Examples below mainly use **SQL Server** syntax.

1. GETDATE()

Returns the current date and time.

```
SELECT GETDATE();
```

Example result:

```
2026-08-14 23:45:30.123
```

2. SYSDATETIME()

Returns the current date and time with higher precision.

```
SELECT SYSDATETIME();
```

Example:

```
2026-08-14 23:45:30.1234567
```

GETDATE() vs SYSDATETIME()

GETDATE()	→ DATETIME
SYSDATETIME()	→ DATETIME2 with higher precision

3. DATEADD()

Adds or subtracts a specified time interval from a date.

Syntax:

```
DATEADD(datepart, number, date)
```

Add days

```
SELECT DATEADD(DAY, 10, '2026-08-14');
```

Result:

```
2026-08-24
```

Add months

```
SELECT DATEADD(MONTH, 2, '2026-08-14');
```

Result:

```
2026-10-14
```

Subtract days

Use a negative number:

```
SELECT DATEADD(DAY, -10, '2026-08-14');
```

Result:

```
2026-08-04
```

Common date parts

```
YEAR  
QUARTER  
MONTH  
DAY  
WEEK  
HOUR
```

MINUTE
SECOND

4. DATEDIFF()

Calculates the difference between two dates.

Syntax:

```
DATEDIFF(datepart, start_date, end_date)
```

Difference in days

```
SELECT DATEDIFF(  
    DAY,  
    '2026-08-01',  
    '2026-08-14'  
);
```

Result:

```
13
```

Difference in months

```
SELECT DATEDIFF(  
    MONTH,  
    '2026-01-01',  
    '2026-08-01'  
);
```

Result:

```
7
```

Difference in years

```
SELECT DATEDIFF(  
    YEAR,  
    '2020-01-01',
```

```
'2026-01-01'  
);
```

Result:

```
6
```

Practical example

Find how many days an order has been pending:

```
SELECT  
    OrderId,  
    DATEDIFF(DAY, OrderDate, GETDATE()) AS DaysPending  
FROM Orders;
```

5. DATEPART()

Extracts a specific part of a date.

```
SELECT DATEPART(YEAR, '2026-08-14');
```

Result:

```
2026
```

```
SELECT DATEPART(MONTH, '2026-08-14');
```

Result:

```
8
```

```
SELECT DATEPART(DAY, '2026-08-14');
```

Result:

```
14
```

You can also extract:

```
SELECT DATEPART(QUARTER, '2026-08-14');
```

Result:

```
3
```

6. YEAR()

Returns the year from a date.

```
SELECT YEAR('2026-08-14');
```

Result:

```
2026
```

Example:

```
SELECT
    OrderId,
    YEAR(OrderDate) AS OrderYear
FROM Orders;
```

7. MONTH()

Returns the month number.

```
SELECT MONTH('2026-08-14');
```

Result:

```
8
```

Example:

```
SELECT
    OrderId,
```

```
    MONTH(OrderDate) AS OrderMonth  
FROM Orders;
```

8. DAY()

Returns the day of the month.

```
SELECT DAY('2026-08-14');
```

Result:

```
14
```

Example:

```
SELECT  
    OrderId,  
    DAY(OrderDate) AS OrderDay  
FROM Orders;
```

9. EOMONTH()

Returns the **last day of a month**.

```
SELECT EOMONTH('2026-08-14');
```

Result:

```
2026-08-31
```

Previous month's end

```
SELECT EOMONTH('2026-08-14', -1);
```

Result:

```
2026-07-31
```

Next month's end

```
SELECT EOMONTH('2026-08-14', 1);
```

Result:

```
2026-09-30
```

This is very useful for **monthly reports and financial applications**.

10. DATENAME()

Returns the name of a date part instead of its number.

```
SELECT DATENAME(MONTH, '2026-08-14');
```

Result:

```
August
```

Get weekday name:

```
SELECT DATENAME(WEEKDAY, '2026-08-14');
```

Result depends on the language/session settings, e.g.:

```
Friday
```

11. ISDATE()

Checks whether a value can be interpreted as a valid date.

```
SELECT ISDATE('2026-08-14');
```

Result:

```
1
```

Invalid date:

```
SELECT ISDATE('Hello');
```

Result:

```
0
```

Useful when validating input before conversion.

12. DATEFROMPARTS()

Creates a date from year, month, and day.

```
SELECT DATEFROMPARTS(2026, 8, 14);
```

Result:

```
2026-08-14
```

Example:

```
SELECT DATEFROMPARTS(2026, 12, 25);
```

Result:

```
2026-12-25
```

13. TIMEFROMPARTS()

Creates a time from individual components.

```
SELECT TIMEFROMPARTS(14, 30, 15, 0, 0);
```

Result:

```
14:30:15
```

Practical Examples

Find employees who joined this year

```
SELECT *
FROM Employees
WHERE YEAR(JoiningDate) = YEAR(GETDATE());
```

Find employees who joined in August

```
SELECT *
FROM Employees
WHERE MONTH(JoiningDate) = 8;
```

Find employees who joined more than 5 years ago

```
SELECT *
FROM Employees
WHERE DATEDIFF(YEAR, JoiningDate, GETDATE()) > 5;
```

Calculate employee experience in days

```
SELECT
    EmployeeName,
    DATEDIFF(DAY, JoiningDate, GETDATE()) AS ExperienceDays
FROM Employees;
```

Calculate an expiry date 30 days after registration

```
SELECT
    UserId,
    DATEADD(DAY, 30, RegistrationDate) AS ExpiryDate
FROM Users;
```

Important SQL Date Functions

Function	Purpose
GETDATE()	Current date/time

Function	Purpose
<code>SYSDATETIME()</code>	Current date/time with high precision
<code>DATEADD()</code>	Add/subtract date interval
<code>DATEDIFF()</code>	Difference between dates
<code>DATEPART()</code>	Extract numeric date part
<code>DATENAME()</code>	Extract named date part
<code>YEAR()</code>	Get year
<code>MONTH()</code>	Get month
<code>DAY()</code>	Get day
<code>EOMONTH()</code>	Last day of month
<code>ISDATE()</code>	Validate date
<code>DATEFROMPARTS()</code>	Create a date
<code>TIMEFROMPARTS()</code>	Create a time

Quick memory trick

GETDATE	→ Current date/time
DATEADD	→ Add/subtract
DATEDIFF	→ Difference
DATEPART	→ Extract number
DATENAME	→ Extract name
YEAR	→ Year
MONTH	→ Month
DAY	→ Day
EOMONTH	→ Month end
ISDATE	→ Validate date

- Conversion Functions

SQL Conversion Functions

Conversion functions are used to convert a value from one data type to another.

For example:

INT	→	VARCHAR
VARCHAR	→	INT
VARCHAR	→	DATE
DATE	→	VARCHAR
DECIMAL	→	INT

The most important conversion functions are:

- `CAST()`
- `CONVERT()`
- `TRY_CAST()`
- `TRY_CONVERT()`

Examples below mainly use **SQL Server** syntax.

1. `CAST()`

`CAST()` converts a value from one data type to another.

Syntax

```
CAST(expression AS data_type)
```

Integer → String

```
SELECT CAST(100 AS VARCHAR(10));
```

Result:

```
100
```

Decimal → Integer

```
SELECT CAST(123.45 AS INT);
```

Result:

```
123
```

The decimal portion is removed.

String → Integer

```
SELECT CAST('500' AS INT);
```

Result:

```
500
```

2. CONVERT()

CONVERT() also converts one data type into another.

Syntax

```
CONVERT(data_type, expression, style)
```

Example:

```
SELECT CONVERT(VARCHAR(10), 100);
```

Result:

```
100
```

CAST() vs CONVERT()

Both can perform type conversion:

```
SELECT CAST(123 AS VARCHAR(10));  
  
SELECT CONVERT(VARCHAR(10), 123);
```

The major difference is that SQL Server's **CONVERT()** provides a **style parameter**, which is particularly useful for date formatting.

3. Date Conversion with CONVERT()

Convert a date to **DD/MM/YYYY**:

```
SELECT CONVERT(VARCHAR(10), GETDATE(), 103);
```

Example:

```
14/08/2026
```

Common SQL Server date styles:

Style	Format	Example
101	MM/DD/YYYY	08/14/2026
103	DD/MM/YYYY	14/08/2026
110	MM-DD-YYYY	08-14-2026
111	YYYY/MM/DD	2026/08/14
112	YYYYMMDD	20260814
120	YYYY-MM-DD HH:MI:SS	2026-08-14 23:45:30

Example:

```
SELECT CONVERT(VARCHAR(8), GETDATE(), 112);
```

Result:

```
20260814
```

4. TRY_CAST()

TRY_CAST() attempts to convert a value.

If conversion succeeds, it returns the converted value.

If conversion fails, it returns **NULL instead of an error**.

Valid conversion

```
SELECT TRY_CAST('100' AS INT);
```

Result:

```
100
```

Invalid conversion

```
SELECT TRY_CAST('ABC' AS INT);
```

Result:

```
NULL
```

This is very useful when processing data from external sources.

5. TRY_CONVERT()

TRY_CONVERT() works similarly to TRY_CAST().

```
SELECT TRY_CONVERT(INT, '500');
```

Result:

```
500
```

Invalid conversion:

```
SELECT TRY_CONVERT(INT, 'ABC');
```

Result:

```
NULL
```

6. PARSE()

PARSE() converts a string to a specified data type using .NET culture information.

```
SELECT PARSE('14/08/2026' AS DATE USING 'en-GB');
```

Result:

```
2026-08-14
```

It is useful for **culture-specific formats**, but generally has more overhead than CAST() and CONVERT().

7. TRY_PARSE()

`TRY_PARSE()` is similar to `PARSE()`, but returns `NULL` when conversion fails.

```
SELECT TRY_PARSE('14/08/2026' AS DATE USING 'en-GB');
```

Result:

```
2026-08-14
```

Invalid value:

```
SELECT TRY_PARSE('ABC' AS DATE USING 'en-GB');
```

Result:

```
NULL
```

Practical Example

Suppose an imported table contains prices as strings:

ProductName	Price
Laptop	'55000'
Mobile	'25000'
Keyboard	'1500'

You can convert `Price` to a number:

```
SELECT
    ProductName,
    CAST(Price AS DECIMAL(10,2)) AS NumericPrice
FROM Products;
```

Handling Bad Data

Suppose:

ProductName	Price
Laptop	'55000'
Mobile	'25000'
Keyboard	'ABC'

Using `CAST()`:

```
SELECT CAST(Price AS INT)
FROM Products;
```

The query can fail because `'ABC'` cannot be converted to `INT`.

Instead:

```
SELECT TRY_CAST(Price AS INT) AS NumericPrice
FROM Products;
```

Result:

```
55000
25000
NULL
```

This is a **very common real-world use case**.

Conversion in `WHERE`

You can convert a column before comparing it:

```
SELECT *
FROM Products
WHERE TRY_CAST(Price AS DECIMAL(10,2)) > 10000;
```

Conversion in `JOIN`

Sometimes two tables store the same value using different data types.

For example:

```
Customers.CustomerId → INT
```

Orders.CustomerId → VARCHAR

You could convert:

```
SELECT *
FROM Customers C
JOIN Orders O
  ON C.CustomerId = TRY_CAST(O.CustomerId AS INT);
```

However, frequent conversions on columns can hurt performance and may prevent efficient index usage.
Matching data types across related columns is preferable.

Important Conversion Functions

Function	Purpose
CAST()	Standard data-type conversion
CONVERT()	SQL Server conversion + formatting styles
TRY_CAST()	Conversion; returns NULL on failure
TRY_CONVERT()	Conversion; returns NULL on failure
PARSE()	Culture-aware conversion
TRY_PARSE()	Culture-aware conversion; returns NULL on failure

Interview Comparison

CAST()
↓
Conversion fails → Error

TRY_CAST()
↓
Conversion fails → NULL

CONVERT()
↓
Conversion + SQL Server style formatting

TRY_CONVERT()
↓
Conversion + NULL on failure

Most important to learn first


```
CAST()  
CONVERT()  
TRY_CAST()  
TRY_CONVERT()
```

For SQL Server interviews, **CAST()** vs **CONVERT()** and **CAST()** vs **TRY_CAST()** are especially important.

- NULL Functions

SQL NULL Functions

NULL functions are used to handle missing, unknown, or undefined values in SQL.

Important functions are:

- ISNULL()
- COALESCE()
- NULLIF()
- IS NULL
- IS NOT NULL

Examples below mainly use **SQL Server** syntax.

1. ISNULL()

ISNULL() replaces a **NULL** value with a specified replacement value.

Syntax

```
ISNULL(expression, replacement_value)
```

Example

```
SELECT ISNULL(NULL, 'No Value');
```

Result:

```
No Value
```

With a table

Suppose:

EmployeeName	Phone
John	9876543210
Alice	NULL
Bob	9123456789

Query:

```
SELECT
    EmployeeName,
    ISNULL(Phone, 'Not Available') AS Phone
FROM Employees;
```

Result:

EmployeeName	Phone
John	9876543210
Alice	Not Available
Bob	9123456789

2. COALESCE()

COALESCE() returns the **first non-NULL value** from a list of expressions.

Syntax

```
COALESCE(value1, value2, value3, ...)
```

Example:

```
SELECT COALESCE(NULL, NULL, 'Hello', 'World');
```

Result:

Hello

SQL checks from left to right:

```
NULL
↓
NULL
```

↓

Hello ← First non-NULL value

Practical example

Suppose an employee has multiple contact numbers:

Phone	Mobile	OfficePhone
NULL	9876543210	9123456789

Query:

```
SELECT
    EmployeeName,
    COALESCE(Phone, Mobile, OfficePhone, 'No Contact') AS ContactNumber
FROM Employees;
```

Result:

9876543210

Because **Phone** is **NULL**, SQL checks **Mobile**.

3. NULLIF()

NULLIF() returns **NULL** if two expressions are equal.

Syntax

```
NULLIF(expression1, expression2)
```

Example:

```
SELECT NULLIF(10, 10);
```

Result:

NULL

Because:

```
10 = 10
```

If they are different:

```
SELECT NULLIF(10, 20);
```

Result:

```
10
```

4. `NULLIF()` to Avoid Divide-by-Zero

This is one of the most useful practical applications.

Suppose:

Sales	Quantity
10000	5
5000	0

This query can produce a divide-by-zero error:

```
SELECT Sales / Quantity  
FROM Orders;
```

Use `NULLIF()`:

```
SELECT  
    Sales / NULLIF(Quantity, 0) AS PricePerUnit  
FROM Orders;
```

When:

```
Quantity = 0
```

this:

```
NULLIF(Quantity, 0)
```

returns:

```
NULL
```

Therefore:

```
5000 / NULL
```

returns **NULL** instead of causing a divide-by-zero error.

5. IS NULL

IS NULL checks whether a column contains **NULL**.

Example

```
SELECT *  
FROM Employees  
WHERE Phone IS NULL;
```

This returns employees whose phone number is missing.

Important

Do **not** use:

```
WHERE Phone = NULL;
```

This is incorrect.

Use:

```
WHERE Phone IS NULL;
```

6. IS NOT NULL

Checks whether a value is **not NULL**.

```
SELECT *  
FROM Employees  
WHERE Phone IS NOT NULL;
```

This returns employees who have a phone number.

ISNULL() vs COALESCE()

Both can replace **NULL**, but there are important differences.

ISNULL()

```
SELECT ISNULL(Phone, 'Not Available')
FROM Employees;
```

It accepts **two arguments** in SQL Server.

COALESCE()

```
SELECT COALESCE(
    Phone,
    Mobile,
    OfficePhone,
    'Not Available'
)
FROM Employees;
```

It can accept **multiple arguments**.

Comparison

Feature	ISNULL()	COALESCE()
Arguments	2	Multiple
Standard SQL	No, SQL Server-specific	Yes
Returns first non-NULL	No	Yes
Common in SQL Server	Very common	Very common
Multiple fallback values	✗	✓

COALESCE() Example

Consider:

Phone	Mobile	Email
NULL	NULL	john@gmail.com

Query:

```
SELECT COALESCE(  
    Phone,  
    Mobile,  
    Email,  
    'No Contact'  
) AS Contact  
FROM Employees;
```

Result:

```
john@gmail.com
```

Because:

```
Phone  → NULL  
Mobile → NULL  
Email  → john@gmail.com ← First non-NULL
```

Important NULL Concept

NULL is **not the same as zero or an empty string**.

```
NULL    → Unknown / missing  
0       → Numeric zero  
' '     → Empty string
```

For example:

```
SELECT  
    CASE  
        WHEN Salary IS NULL THEN 'Salary Missing'  
        WHEN Salary = 0 THEN 'Salary is Zero'  
        ELSE 'Salary Available'  
    END AS SalaryStatus  
FROM Employees;
```

Practical Example

Suppose:

EmployeeName	Salary	Bonus
John	50000	5000
Alice	60000	NULL
Bob	NULL	3000

You can calculate total compensation:

```
SELECT
    EmployeeName,
    ISNULL(Salary, 0) + ISNULL(Bonus, 0) AS TotalCompensation
FROM Employees;
```

Result:

EmployeeName	TotalCompensation
John	55000
Alice	60000
Bob	3000

Without `ISNULL()`, arithmetic involving `NULL` can produce `NULL`.

Quick Summary

Function / Operator	Purpose
<code>ISNULL()</code>	Replace <code>NULL</code> with a value
<code>COALESCE()</code>	Return first non-NULL value
<code>NULLIF()</code>	Return <code>NULL</code> when two values are equal
<code>IS NULL</code>	Find NULL values
<code>IS NOT NULL</code>	Find non-NULL values

Easy way to remember

```
ISNULL()
  ↓
Replace NULL

COALESCE()
  ↓
First non-NULL

NULLIF()
  ↓
```


Equal → NULL

IS NULL

↓

Check for NULL

IS NOT NULL

↓

Check for value

Most important for interviews

Focus especially on:

```
ISNULL()  
COALESCE()  
NULLIF()  
IS NULL  
IS NOT NULL
```

And remember the classic interview question:

ISNULL() vs **COALESCE()** → **ISNULL()** is SQL Server-specific and takes two arguments; **COALESCE()** is standard SQL and can evaluate multiple expressions.

16. Window Functions

- **ROW_NUMBER()**

ROW_NUMBER() in SQL

ROW_NUMBER() is a **window function** that assigns a **unique sequential number** to each row in a result set.

The numbering starts from **1**.

Syntax

```
ROW_NUMBER() OVER (  
    [PARTITION BY column]  
    ORDER BY column  
)
```

- **ORDER BY** → determines the numbering order.
- **PARTITION BY** → optionally restarts numbering for each group.

1. Basic Example

Suppose we have:

Employees

EmployeeId	Name	Department	Salary
1	Amit	IT	60000
2	Ravi	IT	75000
3	Priya	HR	65000
4	Neha	HR	80000
5	John	IT	75000

Query:

```
SELECT
  Name,
  Department,
  Salary,
  ROW_NUMBER() OVER (
    ORDER BY Salary DESC
  ) AS RowNum
FROM Employees;
```

Result:

Name	Department	Salary	RowNum
Neha	HR	80000	1
Ravi	IT	75000	2
John	IT	75000	3
Priya	HR	65000	4
Amit	IT	60000	5

Notice that **Ravi and John have the same salary**, but they still get different row numbers.

```
Neha → 1
Ravi → 2
John → 3
Priya → 4
Amit → 5
```

2. ROW_NUMBER() with PARTITION BY

PARTITION BY divides the data into groups and **restarts numbering for each group**.

```
SELECT
    Name,
    Department,
    Salary,
    ROW_NUMBER() OVER (
        PARTITION BY Department
        ORDER BY Salary DESC
    ) AS RowNum
FROM Employees;
```

Result:

Name	Department	Salary	RowNum
Neha	HR	80000	1
Priya	HR	65000	2
Ravi	IT	75000	1
John	IT	75000	2
Amit	IT	60000	3

Here:

```
HR
  Neha → 1
  Priya → 2

IT
  Ravi → 1
  John → 2
  Amit → 3
```

So:

PARTITION BY = "restart the row numbering for each group."

3. Find the Highest-Paid Employee in Each Department

This is one of the **most common uses** of **ROW_NUMBER()**.

```
WITH RankedEmployees AS
(
    SELECT
        EmployeeId,
```

```
Name,  
Department,  
Salary,  
ROW_NUMBER() OVER (  
    PARTITION BY Department  
    ORDER BY Salary DESC  
) AS RowNum  
FROM Employees  
)  
SELECT  
    EmployeeId,  
    Name,  
    Department,  
    Salary  
FROM RankedEmployees  
WHERE RowNum = 1;
```

Result:

EmployeeId	Name	Department	Salary
4	Neha	HR	80000
2	Ravi	IT	75000

Why?

For each department:

```
HR:  
Neha  → 1  
Priya → 2  
  
IT:  
Ravi  → 1  
John  → 2  
Amit  → 3
```

Then:

```
WHERE RowNum = 1
```

selects the first employee from each department.

4. Find the Second-Highest Employee in Each Department

```
WITH RankedEmployees AS
(
    SELECT
        Name,
        Department,
        Salary,
        ROW_NUMBER() OVER (
            PARTITION BY Department
            ORDER BY Salary DESC
        ) AS RowNum
    FROM Employees
)
SELECT *
FROM RankedEmployees
WHERE RowNum = 2;
```

Result:

Name	Department	Salary
Priya	HR	65000
John	IT	75000

5. Remove Duplicate Records

`ROW_NUMBER()` is commonly used to identify duplicates.

Suppose:

CustomerId	Email
1	a@test.com
2	b@test.com
3	a@test.com
4	c@test.com
5	b@test.com

You can number records within each email:

```
SELECT
    CustomerId,
    Email,
    ROW_NUMBER() OVER (
        PARTITION BY Email
        ORDER BY CustomerId
```

```
) AS RowNum  
FROM Customers;
```

Result:

CustomerId	Email	RowNum
1	a@test.com	1
3	a@test.com	2
2	b@test.com	1
5	b@test.com	2
4	c@test.com	1

The records where:

```
RowNum > 1
```

are duplicates.

6. Pagination

`ROW_NUMBER()` can also be used for pagination.

```
WITH EmployeeList AS  
(  
    SELECT  
        EmployeeId,  
        Name,  
        Salary,  
        ROW_NUMBER() OVER (  
            ORDER BY EmployeeId  
        ) AS RowNum  
    FROM Employees  
)  
SELECT *  
FROM EmployeeList  
WHERE RowNum BETWEEN 11 AND 20;
```

This returns rows **11–20**.

In modern SQL Server, however, `OFFSET/FETCH` is generally preferred for straightforward pagination.

7. Multiple Columns in `ORDER BY`

You should often add a second column when the primary sorting column can contain duplicates.

Instead of:

```
ROW_NUMBER() OVER (  
    ORDER BY Salary DESC  
)
```

use:

```
ROW_NUMBER() OVER (  
    ORDER BY Salary DESC, EmployeeId  
)
```

This makes the ordering deterministic when salaries are equal.

For example:

```
Ravi → Salary 75000, EmployeeId 2  
John → Salary 75000, EmployeeId 5
```

The database can consistently determine who gets 2 and who gets 3.

ROW_NUMBER() VS RANK() VS DENSE_RANK()

This is very important.

Suppose salaries are:

```
80000  
75000  
75000  
65000
```

ROW_NUMBER()

```
1  
2  
3  
4
```

Every row gets a unique number.

RANK()

```
1  
2  
2  
4
```

Ties receive the same rank, and gaps occur.

DENSE_RANK()

```
1  
2  
2  
3
```

Ties receive the same rank, but there are no gaps.

Important Point

ROW_NUMBER() **does not mean salary rank.**

It simply means:

"Give every row a sequential number according to the specified ordering."

For example:

```
ROW_NUMBER() OVER (  
    ORDER BY Salary DESC  
)
```

means:

Number the employees from highest salary to lowest salary.

Whereas:

```
ROW_NUMBER() OVER (  
    PARTITION BY Department  
    ORDER BY Salary DESC  
)
```

means:

Number employees from highest salary to lowest salary **separately within each department.**

Real-world uses

`ROW_NUMBER()` is especially useful for:

- Finding the first/latest record
- Finding top N records per group
- Removing duplicates
- Pagination
- Selecting the latest transaction per customer
- Selecting the highest-paid employee per department
- Generating sequential numbers
- Comparing or filtering ordered records
- `RANK()`

`RANK()` in SQL

`RANK()` is a **window function** that assigns a rank to each row based on the specified ordering.

The key feature is:

Rows with the same value receive the same rank, and gaps are created after ties.

Syntax

```
RANK() OVER (  
    [PARTITION BY column]  
    ORDER BY column  
)
```

- `ORDER BY` → determines the ranking order.
- `PARTITION BY` → optionally creates separate rankings for each group.

1. Basic Example

Suppose we have:

Employees

EmployeeId	Name	Department	Salary
1	Amit	IT	60000
2	Ravi	IT	75000
3	Priya	HR	65000

EmployeeId	Name	Department	Salary
4	Neha	HR	80000
5	John	IT	75000

Query:

```
SELECT
    Name,
    Salary,
    RANK() OVER (
        ORDER BY Salary DESC
    ) AS SalaryRank
FROM Employees;
```

Result:

Name	Salary	SalaryRank
Neha	80000	1
Ravi	75000	2
John	75000	2
Priya	65000	4
Amit	60000	5

Notice:

```
80000 → Rank 1
75000 → Rank 2
75000 → Rank 2
65000 → Rank 4
60000 → Rank 5
```

There is **no rank 3** because two employees share rank 2.

2. Why Does `RANK()` Skip a Number?

Suppose the salaries are:

```
100000
90000
90000
```

```
90000
80000
```

Using `RANK()`:

```
100000 → 1
90000  → 2
90000  → 2
90000  → 2
80000  → 5
```

Three employees occupy rank 2.

Therefore, the next rank becomes:

```
2 + 3 = 5
```

This is why `RANK()` produces gaps.

3. `RANK()` with `PARTITION BY`

You can create separate rankings within each department.

```
SELECT
    Name,
    Department,
    Salary,
    RANK() OVER (
        PARTITION BY Department
        ORDER BY Salary DESC
    ) AS SalaryRank
FROM Employees;
```

Result:

Name	Department	Salary	SalaryRank
Neha	HR	80000	1
Priya	HR	65000	2
Ravi	IT	75000	1
John	IT	75000	1
Amit	IT	60000	3

Notice that ranking restarts for each department.

```
HR
Neha  → 1
Priya → 2

IT
Ravi  → 1
John  → 1
Amit  → 3
```

4. Find the Highest-Paid Employee

```
SELECT *
FROM
(
    SELECT
        Name,
        Department,
        Salary,
        RANK() OVER (
            PARTITION BY Department
            ORDER BY Salary DESC
        ) AS SalaryRank
    FROM Employees
) E
WHERE SalaryRank = 1;
```

Result:

Name	Department	Salary
Neha	HR	80000
Ravi	IT	75000
John	IT	75000

Notice something important:

Both Ravi and John are returned because they have the same highest salary in IT.

This is one of the major differences between `RANK()` and `ROW_NUMBER()`.

5. Find the Second-Highest Salary

```
SELECT *
FROM
(
    SELECT
        Name,
        Department,
        Salary,
        RANK() OVER (
            ORDER BY Salary DESC
        ) AS SalaryRank
    FROM Employees
) E
WHERE SalaryRank = 2;
```

Result:

Name	Salary
Ravi	75000
John	75000

Both employees receive rank 2.

6. RANK() vs ROW_NUMBER()

Suppose:

```
Salary
-----
80000
75000
75000
65000
```

ROW_NUMBER()

```
ROW_NUMBER() OVER (
    ORDER BY Salary DESC
)
```

Result:

```
80000 → 1
75000 → 2
```

```
75000 → 3
65000 → 4
```

Every row gets a unique number.

RANK()

```
RANK() OVER (
    ORDER BY Salary DESC
)
```

Result:

```
80000 → 1
75000 → 2
75000 → 2
65000 → 4
```

Ties receive the same rank.

7. RANK() vs DENSE_RANK()

This is another common interview question.

Suppose:

```
Salary
-----
80000
75000
75000
65000
```

RANK()

```
1
2
2
4
```

DENSE_RANK()

```
1
2
```

```
2  
3
```

The difference is:

```
RANK()  
→ gaps after ties  
  
DENSE_RANK()  
→ no gaps after ties
```

8. Top 3 Employees

You can use `RANK()` to find employees in the top 3 ranks.

```
SELECT *  
FROM  
(  
    SELECT  
        Name,  
        Salary,  
        RANK() OVER (  
            ORDER BY Salary DESC  
        ) AS SalaryRank  
    FROM Employees  
) E  
WHERE SalaryRank <= 3;
```

However, remember that **ties can cause more than 3 rows to be returned**.

For example:

Salary	Rank
80000	1
75000	2
75000	2
70000	4

`WHERE SalaryRank <= 3` returns the first three rows here, but if multiple people share rank 3, all of them are included.

9. Ranking Products by Sales

`RANK()` isn't limited to employees and salaries.

```
SELECT
    ProductName,
    TotalSales,
    RANK() OVER (
        ORDER BY TotalSales DESC
    ) AS SalesRank
FROM ProductSales;
```

Example:

ProductName	TotalSales	SalesRank
Laptop	500000	1
Mobile	400000	2
Tablet	400000	2
Monitor	250000	4

10. Ranking Customers Within Each Region

```
SELECT
    CustomerName,
    Region,
    TotalPurchase,
    RANK() OVER (
        PARTITION BY Region
        ORDER BY TotalPurchase DESC
    ) AS CustomerRank
FROM Customers;
```

This gives each customer a rank **within their region**.

Key Difference

Function	Example with tie	Result
ROW_NUMBER()	100, 90, 90, 80	1, 2, 3, 4
RANK()	100, 90, 90, 80	1, 2, 2, 4
DENSE_RANK()	100, 90, 90, 80	1, 2, 2, 3

Remember

`ROW_NUMBER()`

→ Every row gets a different number.

`RANK()`

→ Same values get the same rank + gaps.

`DENSE_RANK()`

→ Same values get the same rank + no gaps.

Real-world use: If you want **all employees tied for the highest salary**, use `RANK()` (or `DENSE_RANK()`). If you want **exactly one employee**, use `ROW_NUMBER()` with a deterministic tie-breaker.

- `DENSE_RANK()`

`DENSE_RANK()` in SQL

`DENSE_RANK()` is a **window function** used to assign rankings to rows.

The important feature is:

Rows with the same value receive the same rank, and unlike `RANK()`, no gaps are created after ties.

Syntax

```
DENSE_RANK() OVER (  
    [PARTITION BY column]  
    ORDER BY column  
)
```

- `ORDER BY` → determines the ranking order.
- `PARTITION BY` → creates separate rankings for each group.

1. Basic Example

Suppose we have:

Employees

EmployeeId	Name	Department	Salary
1	Amit	IT	60000
2	Ravi	IT	75000
3	Priya	HR	65000
4	Neha	HR	80000

EmployeeId	Name	Department	Salary
5	John	IT	75000

Query:

```
SELECT
  Name,
  Salary,
  DENSE_RANK() OVER (
    ORDER BY Salary DESC
  ) AS SalaryRank
FROM Employees;
```

Result:

Name	Salary	SalaryRank
Neha	80000	1
Ravi	75000	2
John	75000	2
Priya	65000	3
Amit	60000	4

Notice:

```
80000 → 1
75000 → 2
75000 → 2
65000 → 3
60000 → 4
```

There is **no rank 3 skipped**.

2. Why Is It Called "Dense"?

Consider these salaries:

```
100000
90000
90000
90000
80000
70000
```

With `DENSE_RANK()`:

```
100000 → 1
90000  → 2
90000  → 2
90000  → 2
80000  → 3
70000  → 4
```

The ranking remains **continuous**:

```
1 → 2 → 3 → 4
```

That's the "dense" part.

3. `DENSE_RANK()` with `PARTITION BY`

You can rank employees separately within each department.

```
SELECT
  Name,
  Department,
  Salary,
  DENSE_RANK() OVER (
    PARTITION BY Department
    ORDER BY Salary DESC
  ) AS SalaryRank
FROM Employees;
```

Result:

Name	Department	Salary	SalaryRank
Neha	HR	80000	1
Priya	HR	65000	2
Ravi	IT	75000	1
John	IT	75000	1
Amit	IT	60000	2

The ranking restarts for each department.

```
HR
Neha → 1
```

Priya → 2

IT

Ravi → 1

John → 1

Amit → 2

4. Find the Second-Highest Salary

One of the most common uses of `DENSE_RANK()` is finding the **second-highest distinct salary**.

```
WITH RankedEmployees AS
(
    SELECT
        Name,
        Salary,
        DENSE_RANK() OVER (
            ORDER BY Salary DESC
        ) AS SalaryRank
    FROM Employees
)
SELECT *
FROM RankedEmployees
WHERE SalaryRank = 2;
```

Result:

Name	Salary
Ravi	75000
John	75000

The second-highest **distinct** salary is 75000.

5. Find the Third-Highest Salary

```
WITH RankedEmployees AS
(
    SELECT
        Name,
        Salary,
        DENSE_RANK() OVER (
            ORDER BY Salary DESC
        ) AS SalaryRank
    FROM Employees
```

```
)  
SELECT *  
FROM RankedEmployees  
WHERE SalaryRank = 3;
```

Result:

Name	Salary
Priya	65000

6. Find Top 3 Distinct Salary Levels

```
WITH RankedEmployees AS  
(  
    SELECT  
        Name,  
        Salary,  
        DENSE_RANK() OVER (  
            ORDER BY Salary DESC  
        ) AS SalaryRank  
    FROM Employees  
)  
SELECT *  
FROM RankedEmployees  
WHERE SalaryRank <= 3;
```

This returns employees whose salaries belong to the **top 3 distinct salary levels**.

7. DENSE_RANK() vs RANK()

This is an important interview question.

Suppose:

```
Salary  
-----  
100000  
90000  
90000  
80000  
70000
```

RANK()

```
100000 → 1
90000 → 2
90000 → 2
80000 → 4
70000 → 5
```

DENSE_RANK()

```
100000 → 1
90000 → 2
90000 → 2
80000 → 3
70000 → 4
```

The difference:

```
RANK()
→ 1, 2, 2, 4, 5

DENSE_RANK()
→ 1, 2, 2, 3, 4
```

8. DENSE_RANK() vs ROW_NUMBER()

Suppose:

```
Salary
-----
100000
90000
90000
80000
```

ROW_NUMBER()

```
100000 → 1
90000 → 2
90000 → 3
80000 → 4
```

DENSE_RANK()

```
100000 → 1
90000 → 2
90000 → 2
80000 → 3
```

So:

`ROW_NUMBER()` gives every row a unique number.

`DENSE_RANK()` gives equal values the same rank.

9. Real-World Example: Student Scores

Suppose:

Student	Score
Amit	95
Ravi	90
Priya	90
Neha	85
John	80

```
SELECT
  Student,
  Score,
  DENSE_RANK() OVER (
    ORDER BY Score DESC
  ) AS Rank
FROM Students;
```

Result:

Student	Score	Rank
Amit	95	1
Ravi	90	2
Priya	90	2
Neha	85	3
John	80	4

This is useful when ranking students based on **distinct score levels**.

10. When Should You Use `DENSE_RANK()`?

Use `DENSE_RANK()` when you care about **distinct ranking levels**.

Common examples:

- Find the second-highest salary
- Find the third-highest salary
- Rank products by sales
- Rank students by marks
- Rank employees by performance
- Find top N distinct values
- Rank customers within regions
- Find highest/second-highest/third-highest values per group

Quick Comparison

Function	Ties	Example
<code>ROW_NUMBER()</code>	Different numbers	1, 2, 3, 4
<code>RANK()</code>	Same rank + gaps	1, 2, 2, 4
<code>DENSE_RANK()</code>	Same rank + no gaps	1, 2, 2, 3

Easy way to remember

`ROW_NUMBER()`
→ Every row is unique.

`RANK()`
→ Ties cause gaps.

`DENSE_RANK()`
→ Ties do NOT cause gaps.

Interview tip: If the question says "**second-highest distinct salary**", `DENSE_RANK()` is often the clearest window-function solution.

- `LEAD()`

`LEAD()` in SQL

`LEAD()` is a **window function** that allows you to access a value from a **following row** without using a self-join.

In simple words:

LEAD() looks ahead to the next row.

Syntax

```
LEAD(column, offset, default_value) OVER (  
    ORDER BY column  
)
```

Parameters

Parameter	Meaning
column	Column whose future value you want
offset	How many rows ahead; default is 1
default_value	Value returned when there is no following row
ORDER BY	Determines the row sequence

Basic usage:

```
LEAD(Salary) OVER (  
    ORDER BY EmployeeId  
)
```

1. Basic Example

Suppose we have:

Employees

EmployeeId	Name	Salary
1	Amit	60000
2	Ravi	75000
3	Priya	65000
4	Neha	80000
5	John	70000

Query:

```
SELECT
    EmployeeId,
    Name,
    Salary,
    LEAD(Salary) OVER (
        ORDER BY EmployeeId
    ) AS NextSalary
FROM Employees;
```

Result:

EmployeeId	Name	Salary	NextSalary
1	Amit	60000	75000
2	Ravi	75000	65000
3	Priya	65000	80000
4	Neha	80000	70000
5	John	70000	NULL

For Amit:

```
Current row → Amit → 60000
Next row    → Ravi  → 75000
```

Therefore:

```
LEAD(Salary) → 75000
```

The last employee has no next row, so `LEAD()` returns `NULL`.

2. `LEAD()` with an Offset

By default:

```
LEAD(Salary)
```

means:

```
Look 1 row ahead.
```

You can specify how many rows ahead you want.

```
SELECT
    Name,
    Salary,
    LEAD(Salary, 2) OVER (
        ORDER BY EmployeeId
    ) AS SalaryAfterTwoRows
FROM Employees;
```

Result:

Name	Salary	SalaryAfterTwoRows
Amit	60000	65000
Ravi	75000	80000
Priya	65000	70000
Neha	80000	NULL
John	70000	NULL

For Amit:

```
Amit
↓
Ravi
↓
Priya
```

Two rows ahead = Priya's salary.

3. LEAD() with a Default Value

You can specify a value instead of **NULL** when there is no following row.

```
SELECT
    Name,
    Salary,
    LEAD(Salary, 1, 0) OVER (
        ORDER BY EmployeeId
    ) AS NextSalary
FROM Employees;
```

Result:

Name	Salary	NextSalary
Amit	60000	75000
Ravi	75000	65000
Priya	65000	80000
Neha	80000	70000
John	70000	0

For John, there is no next row, so:

```
NextSalary = 0
```

instead of **NULL**.

4. Calculate Difference with the Next Row

One of the most useful applications of **LEAD()** is comparing the current row with the next row.

```
SELECT
    Name,
    Salary,
    LEAD(Salary) OVER (
        ORDER BY EmployeeId
    ) - Salary AS SalaryDifference
FROM Employees;
```

Result:

Name	Salary	SalaryDifference
Amit	60000	15000
Ravi	75000	-10000
Priya	65000	15000
Neha	80000	-10000
John	70000	NULL

For Amit:

```
Next salary = 75000
Current salary = 60000
```

```
75000 - 60000 = 15000
```

5. LEAD() with PARTITION BY

LEAD() can look ahead **within each group**.

Suppose employees belong to different departments:

```
SELECT
    Name,
    Department,
    Salary,
    LEAD(Salary) OVER (
        PARTITION BY Department
        ORDER BY Salary DESC
    ) AS NextSalary
FROM Employees;
```

The calculation happens separately for each department.

For example:

```
IT
75000 → next IT salary
75000 → next IT salary
60000 → NULL

HR
80000 → next HR salary
65000 → NULL
```

LEAD() does **not cross the partition boundary**.

6. Compare Current Salary with Next Salary

You can calculate whether the next employee has a higher or lower salary.

```
SELECT
    Name,
    Salary,
    LEAD(Salary) OVER (
        ORDER BY EmployeeId
    ) AS NextSalary,
    CASE
```

```
    WHEN LEAD(Salary) OVER (  
        ORDER BY EmployeeId  
    ) > Salary  
    THEN 'Higher'  
    WHEN LEAD(Salary) OVER (  
        ORDER BY EmployeeId  
    ) < Salary  
    THEN 'Lower'  
    ELSE 'Same'  
END AS Comparison  
FROM Employees;
```

Example result:

Name	Salary	NextSalary	Comparison
Amit	60000	75000	Higher
Ravi	75000	65000	Lower
Priya	65000	80000	Higher
Neha	80000	70000	Lower
John	70000	NULL	Same

In production SQL, you may prefer calculating `LEAD()` once in a CTE/subquery rather than repeating it.

7. Real-World Example: Sales

Suppose:

MonthlySales

Month	Sales
January	100000
February	120000
March	115000
April	140000

Query:

```
SELECT  
    Month,  
    Sales,  
    LEAD(Sales) OVER (  
        ORDER BY Month
```

```
) AS NextMonthSales  
FROM MonthlySales;
```

Result:

Month	Sales	NextMonthSales
January	100000	120000
February	120000	115000
March	115000	140000
April	140000	NULL

8. Calculate Future Sales Growth

```
SELECT  
    Month,  
    Sales,  
    LEAD(Sales) OVER (  
        ORDER BY Month  
    ) - Sales AS SalesChange  
FROM MonthlySales;
```

You can also calculate percentage change:

```
SELECT  
    Month,  
    Sales,  
    (  
        (LEAD(Sales) OVER (ORDER BY Month) - Sales) * 100.0  
        / NULLIF(Sales, 0)  
    ) AS PercentageChange  
FROM MonthlySales;
```

9. LEAD() vs LAG()

This is an important interview question.

LEAD()

Looks **forward**.

Current → Next

```
LEAD(Salary) OVER (  
    ORDER BY EmployeeId  
)
```

LAG()

Looks **backward**.

Previous ← Current

```
LAG(Salary) OVER (  
    ORDER BY EmployeeId  
)
```

Example:

Employee	Salary
Amit	60000
Ravi	75000
Priya	65000

For Ravi:

```
LAG() → 60000  
LEAD() → 65000
```

10. LEAD() vs Self Join

Without **LEAD()**, comparing a row with the next row often requires a self-join.

With **LEAD()**:

```
SELECT  
    EmployeeId,  
    Name,  
    Salary,  
    LEAD(Salary) OVER (  
        ORDER BY EmployeeId  
    ) AS NextSalary  
FROM Employees;
```


This is much simpler and is specifically designed for **row-to-row comparisons**.

Common Uses of **LEAD()**

LEAD() is useful for:

- Comparing current and next records
- Sales trend analysis
- Finding the next transaction
- Finding the next login/event
- Calculating time between events
- Detecting changes in status
- Calculating future values
- Time-series analysis
- Comparing current and next salary/price
- Finding gaps between dates

Easy way to remember

LAG()
→ Look backward

LEAD()
→ Look forward

For example:

Previous	Current	Next
↑	↑	↑
LAG()		LEAD()

- **LAG()**

LAG() in SQL

LAG() is a **window function** used to access a value from a **previous row** without using a self-join.

In simple terms:

LAG() looks backward.

Syntax

```
LAG(column, offset, default_value) OVER (  
    ORDER BY column  
)
```

Parameters

Parameter	Meaning
column	Column whose previous value you want
offset	Number of rows to look backward; default is 1
default_value	Value returned when no previous row exists
ORDER BY	Determines the row sequence

Basic example:

```
LAG(Salary) OVER (  
    ORDER BY EmployeeId  
)
```

1. Basic Example

Suppose we have:

Employees

EmployeeId	Name	Salary
1	Amit	60000
2	Ravi	75000
3	Priya	65000
4	Neha	80000
5	John	70000

Query:

```
SELECT  
    EmployeeId,  
    Name,  
    Salary,  
    LAG(Salary) OVER (  
        ORDER BY EmployeeId
```

```
) AS PreviousSalary  
FROM Employees;
```

Result:

EmployeeId	Name	Salary	PreviousSalary
1	Amit	60000	NULL
2	Ravi	75000	60000
3	Priya	65000	75000
4	Neha	80000	65000
5	John	70000	80000

For Ravi:

```
Current row → Ravi → 75000  
Previous row → Amit → 60000
```

Therefore:

```
LAG(Salary) → 60000
```

The first row has no previous row, so it returns **NULL**.

2. LAG() with Offset

By default:

```
LAG(Salary)
```

looks **1 row backward**.

You can specify an offset.

```
SELECT  
    Name,  
    Salary,  
    LAG(Salary, 2) OVER (  
        ORDER BY EmployeeId  
    ) AS SalaryTwoRowsAgo  
FROM Employees;
```

Result:

Name	Salary	SalaryTwoRowsAgo
Amit	60000	NULL
Ravi	75000	NULL
Priya	65000	60000
Neha	80000	75000
John	70000	65000

For Priya:

```
Priya
  ↑
Ravi
  ↑
Amit
```

Two rows backward = Amit's salary.

3. LAG() with Default Value

The first row normally returns **NULL**.

You can provide a default value:

```
SELECT
    Name,
    Salary,
    LAG(Salary, 1, 0) OVER (
        ORDER BY EmployeeId
    ) AS PreviousSalary
FROM Employees;
```

Result:

Name	Salary	PreviousSalary
Amit	60000	0
Ravi	75000	60000
Priya	65000	75000
Neha	80000	65000

Name	Salary	PreviousSalary
John	70000	80000

For Amit, there is no previous row, so 0 is returned.

4. Calculate Difference from Previous Row

One of the most common uses of `LAG()` is comparing the current row with the previous row.

```
SELECT
    Name,
    Salary,
    Salary - LAG(Salary) OVER (
        ORDER BY EmployeeId
    ) AS SalaryDifference
FROM Employees;
```

Result:

Name	Salary	SalaryDifference
Amit	60000	NULL
Ravi	75000	15000
Priya	65000	-10000
Neha	80000	15000
John	70000	-10000

For Ravi:

```
Current Salary  = 75000
Previous Salary = 60000

75000 - 60000 = 15000
```

5. Calculate Percentage Change

`LAG()` is very useful for calculating growth percentages.

```
SELECT
    Name,
    Salary,
```

```
(
    (Salary - LAG(Salary) OVER (ORDER BY EmployeeId))
    * 100.0
    / NULLIF(LAG(Salary) OVER (ORDER BY EmployeeId), 0)
) AS PercentageChange
FROM Employees;
```

For example:

```
Previous salary = 60000
Current salary  = 75000

Percentage change
= (75000 - 60000) / 60000 × 100
= 25%
```

6. LAG() with PARTITION BY

LAG() can look backward **within each group**.

```
SELECT
    Name,
    Department,
    Salary,
    LAG(Salary) OVER (
        PARTITION BY Department
        ORDER BY Salary
    ) AS PreviousSalary
FROM Employees;
```

The calculation is performed separately for each department.

For example:

```
HR
65000 → NULL
80000 → 65000

IT
60000 → NULL
75000 → 60000
75000 → 75000
```

LAG() does not move from one department into another because of PARTITION BY.

7. Real-World Example: Monthly Sales

Suppose:

Month	Sales
January	100000
February	120000
March	115000
April	140000

Query:

```
SELECT
    Month,
    Sales,
    LAG(Sales) OVER (
        ORDER BY Month
    ) AS PreviousMonthSales
FROM MonthlySales;
```

Result:

Month	Sales	PreviousMonthSales
January	100000	NULL
February	120000	100000
March	115000	120000
April	140000	115000

8. Calculate Monthly Sales Change

```
SELECT
    Month,
    Sales,
    Sales - LAG(Sales) OVER (
        ORDER BY Month
    ) AS SalesChange
FROM MonthlySales;
```

Result:

Month	Sales	SalesChange
January	100000	NULL
February	120000	20000
March	115000	-5000
April	140000	25000

This immediately shows:

```
January → No previous month
February → +20000
March → -5000
April → +25000
```

9. Calculate Month-over-Month Growth

```
SELECT
  Month,
  Sales,
  ROUND(
    (
      Sales - LAG(Sales) OVER (ORDER BY Month)
    ) * 100.0
    / NULLIF(
      LAG(Sales) OVER (ORDER BY Month),
      0
    ),
    2
  ) AS GrowthPercentage
FROM MonthlySales;
```

Example:

```
January → NULL
February → 20.00%
March → -4.17%
April → 21.74%
```

This is a very common analytics use of `LAG()`.

10. Find Changes in Status

Suppose we have:

Date	Status
Jan 1	Pending
Jan 2	Pending
Jan 3	Approved
Jan 4	Approved
Jan 5	Rejected

You can compare the current status with the previous status:

```
SELECT
    Date,
    Status,
    LAG(Status) OVER (
        ORDER BY Date
    ) AS PreviousStatus
FROM Orders;
```

Result:

Date	Status	PreviousStatus
Jan 1	Pending	NULL
Jan 2	Pending	Pending
Jan 3	Approved	Pending
Jan 4	Approved	Approved
Jan 5	Rejected	Approved

You can then detect status changes:

```
SELECT *
FROM
(
    SELECT
        Date,
        Status,
        LAG(Status) OVER (
            ORDER BY Date
        ) AS PreviousStatus
    FROM Orders
) O
WHERE Status <> PreviousStatus;
```

11. LAG() vs LEAD()

Very important for interviews:

Function	Looks
LAG()	Backward
LEAD()	Forward

Example:

Previous	Current	Next
↑	↑	↑
LAG()		LEAD()

For this data:

```
Amit → 60000  
Ravi → 75000  
Priya → 65000
```

When processing Ravi:

```
LAG() → 60000  
Current → 75000  
LEAD() → 65000
```

12. LAG() vs Self Join

Without LAG(), comparing a row with the previous row often requires a self-join.

With LAG():

```
SELECT  
    EmployeeId,  
    Name,  
    Salary,  
    LAG(Salary) OVER (  
        ORDER BY EmployeeId  
    ) AS PreviousSalary  
FROM Employees;
```

This is much cleaner for row-to-row comparisons.

Common Uses of LAG()

LAG() is commonly used for:

- Month-over-month growth
- Year-over-year comparison
- Comparing current and previous salary
- Comparing current and previous price
- Detecting status changes
- Detecting changes in account balance
- Finding previous transactions
- Time-series analysis
- Calculating differences between events
- Finding gaps between dates

Easy way to remember

LAG()
→ Look backward

LEAD()
→ Look forward

Interview shortcut

LAG(value) → Previous row's value
LEAD(value) → Next row's value

For example:

```
SELECT
    Month,
    Sales,
    LAG(Sales) OVER (ORDER BY Month) AS PreviousSales,
    LEAD(Sales) OVER (ORDER BY Month) AS NextSales
FROM MonthlySales;
```

This lets you see the **previous, current, and next values in the same row**.

- NTILE()

NTILE() in SQL

NTILE() is a **window function** that divides the rows of a result set into a specified number of **approximately equal groups**, called **buckets**.

In simple terms:

NTILE(n) divides ordered rows into n groups.

Syntax

```
NTILE(number_of_buckets) OVER (  
    ORDER BY column  
)
```

With partitioning:

```
NTILE(number_of_buckets) OVER (  
    PARTITION BY column  
    ORDER BY column  
)
```

1. Basic Example

Suppose we have 8 employees:

EmployeeId	Name	Salary
1	Amit	50000
2	Ravi	60000
3	Priya	65000
4	Neha	70000
5	John	75000
6	Sara	80000
7	Mike	90000
8	David	100000

We want to divide them into **4 groups**.

```
SELECT  
    Name,  
    Salary,  
    NTILE(4) OVER (  
        ORDER BY Salary DESC
```

```
) AS SalaryGroup  
FROM Employees;
```

Result:

Name	Salary	SalaryGroup
David	100000	1
Mike	90000	1
Sara	80000	2
John	75000	2
Neha	70000	3
Priya	65000	3
Ravi	60000	4
Amit	50000	4

The employees are divided into:

```
Group 1 → Top 25%  
Group 2 → Next 25%  
Group 3 → Next 25%  
Group 4 → Bottom 25%
```

2. NTILE(2) — Two Groups

If you use:

```
NTILE(2)
```

the rows are divided into **2 buckets**.

```
SELECT  
    Name,  
    Salary,  
    NTILE(2) OVER (  
        ORDER BY Salary DESC  
    ) AS SalaryGroup  
FROM Employees;
```

Result:

Name	Salary	Group
David	100000	1
Mike	90000	1
Sara	80000	1
John	75000	1
Neha	70000	2
Priya	65000	2
Ravi	60000	2
Amit	50000	2

Conceptually:

Group 1 → Top 50%
Group 2 → Bottom 50%

3. `NTILE(4)` — Quartiles

`NTILE(4)` is commonly used for **quartiles**.

```
SELECT
    Name,
    Salary,
    NTILE(4) OVER (
        ORDER BY Salary DESC
    ) AS Quartile
FROM Employees;
```

Conceptually:

1 → Top 25%
2 → 25% - 50%
3 → 50% - 75%
4 → Bottom 25%

This is useful for:

- Employee performance
- Customer segmentation
- Salary analysis
- Sales analysis

- Risk analysis

4. NTILE(10) — Deciles

NTILE(10) divides rows into **10 groups**.

```
SELECT
    Name,
    Salary,
    NTILE(10) OVER (
        ORDER BY Salary DESC
    ) AS SalaryDecile
FROM Employees;
```

Conceptually:

```
1  → Top 10%
2  → 10% - 20%
3  → 20% - 30%
...
10 → Bottom 10%
```

5. NTILE(100) — Percentile-Like Buckets

You can use:

```
NTILE(100)
```

to create 100 approximately equal groups.

```
SELECT
    Name,
    Salary,
    NTILE(100) OVER (
        ORDER BY Salary DESC
    ) AS PercentileGroup
FROM Employees;
```

However, remember:

NTILE(100) creates **100 buckets**; it is not exactly the same thing as calculating a statistical percentile.

6. What Happens When Rows Don't Divide Evenly?

This is important.

Suppose you have **10 rows** and use:

```
NTILE(4)
```

10 rows cannot be divided equally into 4 groups.

SQL distributes the extra rows among the earlier buckets.

Conceptually:

```
10 rows / 4 buckets
```

```
Bucket 1 → 3 rows
```

```
Bucket 2 → 3 rows
```

```
Bucket 3 → 2 rows
```

```
Bucket 4 → 2 rows
```

So the groups are **as equal as possible**.

7. NTILE() with PARTITION BY

You can divide rows into buckets separately within each group.

Suppose employees belong to departments:

```
SELECT
  Name,
  Department,
  Salary,
  NTILE(2) OVER (
    PARTITION BY Department
    ORDER BY Salary DESC
  ) AS SalaryGroup
FROM Employees;
```

Now each department gets its own buckets.

For example:

```
HR
```

```
Group 1 → Top 50% of HR
```

```
Group 2 → Bottom 50% of HR
```


IT
Group 1 → Top 50% of IT
Group 2 → Bottom 50% of IT

PARTITION BY effectively resets the **NTILE()** calculation for each department.

8. Find Top 25% Employees

You can use **NTILE(4)** and select bucket 1.

```
WITH EmployeeGroups AS
(
    SELECT
        Name,
        Salary,
        NTILE(4) OVER (
            ORDER BY Salary DESC
        ) AS SalaryGroup
    FROM Employees
)
SELECT *
FROM EmployeeGroups
WHERE SalaryGroup = 1;
```

This returns employees in the **top quartile**.

9. Customer Segmentation

Suppose you have customers and their total purchases:

```
SELECT
    CustomerId,
    CustomerName,
    TotalPurchase,
    NTILE(4) OVER (
        ORDER BY TotalPurchase DESC
    ) AS CustomerGroup
FROM Customers;
```

You could interpret the groups as:

Group 1 → High-value customers
Group 2 → Above-average customers

```
Group 3 → Lower-value customers
Group 4 → Low-value customers
```

This is useful for customer segmentation.

10. NTILE() vs RANK()

These functions solve different problems.

RANK()

Ranks rows based on their values.

```
RANK() OVER (
    ORDER BY Salary DESC
)
```

Example:

```
100000 → 1
90000  → 2
90000  → 2
80000  → 4
```

NTILE(4)

Divides rows into four groups.

```
NTILE(4) OVER (
    ORDER BY Salary DESC
)
```

Example with 8 rows:

```
Rows 1-2 → Group 1
Rows 3-4 → Group 2
Rows 5-6 → Group 3
Rows 7-8 → Group 4
```

So:

```
RANK()
→ "What position does this row have?"
```

```
NTILE()  
→ "Which bucket does this row belong to?"
```

11. NTILE() vs ROW_NUMBER()

ROW_NUMBER() assigns a unique number:

```
1  
2  
3  
4  
5  
6  
7  
8
```

NTILE(4) assigns bucket numbers:

```
1  
1  
2  
2  
3  
3  
4  
4
```

Therefore:

```
ROW_NUMBER()  
→ Individual row position  
  
NTILE()  
→ Group/bucket position
```

12. Important: Ordering Matters

Consider:

```
NTILE(4) OVER (  
    ORDER BY Salary DESC  
)
```

The highest salaries go into the first bucket.

If you change it to:

```
NTILE(4) OVER (  
    ORDER BY Salary ASC  
)
```

the lowest salaries go into the first bucket.

Descending

Group 1 → Highest values
Group 4 → Lowest values

Ascending

Group 1 → Lowest values
Group 4 → Highest values

Quick Comparison

Function	Purpose
ROW_NUMBER()	Assign unique row numbers
RANK()	Rank values, gaps after ties
DENSE_RANK()	Rank values, no gaps
NTILE(n)	Divide rows into n buckets
LAG()	Get previous row's value
LEAD()	Get next row's value

Easy way to remember

ROW_NUMBER()
→ Number the rows

RANK()
→ Rank the rows

DENSE_RANK()
→ Rank without gaps

```
NTILE()  
→ Divide rows into groups  
  
LAG()  
→ Look backward  
  
LEAD()  
→ Look forward
```

Common real-world uses

NTILE() is especially useful for:

- **Quartiles** → **NTILE(4)**
- **Deciles** → **NTILE(10)**
- **Customer segmentation**
- **Salary bands**
- **Performance groups**
- **Top/bottom percentage groups**
- **Sales analysis**
- **Risk classification**
- Aggregate Window Functions

Aggregate Window Functions in SQL

Aggregate Window Functions combine normal aggregate functions such as **SUM()**, **AVG()**, **COUNT()**, **MIN()**, and **MAX()** with the **OVER()** clause.

The key idea is:

GROUP BY combines rows, while aggregate window functions calculate across rows without removing the individual rows.

Common Aggregate Window Functions

```
SUM()    → Total  
AVG()    → Average  
COUNT() → Number of rows  
MIN()    → Minimum  
MAX()    → Maximum
```

Basic syntax:

```
AGGREGATE_FUNCTION(column) OVER (  
    [PARTITION BY column]  
    [ORDER BY column]  
)
```

1. SUM() OVER()

SUM() calculates a total across the window.

Suppose:

Employees

EmployeeId	Name	Department	Salary
1	Amit	IT	60000
2	Ravi	IT	75000
3	Priya	HR	65000
4	Neha	HR	80000
5	John	IT	70000

Total salary of all employees

```
SELECT  
    Name,  
    Salary,  
    SUM(Salary) OVER () AS TotalSalary  
FROM Employees;
```

Result:

Name	Salary	TotalSalary
Amit	60000	350000
Ravi	75000	350000
Priya	65000	350000
Neha	80000	350000
John	70000	350000

Every row remains in the result.

2. SUM() with PARTITION BY

Calculate the total salary for each department.

```
SELECT
    Name,
    Department,
    Salary,
    SUM(Salary) OVER (
        PARTITION BY Department
    ) AS DepartmentTotal
FROM Employees;
```

Result:

Name	Department	Salary	DepartmentTotal
Amit	IT	60000	205000
Ravi	IT	75000	205000
John	IT	70000	205000
Priya	HR	65000	145000
Neha	HR	80000	145000

The calculation is performed separately for each department.

3. Running Total

One of the most important uses of aggregate window functions is calculating a **running total**.

Suppose:

Sales

SaleId	SaleDate	Amount
1	2026-01-01	1000
2	2026-01-02	1500
3	2026-01-03	1200
4	2026-01-04	1800

Query:

```
SELECT
    SaleDate,
    Amount,
    SUM(Amount) OVER (
        ORDER BY SaleDate
    ) AS RunningTotal
FROM Sales;
```

Result:

SaleDate	Amount	RunningTotal
2026-01-01	1000	1000
2026-01-02	1500	2500
2026-01-03	1200	3700
2026-01-04	1800	5500

Calculation:

```
1000
1000 + 1500 = 2500
2500 + 1200 = 3700
3700 + 1800 = 5500
```

4. AVG() OVER()

AVG() calculates an average while keeping every row.

```
SELECT
    Name,
    Department,
    Salary,
    AVG(Salary) OVER () AS AverageSalary
FROM Employees;
```

Result:

Name	Salary	AverageSalary
Amit	60000	70000
Ravi	75000	70000
Priya	65000	70000

Name	Salary	AverageSalary
Neha	80000	70000
John	70000	70000

5. Department Average

Use **PARTITION BY** to calculate the average separately for each department.

```
SELECT
    Name,
    Department,
    Salary,
    AVG(Salary) OVER (
        PARTITION BY Department
    ) AS DepartmentAverage
FROM Employees;
```

Result:

Name	Department	Salary	DepartmentAverage
Amit	IT	60000	68333.33
Ravi	IT	75000	68333.33
John	IT	70000	68333.33
Priya	HR	65000	72500
Neha	HR	80000	72500

6. COUNT() OVER()

Count the total number of employees.

```
SELECT
    Name,
    Department,
    COUNT(*) OVER () AS TotalEmployees
FROM Employees;
```

Result:

Name	Department	TotalEmployees
Amit	IT	5
Ravi	IT	5
Priya	HR	5
Neha	HR	5
John	IT	5

7. Count Employees Per Department

```
SELECT
    Name,
    Department,
    COUNT(*) OVER (
        PARTITION BY Department
    ) AS DepartmentEmployeeCount
FROM Employees;
```

Result:

Name	Department	DepartmentEmployeeCount
Amit	IT	3
Ravi	IT	3
John	IT	3
Priya	HR	2
Neha	HR	2

This is similar to:

```
GROUP BY Department
```

but the individual employee rows are still available.

8. MIN() OVER()

Find the minimum salary while keeping every employee.

```
SELECT
    Name,
    Salary,
    MIN(Salary) OVER () AS MinimumSalary
FROM Employees;
```

Result:

Name	Salary	MinimumSalary
Amit	60000	60000
Ravi	75000	60000
Priya	65000	60000
Neha	80000	60000
John	70000	60000

9. Minimum Salary Per Department

```
SELECT
    Name,
    Department,
    Salary,
    MIN(Salary) OVER (
        PARTITION BY Department
    ) AS DepartmentMinimum
FROM Employees;
```

10. MAX() OVER()

Find the maximum salary.

```
SELECT
    Name,
    Salary,
    MAX(Salary) OVER () AS MaximumSalary
FROM Employees;
```

Result:

Name	Salary	MaximumSalary
Amit	60000	80000
Ravi	75000	80000
Priya	65000	80000
Neha	80000	80000
John	70000	80000

11. Maximum Salary Per Department

```
SELECT
    Name,
    Department,
    Salary,
    MAX(Salary) OVER (
        PARTITION BY Department
    ) AS DepartmentMaximum
FROM Employees;
```

Result:

Name	Department	Salary	DepartmentMaximum
Amit	IT	60000	75000
Ravi	IT	75000	75000
John	IT	70000	75000
Priya	HR	65000	80000
Neha	HR	80000	80000

12. Compare Employee Salary with Department Average

This is a very useful real-world example.

```
SELECT
    Name,
    Department,
    Salary,
    AVG(Salary) OVER (
        PARTITION BY Department
```

```
) AS DepartmentAverage,  
Salary - AVG(Salary) OVER (  
    PARTITION BY Department  
) AS DifferenceFromAverage  
FROM Employees;
```

This lets you determine whether an employee's salary is above or below their department's average.

13. Percentage of Department Total

Suppose you want to know what percentage of a department's total salary each employee represents.

```
SELECT  
    Name,  
    Department,  
    Salary,  
    SUM(Salary) OVER (  
        PARTITION BY Department  
    ) AS DepartmentTotal,  
    Salary * 100.0 /  
        SUM(Salary) OVER (  
            PARTITION BY Department  
        ) AS PercentageOfDepartment  
FROM Employees;
```

For Amit:

```
Amit salary = 60000  
IT total    = 205000  
  
60000 / 205000 × 100  
≈ 29.27%
```

14. Running Average

You can combine `AVG()` with `ORDER BY`.

```
SELECT  
    SaleDate,  
    Amount,  
    AVG(Amount) OVER (  
        ORDER BY SaleDate  
    ) AS RunningAverage  
FROM Sales;
```

Example:

Date	Amount	RunningAverage
Jan 1	1000	1000
Jan 2	1500	1250
Jan 3	1200	1233.33
Jan 4	1800	1375

15. Window Frame

You can control exactly which rows are included in the calculation.

For example, a **3-row moving average**:

```
SELECT
    SaleDate,
    Amount,
    AVG(Amount) OVER (
        ORDER BY SaleDate
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS MovingAverage
FROM Sales;
```

For each row, SQL considers:

```
Previous 2 rows + Current row
```

This is useful for:

- Moving averages
- Financial analysis
- Time-series analysis
- Trend detection

16. GROUP BY vs Aggregate Window Function

GROUP BY

```
SELECT
    Department,
    SUM(Salary) AS TotalSalary
```

```
FROM Employees
GROUP BY Department;
```

Result:

Department	TotalSalary
IT	205000
HR	145000

The employee rows are grouped together.

Window Function

```
SELECT
    Name,
    Department,
    Salary,
    SUM(Salary) OVER (
        PARTITION BY Department
    ) AS DepartmentTotal
FROM Employees;
```

Result:

Name	Department	Salary	DepartmentTotal
Amit	IT	60000	205000
Ravi	IT	75000	205000
John	IT	70000	205000
Priya	HR	65000	145000
Neha	HR	80000	145000

Main difference

GROUP BY
→ Combines rows.

Aggregate Window Function
→ Keeps rows and calculates across them.

17. PARTITION BY vs ORDER BY

These two are extremely important.

Without **PARTITION BY**

```
SUM(Salary) OVER ()
```

Calculates over **all rows**.

With **PARTITION BY**

```
SUM(Salary) OVER (  
    PARTITION BY Department  
)
```

Calculates separately for **each department**.

With **ORDER BY**

```
SUM(Salary) OVER (  
    ORDER BY Salary  
)
```

Calculates progressively, producing a **running total**.

So remember:

```
OVER ()  
    → Entire result  
  
OVER (PARTITION BY Department)  
    → Each department  
  
OVER (ORDER BY Salary)  
    → Running calculation
```

Quick Reference

Function	Purpose
SUM() OVER()	Total / running total
AVG() OVER()	Average / running average
COUNT() OVER()	Count rows
MIN() OVER()	Minimum value

Function	Purpose
<code>MAX()</code> <code>OVER()</code>	Maximum value

Most important patterns

```
-- Total
SUM(Salary) OVER ()

-- Department total
SUM(Salary) OVER (
  PARTITION BY Department
)

-- Running total
SUM(Salary) OVER (
  ORDER BY SaleDate
)

-- Department average
AVG(Salary) OVER (
  PARTITION BY Department
)

-- Count per department
COUNT(*) OVER (
  PARTITION BY Department
)

-- Min per department
MIN(Salary) OVER (
  PARTITION BY Department
)

-- Max per department
MAX(Salary) OVER (
  PARTITION BY Department
)
```

Interview rule to remember

Aggregate + `OVER()` = calculate across rows without collapsing them.

This makes aggregate window functions extremely useful for **running totals, moving averages, department-level comparisons, percentages, rankings, and analytical SQL queries.**

17. Common Table Expressions (CTEs)

- Simple CTE

Simple CTE in SQL

A **Simple CTE (Common Table Expression)** is a temporary named result set that you can use inside a single SQL statement.

It is created using the **WITH** keyword.

1. Basic Syntax

```
WITH CTE_Name AS
(
    SELECT column1, column2
    FROM TableName
    WHERE condition
)
SELECT *
FROM CTE_Name;
```

Think of it as:

First create a temporary query result → then query that result.

2. Simple Example

Suppose we have an **Employees** table:

Id	Name	Department	Salary
1	Rahul	IT	60000
2	Amit	IT	75000
3	Priya	HR	50000
4	Neha	IT	80000
5	Raj	HR	55000

We want employees whose salary is greater than **70000**.

Using CTE

```
WITH HighSalaryEmployees AS
(
    SELECT Id, Name, Department, Salary
    FROM Employees
    WHERE Salary > 70000
)
SELECT *
FROM HighSalaryEmployees;
```

Result

Id	Name	Department	Salary
2	Amit	IT	75000
4	Neha	IT	80000

3. CTE with **WHERE**

```
WITH ITEmployees AS
(
    SELECT *
    FROM Employees
    WHERE Department = 'IT'
)
SELECT *
FROM ITEmployees;
```

Result:

Id	Name	Department	Salary
1	Rahul	IT	60000
2	Amit	IT	75000
4	Neha	IT	80000

The CTE acts like a temporary result called **ITEmployees**.

4. CTE with Aggregation

You can use aggregate functions inside a CTE.

For example, calculate average salary by department:

```
WITH DepartmentSalary AS
(
    SELECT
        Department,
        AVG(Salary) AS AverageSalary
    FROM Employees
    GROUP BY Department
)
```

```
SELECT *  
FROM DepartmentSalary;
```

Result:

Department	AverageSalary
IT	71666.67
HR	52500.00

You can then filter that result:

```
WITH DepartmentSalary AS  
(  
    SELECT  
        Department,  
        AVG(Salary) AS AverageSalary  
    FROM Employees  
    GROUP BY Department  
)  
SELECT *  
FROM DepartmentSalary  
WHERE AverageSalary > 60000;
```

Result:

Department	AverageSalary
IT	71666.67

5. Multiple CTEs

You can define more than one CTE.

```
WITH ITEmployees AS  
(  
    SELECT *  
    FROM Employees  
    WHERE Department = 'IT'  
)  
HighSalary AS  
(  
    SELECT *  
    FROM ITEmployees  
    WHERE Salary > 70000  
)
```

```
SELECT *  
FROM HighSalary;
```

Execution flow:

```
Employees  
  ↓  
ITEmployees  
  ↓  
HighSalary  
  ↓  
Final SELECT
```

6. CTE vs Temporary Table

A CTE:

```
WITH HighSalary AS  
(  
    SELECT *  
    FROM Employees  
    WHERE Salary > 70000  
)  
SELECT *  
FROM HighSalary;
```

A temporary table:

```
CREATE TABLE #HighSalary  
(  
    Id INT,  
    Name VARCHAR(100),  
    Salary DECIMAL(10,2)  
);  
  
INSERT INTO #HighSalary  
SELECT Id, Name, Salary  
FROM Employees  
WHERE Salary > 70000;  
  
SELECT *  
FROM #HighSalary;
```

Main difference:

CTE	Temporary Table
Exists for one statement	Can exist for a session/transaction scope
No explicit storage structure required	Creates a temporary table
Very useful for readable queries	Useful for intermediate data reused across statements
Supports recursive queries	Does not itself provide recursion
Simple syntax	More setup

7. CTE vs Subquery

Without CTE:

```
SELECT *
FROM
(
    SELECT *
    FROM Employees
    WHERE Department = 'IT'
) AS ITEmployees
WHERE Salary > 70000;
```

With CTE:

```
WITH ITEmployees AS
(
    SELECT *
    FROM Employees
    WHERE Department = 'IT'
)
SELECT *
FROM ITEmployees
WHERE Salary > 70000;
```

The CTE version is usually easier to read, especially when the query becomes complicated.

Important Points

1. CTE starts with **WITH**.
2. CTE must be followed by a query that uses it.
3. A normal CTE does **not** persist after the SQL statement finishes.
4. A CTE can contain **SELECT**, joins, filtering, grouping, window functions, etc.
5. A CTE can be followed by **SELECT**, **INSERT**, **UPDATE**, or **DELETE** depending on the database.
6. A CTE is mainly about **query organization and readability**, not automatically improving performance.

7. A recursive CTE is a special type of CTE used for hierarchical data.

Most important pattern to remember

```
WITH CTE_Name AS
(
    SELECT ...
    FROM ...
    WHERE ...
)
SELECT ...
FROM CTE_Name;
```

Simple CTE = name a query result and use that name in the main query.

- Recursive CTE

Recursive CTE in SQL

A **Recursive CTE (Common Table Expression)** is a CTE that **refers to itself**.

It is mainly used when data has a **parent-child relationship** or a hierarchy.

Common examples:

- Employee → Manager
- Category → Subcategory
- Folder → Subfolder
- Organization hierarchy
- Family tree
- Menu → Submenu

1. Basic Structure

A recursive CTE has two parts:

```
WITH CTE_Name AS
(
    -- 1. Anchor Query
    SELECT ...

    UNION ALL

    -- 2. Recursive Query
    SELECT ...
    FROM TableName
    JOIN CTE_Name
```

```
        ON ...  
)  
SELECT *  
FROM CTE_Name;
```

Two important parts

Anchor Query

Gets the starting/root records.

Recursive Query

Uses the previous result to find the next level.

2. Employee Hierarchy Example

Suppose we have:

Employees

Id	Name	ManagerId
1	CEO	NULL
2	Rahul	1
3	Amit	1
4	Priya	2
5	Neha	2
6	Raj	4

The hierarchy is:

```
CEO  
├── Rahul  
│   ├── Priya  
│   │   └── Raj  
│   └── Neha  
└── Amit
```

Here:

- **Id** identifies the employee.
- **ManagerId** identifies the employee's manager.
- **ManagerId = NULL** means the employee is at the root.

3. Recursive CTE Query

```
WITH EmployeeHierarchy AS
(
    -- Anchor Query
    SELECT
        Id,
        Name,
        ManagerId,
        0 AS Level
    FROM Employees
    WHERE ManagerId IS NULL

    UNION ALL

    -- Recursive Query
    SELECT
        e.Id,
        e.Name,
        e.ManagerId,
        eh.Level + 1
    FROM Employees e
    INNER JOIN EmployeeHierarchy eh
        ON e.ManagerId = eh.Id
)
SELECT *
FROM EmployeeHierarchy
ORDER BY Level, Id;
```

Result

Id	Name	ManagerId	Level
1	CEO	NULL	0
2	Rahul	1	1
3	Amit	1	1
4	Priya	2	2
5	Neha	2	2
6	Raj	4	3

4. How Recursion Works

Let's understand the query step by step.

Step 1 — Anchor Query

```
SELECT
    Id,
    Name,
    ManagerId,
    0 AS Level
FROM Employees
WHERE ManagerId IS NULL;
```

Finds the root employee:

CEO

So:

Id	Name	ManagerId	Level
1	CEO	NULL	0

Step 2 — First Recursion

The recursive part is:

```
SELECT
    e.Id,
    e.Name,
    e.ManagerId,
    eh.Level + 1
FROM Employees e
INNER JOIN EmployeeHierarchy eh
    ON e.ManagerId = eh.Id;
```

The current employee is:

CEO (Id = 1)

Find employees where:

ManagerId = 1

We get:

Rahul
Amit

Their level becomes:

1

Step 3 — Second Recursion

Now the CTE has:

Rahul
Amit

Find employees whose **ManagerId** matches their IDs.

Rahul's ID is 2.

ManagerId = 2

gives:

Priya
Neha

Their level becomes:

2

Step 4 — Third Recursion

Priya's ID is 4.

Find:

ManagerId = 4

We get:

Raj

Level:

```
3
```

Step 5 — Stop

There are no employees reporting to Raj.

Therefore recursion stops.

5. Understanding `Level`

The `Level` column is very useful.

```
0 AS Level
```

The root starts at `0`.

Then:

```
eh.Level + 1
```

increments the level for every child.

Result:

```
CEO      → Level 0
Rahul    → Level 1
Amit     → Level 1
Priya    → Level 2
Neha     → Level 2
Raj      → Level 3
```

You can use this to display hierarchy depth.

6. Display the Hierarchy

You can create an indented hierarchy:

```
WITH EmployeeHierarchy AS
(
  SELECT
    Id,
    Name,
```

```
        ManagerId,
        0 AS Level
FROM Employees
WHERE ManagerId IS NULL

UNION ALL

SELECT
    e.Id,
    e.Name,
    e.ManagerId,
    eh.Level + 1
FROM Employees e
INNER JOIN EmployeeHierarchy eh
    ON e.ManagerId = eh.Id
)
SELECT
    REPLICATE(' ', Level) + Name AS Employee,
    Level
FROM EmployeeHierarchy
ORDER BY Level, Id;
```

Possible output:

```
Employee
-----
CEO
  Rahul
  Amit
    Priya
    Neha
      Raj
```

`REPLICATE()` here is SQL Server syntax. Other databases use different functions for indentation.

7. Recursive CTE for Numbers

Recursive CTEs aren't limited to employee hierarchies.

For example, generate numbers from 1 to 10:

```
WITH Numbers AS
(
    SELECT 1 AS Number

    UNION ALL

    SELECT Number + 1
    FROM Numbers
```

```
    WHERE Number < 10
)
SELECT *
FROM Numbers;
```

Result:

```
Number
-----
1
2
3
4
5
6
7
8
9
10
```

Here:

```
SELECT 1 AS Number
```

is the **anchor**.

And:

```
SELECT Number + 1
FROM Numbers
WHERE Number < 10
```

is the **recursive part**.

8. Recursive CTE for Categories

Suppose:

```
Categories
-----
Id | Name          | ParentId
---|-----|-----
1 | Electronics | NULL
2 | Computers   | 1
3 | Laptops     | 2
```

4	Phones	1
5	Android	4

You can retrieve the complete hierarchy:

```
WITH CategoryHierarchy AS
(
    SELECT
        Id,
        Name,
        ParentId,
        0 AS Level
    FROM Categories
    WHERE ParentId IS NULL

    UNION ALL

    SELECT
        c.Id,
        c.Name,
        c.ParentId,
        ch.Level + 1
    FROM Categories c
    INNER JOIN CategoryHierarchy ch
        ON c.ParentId = ch.Id
)
SELECT *
FROM CategoryHierarchy
ORDER BY Level, Id;
```

Result:

Electronics	Level 0
Computers	Level 1
Phones	Level 1
Laptops	Level 2
Android	Level 2

9. Recursive CTE vs Simple CTE

Simple CTE	Recursive CTE
Does not reference itself	References itself
Used to simplify queries	Used for hierarchical/iterative data
Usually one logical query step	Repeats until no more rows

Simple CTE	Recursive CTE
Easier to understand	More complex
Does not require UNION ALL	Usually uses UNION ALL
Example: filtering employees	Example: employee hierarchy

10. Important Interview Question

What are the two components of a recursive CTE?

Answer:

1. **Anchor member** — establishes the initial result.
2. **Recursive member** — references the CTE itself to retrieve subsequent results.

For example:

```
WITH EmployeeHierarchy AS
(
    -- Anchor
    SELECT ...
    FROM Employees
    WHERE ManagerId IS NULL

    UNION ALL

    -- Recursive member
    SELECT ...
    FROM Employees e
    JOIN EmployeeHierarchy eh
        ON e.ManagerId = eh.Id
)
SELECT *
FROM EmployeeHierarchy;
```

Easy way to remember

```
Anchor
  ↓
Level 1
  ↓
Level 2
  ↓
Level 3
  ↓
...
  ↓
No more children
```


↓

STOP

Recursive CTE = a CTE that repeatedly uses its previous result to find the next level of data.

18. Stored Procedures & Functions

- Create Procedure

CREATE PROCEDURE in SQL

A **Stored Procedure** is a precompiled, named set of SQL statements stored in the database. It is useful when you want to **reuse the same database operation multiple times**.

Examples below use **SQL Server (T-SQL)** syntax.

1. Basic CREATE PROCEDURE Syntax

```
CREATE PROCEDURE procedure_name
AS
BEGIN
    -- SQL statements
END;
```

Example

```
CREATE PROCEDURE GetAllEmployees
AS
BEGIN
    SELECT *
    FROM Employees;
END;
```

This creates a procedure named `GetAllEmployees`.

2. Procedure with Input Parameter

You can pass values to a procedure using parameters.

```
CREATE PROCEDURE GetEmployeeById
    @EmployeeId INT
AS
BEGIN
    SELECT *
    FROM Employees
```

```
WHERE EmployeeId = @EmployeeId;  
END;
```

Here:

```
@EmployeeId
```

is an input parameter.

3. Procedure with Multiple Parameters

```
CREATE PROCEDURE GetEmployees  
    @Department VARCHAR(50),  
    @MinimumSalary DECIMAL(10,2)  
AS  
BEGIN  
    SELECT EmployeeId, Name, Salary  
    FROM Employees  
    WHERE Department = @Department  
        AND Salary >= @MinimumSalary;  
END;
```

The procedure accepts:

- Department
- Minimum salary

4. Procedure with INSERT

Procedures are commonly used for inserting records.

```
CREATE PROCEDURE AddEmployee  
    @EmployeeId INT,  
    @Name VARCHAR(100),  
    @Salary DECIMAL(10,2),  
    @Department VARCHAR(50)  
AS  
BEGIN  
    INSERT INTO Employees  
    (  
        EmployeeId,  
        Name,  
        Salary,  
        Department  
    )  
    VALUES  
    (  

```

```
        @EmployeeId,  
        @Name,  
        @Salary,  
        @Department  
    );  
END;
```

5. Procedure with UPDATE

```
CREATE PROCEDURE UpdateEmployeeSalary  
    @EmployeeId INT,  
    @NewSalary DECIMAL(10,2)  
AS  
BEGIN  
    UPDATE Employees  
    SET Salary = @NewSalary  
    WHERE EmployeeId = @EmployeeId;  
END;
```

6. Procedure with DELETE

```
CREATE PROCEDURE DeleteEmployee  
    @EmployeeId INT  
AS  
BEGIN  
    DELETE FROM Employees  
    WHERE EmployeeId = @EmployeeId;  
END;
```

7. Using IF Condition

A procedure can contain business logic.

```
CREATE PROCEDURE CheckEmployeeSalary  
    @EmployeeId INT  
AS  
BEGIN  
    DECLARE @Salary DECIMAL(10,2);  
  
    SELECT @Salary = Salary  
    FROM Employees  
    WHERE EmployeeId = @EmployeeId;  
  
    IF @Salary >= 50000  
    BEGIN  
        SELECT 'High Salary' AS Result;
```

```
END
ELSE
BEGIN
    SELECT 'Low Salary' AS Result;
END
END;
```

8. Procedure with Transaction

For operations where multiple statements must succeed together:

```
CREATE PROCEDURE TransferMoney
    @FromAccount INT,
    @ToAccount INT,
    @Amount DECIMAL(18,2)
AS
BEGIN
    BEGIN TRANSACTION;

    UPDATE Accounts
    SET Balance = Balance - @Amount
    WHERE AccountId = @FromAccount;

    UPDATE Accounts
    SET Balance = Balance + @Amount
    WHERE AccountId = @ToAccount;

    COMMIT TRANSACTION;
END;
```

In production code, you would normally add `TRY...CATCH` and rollback handling.

9. Create or Alter Procedure

If the procedure already exists, SQL Server supports:

```
CREATE OR ALTER PROCEDURE GetAllEmployees
AS
BEGIN
    SELECT *
    FROM Employees;
END;
```

This is often more convenient during development because you don't need to manually drop the procedure first.

10. Procedure Naming Convention

Common naming styles include:

```
GetEmployeeById  
GetAllEmployees  
AddEmployee  
UpdateEmployee  
DeleteEmployee  
TransferMoney
```

Avoid unnecessary prefixes such as:

```
sp_GetEmployee
```

In SQL Server, `sp_` is associated with system stored procedures, so it is generally better to avoid using that prefix for your own procedures.

Complete Example

Table

```
CREATE TABLE Employees  
(  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Salary DECIMAL(10,2),  
    Department VARCHAR(50)  
);
```

Procedure

```
CREATE PROCEDURE GetEmployeesByDepartment  
    @Department VARCHAR(50)  
AS  
BEGIN  
    SELECT  
        EmployeeId,  
        Name,  
        Salary,  
        Department  
    FROM Employees  
    WHERE Department = @Department;  
END;
```

Execute

```
EXEC GetEmployeesByDepartment  
    @Department = 'IT';
```

Flow

```
CREATE PROCEDURE  
    ↓  
Store procedure in database  
    ↓  
EXEC procedure  
    ↓  
Pass parameters  
    ↓  
SQL statements execute  
    ↓  
Result returned
```

Key Interview Point

Stored Procedure = reusable database program that can accept parameters and perform operations such as SELECT, INSERT, UPDATE, DELETE, transactions, and business logic.

- Execute Procedure

EXECUTE PROCEDURE in SQL

After creating a stored procedure, you use **EXEC** or **EXECUTE** to run it.

Examples below use **SQL Server (T-SQL)**.

1. Basic Syntax

```
EXEC procedure_name;
```

or

```
EXECUTE procedure_name;
```

Both are valid.

Example

Suppose we have:

```
CREATE PROCEDURE GetAllEmployees
AS
BEGIN
    SELECT *
    FROM Employees;
END;
```

Execute it:

```
EXEC GetAllEmployees;
```

or:

```
EXECUTE GetAllEmployees;
```

2. Execute Procedure with Parameter

Procedure:

```
CREATE PROCEDURE GetEmployeeById
    @EmployeeId INT
AS
BEGIN
    SELECT *
    FROM Employees
    WHERE EmployeeId = @EmployeeId;
END;
```

Execute using a named parameter:

```
EXEC GetEmployeeById
    @EmployeeId = 101;
```

You can also use:

```
EXEC GetEmployeeById 101;
```

3. Multiple Parameters

Procedure:

```
CREATE PROCEDURE GetEmployees
    @Department VARCHAR(50),
    @MinimumSalary DECIMAL(10,2)
AS
BEGIN
    SELECT *
    FROM Employees
    WHERE Department = @Department
        AND Salary >= @MinimumSalary;
END;
```

Execute:

```
EXEC GetEmployees
    @Department = 'IT',
    @MinimumSalary = 50000;
```

Positional Parameters

You can also pass parameters by position:

```
EXEC GetEmployees 'IT', 50000;
```

Named parameters are generally easier to read and safer, especially when there are many parameters.

4. Execute Procedure with INSERT

Procedure:

```
CREATE PROCEDURE AddEmployee
    @EmployeeId INT,
    @Name VARCHAR(100),
    @Salary DECIMAL(10,2),
    @Department VARCHAR(50)
AS
BEGIN
    INSERT INTO Employees
    VALUES
    (
        @EmployeeId,
        @Name,
        @Salary,
        @Department
    );
END;
```

Execute:


```
EXEC AddEmployee
    @EmployeeId = 101,
    @Name = 'John',
    @Salary = 60000,
    @Department = 'IT';
```

5. Execute Procedure with UPDATE

Procedure:

```
CREATE PROCEDURE UpdateEmployeeSalary
    @EmployeeId INT,
    @NewSalary DECIMAL(10,2)
AS
BEGIN
    UPDATE Employees
    SET Salary = @NewSalary
    WHERE EmployeeId = @EmployeeId;
END;
```

Execute:

```
EXEC UpdateEmployeeSalary
    @EmployeeId = 101,
    @NewSalary = 75000;
```

6. Execute Procedure with Output Parameter

A procedure can return a value through an **OUTPUT** parameter.

Procedure

```
CREATE PROCEDURE GetEmployeeSalary
    @EmployeeId INT,
    @Salary DECIMAL(10,2) OUTPUT
AS
BEGIN
    SELECT @Salary = Salary
    FROM Employees
    WHERE EmployeeId = @EmployeeId;
END;
```

Execute

First create a variable:

```
DECLARE @EmployeeSalary DECIMAL(10,2);
```

Then execute:

```
EXEC GetEmployeeSalary  
    @EmployeeId = 101,  
    @Salary = @EmployeeSalary OUTPUT;
```

Display the result:

```
SELECT @EmployeeSalary AS Salary;
```

7. Execute Procedure from Another Procedure

One stored procedure can execute another stored procedure.

```
CREATE PROCEDURE ProcessEmployee  
AS  
BEGIN  
    EXEC GetAllEmployees;  
END;
```

Then:

```
EXEC ProcessEmployee;
```

8. Execute Procedure with Default Parameter

You can define a default value.

```
CREATE PROCEDURE GetEmployeesByDepartment  
    @Department VARCHAR(50) = 'IT'  
AS  
BEGIN  
    SELECT *  
    FROM Employees  
    WHERE Department = @Department;  
END;
```

Now you can execute without passing a parameter:

```
EXEC GetEmployeesByDepartment;
```

It automatically uses:

```
IT
```

Or override it:

```
EXEC GetEmployeesByDepartment  
    @Department = 'HR';
```

9. EXEC vs EXECUTE

Both are equivalent in SQL Server:

```
EXEC GetAllEmployees;
```

```
EXECUTE GetAllEmployees;
```

Most developers commonly use:

```
EXEC GetAllEmployees;
```

10. Complete Example

Create

```
CREATE PROCEDURE GetEmployeesBySalary  
    @MinimumSalary DECIMAL(10,2)  
AS  
BEGIN  
    SELECT  
        EmployeeId,  
        Name,  
        Salary,  
        Department  
    FROM Employees  
    WHERE Salary >= @MinimumSalary;  
END;
```

Execute

```
EXEC GetEmployeesBySalary
    @MinimumSalary = 50000;
```

Result

EmployeeId	Name	Salary	Department
-----	-----	-----	-----
101	John	60000	IT
102	Alice	75000	HR

Important Interview Difference

Action	Syntax
Create procedure	CREATE PROCEDURE
Run procedure	EXEC / EXECUTE
Pass parameter	@Parameter = value
Output parameter	OUTPUT
Modify procedure	ALTER PROCEDURE
Delete procedure	DROP PROCEDURE

Remember

CREATE PROCEDURE	→ Create and store the procedure
EXEC	→ Execute the procedure
@Parameter	→ Pass input
OUTPUT	→ Receive output

- User-Defined Functions (UDFs)

User-Defined Functions (UDFs) in SQL

A **User-Defined Function (UDF)** is a custom function created by the developer to perform reusable logic and **return a value or a table**.

Examples below use **SQL Server (T-SQL)**.

1. Why Use UDFs?

UDFs are useful when you have logic that needs to be reused multiple times.

For example:

```
Monthly Salary → Annual Salary
```

Instead of writing:

```
Salary * 12
```

everywhere, you can create:

```
dbo.GetAnnualSalary(Salary)
```

2. Types of UDFs

There are mainly two types:

1. Scalar-Valued Function

Returns **one value**.

```
Input → Function → Single Value
```

2. Table-Valued Function

Returns a **table**.

```
Input → Function → Table
```

Table-valued functions can further be:

- Inline Table-Valued Function
- Multi-Statement Table-Valued Function

3. Scalar-Valued UDF

A scalar function returns a single value.

Syntax

```
CREATE FUNCTION function_name  
(
```

```
    @parameter datatype
)
RETURNS return_datatype
AS
BEGIN
    -- Logic
    RETURN value;
END;
```

Example

Create a function to calculate annual salary:

```
CREATE FUNCTION GetAnnualSalary
(
    @MonthlySalary DECIMAL(10,2)
)
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @MonthlySalary * 12;
END;
```

4. Calling a Scalar Function

Unlike a stored procedure, a function can be called inside a **SELECT**.

```
SELECT dbo.GetAnnualSalary(50000);
```

Result:

```
600000
```

You can also use it with a table:

```
SELECT
    Name,
    Salary,
    dbo.GetAnnualSalary(Salary) AS AnnualSalary
FROM Employees;
```

Example result:

Name	Salary	AnnualSalary
John	50000	600000
Alice	60000	720000
Bob	75000	900000

5. UDF with Multiple Parameters

You can pass multiple parameters.

```
CREATE FUNCTION CalculateInterest
(
    @Principal DECIMAL(18,2),
    @Rate DECIMAL(5,2),
    @Years INT
)
RETURNS DECIMAL(18,2)
AS
BEGIN
    RETURN (@Principal * @Rate * @Years) / 100;
END;
```

Call it:

```
SELECT dbo.CalculateInterest(100000, 7.5, 2);
```

Result:

```
15000
```

6. UDF for String Manipulation

```
CREATE FUNCTION GetFullName
(
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50)
)
RETURNS VARCHAR(101)
AS
BEGIN
```

```
    RETURN @FirstName + ' ' + @LastName;  
END;
```

Use it:

```
SELECT dbo.GetFullName('John', 'Smith');
```

Result:

```
John Smith
```

7. Inline Table-Valued Function

An **Inline Table-Valued Function (iTVF)** returns a table using a single **SELECT**.

Syntax

```
CREATE FUNCTION function_name  
(  
    @parameter datatype  
)  
RETURNS TABLE  
AS  
RETURN  
(  
    SELECT ...  
);
```

Example

```
CREATE FUNCTION GetEmployeesByDepartment  
(  
    @Department VARCHAR(50)  
)  
RETURNS TABLE  
AS  
RETURN  
(  
    SELECT  
        EmployeeId,  
        Name,  
        Salary,  
        Department  
    FROM Employees
```



```
WHERE Department = @Department  
);
```

Call it:

```
SELECT *  
FROM dbo.GetEmployeesByDepartment('IT');
```

8. Use Table-Valued Function with WHERE

Because it returns a table, you can query it like a table:

```
SELECT *  
FROM dbo.GetEmployeesByDepartment('IT')  
WHERE Salary > 50000;
```

9. Use Table-Valued Function with JOIN

You can also join its result with another table:

```
SELECT  
    E.Name,  
    E.Salary,  
    D.DepartmentName  
FROM dbo.GetEmployeesByDepartment('IT') E  
INNER JOIN Departments D  
    ON E.Department = D.DepartmentCode;
```

10. Multi-Statement Table-Valued Function

A **Multi-Statement Table-Valued Function (MSTVF)** allows you to declare a table variable, insert data into it, perform multiple operations, and return it.

Example

```
CREATE FUNCTION GetHighSalaryEmployees  
(  
    @MinimumSalary DECIMAL(10,2)  
)  
RETURNS @Employees TABLE
```

```
(
    EmployeeId INT,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
)
AS
BEGIN

    INSERT INTO @Employees
    SELECT
        EmployeeId,
        Name,
        Salary
    FROM Employees
    WHERE Salary >= @MinimumSalary;

    RETURN;
END;
```

Call it:

```
SELECT *
FROM dbo.GetHighSalaryEmployees(50000);
```

11. UDF vs Stored Procedure

This is an important interview topic.

Feature	UDF	Stored Procedure
Must return something	Yes	No
Return scalar value	Yes	Can use output parameters/result sets
Return table	Yes	Can return result sets
Called with SELECT	Yes	No
Called with EXEC	No	Yes
Used in WHERE	Yes	No
Used in expressions	Yes	No
Parameters	Yes	Yes
Main purpose	Reusable calculation/data	Perform operations/business processes

Example

Function:

```
SELECT dbo.GetAnnualSalary(Salary)
FROM Employees;
```

Procedure:

```
EXEC GetAllEmployees;
```

12. Function vs Procedure — Simple Rule

Think of it this way:

Function

"Give me something."

```
SELECT dbo.GetAnnualSalary(50000);
```

Procedure

"Do something."

```
EXEC AddEmployee
    @EmployeeId = 101,
    @Name = 'John',
    @Salary = 50000,
    @Department = 'IT';
```

13. Important Restrictions

SQL Server UDFs have restrictions compared with stored procedures. In particular, you generally **should not use a UDF for operations that modify database state**, such as ordinary **INSERT**, **UPDATE**, or **DELETE** against permanent tables.

For example, this is not the normal use of a UDF:

```
CREATE FUNCTION AddEmployee(...)
```

Instead, use a **stored procedure** for database modifications.

14. Real-World Example

Suppose you have an e-commerce application.

You could create:

Function — Calculate Discount

```
CREATE FUNCTION CalculateDiscount
(
    @Amount DECIMAL(18,2),
    @DiscountPercentage DECIMAL(5,2)
)
RETURNS DECIMAL(18,2)
AS
BEGIN
    RETURN @Amount * @DiscountPercentage / 100;
END;
```

Use:

```
SELECT dbo.CalculateDiscount(10000, 10);
```

Result:

```
1000
```

Then calculate final price:

```
SELECT
    10000 - dbo.CalculateDiscount(10000, 10) AS FinalPrice;
```

Result:

```
9000
```

15. UDF Syntax to Remember

Scalar Function

```
CREATE FUNCTION FunctionName
(
```

```
    @Parameter datatype
)
RETURNS datatype
AS
BEGIN
    RETURN value;
END;
```

Inline Table-Valued Function

```
CREATE FUNCTION FunctionName
(
    @Parameter datatype
)
RETURNS TABLE
AS
RETURN
(
    SELECT ...
);
```

Multi-Statement Table-Valued Function

```
CREATE FUNCTION FunctionName
(
    @Parameter datatype
)
RETURNS @Result TABLE
(
    Column1 datatype,
    Column2 datatype
)
AS
BEGIN
    INSERT INTO @Result
    SELECT ...;

    RETURN;
END;
```

Key interview point: A UDF is primarily for **reusable calculations or reusable query logic**, while a stored procedure is primarily for **executing database operations or business processes**.

19. Triggers

- BEFORE Trigger

BEFORE Trigger in SQL

A **BEFORE Trigger** is a database trigger that executes **before** an **INSERT**, **UPDATE**, or **DELETE** operation takes place.

It is mainly used to **validate or modify data before it is stored**.

Important: MySQL and Oracle support **BEFORE** triggers. **SQL Server does not support BEFORE triggers**; it uses **AFTER** and **INSTEAD OF** triggers.

1. Basic Syntax — MySQL

```
CREATE TRIGGER trigger_name
BEFORE INSERT ON table_name
FOR EACH ROW
BEGIN
    -- trigger logic
END;
```

For example:

```
CREATE TABLE Employees
(
    EmployeeId INT,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

Create a BEFORE trigger:

```
CREATE TRIGGER trg_CheckSalary
BEFORE INSERT ON Employees
FOR EACH ROW
BEGIN
    IF NEW.Salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END;
```

Now try:

```
INSERT INTO Employees
VALUES (1, 'John', -5000);
```

The trigger executes **before** the insertion and rejects the data.

2. Modifying Data Before INSERT

A BEFORE trigger can also modify the value.

```
CREATE TRIGGER trg_SetSalary
BEFORE INSERT ON Employees
FOR EACH ROW
BEGIN
    IF NEW.Salary IS NULL THEN
        SET NEW.Salary = 25000;
    END IF;
END;
```

Now:

```
INSERT INTO Employees
(EmployeeId, Name, Salary)
VALUES
(1, 'John', NULL);
```

The trigger changes the salary to 25000 before the row is inserted.

Result:

EmployeeId	Name	Salary
1	John	25000

3. BEFORE UPDATE Trigger

A BEFORE trigger can also execute before an update.

```
CREATE TRIGGER trg_CheckSalaryUpdate
BEFORE UPDATE ON Employees
FOR EACH ROW
BEGIN
    IF NEW.Salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END;
```

Now:

```
UPDATE Employees
SET Salary = -1000
WHERE EmployeeId = 1;
```

The trigger prevents the invalid update.

4. NEW and OLD

In MySQL triggers, special keywords are available.

NEW

Represents the **new value**.

Example:

```
NEW.Salary
```

Used commonly with:

- INSERT
- UPDATE

OLD

Represents the **existing value** before the change.

Used commonly with:

- UPDATE
- DELETE

Example:

```
UPDATE
```

```
OLD.Salary → 30000
```

```
NEW.Salary → 40000
```

5. BEFORE INSERT

```
INSERT request
  ↓
BEFORE Trigger
  ↓
Validate / Modify
  ↓
Insert row
```

Example:


```
INSERT INTO Employees  
VALUES (1, 'John', 50000);
```

The trigger gets a chance to inspect or modify the data **before** it reaches the table.

6. BEFORE UPDATE

```
UPDATE request  
  ↓  
BEFORE Trigger  
  ↓  
Validate / Modify  
  ↓  
Update row
```

For example:

```
UPDATE Employees  
SET Salary = 60000  
WHERE EmployeeId = 1;
```

The trigger can check whether the new salary is acceptable.

7. BEFORE DELETE

A BEFORE DELETE trigger can perform logic before deleting a row.

For example:

```
CREATE TRIGGER trg_PreventDelete  
BEFORE DELETE ON Employees  
FOR EACH ROW  
BEGIN  
  IF OLD.EmployeeId = 1 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'This employee cannot be deleted';  
  END IF;  
END;
```

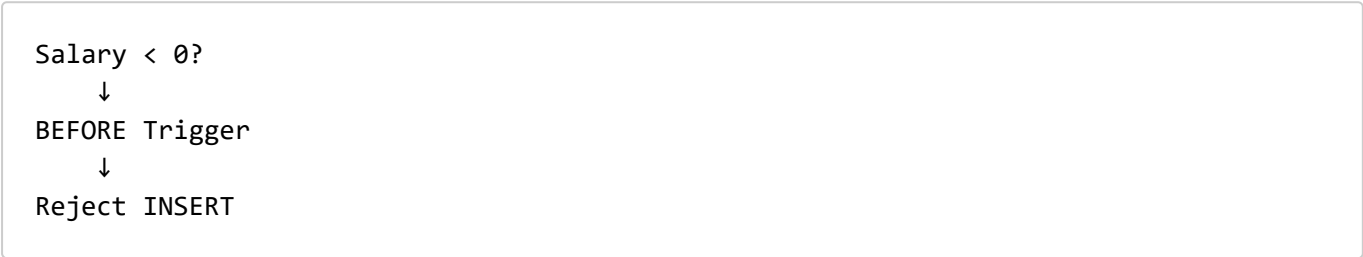
Here **OLD** represents the row that is about to be deleted.

Common Uses

BEFORE triggers are useful for:

- Validating data
- Preventing invalid values
- Automatically modifying values
- Normalizing data
- Enforcing business rules
- Preventing certain operations

For example:



BEFORE vs AFTER

BEFORE Trigger	AFTER Trigger
Runs before operation	Runs after operation
Can validate data	Good for auditing
Can modify incoming values	Used for related operations
Can prevent operation	Operation has already occurred
NEW can be modified in MySQL	Usually used to react to completed change

Interview Definition

A **BEFORE trigger** is a trigger that automatically executes before an **INSERT, UPDATE, or DELETE operation**, usually to validate or modify data before the operation is completed.

- AFTER Trigger

AFTER Trigger in SQL

An **AFTER Trigger** is a database trigger that executes **after** an **INSERT, UPDATE, or DELETE** operation has successfully occurred.

It is commonly used for:

- Auditing changes
- Maintaining history tables
- Updating related tables
- Logging operations

- Maintaining summary data

SQL Server supports AFTER triggers. In SQL Server, they are also called **AFTER triggers** or **FOR triggers**.

1. Basic Syntax

SQL Server

```
CREATE TRIGGER trigger_name
ON table_name
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    -- Trigger logic
END;
```

For example:

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

Create an audit table:

```
CREATE TABLE EmployeeAudit
(
    AuditId INT IDENTITY PRIMARY KEY,
    EmployeeId INT,
    Action VARCHAR(20),
    ActionDate DATETIME
);
```

Now create an AFTER INSERT trigger:

```
CREATE TRIGGER trg_Employee_Insert
ON Employees
AFTER INSERT
AS
BEGIN
    INSERT INTO EmployeeAudit
    (
        EmployeeId,
        Action,
```

```
        ActionDate
    )
    SELECT
        EmployeeId,
        'INSERT',
        GETDATE()
    FROM inserted;
END;
```

2. How It Works

When you execute:

```
INSERT INTO Employees
VALUES (1, 'John', 50000);
```

The process is:

```
INSERT
↓
Employee row inserted
↓
AFTER Trigger
↓
Audit record inserted
```

The audit table will contain:

AuditId	EmployeeId	Action	ActionDate
1	1	INSERT	2026-08-14 ...

3. **inserted** and **deleted** Tables

SQL Server provides two special temporary logical tables inside triggers:

inserted

Contains the **new data**.

Used for:

- **INSERT**
- **UPDATE**

deleted

Contains the **old data**.

Used for:

- **DELETE**
- **UPDATE**

For an update:

Before UPDATE

deleted

Salary = 50000

After UPDATE

inserted

Salary = 60000

So:

deleted → Old values

inserted → New values

4. AFTER INSERT Trigger

Example:

```
CREATE TRIGGER trg_AfterEmployeeInsert
ON Employees
AFTER INSERT
AS
BEGIN
    INSERT INTO EmployeeAudit
    (
        EmployeeId,
        Action,
        ActionDate
    )
    SELECT
        EmployeeId,
        'INSERT',
        GETDATE()
```

```
FROM inserted;  
END;
```

When:

```
INSERT INTO Employees  
VALUES (2, 'David', 60000);
```

The trigger automatically creates an audit record.

5. AFTER UPDATE Trigger

Suppose we want to track salary changes.

Create an audit table:

```
CREATE TABLE SalaryAudit  
(  
    AuditId INT IDENTITY PRIMARY KEY,  
    EmployeeId INT,  
    OldSalary DECIMAL(10,2),  
    NewSalary DECIMAL(10,2),  
    ChangedDate DATETIME  
);
```

Create the trigger:

```
CREATE TRIGGER trg_SalaryUpdate  
ON Employees  
AFTER UPDATE  
AS  
BEGIN  
    INSERT INTO SalaryAudit  
    (  
        EmployeeId,  
        OldSalary,  
        NewSalary,  
        ChangedDate  
    )  
    SELECT  
        d.EmployeeId,  
        d.Salary,  
        i.Salary,  
        GETDATE()  
    FROM deleted d  
    INNER JOIN inserted i  
        ON d.EmployeeId = i.EmployeeId
```

```
WHERE d.Salary <> i.Salary;  
END;
```

Now:

```
UPDATE Employees  
SET Salary = 70000  
WHERE EmployeeId = 1;
```

The trigger records:

EmployeeId	OldSalary	NewSalary
1	50000	70000

6. AFTER DELETE Trigger

We can also record deleted employees.

```
CREATE TRIGGER trg_EmployeeDelete  
ON Employees  
AFTER DELETE  
AS  
BEGIN  
    INSERT INTO EmployeeAudit  
    (  
        EmployeeId,  
        Action,  
        ActionDate  
    )  
    SELECT  
        EmployeeId,  
        'DELETE',  
        GETDATE()  
    FROM deleted;  
END;
```

Now:

```
DELETE FROM Employees  
WHERE EmployeeId = 1;
```

The deleted employee's ID is recorded in `EmployeeAudit`.

7. Important: Triggers Handle Multiple Rows

One of the most important SQL Server interview points is that a trigger executes **once per statement**, not once per row.

For example:

```
UPDATE Employees
SET Salary = Salary * 1.10
WHERE Department = 'IT';
```

Suppose 100 employees are updated.

The trigger executes **once**, and the **inserted/deleted** tables contain all affected rows.

Therefore, avoid code like:

```
SELECT @EmployeeId = EmployeeId
FROM inserted;
```

if you expect multiple rows.

Prefer set-based operations:

```
INSERT INTO EmployeeAudit
(
    EmployeeId,
    Action,
    ActionDate
)
SELECT
    EmployeeId,
    'UPDATE',
    GETDATE()
FROM inserted;
```

8. AFTER Trigger Flow

```
INSERT / UPDATE / DELETE
    ↓
Table Operation
    ↓
Data Changed
    ↓
AFTER Trigger
```


↓

Trigger Logic

9. BEFORE vs AFTER vs INSTEAD OF

Trigger	Execution
BEFORE	Before the operation
AFTER	After the operation
INSTEAD OF	Replaces the operation

For SQL Server:



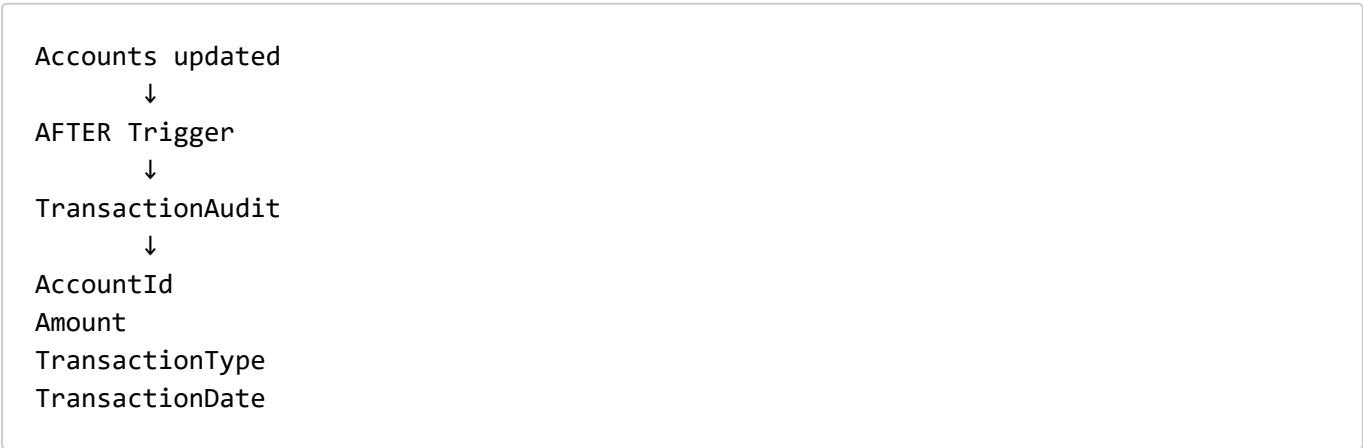
Real-World Example

Suppose you have a banking application:



A customer transfers ₹10,000.

After the transaction:



This allows you to maintain an audit trail automatically.

Advantages

- Automatic execution
- Useful for auditing
- Centralizes certain database rules
- Maintains history automatically
- Keeps related data synchronized

Disadvantages

- Can make database logic difficult to understand
- May negatively affect performance
- Can create unexpected side effects
- Debugging can be difficult
- Excessive trigger usage can make applications harder to maintain

Interview Definition

An AFTER trigger is a database trigger that automatically executes after an INSERT, UPDATE, or DELETE operation has been performed. It is commonly used for auditing, logging, maintaining history, and synchronizing related data.

- INSTEAD OF Trigger

INSTEAD OF Trigger in SQL

An **INSTEAD OF Trigger** executes **instead of** the original **INSERT**, **UPDATE**, or **DELETE** operation.

In simple words:

The database receives the operation, but the trigger intercepts it and executes its own logic instead.

It is particularly useful with **views**, especially when you want to make a complex view updatable.

1. Basic Syntax — SQL Server

```
CREATE TRIGGER trigger_name
ON table_or_view
INSTEAD OF INSERT, UPDATE, DELETE
AS
BEGIN
    -- Custom logic
END;
```

For example:

```
CREATE TRIGGER trg_EmployeeDelete
ON Employees
INSTEAD OF DELETE
AS
BEGIN
    -- Custom delete logic
END;
```

2. Simple Example

Create a table:

```
CREATE TABLE Employees
(
    EmployeeId INT PRIMARY KEY,
    Name VARCHAR(100),
    Department VARCHAR(100)
);
```

Insert some data:

```
INSERT INTO Employees
VALUES
(1, 'John', 'IT'),
(2, 'David', 'HR');
```

Now create a view:

```
CREATE VIEW EmployeeView
AS
SELECT
    EmployeeId,
    Name,
    Department
FROM Employees;
```

3. INSTEAD OF INSERT

Create a trigger on the view:

```
CREATE TRIGGER trg_EmployeeViewInsert
ON EmployeeView
INSTEAD OF INSERT
AS
BEGIN
    INSERT INTO Employees
    (
        EmployeeId,
        Name,
        Department
    )
    SELECT
        EmployeeId,
        Name,
        Department
    FROM inserted;
END;
```

Now execute:

```
INSERT INTO EmployeeView
VALUES (3, 'Mike', 'Finance');
```

Normally, the INSERT would target the view.

But because of the trigger:

```
INSERT INTO EmployeeView
    ↓
INSTEAD OF Trigger
    ↓
Original INSERT is intercepted
    ↓
Trigger executes
    ↓
INSERT INTO Employees
```

The data is ultimately inserted into the **Employees** table.

4. INSTEAD OF DELETE

Suppose we don't want employees to be physically deleted.

Instead, we want to mark them as inactive.

First add a column:

```
ALTER TABLE Employees
ADD IsActive BIT DEFAULT 1;
```

Now create a trigger:

```
CREATE TRIGGER trg_PreventEmployeeDelete
ON Employees
INSTEAD OF DELETE
AS
BEGIN
    UPDATE e
    SET IsActive = 0
    FROM Employees e
    INNER JOIN deleted d
        ON e.EmployeeId = d.EmployeeId;
END;
```

Now:

```
DELETE FROM Employees
WHERE EmployeeId = 1;
```

The employee is **not actually deleted**.

Instead:

```
DELETE request
    ↓
INSTEAD OF Trigger
    ↓
DELETE cancelled/intercepted
    ↓
IsActive = 0
```

This technique is sometimes called a **soft delete**.

5. **inserted** and **deleted**

SQL Server provides two logical tables inside triggers.

Logical Table	Contains
inserted	New values
deleted	Old/deleted values

For example, with:

```
DELETE FROM Employees
WHERE EmployeeId = 1;
```

The `deleted` table contains:

EmployeeId	Name	Department
1	John	IT

The trigger can use this information:

```
SELECT *
FROM deleted;
```

6. INSTEAD OF UPDATE

An INSTEAD OF trigger can also intercept an update.

```
CREATE TRIGGER trg_EmployeeUpdate
ON EmployeeView
INSTEAD OF UPDATE
AS
BEGIN
    UPDATE e
    SET
        Name = i.Name,
        Department = i.Department
    FROM Employees e
    INNER JOIN inserted i
        ON e.EmployeeId = i.EmployeeId;
END;
```

Now:

```
UPDATE EmployeeView
SET Department = 'Finance'
WHERE EmployeeId = 1;
```

Instead of directly updating the view, the trigger controls how the underlying table is updated.

7. Why Use INSTEAD OF Triggers?

1. Complex Views

Suppose a view joins multiple tables:

```
Employees
  +
Departments
  +
Locations
  ↓
EmployeeView
```

Directly updating such a view may not be possible or may not behave as desired.

An **INSTEAD OF** trigger can determine how the update should be distributed among the underlying tables.

2. Soft Delete

Instead of:

```
DELETE FROM Employees;
```

the trigger can perform:

```
UPDATE Employees
SET IsActive = 0;
```

3. Custom Business Logic

For example:

```
DELETE Customer
  ↓
INSTEAD OF Trigger
  ↓
Check outstanding payments
  ↓
If payment exists → prevent deletion
Otherwise → delete
```

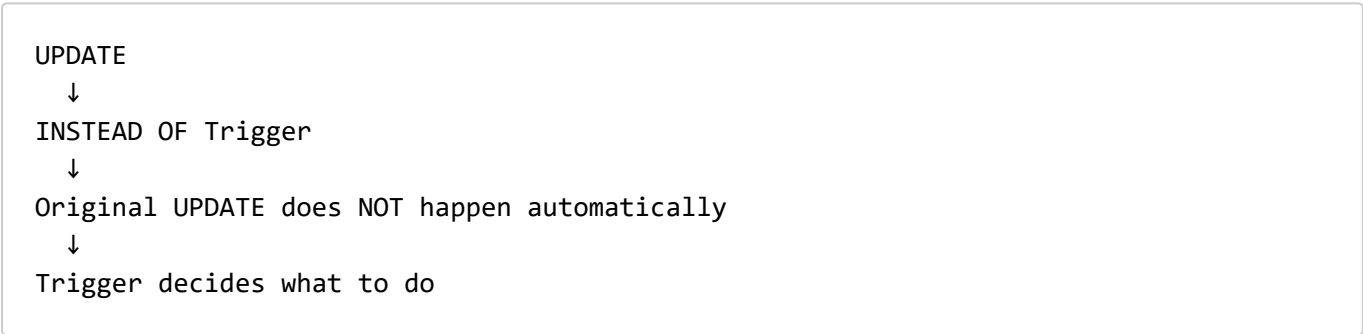
8. AFTER vs INSTEAD OF

This is an important interview distinction.

AFTER Trigger



INSTEAD OF Trigger



9. Key Difference

Feature	AFTER	INSTEAD OF
Original operation	Happens	Intercepted
Trigger execution	After operation	Instead of operation
Common use	Auditing	Custom operation
Commonly used on tables	Yes	Yes
Commonly used on views	Sometimes	Very common
Must perform the operation manually?	No	Usually yes

10. Important Interview Point

An **INSTEAD OF** trigger does not automatically perform the original operation.

For example:


```
DELETE FROM Employees
WHERE EmployeeId = 1;
```

If an **INSTEAD OF DELETE** trigger exists, SQL Server executes the trigger **instead of performing the DELETE**.

If you want the row deleted, the trigger must explicitly execute the required **DELETE**.

Simple Memory Trick

```
BEFORE
→ Do something BEFORE the operation

AFTER
→ Do something AFTER the operation

INSTEAD OF
→ Do something IN PLACE OF the operation
```

Interview Definition

An INSTEAD OF trigger is a trigger that intercepts an INSERT, UPDATE, or DELETE operation and executes custom logic in place of the original operation. It is commonly used with views, complex data modifications, and soft-delete scenarios.

20. Transactions (TCL)

Transactions (TCL) in SQL

A **Transaction** is a group of one or more SQL operations treated as a **single unit of work**.

The main goal is to ensure that either **all required operations succeed** or the changes are **undone**.

Example: Bank transfer

```
Account A: ₹50,000
Account B: ₹20,000

Transfer ₹10,000

1. Deduct ₹10,000 from A
2. Add ₹10,000 to B
```

Both operations should succeed together.

```
BEGIN
↓
```

```
Deduct from A
↓
Add to B
↓
COMMIT
```

If something fails:

```
BEGIN
↓
Deduct from A
↓
ERROR
↓
ROLLBACK
```

1. BEGIN

BEGIN starts a transaction.

SQL Server:

```
BEGIN TRANSACTION;
```

Short form:

```
BEGIN TRAN;
```

Example:

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 10000
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 10000
WHERE AccountId = 2;
```

The changes are not permanently committed yet.

2. COMMIT

COMMIT permanently saves the changes made by the transaction.

```
COMMIT TRANSACTION;
```

Example:

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 10000
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 10000
WHERE AccountId = 2;

COMMIT TRANSACTION;
```

After **COMMIT**:

```
Changes
  ↓
Permanently saved
```

You generally cannot use **ROLLBACK** to undo those changes after the transaction has been committed.

3. ROLLBACK

ROLLBACK cancels the transaction and reverses changes made during the transaction.

```
ROLLBACK TRANSACTION;
```

Example:

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 10000
WHERE AccountId = 1;

-- Something went wrong

ROLLBACK TRANSACTION;
```

The balance change is undone.

Flow

```
BEGIN
↓
UPDATE
↓
Error
↓
ROLLBACK
↓
Changes undone
```

4. SAVEPOINT

A **SAVEPOINT** creates a point inside a transaction to which you can partially roll back.

In SQL Server, the equivalent syntax is:

```
SAVE TRANSACTION SavePointName;
```

Example:

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 10000
WHERE AccountId = 1;

SAVE TRANSACTION TransferStep;

UPDATE Accounts
SET Balance = Balance + 10000
WHERE AccountId = 2;

ROLLBACK TRANSACTION TransferStep;

COMMIT TRANSACTION;
```

Here, the rollback goes back to the savepoint instead of rolling back the entire transaction.

5. Savepoint Example

Imagine:

```

BEGIN TRANSACTION
  ↓
Operation 1
  ↓
SAVEPOINT A
  ↓
Operation 2
  ↓
Operation 3
  ↓
ROLLBACK TO A
  ↓
Operation 2 & 3 undone
  ↓
Operation 1 remains
  ↓
COMMIT

```

This is useful when a transaction contains multiple logical steps and you only want to undo part of the work.

6. Complete Example

Suppose we have:

```

CREATE TABLE Accounts
(
  AccountId INT PRIMARY KEY,
  AccountHolder VARCHAR(100),
  Balance DECIMAL(10,2)
);

```

Data:

AccountId	AccountHolder	Balance
1	John	50000
2	David	30000

Perform a transfer:

```

BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 10000

```

```
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 10000
WHERE AccountId = 2;

COMMIT TRANSACTION;
```

Final result:

```
John  → ₹40,000
David → ₹40,000
```

7. Transaction with ROLLBACK

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 10000
WHERE AccountId = 1;

-- Problem occurs

ROLLBACK TRANSACTION;
```

Result:

```
Before: ₹50,000
After:  ₹50,000
```

The update was undone.

8. Transaction with Error Handling

In SQL Server, transactions are commonly combined with `TRY...CATCH`.

```
BEGIN TRY

    BEGIN TRANSACTION;

    UPDATE Accounts
    SET Balance = Balance - 10000
    WHERE AccountId = 1;
```

```
UPDATE Accounts
SET Balance = Balance + 10000
WHERE AccountId = 2;

COMMIT TRANSACTION;

END TRY
BEGIN CATCH

    ROLLBACK TRANSACTION;

    THROW;

END CATCH;
```

This is a common pattern in **ASP.NET Core + SQL Server** applications.

9. TCL Commands

Command	Purpose
BEGIN	Starts a transaction
COMMIT	Permanently saves changes
ROLLBACK	Undoes transaction changes
SAVEPOINT	Creates a point for partial rollback

10. Transaction and ACID

Transactions are closely related to the **ACID** properties.

A — Atomicity

All operations succeed or none do.

```
Transfer:
Debit + Credit
  ↓
Both succeed OR both rollback
```

C — Consistency

The database remains in a valid state.

I — Isolation

Transactions should not improperly interfere with each other.

D — Durability

After **COMMIT**, the changes survive failures.

Easy Way to Remember

```
BEGIN
↓
Start work
↓
SAVEPOINT
↓
More work
↓
Everything OK?
├── YES → COMMIT
└── NO  → ROLLBACK
```

Interview Definition

A transaction is a logical unit of work consisting of one or more SQL operations. TCL commands such as **BEGIN, **COMMIT**, **ROLLBACK**, and **SAVEPOINT** are used to control the transaction and maintain data consistency.**

- **BEGIN**

BEGIN in SQL

BEGIN is used to **start a transaction**. A transaction groups multiple SQL operations into a single logical unit of work.

In **SQL Server**, you normally use:

```
BEGIN TRANSACTION;
```

or the shorter form:

```
BEGIN TRAN;
```

1. Basic Example

Suppose we have an **Accounts** table:


```
CREATE TABLE Accounts
(
    AccountId INT PRIMARY KEY,
    Name VARCHAR(100),
    Balance DECIMAL(10,2)
);
```

Start a transaction:

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 10000
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 10000
WHERE AccountId = 2;

COMMIT TRANSACTION;
```

The two **UPDATE** statements are treated as part of the same transaction.

2. Why Use **BEGIN**?

Without a transaction:

```
Debit Account A
  ↓
ERROR
  ↓
Credit Account B not executed
```

This could leave the database in an incorrect state.

With a transaction:

```
BEGIN
  ↓
Debit Account A
  ↓
Credit Account B
  ↓
COMMIT
```

If something goes wrong:

```
BEGIN
↓
Debit Account A
↓
ERROR
↓
ROLLBACK
```

The previous changes can be undone.

3. BEGIN + COMMIT

```
BEGIN TRANSACTION;

INSERT INTO Employees
(EmployeeId, Name, Salary)
VALUES
(1, 'John', 50000);

UPDATE Employees
SET Salary = 55000
WHERE EmployeeId = 1;

COMMIT TRANSACTION;
```

After **COMMIT**, the changes are permanently saved.

4. BEGIN + ROLLBACK

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 100000
WHERE EmployeeId = 1;

ROLLBACK TRANSACTION;
```

The salary change is undone.

Conceptually:

```
Before BEGIN
Salary = 50000

↓
```

```
BEGIN
```

↓

```
UPDATE
```

```
Salary = 100000
```

↓

```
ROLLBACK
```

↓

```
Salary = 50000
```

5. **BEGIN** with Error Handling

A common SQL Server pattern is:

```
BEGIN TRY
```

```
    BEGIN TRANSACTION;
```

```
    UPDATE Accounts
```

```
    SET Balance = Balance - 10000
```

```
    WHERE AccountId = 1;
```

```
    UPDATE Accounts
```

```
    SET Balance = Balance + 10000
```

```
    WHERE AccountId = 2;
```

```
    COMMIT TRANSACTION;
```

```
END TRY
```

```
BEGIN CATCH
```

```
    IF @@TRANCOUNT > 0
```

```
        ROLLBACK TRANSACTION;
```

```
    THROW;
```

```
END CATCH;
```

Here:

- **BEGIN TRANSACTION** → starts the transaction
- **COMMIT** → saves the changes
- **ROLLBACK** → reverses the changes if an error occurs
- **@@TRANCOUNT** → checks whether a transaction is active

6. BEGIN vs BEGIN TRANSACTION

Be careful: **BEGIN** by itself is not universally the transaction command.

For SQL Server:

```
BEGIN TRANSACTION;
```

For MySQL:

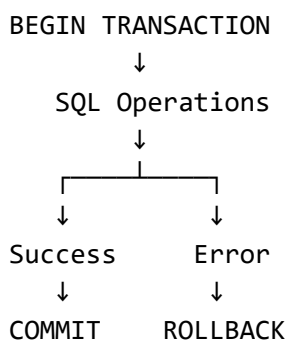
```
START TRANSACTION;
```

For Oracle:

Transactions generally begin implicitly when DML occurs; you don't typically use **BEGIN TRANSACTION** in the same way as SQL Server.

So the exact syntax depends on the database system.

Transaction Flow



Interview Definition

BEGIN TRANSACTION starts a transaction and marks the beginning of a group of SQL operations that can later be permanently saved using **COMMIT** or undone using **ROLLBACK**.

- **COMMIT**

COMMIT in SQL

COMMIT is used to **permanently save all changes made during the current transaction**.

In SQL Server:

```
COMMIT TRANSACTION;
```

Short form:

```
COMMIT;
```

1. Basic Example

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 10000
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 10000
WHERE AccountId = 2;

COMMIT TRANSACTION;
```

The sequence is:

```
BEGIN
  ↓
UPDATE Account 1
  ↓
UPDATE Account 2
  ↓
COMMIT
  ↓
Changes permanently saved
```

2. What Happens Before COMMIT?

Consider:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 60000
WHERE EmployeeId = 1;
```

The update is part of the active transaction.

Then:

```
COMMIT TRANSACTION;
```

The transaction is completed and the changes are committed.

Conceptually:

```
Before COMMIT
  ↓
Transaction changes
  ↓
COMMIT
  ↓
Changes become permanent
```

3. COMMIT vs ROLLBACK

COMMIT

Saves the changes:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 70000
WHERE EmployeeId = 1;

COMMIT;
```

Result:

```
Salary = 70000
```

ROLLBACK

Undoes the changes:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 70000
WHERE EmployeeId = 1;

ROLLBACK;
```

Result:

Salary = previous value

4. Bank Transfer Example

Suppose:

Account A = ₹50,000
Account B = ₹20,000

Transfer ₹10,000:

```
BEGIN TRANSACTION;  
  
UPDATE Accounts  
SET Balance = Balance - 10000  
WHERE AccountId = 1;  
  
UPDATE Accounts  
SET Balance = Balance + 10000  
WHERE AccountId = 2;  
  
COMMIT;
```

Final balances:

Account A = ₹40,000
Account B = ₹30,000

Both operations are committed together.

5. COMMIT with Error Handling

In SQL Server:

```
BEGIN TRY  
  
    BEGIN TRANSACTION;  
  
    UPDATE Accounts  
    SET Balance = Balance - 10000
```

```
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 10000
WHERE AccountId = 2;

COMMIT;

END TRY
BEGIN CATCH

    IF @@TRANCOUNT > 0
        ROLLBACK;

    THROW;

END CATCH;
```

Successful execution

```
BEGIN
↓
Operations
↓
COMMIT
↓
Success
```

Error occurs

```
BEGIN
↓
Operation
↓
ERROR
↓
ROLLBACK
↓
Changes undone
```

6. Important Point

After:

```
COMMIT;
```


the transaction is completed.

You generally cannot then do:

```
ROLLBACK;
```

to undo the already committed transaction.

For example:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 80000
WHERE EmployeeId = 1;

COMMIT;

-- Transaction has already completed
```

If you later want to change the salary back, you would need to perform another transaction/update.

7. COMMIT and ACID

COMMIT is especially important for **Durability**.

The ACID properties are:

```
A → Atomicity
C → Consistency
I → Isolation
D → Durability
```

After a successful COMMIT, the transaction's changes are considered committed and should persist even if the system subsequently fails.

Simple Memory Trick

```
BEGIN
↓
Do work
↓
Everything successful?
↓
YES
```

↓
COMMIT
↓
Save permanently

Interview Definition

COMMIT permanently saves all changes made during the current transaction and ends the transaction. If the transaction is committed, its changes cannot normally be undone using **ROLLBACK**.

- **ROLLBACK**

ROLLBACK in SQL

ROLLBACK is used to **undo changes made during the current transaction**.

It is commonly used when an error occurs or when a transaction should not be completed.

In SQL Server:

```
ROLLBACK TRANSACTION;
```

or simply:

```
ROLLBACK;
```

1. Basic Example

```
BEGIN TRANSACTION;  
  
UPDATE Employees  
SET Salary = 70000  
WHERE EmployeeId = 1;  
  
ROLLBACK TRANSACTION;
```

The salary change is undone.

If the salary was originally:

```
50000
```

After the **UPDATE**:

70000

After **ROLLBACK**:

50000

2. Transaction Flow

```
BEGIN TRANSACTION
  ↓
UPDATE
  ↓
DELETE
  ↓
INSERT
  ↓
ERROR
  ↓
ROLLBACK
  ↓
All transaction changes undone
```

3. Bank Transfer Example

Suppose:

Account A = ₹50,000
Account B = ₹20,000

We want to transfer ₹10,000.

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 10000
WHERE AccountId = 1;

UPDATE Accounts
SET Balance = Balance + 10000
WHERE AccountId = 2;
```

```
ROLLBACK;
```

Because we used `ROLLBACK`, both changes are undone.

Final result:

```
Account A = ₹50,000
Account B = ₹20,000
```

4. `ROLLBACK` After an Error

A common SQL Server pattern is:

```
BEGIN TRY

    BEGIN TRANSACTION;

    UPDATE Accounts
    SET Balance = Balance - 10000
    WHERE AccountId = 1;

    -- Some operation fails here

    UPDATE Accounts
    SET Balance = Balance + 10000
    WHERE AccountId = 2;

    COMMIT;

END TRY
BEGIN CATCH

    ROLLBACK;

    THROW;

END CATCH;
```

If an error occurs:

```
BEGIN
↓
Operation 1
↓
Operation 2
↓
```

```
ERROR
↓
CATCH
↓
ROLLBACK
↓
Undo transaction
```

This prevents a partially completed operation.

5. ROLLBACK vs COMMIT

COMMIT	ROLLBACK
Saves changes	Undoes changes
Ends transaction successfully	Cancels transaction
Makes changes permanent	Restores previous state
Used when everything succeeds	Used when something fails

Example:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 60000
WHERE EmployeeId = 1;

COMMIT;
```

Result:

```
Salary = 60000
```

With rollback:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 60000
WHERE EmployeeId = 1;

ROLLBACK;
```

Result:

```
Salary = previous value
```

6. ROLLBACK to a Savepoint

You can also roll back only part of a transaction.

In SQL Server:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 60000
WHERE EmployeeId = 1;

SAVE TRANSACTION SavePoint1;

UPDATE Employees
SET Salary = 70000
WHERE EmployeeId = 2;

ROLLBACK TRANSACTION SavePoint1;

COMMIT TRANSACTION;
```

Flow:

```
BEGIN
↓
Update Employee 1
↓
SAVEPOINT
↓
Update Employee 2
↓
ROLLBACK TO SAVEPOINT
↓
Employee 2 update undone
↓
Employee 1 update remains
↓
COMMIT
```

So **ROLLBACK** can be used at two levels:

```
ROLLBACK
↓
```

Undo entire transaction

```
ROLLBACK TRANSACTION SavePoint1
```

↓

Undo changes after SavePoint1

7. Important Point

ROLLBACK only affects the **active transaction**.

For example:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 60000
WHERE EmployeeId = 1;

COMMIT;

ROLLBACK;
```

The second **ROLLBACK** cannot undo the already committed update.

Why?

```
BEGIN
↓
UPDATE
↓
COMMIT
↓
Transaction finished
```

The rollback came too late.

8. Real-World Example

In an e-commerce application:

```
Create Order
↓
Reduce Inventory
```

↓
Create Payment

Suppose payment fails:

```
BEGIN
  ↓
Create Order
  ↓
Reduce Inventory
  ↓
Payment FAILED
  ↓
ROLLBACK
```

The order and inventory changes can be rolled back together.

This is one of the major reasons transactions are important in applications such as **banking, payments, orders, and inventory systems**.

Easy Memory Trick

```
COMMIT → "Keep my changes"
ROLLBACK → "Undo my changes"
```

Interview Definition

ROLLBACK cancels the current transaction and undoes the changes made since the transaction began or since a specified savepoint. It is commonly used when an error occurs or a transaction cannot be completed successfully.

- **SAVEPOINT**

SAVEPOINT in SQL

A **SAVEPOINT** creates a named point inside a transaction.

It allows you to **partially roll back a transaction** without undoing all the work done since the transaction began.

Think of it as a **checkpoint** inside a transaction.

1. Basic Concept


```
BEGIN TRANSACTION
  ↓
Operation 1
  ↓
SAVEPOINT A
  ↓
Operation 2
  ↓
Operation 3
  ↓
ROLLBACK TO A
  ↓
Operation 2 & 3 undone
Operation 1 remains
```

2. SQL Server Syntax

In SQL Server, the syntax is:

```
SAVE TRANSACTION SavePointName;
```

For example:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 60000
WHERE EmployeeId = 1;

SAVE TRANSACTION SalaryUpdate;

UPDATE Employees
SET Salary = 70000
WHERE EmployeeId = 2;

ROLLBACK TRANSACTION SalaryUpdate;

COMMIT TRANSACTION;
```

The rollback returns the transaction to the **SalaryUpdate** point.

3. Example

Suppose we start with:

Employee 1 → ₹50,000
Employee 2 → ₹60,000
Employee 3 → ₹70,000

Transaction:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 55000
WHERE EmployeeId = 1;

SAVE TRANSACTION Point1;

UPDATE Employees
SET Salary = 65000
WHERE EmployeeId = 2;

UPDATE Employees
SET Salary = 75000
WHERE EmployeeId = 3;

ROLLBACK TRANSACTION Point1;

COMMIT TRANSACTION;
```

Result:

Employee 1 → ₹55,000
Employee 2 → ₹60,000
Employee 3 → ₹70,000

Why?

```
BEGIN
↓
Employee 1 updated
↓
SAVEPOINT Point1
↓
Employee 2 updated
↓
Employee 3 updated
↓
ROLLBACK TO Point1
↓
Employee 2 update removed
Employee 3 update removed
Employee 1 update remains
```

↓
COMMIT

4. SAVEPOINT VS ROLLBACK

A **SAVEPOINT** does not undo anything by itself.

It only creates a checkpoint.

```
SAVE TRANSACTION Point1;
```

Then you decide whether to roll back to it:

```
ROLLBACK TRANSACTION Point1;
```

So:

```
SAVEPOINT
  ↓
Creates checkpoint

ROLLBACK TO SAVEPOINT
  ↓
Returns to checkpoint
```

5. Bank Transfer Example

Imagine a transaction involving several operations:

```
BEGIN TRANSACTION;

-- Step 1
UPDATE Accounts
SET Balance = Balance - 10000
WHERE AccountId = 1;

SAVE TRANSACTION AfterDebit;

-- Step 2
INSERT INTO Transactions
(AccountId, Amount)
VALUES (1, 10000);
```

```
-- Step 3
-- Some operation fails

ROLLBACK TRANSACTION AfterDebit;

COMMIT TRANSACTION;
```

The rollback removes changes made **after** **AfterDebit**, while keeping the changes made before the savepoint.

6. Multiple Savepoints

You can create multiple savepoints:

```
BEGIN TRANSACTION;

INSERT INTO Orders
VALUES (1, 1000);

SAVE TRANSACTION OrderCreated;

UPDATE Inventory
SET Quantity = Quantity - 1
WHERE ProductId = 10;

SAVE TRANSACTION InventoryUpdated;

INSERT INTO Payments
VALUES (1, 1000);

ROLLBACK TRANSACTION InventoryUpdated;

COMMIT TRANSACTION;
```

Conceptually:

```
BEGIN
↓
Create Order
↓
SAVEPOINT OrderCreated
↓
Update Inventory
↓
SAVEPOINT InventoryUpdated
↓
Create Payment
```

↓

ROLLBACK TO InventoryUpdated

Only work after **InventoryUpdated** is rolled back.

7. SAVEPOINT vs COMMIT vs ROLLBACK

Command	Purpose
BEGIN	Starts transaction
SAVEPOINT	Creates a checkpoint
ROLLBACK	Undoes transaction changes
ROLLBACK TO SAVEPOINT	Undoes changes after checkpoint
COMMIT	Permanently saves transaction

8. Important SQL Server Syntax

For **SQL Server**, remember:

BEGIN TRANSACTION;

SAVE TRANSACTION MyPoint;

ROLLBACK TRANSACTION MyPoint;

COMMIT TRANSACTION;

For databases such as **MySQL** and **PostgreSQL**, you commonly see:

SAVEPOINT MyPoint;

ROLLBACK TO SAVEPOINT MyPoint;

So syntax varies slightly by database.

9. Important Interview Point

A **SAVEPOINT** **does not end the transaction**.

For example:

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = 60000
WHERE EmployeeId = 1;

SAVE TRANSACTION Point1;

UPDATE Employees
SET Salary = 70000
WHERE EmployeeId = 2;

ROLLBACK TRANSACTION Point1;

-- Transaction is still active

COMMIT TRANSACTION;
```

The **ROLLBACK** to the savepoint does **not** terminate the entire transaction.

Easy Memory Trick

```
BEGIN
  ↓
Start transaction

SAVEPOINT
  ↓
Create checkpoint

ROLLBACK TO SAVEPOINT
  ↓
Go back to checkpoint

COMMIT
  ↓
Save everything remaining
```

Interview Definition

A SAVEPOINT is a named checkpoint within a transaction that allows you to roll back only part of the transaction while keeping the transaction active.

21. SQL Security (DCL)

- **GRANT**

GRANT in SQL

GRANT is a **DCL (Data Control Language)** command used to **give permissions or privileges** to a user or role.

Basic Syntax

```
GRANT permission
ON object
TO user_or_role;
```

Example

Suppose we have:

```
CREATE TABLE Employees (
    Id INT,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

Give **John** permission to read the table:

```
GRANT SELECT
ON Employees
TO John;
```

Now **John** can execute:

```
SELECT * FROM Employees;
```

Grant Multiple Permissions

```
GRANT SELECT, INSERT, UPDATE
ON Employees
TO John;
```

John can now:

```
SELECT * FROM Employees;

INSERT INTO Employees
VALUES (1, 'Rahul', 50000);

UPDATE Employees
```

```
SET Salary = 55000  
WHERE Id = 1;
```

Common Permissions

Permission	Allows
SELECT	Read data
INSERT	Add new rows
UPDATE	Modify existing rows
DELETE	Delete rows
EXECUTE	Execute procedures/functions
ALL / ALL PRIVILEGES	All supported privileges

Grant Permission on a View

```
GRANT SELECT  
ON EmployeeView  
TO ReportingUser;
```

The user can query the view:

```
SELECT * FROM EmployeeView;
```

Grant Permission to a Role

Instead of granting permissions individually to every user:

```
GRANT SELECT  
ON Employees  
TO ReportingRole;
```

Users who have that role can receive the permission according to the database's role system.

Grant All Privileges

Some databases support:

```
GRANT ALL PRIVILEGES  
ON Employees
```



```
TO John;
```

This grants all applicable privileges on `Employees`.

GRANT with WITH GRANT OPTION

Some database systems allow a user to **give the same permission to other users**:

```
GRANT SELECT
ON Employees
TO John
WITH GRANT OPTION;
```

Now John can potentially grant `SELECT` on `Employees` to another user:

```
GRANT SELECT
ON Employees
TO David;
```

This should be used carefully because it allows permissions to propagate.

GRANT VS REVOKE

```
GRANT → Give permission
REVOKE → Remove permission
```

Example:

```
GRANT SELECT ON Employees TO John;
```

Later:

```
REVOKE SELECT ON Employees FROM John;
```

Real-world example

A reporting application should only read data:

```
GRANT SELECT
ON Sales
TO ReportingUser;
```

It should **not** receive:

```
INSERT
UPDATE
DELETE
```

This is an example of the **principle of least privilege**—users get only the access they need.

- **REVOKE**

REVOKE in SQL

REVOKE is a **DCL (Data Control Language)** command used to **remove permissions/privileges** from a user or role.

Think of it simply:

```
GRANT → Give permission
REVOKE → Remove permission
```

1. Basic Syntax

```
REVOKE permission
ON object
FROM user_or_role;
```

Example

First, give John permission:

```
GRANT SELECT
ON Employees
TO John;
```

Later, remove that permission:

```
REVOKE SELECT
ON Employees
FROM John;
```

John can no longer use that particular granted privilege to run:

```
SELECT * FROM Employees;
```

assuming he has no other permission path.

2. Revoke Multiple Permissions

You can remove multiple permissions:

```
REVOKE INSERT, UPDATE  
ON Employees  
FROM John;
```

This removes John's **INSERT** and **UPDATE** privileges on **Employees**.

3. Revoke All Privileges

Some database systems support:

```
REVOKE ALL PRIVILEGES  
ON Employees  
FROM John;
```

This removes all applicable privileges previously granted to John on that object.

Note: Exact syntax and behavior vary between SQL Server, PostgreSQL, MySQL, and Oracle.

4. Revoke from a Role

Permissions can also be removed from a role:

```
REVOKE SELECT  
ON Employees  
FROM ReportingRole;
```

Users receiving permissions through that role may consequently lose the permission.

5. REVOKE with GRANT OPTION

Suppose John was given:

```
GRANT SELECT  
ON Employees
```

```
TO John  
WITH GRANT OPTION;
```

John can then grant **SELECT** to another user.

You can remove John's ability to **grant that privilege onward** while potentially retaining his own privilege, depending on the database:

```
REVOKE GRANT OPTION FOR SELECT  
ON Employees  
FROM John;
```

The exact syntax varies by database system.

6. Example Scenario

Suppose:

```
John  
├─ SELECT  
├─ INSERT  
└─ UPDATE
```

You want John to stop modifying data but still allow him to read it.

```
REVOKE INSERT, UPDATE  
ON Employees  
FROM John;
```

Afterward:

```
John  
└─ SELECT
```

So this works:

```
SELECT * FROM Employees;
```

But these operations are no longer allowed through those privileges:

```
INSERT INTO Employees ...;
```

```
UPDATE Employees  
SET Salary = 60000;
```

GRANT VS REVOKE

Command	Purpose	Example
GRANT	Give permission	GRANT SELECT ON Employees TO John;
REVOKE	Remove permission	REVOKE SELECT ON Employees FROM John;

Easy way to remember

```
GRANT  = Give access  
REVOKE = Take access back
```

DCL → **GRANT** + **REVOKE** → **Database Security & Access Control**.

22. Normalization

Normalization in SQL

Normalization is the process of organizing database tables to:

- Reduce **data redundancy**
- Avoid **insert, update, and delete anomalies**
- Improve **data consistency**
- Make relationships between data clearer

The main normal forms are **1NF** → **2NF** → **3NF** → **BCNF**.

1. First Normal Form (1NF)

A table is in **1NF** when:

1. Each column contains **atomic (single) values**
2. There are no repeating groups or arrays
3. Each row is uniquely identifiable

✗ Not in 1NF

StudentId	Name	Courses
1	Rahul	SQL, C#, Java
2	Amit	Python, C++

Courses contains multiple values.

✓ In 1NF

StudentId	Name	Course
1	Rahul	SQL
1	Rahul	C#
1	Rahul	Java
2	Amit	Python
2	Amit	C++

Now each cell contains a single value.

Key idea

1NF = Atomic values

2. Second Normal Form (2NF)

A table is in **2NF** when:

1. It is already in **1NF**
2. There is **no partial dependency**

Partial dependency occurs when a non-key column depends on **only part of a composite primary key**.

Example

Suppose:

StudentCourse

StudentId

CourseId

StudentName

CourseName

Marks

Primary key:

(StudentId, CourseId)

Dependencies:

```
StudentId + CourseId → Marks
StudentId → StudentName
CourseId → CourseName
```

StudentName depends only on **StudentId**.

CourseName depends only on **CourseId**.

Therefore, there is **partial dependency**, so the table is not in 2NF.

✅ Convert to 2NF

Create separate tables:

Student

StudentId	StudentName
1	Rahul
2	Amit

Course

CourseId	CourseName
101	SQL
102	C#

StudentCourse

StudentId	CourseId	Marks
1	101	85
1	102	90
2	101	78

Now:

```
StudentId → StudentName
CourseId → CourseName
(StudentId, CourseId) → Marks
```

Key idea

2NF = 1NF + No partial dependency

3. Third Normal Form (3NF)

A table is in **3NF** when:

1. It is already in **2NF**
2. There is **no transitive dependency**

What is transitive dependency?

A non-key column depends on another non-key column instead of directly depending on the primary key.

Example

```
Employee
-----
EmployeeId
EmployeeName
DepartmentId
DepartmentName
```

Primary key:

```
EmployeeId
```

Dependencies:

```
EmployeeId → EmployeeName
EmployeeId → DepartmentId
DepartmentId → DepartmentName
```

Therefore:

```
EmployeeId → DepartmentId → DepartmentName
```

DepartmentName indirectly depends on **EmployeeId**.

This is a **transitive dependency**.

✗ Before 3NF

EmployeeId	EmployeeName	DepartmentId	DepartmentName
1	Rahul	10	IT
2	Amit	10	IT
3	Priya	20	HR

✓ After 3NF

Employee

EmployeeId	EmployeeName	DepartmentId
1	Rahul	10
2	Amit	10
3	Priya	20

Department

DepartmentId	DepartmentName
10	IT
20	HR

Now:

EmployeeId → EmployeeName
EmployeeId → DepartmentId

DepartmentId → DepartmentName

Key idea

3NF = 2NF + No transitive dependency

4. BCNF

BCNF = Boyce-Codd Normal Form

BCNF is a stronger version of 3NF.

A table is in BCNF when:

For every functional dependency $X \rightarrow Y$, **X must be a super key.**

In simple terms:

Every determinant must be a candidate key/super key.

Example

Consider:

StudentCourseInstructor

Student

Course

Instructor

Suppose:

$(\text{Student}, \text{Course}) \rightarrow \text{Instructor}$

$\text{Instructor} \rightarrow \text{Course}$

If an instructor teaches only one course, then:

$\text{Instructor} \rightarrow \text{Course}$

But **Instructor** is not a candidate key for the entire table.

Therefore, the table can violate BCNF.

Decompose

InstructorCourse

Instructor	Course
I1	SQL
I2	C#

StudentInstructor

Student	Instructor
S1	I1
S2	I2

Now the dependencies are separated appropriately.

Key idea

BCNF = Every determinant must be a candidate key/super key

5. Denormalization

Denormalization is intentionally adding redundancy to a database to improve **read/query performance**.

It is essentially the opposite direction of normalization.

Normalized design

Customer

CustomerId	CustomerName
1	Rahul

Order

OrderId	CustomerId	Amount
101	1	5000

To get the customer name with an order, you need a **JOIN**.

```
SELECT
  o.OrderId,
  c.CustomerName,
  o.Amount
FROM Orders o
JOIN Customers c
  ON o.CustomerId = c.CustomerId;
```

Denormalized design

You could store:

OrderId	CustomerId	CustomerName	Amount
101	1	Rahul	5000

Now reading the order and customer name can be faster because the join may be unnecessary.

But there is redundancy.

If Rahul changes his name, multiple order records might need updating.

Normalization vs Denormalization

Feature	Normalization	Denormalization
Redundancy	Low	Higher
Data consistency	Better	More difficult
Number of tables	More	Fewer

Feature	Normalization	Denormalization
JOINS	More	Fewer
Write performance	Usually better	Can be worse
Read performance	Can require more joins	Often faster
Storage	Less duplication	More duplication
Main goal	Data integrity	Query performance

Easy Way to Remember

```
1NF
↓
Atomic values
↓
2NF
↓
Remove partial dependency
↓
3NF
↓
Remove transitive dependency
↓
BCNF
↓
Every determinant is a candidate key
```

One-line interview definitions

- **1NF:** Every column contains atomic values.
- **2NF:** 1NF + no partial dependency on a composite key.
- **3NF:** 2NF + no transitive dependency.
- **BCNF:** Every determinant must be a candidate key/super key.
- **Denormalization:** Intentionally introducing redundancy to improve read/query performance.
- 1NF

1NF — First Normal Form

1NF (First Normal Form) is the first step of database normalization.

A table is in **1NF** when:

1. Every column contains **atomic (single) values**.
2. There are **no repeating groups**.
3. Each row can be uniquely identified.
4. Each column contains values of the same type/domain.

1. Atomic Values

Atomic value means one cell should contain **one value**.

✗ Not in 1NF

StudentId	StudentName	PhoneNumbers
1	Rahul	9876543210, 9123456789
2	Amit	9988776655

PhoneNumbers contains multiple values in one cell.

✓ In 1NF

StudentId	StudentName	PhoneNumber
1	Rahul	9876543210
1	Rahul	9123456789
2	Amit	9988776655

Each cell now contains a single value.

2. No Repeating Groups

✗ Not in 1NF

```
StudentId | Name | Course1 | Course2 | Course3
-----
1         | Rahul | SQL     | C#      | Java
```

Here **Course1**, **Course2**, and **Course3** are repeating groups.

✓ In 1NF

```
StudentId | Name | Course
-----
1         | Rahul | SQL
1         | Rahul | C#
1         | Rahul | Java
```

Instead of creating multiple columns for the same type of information, use multiple rows.

3. Example with an Employee

✗ Not in 1NF

EmployeeId	EmployeeName	Skills
101	Rahul	C#, SQL, Azure
102	Amit	Java, Python

Skills contains multiple values.

✓ In 1NF

EmployeeId	EmployeeName	Skill
101	Rahul	C#
101	Rahul	SQL
101	Rahul	Azure
102	Amit	Java
102	Amit	Python

Now every cell contains exactly one value.

SQL Example

Create a 1NF table

```
CREATE TABLE StudentCourses
(
    StudentId INT,
    StudentName VARCHAR(100),
    Course VARCHAR(100)
);
```

Insert individual values:

```
INSERT INTO StudentCourses
(StudentId, StudentName, Course)
VALUES
(1, 'Rahul', 'SQL'),
(1, 'Rahul', 'C#'),
(1, 'Rahul', 'Java');
```

Query:

```
SELECT *  
FROM StudentCourses;
```

Result:

StudentId	StudentName	Course
1	Rahul	SQL
1	Rahul	C#
1	Rahul	Java

1NF Interview Question

Q: What is 1NF?

Answer:

1NF means that every column contains atomic values, there are no repeating groups or multi-valued attributes, and each row is uniquely identifiable.

Easy memory trick

1NF = One cell → One value

1NF → Atomicity 2NF → No Partial Dependency 3NF → No Transitive Dependency

- 2NF

2NF — Second Normal Form

2NF (Second Normal Form) is the second step of database normalization.

A table is in **2NF** when:

1. It is already in **1NF**.
2. It has **no partial dependency**.

Simple definition

2NF = 1NF + No Partial Dependency

What is Partial Dependency?

A **partial dependency** occurs when a non-key column depends on **only part of a composite primary key**.

This matters only when the table has a **composite primary key** (a key made from 2 or more columns).

Example

Consider this table:

StudentCourse

StudentId	CourseId	StudentName	CourseName	Marks
1	101	Rahul	SQL	85
1	102	Rahul	C#	90
2	101	Amit	SQL	78

Suppose the primary key is:

(StudentId, CourseId)

Because a student can take multiple courses.

Dependencies

(StudentId, CourseId) → Marks

Marks depends on the **complete key**.

But:

StudentId → StudentName

StudentName depends only on StudentId.

And:

CourseId → CourseName

CourseName depends only on CourseId.

Therefore:

(StudentId, CourseId)
↓
Marks

StudentId

↓
StudentName

CourseId
↓
CourseName

StudentName and CourseName have **partial dependencies**.

So the table is **not in 2NF**.

Converting to 2NF

We split the table into three tables.

1. Student

StudentId	StudentName
1	Rahul
2	Amit

2. Course

CourseId	CourseName
101	SQL
102	C#

3. StudentCourse

StudentId	CourseId	Marks
1	101	85
1	102	90
2	101	78

Now:

StudentId → StudentName

CourseId → CourseName

(StudentId, CourseId) → Marks

Every non-key attribute depends on the **whole key**.

Therefore, the design is in **2NF**.

SQL Example

Student table

```
CREATE TABLE Student
(
    StudentId INT PRIMARY KEY,
    StudentName VARCHAR(100)
);
```

Course table

```
CREATE TABLE Course
(
    CourseId INT PRIMARY KEY,
    CourseName VARCHAR(100)
);
```

StudentCourse table

```
CREATE TABLE StudentCourse
(
    StudentId INT,
    CourseId INT,
    Marks INT,

    PRIMARY KEY (StudentId, CourseId),

    FOREIGN KEY (StudentId)
        REFERENCES Student(StudentId),

    FOREIGN KEY (CourseId)
        REFERENCES Course(CourseId)
);
```

Why 2NF is Important

Without 2NF, you can get **update anomalies**.

For example, if the course name **SQL** changes to **Advanced SQL**, you might have to update many rows:

```
Student 1 → SQL
Student 2 → SQL
Student 3 → SQL
Student 4 → SQL
```

After normalization, the course name exists only once:

```
Course
-----
101 | Advanced SQL
```

So one update is enough.

Important Interview Point

Does 2NF matter for a single-column primary key?

Usually, **no partial dependency is possible** with a single-column primary key.

For example:

```
EmployeeId → EmployeeName
```

There is no "part" of **EmployeeId** being used as a key.

So if a table is in **1NF** and has a **single-column primary key**, it is generally automatically in **2NF**.

Easy Memory Trick

```
1NF → Atomic values
      ↓
2NF → Remove partial dependency
      ↓
3NF → Remove transitive dependency
```

Interview answer

2NF is a normalization form where the table is already in 1NF and every non-key attribute is fully dependent on the entire primary key, with no partial dependency.

- 3NF

3NF — Third Normal Form

3NF (Third Normal Form) is the third step of database normalization.

A table is in **3NF** when:

1. It is already in **2NF**.
2. It has **no transitive dependency**.

Simple definition

3NF = 2NF + No Transitive Dependency

What is Transitive Dependency?

A **transitive dependency** occurs when a non-key column depends on another **non-key column**, rather than directly on the primary key.

For example:

```
EmployeeId → DepartmentId  
DepartmentId → DepartmentName
```

Therefore:

```
EmployeeId → DepartmentId → DepartmentName
```

DepartmentName indirectly depends on **EmployeeId**.

That is a **transitive dependency**.

Example

Consider this table:

Employee

EmployeeId	EmployeeName	DepartmentId	DepartmentName
101	Rahul	10	IT
102	Amit	10	IT
103	Priya	20	HR

Primary key:

```
EmployeeId
```

Dependencies:

```
EmployeeId → EmployeeName
EmployeeId → DepartmentId
DepartmentId → DepartmentName
```

The problem is:

```
EmployeeId
  ↓
DepartmentId
  ↓
DepartmentName
```

DepartmentName depends on **DepartmentId**, which is a **non-key attribute**.

Therefore, this table is **not in 3NF**.

Converting to 3NF

Split the table into two tables.

Employee

EmployeeId	EmployeeName	DepartmentId
101	Rahul	10
102	Amit	10
103	Priya	20

Department

DepartmentId	DepartmentName
10	IT
20	HR

Now the dependencies are:

```
EmployeeId → EmployeeName
EmployeeId → DepartmentId
```

and:

DepartmentId → DepartmentName

There is no transitive dependency inside the **Employee** table.

SQL Example

Department

```
CREATE TABLE Department
(
    DepartmentId INT PRIMARY KEY,
    DepartmentName VARCHAR(100) NOT NULL
);
```

Employee

```
CREATE TABLE Employee
(
    EmployeeId INT PRIMARY KEY,
    EmployeeName VARCHAR(100) NOT NULL,
    DepartmentId INT,

    FOREIGN KEY (DepartmentId)
        REFERENCES Department(DepartmentId)
);
```

Query the information using a **JOIN**:

```
SELECT
    e.EmployeeId,
    e.EmployeeName,
    d.DepartmentName
FROM Employee e
JOIN Department d
    ON e.DepartmentId = d.DepartmentId;
```

Why 3NF is Important

Suppose the IT department changes its name:

IT → Information Technology

✗ Before 3NF

You may need to update many employee records:

```
101 | Rahul | 10 | IT
102 | Amit  | 10 | IT
103 | John  | 10 | IT
```

✓ After 3NF

Only one department record needs to change:

```
10 | Information Technology
```

This reduces:

- Data duplication
- Update anomalies
- Inconsistent data
- Storage requirements

2NF vs 3NF

Normal Form	Removes
1NF	Multi-valued/non-atomic data
2NF	Partial dependency
3NF	Transitive dependency

Easy memory trick

```
1NF → Atomic values
      ↓
2NF → No partial dependency
      ↓
3NF → No transitive dependency
```

Interview answer

3NF is a normalization form where the table is already in 2NF and no non-key attribute depends on another non-key attribute. In other words, there should be no transitive dependency.

- BCNF

BCNF — Boyce-Codd Normal Form

BCNF (Boyce-Codd Normal Form) is a stronger version of **3NF**.

Simple definition

A table is in BCNF if, for every functional dependency $X \rightarrow Y$, X is a super key.

In simpler words:

Every determinant must be a candidate key or a super key.

1. What is a Determinant?

In a dependency:

$A \rightarrow B$

A is called the **determinant** because it determines B .

Example:

$\text{StudentId} \rightarrow \text{StudentName}$

Here:

- StudentId = determinant
- StudentName = dependent attribute

For BCNF:

Determinant $\rightarrow$ Must be a Super Key

Example

Consider:

StudentCourseInstructor

Student	Course	Instructor
Rahul	SQL	John
Amit	SQL	John

Student	Course	Instructor
Priya	C#	David
Raj	C#	David

Assume:

- A student can take multiple courses.
- Each instructor teaches only one course.
- An instructor can teach many students.

Functional dependencies:

(Student, Course) → Instructor
Instructor → Course

The candidate key can be:

(Student, Course)

But look at:

Instructor → Course

Instructor determines **Course**.

However, **Instructor** is **not a super key** for the original table because it doesn't uniquely identify a student.

Therefore, the table **violates BCNF**.

Converting to BCNF

Split the table.

InstructorCourse

Instructor	Course
John	SQL
David	C#

Here:

Instructor → Course

Instructor is now the primary key.

StudentInstructor

Student	Instructor
Rahul	John
Amit	John
Priya	David
Raj	David

Now the problematic dependency has been separated.

BCNF vs 3NF

BCNF is **stricter than 3NF**.

3NF	BCNF
Allows some dependencies where determinant isn't a super key	Does not allow them
Less strict	More strict
Most practical databases stop at 3NF	Used when stronger normalization is required

Important relationship

BCNF ⇒ 3NF

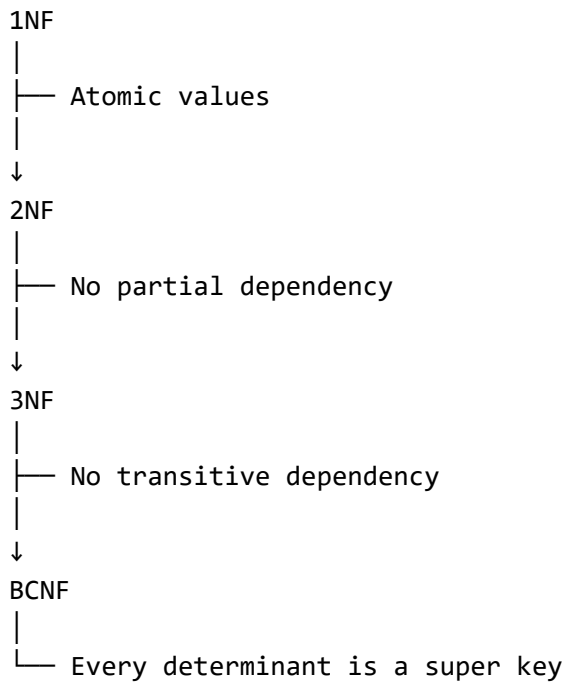
If a table is in **BCNF**, it is also in **3NF**.

But:

3NF ⇏ BCNF

A table can be in 3NF but **not BCNF**.

Quick Comparison



Interview Answer

BCNF, or Boyce-Codd Normal Form, is a stronger form of 3NF. A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, X is a super key.

Easy memory trick

3NF: Remove **transitive dependency** **BCNF:** Every determinant = Super Key

- Denormalization

Denormalization in SQL

Denormalization is the process of **intentionally adding redundant data** to a database to improve **read/query performance**.

It is generally the opposite of normalization.

Normalization → Reduce redundancy Denormalization → Accept some redundancy for performance

Example

Suppose we have normalized tables.

Customer

CustomerId	CustomerName
1	Rahul

CustomerId	CustomerName
2	Amit

Orders

OrderId	CustomerId	Amount
101	1	5000
102	1	3000
103	2	7000

To get customer names with orders, we need a **JOIN**:

```
SELECT
  o.OrderId,
  c.CustomerName,
  o.Amount
FROM Orders o
JOIN Customer c
  ON o.CustomerId = c.CustomerId;
```

Denormalized Table

We could store the customer name directly in **Orders**:

OrderId	CustomerId	CustomerName	Amount
101	1	Rahul	5000
102	1	Rahul	3000
103	2	Amit	7000

Now we can query directly:

```
SELECT
  OrderId,
  CustomerName,
  Amount
FROM Orders;
```

The **JOIN** is no longer required for this query.

However, **CustomerName** is duplicated.

Why Use Denormalization?

Denormalization can be useful when:

1. Read performance is very important

Applications that perform many more reads than writes may benefit.

```
Many SELECTs
  ↓
Fewer JOINS
  ↓
Potentially faster reads
```

2. Complex JOINS are expensive

If a query requires many joins across large tables, storing frequently accessed data together can improve performance.

3. Reporting and analytics

Data warehouses and reporting systems often use denormalized structures.

For example:

```
Sales
├── CustomerName
├── ProductName
├── CategoryName
├── Region
├── SalesAmount
└── Date
```

This can make reporting queries simpler and faster.

Disadvantages

Denormalization introduces **data redundancy**.

1. Duplicate data

```
Order 101 → Rahul
Order 102 → Rahul
Order 103 → Rahul
```

Rahu1 is stored multiple times.

2. Update anomaly

If Rahul changes his name, multiple rows may need to be updated.

```
UPDATE Orders
SET CustomerName = 'Rahu1 Sharma'
WHERE CustomerId = 1;
```

If some rows aren't updated, inconsistent data can result.

3. More storage

Duplicated data requires additional storage.

4. More complex writes

INSERT, UPDATE, and DELETE operations may need additional logic to keep duplicated data synchronized.

Normalization vs Denormalization

Feature	Normalization	Denormalization
Redundancy	Low	Higher
Data consistency	High	More difficult
JOINS	More	Fewer
Read performance	Can be slower	Can be faster
Write complexity	Lower	Higher
Storage	Less	More
Main goal	Data integrity	Read performance

Real-World Example

Imagine an e-commerce application.

A normalized design might have:



```
OrderItem
  ↓
Product
  ↓
Category
```

A reporting query might require several **JOINS**.

For a frequently accessed dashboard, we might create a denormalized table:

```
SalesReport
-----
OrderId
CustomerName
ProductName
CategoryName
Quantity
Price
TotalAmount
OrderDate
Region
```

The dashboard can query this table directly.

Important Point

Denormalization does NOT mean randomly duplicating data.

It should be done when there is a clear performance or architectural reason.

Common techniques include:

- Storing calculated/derived values
- Duplicating frequently accessed attributes
- Pre-joining data
- Summary/aggregate tables
- Materialized views
- Caching frequently accessed data

Interview Answer

Denormalization is the intentional introduction of redundant data into a database to improve read performance and reduce expensive joins. It can improve query performance but increases storage requirements and the risk of data inconsistency.

Easy memory trick

Normalization



Less duplication

Better consistency

Denormalization



More duplication

Faster/simpler reads

Normalization → Data integrity Denormalization → Performance

23. Performance Optimization

- Query Optimization

Query Optimization in SQL

Query Optimization is the process of improving a SQL query so that it executes **faster, uses less CPU/memory/I/O, and scales better with large amounts of data.**

The goal is not simply to make the SQL shorter. The goal is to help the database choose an efficient execution strategy.

1. Select Only Required Columns

✗ Avoid

```
SELECT *  
FROM Employees;
```

If you only need the employee name and salary:

✓ Better

```
SELECT Name, Salary  
FROM Employees;
```

Why?

- Less data read from storage
 - Less memory usage
 - Less network traffic
 - Faster result processing
-

2. Filter Rows Using WHERE

Instead of retrieving everything and filtering later:

```
SELECT Name, Salary
FROM Employees
WHERE DepartmentId = 10;
```

The database can focus on the required rows.

An index can make this even faster:

```
CREATE INDEX IX_Employees_DepartmentId
ON Employees(DepartmentId);
```

3. Avoid Unnecessary Functions on Columns

Suppose you have:

```
JoiningDate DATE
```

✗ Less efficient

```
SELECT *
FROM Employees
WHERE YEAR(JoiningDate) = 2025;
```

The database may have difficulty using a normal index on `JoiningDate`.

✓ Better

```
SELECT *
FROM Employees
WHERE JoiningDate >= '2025-01-01'
      AND JoiningDate < '2026-01-01';
```

This form is generally more **index-friendly**.

4. Use Proper Indexes

Suppose:

```
SELECT *  
FROM Employees  
WHERE EmployeeId = 100;
```

If `EmployeeId` is indexed, the database can potentially locate the row directly.

```
CREATE INDEX IX_Employees_EmployeeId  
ON Employees(EmployeeId);
```

Instead of:

```
Scan many rows  
  ↓  
Check EmployeeId  
  ↓  
Find matching row
```

the database may perform:

```
Index  
  ↓  
EmployeeId = 100  
  ↓  
Matching row
```

5. Avoid `SELECT *` in Large Queries



```
SELECT *  
FROM Orders  
WHERE CustomerId = 100;
```



```
SELECT OrderId, OrderDate, TotalAmount  
FROM Orders  
WHERE CustomerId = 100;
```

This becomes particularly important when tables contain large columns such as:

- `VARCHAR(MAX)`
- `TEXT`
- `BLOB`
- JSON/XML data

6. Optimize JOINS

Suppose:

```
SELECT c.Name, o.OrderDate
FROM Customers c
JOIN Orders o
    ON c.CustomerId = o.CustomerId;
```

Make sure the join columns are appropriately indexed.

For example:

```
CREATE INDEX IX_Orders_CustomerId
ON Orders(CustomerId);
```

Poorly optimized joins can become extremely expensive when tables contain millions of rows.

7. Avoid Unnecessary Joins



```
SELECT c.Name
FROM Customers c
JOIN Orders o
    ON c.CustomerId = o.CustomerId;
```

If you don't actually need any information from `Orders`, ask whether the join is necessary.

If your intention is simply to find customers who have orders, `EXISTS` may be more appropriate:

```
SELECT c.Name
FROM Customers c
WHERE EXISTS
(
    SELECT 1
    FROM Orders o
```

```
WHERE o.CustomerId = c.CustomerId
);
```

8. Use **EXISTS** for Existence Checks

Suppose you want customers who have at least one order.

```
SELECT c.Name
FROM Customers c
WHERE EXISTS
(
    SELECT 1
    FROM Orders o
    WHERE o.CustomerId = c.CustomerId
);
```

EXISTS communicates that you only care whether a matching row exists.

9. Be Careful With **IN**

Example:

```
SELECT *
FROM Employees
WHERE DepartmentId IN (1, 2, 3);
```

This is perfectly reasonable for a small list.

For more complex subqueries, compare **IN**, **EXISTS**, and joins using the actual execution plan rather than assuming one is always faster.

10. Avoid Leading Wildcards



```
SELECT *
FROM Employees
WHERE Name LIKE '%John%';
```

A normal B-tree index generally cannot perform an efficient seek for a pattern beginning with %.



```
SELECT *  
FROM Employees  
WHERE Name LIKE 'John%';
```

This can often use an index on **Name**.

For true "contains" searches, consider database-specific full-text/search capabilities.

11. Optimize **ORDER BY**

Suppose:

```
SELECT EmployeeId, Name, Salary  
FROM Employees  
ORDER BY Salary;
```

If the query frequently sorts by **Salary**, an appropriate index may help:

```
CREATE INDEX IX_Employees_Salary  
ON Employees(Salary);
```

But don't automatically create an index for every **ORDER BY**. Check the workload and execution plan.

12. Optimize **GROUP BY**

Example:

```
SELECT DepartmentId, COUNT(*)  
FROM Employees  
GROUP BY DepartmentId;
```

If **DepartmentId** is frequently used for filtering/grouping, an index may help depending on the query and database optimizer.

13. Use **HAVING** Correctly

WHERE filters rows **before grouping**.

HAVING filters groups **after grouping**.

Better

```
SELECT DepartmentId, COUNT(*) AS EmployeeCount
FROM Employees
WHERE Salary > 50000
GROUP BY DepartmentId
HAVING COUNT(*) > 5;
```

Here:

```
WHERE
↓
Filter rows
↓
GROUP BY
↓
Create groups
↓
HAVING
↓
Filter groups
```

Filtering with **WHERE** as early as possible can reduce the amount of data that needs to be grouped.

14. Avoid Unnecessary **DISTINCT**



```
SELECT DISTINCT DepartmentId
FROM Employees;
```

DISTINCT may require sorting or hashing.

Don't use it simply to hide duplicate rows caused by an incorrect join.

First determine **why duplicates are occurring**.

15. Avoid Unnecessary Sorting

Operations such as:

```
ORDER BY
DISTINCT
```

```
GROUP BY
```

can require significant work on large datasets.

Only use them when the result actually requires them.

16. Use Pagination Carefully

Basic pagination

```
SELECT EmployeeId, Name
FROM Employees
ORDER BY EmployeeId
OFFSET 1000 ROWS
FETCH NEXT 20 ROWS ONLY;
```

For very deep pages, **OFFSET** can become expensive because the database may still need to process many preceding rows.

For large datasets, **keyset/seek pagination** is often better.

```
SELECT TOP 20 EmployeeId, Name
FROM Employees
WHERE EmployeeId > 1000
ORDER BY EmployeeId;
```

17. Avoid Repeated Queries

✗ Application code

```
Get all customers
  ↓
For each customer
  ↓
Query their orders
```

If there are 10,000 customers, this can result in thousands of database calls.

This is commonly called the **N+1 query problem**.

Instead, retrieve the required data using an appropriate join, aggregation, or batch query.

18. Use Execution Plans

One of the most important optimization techniques is:

```
Query
↓
Execution Plan
↓
Find expensive operation
↓
Optimize
↓
Run again
↓
Compare
```

Look for operations such as:

- Table Scan
- Index Scan
- Index Seek
- Key Lookup
- Sort
- Hash Match
- Nested Loops
- Merge Join
- Excessive reads

19. Measure Before and After

Don't assume a query is faster just because it looks better.

For example:

```
SELECT *
FROM Employees
WHERE DepartmentId = 10;
```

After optimization:

```
SELECT EmployeeId, Name, Salary
FROM Employees
WHERE DepartmentId = 10;
```

Compare:

Before
CPU time: 500 ms
Logical reads: 50,000
Execution: 700 ms

After
CPU time: 100 ms
Logical reads: 8,000
Execution: 150 ms

The important thing is **measured improvement**.

20. Practical Query Optimization Checklist

When a query is slow, check in this order:

1. Is the query returning unnecessary data?
↓
2. Can `SELECT *` be avoided?
↓
3. Can `WHERE` filter earlier?
↓
4. Are `JOINS` necessary?
↓
5. Are `JOIN/filter` columns indexed?
↓
6. Are functions being applied to indexed columns?
↓
7. Is there an unnecessary `DISTINCT`?
↓
8. Is unnecessary sorting occurring?
↓
9. Is pagination efficient?
↓
10. What does the execution plan show?
↓
11. Measure again after changes

Interview Definition

Query optimization is the process of improving a SQL query so that the database can execute it efficiently with minimal CPU, memory, I/O, and execution time. Common techniques include selecting only required columns, filtering early, optimizing joins, using appropriate indexes, avoiding unnecessary functions and sorting, and analyzing execution plans.

- Execution Plans

Execution Plans in SQL

An **Execution Plan** is the strategy chosen by the database engine to execute a SQL query.

It shows **how the database gets the requested data**, including:

- Which indexes are used
- Whether a table is scanned
- How tables are joined
- How data is sorted
- Estimated/actual rows
- CPU and I/O costs
- Expensive operations

Execution plans are one of the most important tools for **SQL performance optimization**.

1. Simple Example

Consider:

```
SELECT *  
FROM Employees  
WHERE EmployeeId = 100;
```

If **EmployeeId** has an index, the database might use:

```
Query  
↓  
Index Seek  
↓  
Find EmployeeId = 100  
↓  
Return Row
```

Without a useful index:

```
Query  
↓  
Table Scan  
↓  
Check Row 1  
Check Row 2  
Check Row 3  
...  
Check Row N
```

↓

Return matching row

For a table with millions of rows, the second approach can be expensive.

2. Why Execution Plans Matter

Suppose this query is slow:

```
SELECT *  
FROM Orders  
WHERE CustomerId = 100;
```

Instead of guessing:

```
"Maybe I need an index."  
"Maybe I need a JOIN."  
"Maybe SQL is slow."
```

Look at the execution plan.

It might show:

```
Table Scan  
↓  
10,000,000 rows examined  
↓  
100 rows returned
```

This immediately tells you that the database is examining far more data than necessary.

3. Estimated vs Actual Execution Plan

There are two important types.

Estimated Execution Plan

The database predicts how it will execute the query **without actually running it**.

It estimates:

- Number of rows
- Cost of operations
- Index usage

- Join strategy

Actual Execution Plan

The query is actually executed, and the plan contains runtime information such as:

- Actual rows processed
- Actual execution details
- Runtime operators
- Warnings

For troubleshooting performance, the **actual execution plan** is often more useful.

4. Common Execution Plan Operators

Table Scan

Reads rows from the entire table.

```
Table
↓
Scan every row
↓
Filter
```

Example:

```
SELECT *
FROM Employees
WHERE Name = 'John';
```

If there is no useful index on **Name**, the database may scan the table.

Index Seek

Efficiently navigates an index to find matching rows.

```
Index
↓
Find required key
↓
Retrieve matching rows
```

For example:

```
SELECT *  
FROM Employees  
WHERE EmployeeId = 100;
```

with an appropriate index can result in an **Index Seek**.

Generally, a seek is desirable when the query is selective.

Index Scan

Reads a significant portion of an index.

```
Index  
↓  
Read many/all index entries  
↓  
Filter/process
```

An index scan isn't automatically bad.

For example, if you need a large percentage of a table's rows, scanning an index can be perfectly reasonable.

5. Table Scan vs Index Seek

Imagine a table contains:

```
10,000,000 employees
```

Query:

```
SELECT *  
FROM Employees  
WHERE EmployeeId = 5000000;
```

Table Scan

```
10,000,000 rows  
↓  
Check each row  
↓  
Find 1 row
```

Index Seek

```
Index
↓
EmployeeId = 5000000
↓
Find matching row
```

The second approach can be dramatically faster.

6. Nested Loops Join

Nested Loops is commonly used when one input is relatively small and the other input can be efficiently searched.

Example:

```
SELECT c.Name, o.OrderId
FROM Customers c
JOIN Orders o
  ON c.CustomerId = o.CustomerId;
```

Conceptually:

```
Customers
↓
Take one customer
↓
Search Orders
↓
Take next customer
↓
Search Orders
↓
...
```

Nested Loops can be excellent when the outer input is small and the inner side has an efficient index.

7. Hash Match

A **Hash Match** can be useful for joining or aggregating larger datasets.

Conceptually:

```
Table A
↓
```

Build Hash Table

Table B



Probe Hash Table



Matching rows

Hash operations can consume significant memory.

A Hash Match is **not automatically a problem**. The important question is whether it is appropriate for the data and query.

8. Merge Join

A Merge Join works efficiently when both inputs are appropriately ordered.

Conceptually:

Input A → Sorted



→ Merge Join



Input B → Sorted

It can be very efficient for large datasets when the required ordering is already available.

9. Sort Operator

A Sort operation orders rows.

Example:

```
SELECT *  
FROM Employees  
ORDER BY Salary;
```

The execution plan may contain:

Table/Index



Sort



Result

Sorting millions of rows can consume significant CPU and memory.

An appropriate index can sometimes reduce or eliminate the need for an explicit sort.

10. Key Lookup

A **Key Lookup** occurs when the database finds rows using a nonclustered index but needs to go back to the base table to retrieve additional columns.

Example:

```
SELECT EmployeeId, Name, Salary
FROM Employees
WHERE DepartmentId = 10;
```

Suppose there is an index only on:

```
DepartmentId
```

The database may do:

```
Index Seek
  ↓
Find DepartmentId = 10
  ↓
Key Lookup
  ↓
Retrieve Name, Salary
```

If this happens for a large number of rows, it can become expensive.

A covering index may help:

```
CREATE INDEX IX_Employees_Department
ON Employees(DepartmentId)
INCLUDE (Name, Salary);
```

Whether this is worthwhile depends on the workload and write/storage costs.

11. Estimated Cost

Execution plans often show an estimated cost for operators.

For example:

Index Seek	5%
Key Lookup	10%
Sort	60%
Nested Loops	25%

The expensive operator deserves investigation.

However, **don't blindly optimize the operator with the highest percentage**. Estimated costs are relative optimizer estimates, not actual elapsed time.

12. Estimated Rows vs Actual Rows

This is extremely important.

Suppose the optimizer estimates:

Estimated rows: 100

but execution produces:

Actual rows: 1,000,000

That's a huge mismatch.

It can lead to a poor plan, such as choosing Nested Loops when a different join strategy would have been better.

Possible causes include:

- Outdated statistics
- Data distribution changes
- Correlated predicates
- Parameter-sensitive behavior
- Incorrect cardinality assumptions

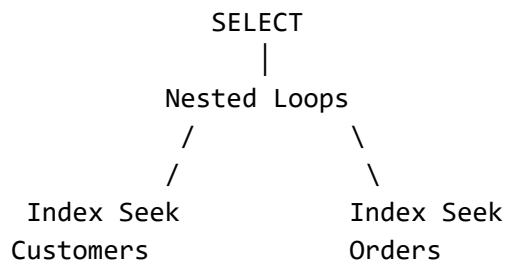
13. Execution Plan Example

Query:

```
SELECT c.Name, o.OrderDate
FROM Customers c
JOIN Orders o
```

```
ON c.CustomerId = o.CustomerId
WHERE c.CustomerId = 100;
```

A simplified plan could look like:



This can be efficient because:

1. Find customer 100
2. Find orders belonging to customer 100
3. Join the results

14. Finding Performance Problems

When analyzing an execution plan, pay attention to:

● Table Scans

Table Scan

Especially problematic when a huge table is being scanned to return very few rows.

● Large Sorts

Sort
10,000,000 rows

Investigate whether ordering can be improved through indexing or query design.

● Excessive Key Lookups

Index Seek
↓
Millions of Key Lookups

Consider whether a covering index is appropriate.

● Large row-estimate mismatches

```
Estimated: 100  
Actual:    1,000,000
```

Investigate statistics and query behavior.

● Excessive memory grants/spills

Large sorts or hash operations can require substantial memory and may spill to disk if insufficient memory is available.

15. SQL Server Example

In SQL Server Management Studio, you can enable the actual execution plan with:

```
Query  
↓  
Include Actual Execution Plan  
↓  
Execute Query
```

The shortcut is:

```
Ctrl + M
```

Then execute:

```
SELECT *  
FROM Employees  
WHERE DepartmentId = 10;
```

The execution plan will appear alongside the query results.

16. Query Optimization Using the Plan

Suppose the plan shows:

```
Table Scan  
10 million rows
```

You investigate and discover there is no suitable index.

You might create:

```
CREATE INDEX IX_Employees_DepartmentId
ON Employees(DepartmentId);
```

Run the query again.

The plan may change to:

```
Index Seek
  ↓
Matching rows
```

Then compare:

```
Before:
Logical Reads → High
CPU           → High
Execution Time → High

After:
Logical Reads → Lower
CPU           → Lower
Execution Time → Lower
```

17. Important Rule

Don't think:

```
Index Seek = Always Good
Table Scan = Always Bad
```

That's incorrect.

For example:

```
SELECT *
FROM Employees;
```

If you need **every row**, scanning the table may be the most efficient option.

Similarly:

Index Scan

can be perfectly reasonable when a large percentage of the data is required.

The correct question is:

Is the chosen execution plan appropriate for the query and amount of data being processed?

Interview Answer

What is an execution plan?

An execution plan is the database optimizer's strategy for executing a SQL query. It shows operations such as table scans, index seeks, joins, sorts, and lookups, along with estimated or actual row counts and costs. We use execution plans to identify performance bottlenecks and determine whether query rewriting, indexing, statistics maintenance, or other optimization is needed.

- Index Optimization

Index Optimization in SQL

Index optimization is the process of designing, modifying, and maintaining indexes so that queries can retrieve data efficiently while avoiding unnecessary storage and write overhead.

The goal is to achieve a good balance between:

Fast SELECT queries
↕
INSERT / UPDATE / DELETE performance
↕
Storage usage

1. Why Indexes Improve Performance

Suppose you have 10 million employees:

```
SELECT *  
FROM Employees  
WHERE EmployeeId = 5000000;
```

Without a suitable index:

```
Table
↓
Check millions of rows
↓
Find EmployeeId = 5000000
```

With an index:

```
Index
↓
Find EmployeeId = 5000000
↓
Retrieve row
```

This can dramatically reduce the amount of data the database needs to examine.

2. Create an Index on Frequently Filtered Columns

Suppose you frequently run:

```
SELECT *
FROM Employees
WHERE DepartmentId = 10;
```

Create an index:

```
CREATE INDEX IX_Employees_DepartmentId
ON Employees(DepartmentId);
```

Now the optimizer can potentially use the index to find employees in department 10.

3. Don't Index Every Column

A common mistake is:

```
Column A → Index
Column B → Index
Column C → Index
Column D → Index
Column E → Index
...
```

More indexes are **not automatically better**.

Every index requires:

- Storage
- Maintenance
- Additional work during **INSERT**
- Additional work during **UPDATE**
- Additional work during **DELETE**

For example:

```
INSERT INTO Employees  
VALUES (1001, 'John', 10, 50000);
```

The database may need to update several indexes.

Therefore:

Create indexes based on actual query workload, not simply because a column exists.

4. Composite Indexes

A composite index contains multiple columns.

Example:

```
CREATE INDEX IX_Employees_Department_Salary  
ON Employees(DepartmentId, Salary);
```

This can help queries such as:

```
SELECT *  
FROM Employees  
WHERE DepartmentId = 10  
AND Salary > 50000;
```

The index is ordered conceptually by:

```
DepartmentId  
↓  
Salary
```

5. Column Order Matters

Consider:

```
CREATE INDEX IX_Employees
ON Employees(DepartmentId, Salary);
```

The **leading column** is:

```
DepartmentId
```

Therefore, this query can generally benefit:

```
WHERE DepartmentId = 10
```

and:

```
WHERE DepartmentId = 10
AND Salary > 50000
```

But a query filtering only on:

```
WHERE Salary > 50000
```

may not benefit nearly as much from this particular index.

This is called the **leftmost/leading-column principle** for traditional B-tree indexes.

6. Choose Index Columns Based on Query Patterns

Suppose you frequently execute:

```
SELECT OrderId, OrderDate, TotalAmount
FROM Orders
WHERE CustomerId = 100
ORDER BY OrderDate DESC;
```

A useful index could be:

```
CREATE INDEX IX_Orders_Customer_Date
```



```
ON Orders(CustomerId, OrderDate DESC);
```

This matches both:

```
WHERE CustomerId
  ↓
ORDER BY OrderDate
```

The exact benefit depends on the database engine and existing indexes.

7. Covering Indexes

A **covering index** contains all the columns needed by a particular query.

Suppose:

```
SELECT EmployeeId, Name, Salary
FROM Employees
WHERE DepartmentId = 10;
```

You could create:

```
CREATE INDEX IX_Employees_Department
ON Employees(DepartmentId)
INCLUDE (EmployeeId, Name, Salary);
```

Conceptually:

```
Index
├─ DepartmentId  ← search
├─ EmployeeId    ← return
├─ Name          ← return
└─ Salary        ← return
```

The database may be able to satisfy the query entirely from the index, avoiding lookups to the base table.

But don't overuse covering indexes

A very wide index can:

- Consume more storage
- Increase write cost
- Increase maintenance
- Increase memory pressure

8. Index Seek vs Index Scan

When analyzing an execution plan:

Index Seek

```
Index
↓
Locate required range
↓
Return rows
```

Usually desirable for selective lookups.

Index Scan

```
Index
↓
Read many/all index entries
↓
Return rows
```

Not automatically bad.

For example, if a query needs 80% of a table's rows, scanning may be cheaper than repeatedly seeking.

9. Avoid Functions on Indexed Columns

Suppose:

```
CREATE INDEX IX_Employees_Name
ON Employees(Name);
```

Potentially problematic

```
SELECT *
FROM Employees
WHERE UPPER(Name) = 'JOHN';
```

The expression changes the indexed value and may prevent a normal index seek, depending on the database.

Prefer an index-friendly predicate where possible, or use a database-appropriate expression/computed-column index if the search requirement genuinely needs the expression.

10. Avoid Leading Wildcards

Suppose:

```
CREATE INDEX IX_Employees_Name
ON Employees(Name);
```



```
WHERE Name LIKE '%John%'
```

A conventional B-tree index generally can't efficiently seek to a pattern that begins with %.



```
WHERE Name LIKE 'John%'
```

This can often use the index.

For contains/fuzzy searching, consider full-text or specialized search indexes.

11. Index Foreign Keys Used in Joins

Suppose:

```
SELECT c.Name, o.OrderId
FROM Customers c
JOIN Orders o
  ON c.CustomerId = o.CustomerId;
```

If `Orders.CustomerId` is frequently used for joins and filtering, an index can help:

```
CREATE INDEX IX_Orders_CustomerId
ON Orders(CustomerId);
```

This is particularly useful when the foreign-key column is not already indexed by another suitable index.

12. Index `ORDER BY` Columns When Appropriate

Query:

```
SELECT OrderId, OrderDate, TotalAmount
FROM Orders
WHERE CustomerId = 100
ORDER BY OrderDate DESC;
```

Potential index:

```
CREATE INDEX IX_Orders_Customer_Date
ON Orders(CustomerId, OrderDate DESC);
```

This may allow the database to retrieve matching rows in the required order and reduce sorting work.

13. Index Selectivity

Selectivity describes how effectively a column distinguishes rows.

Suppose:

```
EmployeeId:
1
2
3
4
5
...
```

Almost every value is unique.

High selectivity:

```
EmployeeId → High
```

Now consider:

```
Gender:
Male
Female
Male
Female
...
```

Low selectivity:

Gender → Low

Highly selective columns are often better candidates for indexes used in equality searches.

However, **low-cardinality columns aren't automatically useless**. Their usefulness depends on the database, table size, query patterns, data distribution, and whether the index is composite or filtered/partial.

14. Unique Index

If values must be unique:

```
CREATE UNIQUE INDEX IX_Employees_Email  
ON Employees(Email);
```

This provides both:

1. Efficient lookup
2. Uniqueness enforcement

Example:

```
SELECT *  
FROM Employees  
WHERE Email = 'john@example.com';
```

A unique index can be very effective for this type of lookup.

15. Clustered Index

A clustered index determines how table data is organized according to the database engine's storage model.

In SQL Server, a table can have **only one clustered index** because the rows can have only one clustered ordering.

Example:

```
CREATE CLUSTERED INDEX IX_Employees_EmployeeId  
ON Employees(EmployeeId);
```

Clustered indexes are especially important when queries frequently retrieve ranges of rows based on the indexed key.

16. Non-Clustered Index

A non-clustered index is a separate index structure that points to the underlying table rows.

Example:

```
CREATE INDEX IX_Employees_DepartmentId
ON Employees(DepartmentId);
```

A table can have multiple non-clustered indexes, subject to practical storage and maintenance considerations.

17. Filtered Index

Some database systems support filtered/partial indexes.

For example, in SQL Server:

```
CREATE INDEX IX_Employees_Active
ON Employees(DepartmentId)
WHERE IsActive = 1;
```

If your workload frequently queries:

```
SELECT *
FROM Employees
WHERE IsActive = 1
AND DepartmentId = 10;
```

a filtered index can be much smaller than indexing every row.

18. Remove Unused or Duplicate Indexes

Suppose you have:

```
Index 1:
(DepartmentId)

Index 2:
(DepartmentId, Salary)

Index 3:
(DepartmentId, Salary, JoiningDate)
```

These indexes overlap.

You should not automatically keep all of them.

Analyze:

- Query workload
- Execution plans
- Index usage
- Write frequency
- Storage
- Existing index definitions

Then determine whether some indexes are redundant.

19. Monitor Index Usage

In SQL Server, you can investigate index usage through DMVs such as:

```
SELECT
    OBJECT_NAME(i.object_id) AS TableName,
    i.name AS IndexName,
    s.user_seeks,
    s.user_scans,
    s.user_lookups,
    s.user_updates
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats s
    ON i.object_id = s.object_id
    AND i.index_id = s.index_id
WHERE i.object_id = OBJECT_ID('dbo.Employees');
```

This can help identify indexes that:

```
Are heavily used
    ↓
Keep

Are rarely used
    ↓
Investigate

Have high maintenance cost
    ↓
Investigate/remove if appropriate
```

Be careful: usage statistics reset under certain conditions, so don't remove an index based on a short observation window.

20. Index Fragmentation

Indexes can become fragmented as data is inserted, updated, and deleted.

In SQL Server, you can inspect fragmentation using:

```
SELECT
    OBJECT_NAME(object_id) AS TableName,
    index_id,
    avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats
(
    DB_ID(),
    NULL,
    NULL,
    NULL,
    'LIMITED'
);
```

Depending on fragmentation and workload, maintenance may involve:

```
ALTER INDEX IX_Employees_DepartmentId
ON Employees
REORGANIZE;
```

or:

```
ALTER INDEX IX_Employees_DepartmentId
ON Employees
REBUILD;
```

The appropriate maintenance strategy depends on the database engine, version, workload, and fragmentation level.

21. Don't Blindly Follow "Missing Index" Suggestions

Execution plans may suggest:

```
Missing Index:
CREATE INDEX ...
```

Treat this as a **recommendation**, not an instruction.

The suggested index may overlap with an existing index or may create excessive write/storage overhead.

Always consider:

```
Existing indexes
      +
Query workload
      +
Read/write ratio
      +
Storage
      +
Execution plans
```

22. Index Optimization Process

A practical process is:

```
Slow Query
  ↓
Measure performance
  ↓
Check Execution Plan
  ↓
Identify scans / lookups / sorts
  ↓
Check existing indexes
  ↓
Analyze query workload
  ↓
Create / modify / remove index
  ↓
Test again
  ↓
Compare CPU / I/O / execution time
```

Example: Complete Optimization

Query

```
SELECT EmployeeId, Name, Salary
FROM Employees
WHERE DepartmentId = 10
ORDER BY Salary DESC;
```

Suppose the table contains:

10 million rows

and the execution plan shows:

```
Table Scan
  ↓
Filter
  ↓
Sort
  ↓
Result
```

Potential index:

```
CREATE INDEX IX_Employees_Department_Salary
ON Employees(DepartmentId, Salary DESC)
INCLUDE (EmployeeId, Name);
```

Potential new plan:

```
Index Seek
  ↓
Matching DepartmentId
  ↓
Rows already ordered by Salary
  ↓
Return required columns
```

This can reduce:

- Rows read
- Sorting work
- Lookups
- I/O
- CPU

But you should **verify the improvement with the actual execution plan and measurements.**

Interview Answer

Index optimization is the process of creating, modifying, and maintaining indexes based on actual query workloads to improve query performance while controlling storage and write overhead. It includes choosing appropriate index columns and column order, using composite

and covering indexes where appropriate, removing redundant indexes, monitoring index usage and fragmentation, and validating changes through execution plans and performance measurements.

- Avoiding Full Table Scans

Avoiding Full Table Scans in SQL

A **Full Table Scan** happens when the database reads a large portion or all rows of a table to find the required data.

For a small table, this may be perfectly fine. For a table containing millions of rows, it can become expensive.

1. Full Table Scan Example

Suppose **Employees** contains **10 million rows**:

```
SELECT *  
FROM Employees  
WHERE EmployeeId = 5000000;
```

If there is no suitable index, the database may do:

```
Employees  
  ↓  
Read Row 1  
Read Row 2  
Read Row 3  
...  
Read Row 5,000,000  
...  
Read Row 10,000,000  
  ↓  
Return matching row
```

This is a **Full Table Scan**.

2. Use an Appropriate Index

Create an index on frequently searched columns:

```
CREATE INDEX IX_Employees_EmployeeId  
ON Employees(EmployeeId);
```

Now the database can potentially do:

```
Index
↓
EmployeeId = 5000000
↓
Find matching row
↓
Return result
```

This is typically much more efficient for a highly selective lookup.

3. Index **WHERE** Columns

Suppose you frequently execute:

```
SELECT EmployeeId, Name, Salary
FROM Employees
WHERE DepartmentId = 10;
```

Consider:

```
CREATE INDEX IX_Employees_DepartmentId
ON Employees(DepartmentId);
```

The database can potentially use the index to locate matching rows instead of scanning the entire table.

4. Index Columns Used in JOINS

Consider:

```
SELECT c.Name, o.OrderDate
FROM Customers c
JOIN Orders o
  ON c.CustomerId = o.CustomerId;
```

An index on the join column can help:

```
CREATE INDEX IX_Orders_CustomerId
ON Orders(CustomerId);
```

This can make finding matching orders much more efficient.

5. Avoid Functions on Indexed Columns

Suppose:

```
CREATE INDEX IX_Employees_JoiningDate
ON Employees(JoiningDate);
```

✗ Potentially inefficient

```
SELECT *
FROM Employees
WHERE YEAR(JoiningDate) = 2025;
```

The function is applied to every `JoiningDate`.

✓ Better

```
SELECT *
FROM Employees
WHERE JoiningDate >= '2025-01-01'
AND JoiningDate < '2026-01-01';
```

This is generally more **SARGable** and gives the optimizer a better opportunity to use the index.

6. Understand SARGability

SARGable means a predicate can be efficiently transformed into an index search/range operation.

SARGable

```
WHERE Salary >= 50000
```

Potentially non-SARGable

```
WHERE Salary + 1000 >= 50000
```

Prefer:

```
WHERE Salary >= 49000
```

when the equivalent transformation is valid.

Another example:



```
WHERE YEAR(OrderDate) = 2026
```



```
WHERE OrderDate >= '2026-01-01'
AND OrderDate < '2027-01-01'
```

7. Avoid Leading Wildcards

Suppose:

```
CREATE INDEX IX_Employees_Name
ON Employees(Name);
```



```
SELECT *
FROM Employees
WHERE Name LIKE '%John%';
```

A normal B-tree index generally cannot efficiently seek to a pattern beginning with %.



```
SELECT *
FROM Employees
WHERE Name LIKE 'John%';
```

This can often use an index.

If you genuinely need "contains" searching, use an appropriate full-text or specialized search solution.

8. Avoid Implicit Conversions

Suppose:

```
EmployeeId INT
```

and you query:

```
SELECT *  
FROM Employees  
WHERE EmployeeId = '100';
```

Depending on the database and data types involved, implicit conversion can interfere with efficient index usage.

Prefer matching data types:

```
SELECT *  
FROM Employees  
WHERE EmployeeId = 100;
```

9. Don't Use **SELECT *** Unnecessarily



```
SELECT *  
FROM Employees  
WHERE DepartmentId = 10;
```



```
SELECT EmployeeId, Name, Salary  
FROM Employees  
WHERE DepartmentId = 10;
```

Selecting only required columns can reduce I/O and can make **covering indexes** more practical.

10. Use Covering Indexes When Appropriate

Query:

```
SELECT EmployeeId, Name, Salary  
FROM Employees  
WHERE DepartmentId = 10;
```

Possible index:

```
CREATE INDEX IX_Employees_Department
ON Employees(DepartmentId)
INCLUDE (EmployeeId, Name, Salary);
```

Conceptually:

```
Index Seek
  ↓
DepartmentId = 10
  ↓
EmployeeId
Name
Salary
  ↓
Result
```

The database may be able to answer the query entirely from the index, avoiding additional table lookups.

11. Avoid Unnecessary **DISTINCT**



```
SELECT DISTINCT DepartmentId
FROM Employees;
```

DISTINCT can require additional sorting or hashing.

Don't use **DISTINCT** simply to hide duplicates caused by an incorrect join.

Fix the underlying query when possible.

12. Avoid Unnecessary Sorting

For example:

```
SELECT *
FROM Employees
ORDER BY Salary;
```

Sorting millions of rows can be expensive.

If the query is frequently used and appropriate for your workload, an index such as:

```
CREATE INDEX IX_Employees_Salary
ON Employees(Salary);
```

may help reduce sorting work.

But don't create an index solely because you see an **ORDER BY**; verify it with the execution plan and workload.

13. Filter Data Early

Suppose:

```
SELECT DepartmentId, COUNT(*)
FROM Employees
WHERE Salary > 50000
GROUP BY DepartmentId;
```

The database can filter rows before grouping:

```
Employees
↓
WHERE Salary > 50000
↓
GROUP BY DepartmentId
↓
COUNT
```

This can be more efficient than processing unnecessary rows first.

14. Use **EXISTS** for Existence Checks

Suppose you want customers who have orders.

```
SELECT c.CustomerId, c.Name
FROM Customers c
WHERE EXISTS
(
    SELECT 1
    FROM Orders o
    WHERE o.CustomerId = c.CustomerId
);
```

With a suitable index:

```
CREATE INDEX IX_Orders_CustomerId
ON Orders(CustomerId);
```

the database can efficiently check for matching orders.

15. Pagination

Suppose you need only 20 rows.

SQL Server

```
SELECT EmployeeId, Name
FROM Employees
ORDER BY EmployeeId
OFFSET 0 ROWS
FETCH NEXT 20 ROWS ONLY;
```

For very deep pages, large **OFFSET** values can become expensive.

Instead, **keyset pagination** can be more efficient:

```
SELECT TOP 20 EmployeeId, Name
FROM Employees
WHERE EmployeeId > 10000
ORDER BY EmployeeId;
```

This works particularly well when **EmployeeId** is indexed.

16. Partitioning for Very Large Tables

For extremely large tables, **partitioning** can reduce the amount of data that needs to be accessed.

For example, an **Orders** table could be partitioned by year:

```
Orders
├─ 2023 partition
├─ 2024 partition
├─ 2025 partition
└─ 2026 partition
```

Query:

```
SELECT *  
FROM Orders  
WHERE OrderDate >= '2026-01-01'  
      AND OrderDate < '2027-01-01';
```

The database may be able to access only the relevant partition. This is called **partition pruning/elimination**.

17. Check the Execution Plan

Don't assume that a query is performing a full scan.

Look at the execution plan.

You may see:

Table Scan

or:

Index Scan

or:

Index Seek

For example:

Before:

```
SELECT *  
FROM Orders  
WHERE CustomerId = 100;
```

Table Scan

↓

10,000,000 rows

↓

100 rows returned

After adding an appropriate index:

Index Seek

↓

CustomerId = 100

↓

100 rows

18. Important: Full Table Scan Is Not Always Bad

This is a very important interview point.

Suppose:

```
SELECT *  
FROM Employees;
```

You want **every row**.

An index doesn't necessarily make this query faster.

The optimizer may correctly choose:

Table Scan

because reading the table sequentially is efficient when most or all rows are required.

Therefore:

The goal is not to eliminate every table scan. The goal is to avoid unnecessary scans when a more efficient access path exists.

19. Practical Optimization Process

When you find a slow query:

```
Slow Query  
↓  
Check Execution Plan  
↓  
Is there a Table Scan?  
↓  
Why is it scanning?  
↓  
Check WHERE/JOIN predicates  
↓  
Check data types  
↓  
Check SARGability  
↓
```

```
Check existing indexes
↓
Create/modify appropriate index if justified
↓
Run query again
↓
Compare performance
```

Interview Answer

Avoiding full table scans means reducing unnecessary reads of an entire table. We can achieve this by creating appropriate indexes, writing SARGable predicates, avoiding functions on indexed columns, avoiding leading wildcards, matching data types, optimizing joins, using covering indexes where appropriate, and analyzing execution plans. However, a full table scan is not inherently bad; if a query needs most or all rows, a scan may be the most efficient execution strategy.

24. Advanced SQL

- Recursive Queries

Recursive Queries in SQL

A **recursive query** is a query that repeatedly executes against its own previous result. It is mainly used for **hierarchical data**.

Common examples:

- Employee → Manager hierarchy
- Organization structure
- Category → Subcategory
- Folder → Subfolder
- Parent → Child relationships
- Bill of materials
- Graph/tree traversal

The most common way to write recursive queries is with a **Recursive CTE (Common Table Expression)**.

1. Basic Structure

A recursive CTE has two parts:

```
WITH CTE_Name AS
(
    -- Anchor query
    SELECT ...
```

```
UNION ALL

-- Recursive query
SELECT ...
FROM Table
JOIN CTE_Name ...
)
SELECT *
FROM CTE_Name;
```

Two important parts

Anchor member

The starting point of the recursion.

Recursive member

Uses the previous result to find the next level.

2. Example: Employee Hierarchy

Suppose we have:

Employees		

EmployeeId	EmployeeName	ManagerId
1	John	NULL
2	Alice	1
3	Bob	1
4	David	2
5	Sarah	2
6	Mike	3

Hierarchy:

```
John
├── Alice
│   ├── David
│   └── Sarah
└── Bob
    └── Mike
```

We want to retrieve the complete hierarchy.

```
WITH EmployeeHierarchy AS
(
    -- Anchor: top-level employees
    SELECT
        EmployeeId,
        EmployeeName,
        ManagerId,
        1 AS Level
    FROM Employees
    WHERE ManagerId IS NULL

    UNION ALL

    -- Recursive member
    SELECT
        e.EmployeeId,
        e.EmployeeName,
        e.ManagerId,
        eh.Level + 1
    FROM Employees e
    INNER JOIN EmployeeHierarchy eh
        ON e.ManagerId = eh.EmployeeId
)
SELECT *
FROM EmployeeHierarchy;
```

Result:

EmployeeId	EmployeeName	ManagerId	Level
1	John	NULL	1
2	Alice	1	2
3	Bob	1	2
4	David	2	3
5	Sarah	2	3
6	Mike	3	3

3. How Recursion Works

The query starts with:

```
WHERE ManagerId IS NULL
```

This returns:

John

Then the recursive part searches for employees whose manager is John:

Alice
Bob

Then it searches for employees whose manager is Alice or Bob:

David
Sarah
Mike

The process continues until there are **no more matching child records**.

Conceptually:

```
Level 1
  John
  ↓
Level 2
  Alice   Bob
  ↓       ↓
Level 3
David Sarah Mike
```

4. Adding a Hierarchy Level

A very common use is tracking the depth.

```
WITH EmployeeHierarchy AS
(
    SELECT
        EmployeeId,
        EmployeeName,
        ManagerId,
        1 AS Level
    FROM Employees
    WHERE ManagerId IS NULL

    UNION ALL

    SELECT
        e.EmployeeId,
        e.EmployeeName,
```



```
        e.ManagerId,  
        eh.Level + 1  
FROM Employees e  
JOIN EmployeeHierarchy eh  
    ON e.ManagerId = eh.EmployeeId  
)  
SELECT *  
FROM EmployeeHierarchy;
```

The important expression is:

```
eh.Level + 1
```

Every time recursion goes one level deeper, the level increases.

5. Creating an Organizational Tree

You can also generate an indented hierarchy.

For SQL Server:

```
WITH EmployeeHierarchy AS  
(  
    SELECT  
        EmployeeId,  
        EmployeeName,  
        ManagerId,  
        1 AS Level  
    FROM Employees  
    WHERE ManagerId IS NULL  
  
    UNION ALL  
  
    SELECT  
        e.EmployeeId,  
        e.EmployeeName,  
        e.ManagerId,  
        eh.Level + 1  
    FROM Employees e  
    JOIN EmployeeHierarchy eh  
        ON e.ManagerId = eh.EmployeeId  
)  
SELECT  
    REPLICATE(' ', Level - 1) + EmployeeName AS Employee  
FROM EmployeeHierarchy;
```

Possible output:

```
John
  Alice
    David
    Sarah
  Bob
    Mike
```

6. Recursive Query for Categories

Suppose:

Categories

Id	Name	ParentId
1	Electronics	NULL
2	Phones	1
3	Android	2
4	iPhone	2
5	Laptops	1
6	Gaming	5

We can retrieve all categories:

```
WITH CategoryTree AS
(
  SELECT
    Id,
    Name,
    ParentId,
    1 AS Level
  FROM Categories
  WHERE ParentId IS NULL

  UNION ALL

  SELECT
    c.Id,
    c.Name,
    c.ParentId,
    ct.Level + 1
  FROM Categories c
  INNER JOIN CategoryTree ct
    ON c.ParentId = ct.Id
)
SELECT *
FROM CategoryTree;
```

Result:

```
Electronics
  Phones
    Android
    iPhone
  Laptops
  Gaming
```

7. Recursive Number Generation

Recursive CTEs aren't limited to hierarchical data.

For example, generate numbers from 1 to 10:

```
WITH Numbers AS
(
  SELECT 1 AS Number

  UNION ALL

  SELECT Number + 1
  FROM Numbers
  WHERE Number < 10
)
SELECT *
FROM Numbers;
```

Result:

```
1
2
3
4
5
6
7
8
9
10
```

The recursion stops because of:

```
WHERE Number < 10
```

Without a proper stopping condition, recursive queries can continue indefinitely or hit the database's recursion limit.

8. Recursive CTE Components

Remember this structure:

```
WITH RecursiveCTE AS
(
    -- 1. Anchor
    SELECT ...

    UNION ALL

    -- 2. Recursive member
    SELECT ...
    FROM Table
    JOIN RecursiveCTE ...
    WHERE ...
)
-- 3. Final query
SELECT *
FROM RecursiveCTE;
```

Think of it as:

```
Anchor
  ↓
First result
  ↓
Recursive query
  ↓
Next result
  ↓
Recursive query
  ↓
Next result
  ↓
No more rows
  ↓
Final result
```

9. Recursive Query vs Normal Query

Normal query

```
SELECT *
FROM Employees;
```

Runs once.

Recursive query

```
WITH EmployeeHierarchy AS (...)
SELECT *
FROM EmployeeHierarchy;
```

The recursive member repeatedly processes newly discovered rows.

10. Important Concepts for Interviews

You should remember these terms:

Concept	Meaning
Recursive CTE	CTE that references itself
Anchor member	Starting rows
Recursive member	Finds next-level rows
UNION ALL	Combines anchor and recursive results
Level/Depth	Tracks hierarchy depth
Termination condition	Stops recursion
Parent	Higher-level record
Child	Lower-level record

11. SQL Server Recursion Limit

SQL Server has a default recursion limit of **100 levels**.

You can change it:

```
OPTION (MAXRECURSION 200);
```

For example:

```
WITH Numbers AS
(
```

```
SELECT 1 AS Number

UNION ALL

SELECT Number + 1
FROM Numbers
WHERE Number < 150
)
SELECT *
FROM Numbers
OPTION (MAXRECURSION 200);
```

MAXRECURSION 0 means no recursion limit in SQL Server, but it should be used carefully because a faulty recursive query can run indefinitely.

12. Recursive Query Interview Example

Question: Find all employees working under manager **John**, including employees under John at every level.

A good approach is:

```
WITH EmployeeHierarchy AS
(
    SELECT
        EmployeeId,
        EmployeeName,
        ManagerId
    FROM Employees
    WHERE EmployeeName = 'John'

    UNION ALL

    SELECT
        e.EmployeeId,
        e.EmployeeName,
        e.ManagerId
    FROM Employees e
    INNER JOIN EmployeeHierarchy eh
        ON e.ManagerId = eh.EmployeeId
)
SELECT *
FROM EmployeeHierarchy;
```

This is one of the most important real-world applications of **Recursive CTEs**.

- Pivot & Unpivot

PIVOT & UNPIVOT in SQL

PIVOT and **UNPIVOT** are used to transform data between **rows and columns**.

- **PIVOT** → converts **rows into columns**
- **UNPIVOT** → converts **columns into rows**

They are especially useful for **reports, dashboards, summaries, and data analysis**.

1. PIVOT

Suppose we have this table:

```
Sales
-----
Employee  Month  Amount
John      Jan    1000
John      Feb    1500
John      Mar    1200
Alice     Jan    2000
Alice     Feb    1800
Alice     Mar    2200
```

We want:

```
Employee  Jan    Feb    Mar
-----
John      1000   1500   1200
Alice     2000   1800   2200
```

This is **PIVOT**.

2. SQL Server PIVOT Syntax

```
SELECT *
FROM
(
    SELECT Employee, Month, Amount
    FROM Sales
) AS SourceTable
PIVOT
(
    SUM(Amount)
    FOR Month IN ([Jan], [Feb], [Mar])
) AS PivotTable;
```

Output:

Employee	Jan	Feb	Mar
Alice	2000	1800	2200
John	1000	1500	1200

How it works

```
SUM(Amount)
```

Specifies the aggregation.

```
FOR Month
```

Specifies the column whose values become columns.

```
IN ([Jan], [Feb], [Mar])
```

Specifies the new columns.

3. PIVOT with GROUPING

Suppose:

Sales		
Department	Month	Amount
IT	Jan	10000
IT	Feb	15000
HR	Jan	8000
HR	Feb	9000

Query:

```
SELECT *
FROM
(
    SELECT Department, Month, Amount
    FROM Sales
) AS S
PIVOT
(
    SUM(Amount)
```



```
    FOR Month IN ([Jan], [Feb])
  ) AS P;
```

Result:

Department	Jan	Feb
HR	8000	9000
IT	10000	15000

4. PIVOT with COUNT

PIVOT doesn't have to use `SUM()`.

You can use:

- `SUM()`
- `COUNT()`
- `AVG()`
- `MIN()`
- `MAX()`

Example:

```
SELECT *
FROM
(
    SELECT Department, Month, EmployeeId
    FROM Employees
) AS S
PIVOT
(
    COUNT(EmployeeId)
    FOR Month IN ([Jan], [Feb], [Mar])
) AS P;
```

This can produce:

Department	Jan	Feb	Mar
IT	10	15	12
HR	5	8	7

5. PIVOT using CASE

You can achieve similar results without the **PIVOT** keyword.

```
SELECT
    Employee,
    SUM(CASE WHEN Month = 'Jan' THEN Amount ELSE 0 END) AS Jan,
    SUM(CASE WHEN Month = 'Feb' THEN Amount ELSE 0 END) AS Feb,
    SUM(CASE WHEN Month = 'Mar' THEN Amount ELSE 0 END) AS Mar
FROM Sales
GROUP BY Employee;
```

This approach is extremely common in real-world SQL.

PIVOT vs CASE

PIVOT	CASE
Dedicated pivot syntax	Uses conditional aggregation
Can be concise	Very flexible
SQL Server supports it	Supported by most databases
Useful for report transformations	Often preferred for portability

6. UNPIVOT

UNPIVOT does the opposite of PIVOT.

Suppose we have:

Employee	Jan	Feb	Mar

John	1000	1500	1200
Alice	2000	1800	2200

We want:

Employee	Month	Amount

John	Jan	1000
John	Feb	1500
John	Mar	1200
Alice	Jan	2000
Alice	Feb	1800
Alice	Mar	2200

That's **UNPIVOT**.

7. SQL Server UNPIVOT Syntax

```
SELECT
    Employee,
    Month,
    Amount
FROM
(
    SELECT Employee, Jan, Feb, Mar
    FROM MonthlySales
) AS S
UNPIVOT
(
    Amount
    FOR Month IN ([Jan], [Feb], [Mar])
) AS U;
```

Result:

Employee	Month	Amount
John	Jan	1000
John	Feb	1500
John	Mar	1200
Alice	Jan	2000
Alice	Feb	1800
Alice	Mar	2200

8. PIVOT vs UNPIVOT

PIVOT

Rows

↓

Columns

UNPIVOT

Columns

↓

Rows

Example:

Original

Employee	Month	Amount
John	Jan	1000
John	Feb	1500

PIVOT

Employee	Jan	Feb
John	1000	1500

UNPIVOT

Employee	Month	Amount
John	Jan	1000
John	Feb	1500

9. Practical Example

Suppose an HR report stores employee attendance like this:

Employee	Jan	Feb	Mar	Apr
John	20	18	22	19
Alice	21	20	23	21
Bob	19	22	20	18

For a dashboard, this format might be useful.

But for analytics, you may want:

Employee	Month	DaysPresent
John	Jan	20
John	Feb	18
John	Mar	22
John	Apr	19
Alice	Jan	21
...		

Use **UNPIVOT** to normalize the report-style data into rows.

10. Dynamic PIVOT

A major limitation of normal PIVOT is that you have to specify the columns:

```
IN ([Jan], [Feb], [Mar])
```

What if the months are dynamic?

For example:

```
Jan  
Feb  
Mar  
Apr  
May  
Jun  
...
```

You can build a **Dynamic SQL PIVOT**.

Conceptually:

```
Get column names  
  ↓  
Build SQL string  
  ↓  
Execute Dynamic SQL  
  ↓  
Generate Pivot
```

This is a common combination of:

PIVOT + Dynamic SQL

11. Important Interview Points

PIVOT

Converts rows into columns and usually performs an aggregation.

UNPIVOT

Converts columns into rows.

PIVOT requires an aggregate

Common examples:

```
SUM()  
COUNT()  
AVG()  
MIN()  
MAX()
```

PIVOT is useful for

- Sales reports
- Monthly reports
- Financial reports
- Attendance reports
- Dashboards
- Cross-tab reports

UNPIVOT is useful for

- Converting wide data to long format
- Data migration
- ETL processes
- Data cleaning
- Preparing data for analytics

Quick Memory Trick

```
PIVOT  
P → Put rows into columns  
  
UNPIVOT  
UN → Undo the pivot
```

So remember:

PIVOT = Rows → Columns UNPIVOT = Columns → Rows

- Dynamic SQL

Dynamic SQL

Dynamic SQL means creating and executing an SQL statement **at runtime**, instead of writing the complete SQL statement beforehand.

It is useful when parts of the query—such as **table names, column names, filters, sorting, or PIVOT columns**—are not known until runtime.

1. Static SQL vs Dynamic SQL

Static SQL

The query is fixed:

```
SELECT *  
FROM Employees  
WHERE Department = 'IT';
```

The table, columns, and filter are already known.

Dynamic SQL

The SQL statement is constructed at runtime:

```
DECLARE @SQL NVARCHAR(MAX);  
  
SET @SQL = '  
    SELECT *  
    FROM Employees  
    WHERE Department = ''IT''  
';  
  
EXEC(@SQL);
```

The SQL statement is stored inside a variable and then executed.

2. Basic Dynamic SQL

SQL Server example:

```
DECLARE @SQL NVARCHAR(MAX);  
  
SET @SQL = 'SELECT * FROM Employees';  
  
EXEC(@SQL);
```

Here:

```
@SQL  
↓  
"SELECT * FROM Employees"  
↓  
EXEC()
```

↓

Database executes the query

3. Dynamic WHERE Condition

Suppose the department is determined at runtime:

```
DECLARE @Department VARCHAR(50) = 'IT';

DECLARE @SQL NVARCHAR(MAX);

SET @SQL =
    'SELECT *
    FROM Employees
    WHERE Department = ''' + @Department + '''';

EXEC(@SQL);
```

The generated query becomes:

```
SELECT *
FROM Employees
WHERE Department = 'IT';
```

However, **this approach should generally not be used with untrusted user input.**

A safer approach is parameterized dynamic SQL.

4. `sp_executesql`

In SQL Server, prefer:

```
sp_executesql
```

for dynamic SQL that contains values supplied at runtime.

Example:

```
DECLARE @SQL NVARCHAR(MAX);
DECLARE @Department VARCHAR(50) = 'IT';

SET @SQL = '
    SELECT *
```



```
FROM Employees
WHERE Department = @Dept;
';

EXEC sp_executesql
    @SQL,
    N'@Dept VARCHAR(50)',
    @Dept = @Department;
```

This separates:

```
SQL structure
+
Parameter values
```

and is much safer than concatenating values directly into the SQL string.

5. Dynamic Table Name

Suppose the table name itself is dynamic.

```
DECLARE @TableName SYSNAME = 'Employees';
DECLARE @SQL NVARCHAR(MAX);

SET @SQL =
    'SELECT *
    FROM ' + QUOTENAME(@TableName);

EXEC sp_executesql @SQL;
```

`QUOTENAME()` is important when inserting an identifier such as a table or column name.

For example:

```
QUOTENAME('Employees')
```

produces:

```
[Employees]
```

6. Dynamic Column Name

Suppose the column used for sorting is dynamic.

```
DECLARE @ColumnName SYSNAME = 'Salary';
DECLARE @SQL NVARCHAR(MAX);

SET @SQL = '
    SELECT *
    FROM Employees
    ORDER BY ' + QUOTENAME(@ColumnName);

EXEC sp_executesql @SQL;
```

You **cannot parameterize an identifier** like this:

```
ORDER BY @ColumnName
```

A parameter represents a **value**, not a table or column identifier.

Therefore, for dynamic identifiers, validate/allow-list the input and use `QUOTENAME()`.

7. Dynamic SQL with Multiple Parameters

```
DECLARE @SQL NVARCHAR(MAX);

SET @SQL = '
    SELECT *
    FROM Employees
    WHERE Department = @Dept
    AND Salary >= @MinSalary;
';

EXEC sp_executesql
    @SQL,
    N'@Dept VARCHAR(50), @MinSalary DECIMAL(10,2)',
    @Dept = 'IT',
    @MinSalary = 50000;
```

This is a good pattern for dynamic filtering.

8. Dynamic SQL for PIVOT

Dynamic SQL is particularly useful for **dynamic PIVOT**.

Suppose the months aren't known beforehand.

Instead of:

```
FOR Month IN ([Jan], [Feb], [Mar])
```

you can dynamically generate:

```
[Jan], [Feb], [Mar], [Apr], [May], ...
```

The general process is:

```
Get distinct values
    ↓
Convert values into column list
    ↓
Build PIVOT query
    ↓
Execute Dynamic SQL
```

This is one of the most common real-world uses of Dynamic SQL.

9. Dynamic SQL for Optional Filters

Suppose you have optional search parameters:

```
Department
Minimum Salary
Job Title
```

Instead of creating many different hardcoded queries, you can build the **WHERE** clause dynamically.

Example:

```
DECLARE @SQL NVARCHAR(MAX);

SET @SQL = '
    SELECT *
    FROM Employees
    WHERE 1 = 1
';

IF @Department IS NOT NULL
    SET @SQL += ' AND Department = @Dept';

IF @MinSalary IS NOT NULL
    SET @SQL += ' AND Salary >= @Salary';

EXEC sp_executesql
```

```
@SQL,  
N'@Dept VARCHAR(50), @Salary DECIMAL(10,2)',  
@Dept = @Department,  
@Salary = @MinSalary;
```

This pattern is useful for:

- Search screens
- Reporting systems
- Admin dashboards
- API filtering

10. Dynamic SQL and SQL Injection

This is one of the **most important concepts**.

Bad:

```
SET @SQL =  
    'SELECT *  
    FROM Employees  
    WHERE Name = '' + @UserInput + ''';  
  
EXEC(@SQL);
```

If `@UserInput` comes directly from a user, malicious SQL could potentially be injected.

Better

Use:

```
sp_executesql
```

with parameters:

```
SET @SQL = '  
    SELECT *  
    FROM Employees  
    WHERE Name = @Name;  
';  
  
EXEC sp_executesql  
    @SQL,  
    N'@Name VARCHAR(100)',  
    @Name = @UserInput;
```

11. Parameters vs Identifiers

This distinction is very important.

Values → Parameters

```
WHERE Salary > @Salary
```

Use:

```
sp_executesql
```

Identifiers → Dynamic SQL + Validation

```
FROM [Employees]  
ORDER BY [Salary]
```

Use:

```
QUOTENAME()
```

and preferably an **allow-list** of permitted table/column names.

12. Advantages

Dynamic SQL can provide:

- Flexible queries
 - Dynamic filtering
 - Dynamic table selection
 - Dynamic column selection
 - Dynamic sorting
 - Dynamic PIVOT
 - Flexible reporting
 - Metadata-driven operations
-

13. Disadvantages

Dynamic SQL also has drawbacks:

- More difficult to debug

- More difficult to maintain
- SQL injection risk if poorly implemented
- Can make code harder to understand
- Plan/cache behavior can be complicated
- Errors may only appear at runtime

Therefore, **don't use Dynamic SQL when normal static SQL can solve the problem cleanly.**

14. EXEC() vs sp_executesql

Feature	EXEC()	sp_executesql
Execute dynamic SQL	✓	✓
Parameterized values	Limited	✓
SQL injection protection	Depends on coding	Better with parameters
Reuse execution plans	Less favorable	Better potential
Recommended for parameterized SQL	✗	✓

Example:

```
EXEC(@SQL);
```

versus:

```
EXEC sp_executesql
    @SQL,
    N'@Id INT',
    @Id = 10;
```

For modern SQL Server development, **sp_executesql** is **generally preferred** for parameterized dynamic SQL.

15. Interview Definition

Dynamic SQL is SQL code that is constructed and executed at runtime. It is useful when query elements such as filters, identifiers, or PIVOT columns need to be determined dynamically. In SQL Server, `sp_executesql` with parameters should be preferred for dynamic values to reduce SQL injection risk and improve plan reuse.

Remember

```
Static SQL
  ↓
Query known beforehand

Dynamic SQL
  ↓
Build query at runtime

Values
  ↓
Parameters / sp_executesql

Table or Column names
  ↓
Validated identifiers + QUOTENAME()
```

Key topics to master next: Dynamic PIVOT → `sp_executesql` → SQL Injection → `QUOTENAME()` → Dynamic WHERE clauses → Dynamic ORDER BY.

- Temporary Tables

Temporary Tables in SQL

A **temporary table** is a table used to store **temporary/intermediate data** during database operations.

They are useful when you need to:

- Store intermediate query results
- Break a complex query into multiple steps
- Process data in stages
- Perform calculations on intermediate data
- Build reports
- Work with large transformations

The exact syntax differs between database systems. The examples below primarily use **SQL Server**, where temporary tables are very common.

1. Creating a Temporary Table

In SQL Server, a local temporary table starts with `#`.

```
CREATE TABLE #Employees
(
    EmployeeId INT,
    EmployeeName VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

Insert data:

```
INSERT INTO #Employees
VALUES
(1, 'John', 50000),
(2, 'Alice', 60000),
(3, 'Bob', 55000);
```

Query it:

```
SELECT *
FROM #Employees;
```

2. Example

```
CREATE TABLE #HighSalaryEmployees
(
    EmployeeId INT,
    EmployeeName VARCHAR(100),
    Salary DECIMAL(10,2)
);

INSERT INTO #HighSalaryEmployees
SELECT
    EmployeeId,
    EmployeeName,
    Salary
FROM Employees
WHERE Salary > 50000;

SELECT *
FROM #HighSalaryEmployees;
```

Here the process is:

```
Employees
  ↓
Filter Salary > 50000
  ↓
#HighSalaryEmployees
  ↓
Use temporary result
```

3. Temporary Table with **SELECT INTO**

You can create and populate a temporary table in one statement:

```
SELECT
    EmployeeId,
    EmployeeName,
    Salary
INTO #HighSalaryEmployees
FROM Employees
WHERE Salary > 50000;
```

Then:

```
SELECT *
FROM #HighSalaryEmployees;
```

This is convenient when you don't need to explicitly define the table structure first.

4. Local Temporary Table

A local temporary table uses:

```
#TableName
```

Example:

```
CREATE TABLE #Orders
(
    OrderId INT,
    Amount DECIMAL(10,2)
);
```

It is generally available only within the **current session/connection**.

When the session ends, SQL Server automatically removes it.

You can also explicitly remove it:

```
DROP TABLE #Orders;
```

5. Global Temporary Table

A global temporary table uses:

```
##TableName
```

Example:

```
CREATE TABLE ##SharedOrders
(
    OrderId INT,
    Amount DECIMAL(10,2)
);
```

Unlike a local temporary table, a global temporary table can be accessed by **multiple sessions** while it exists.

```
DROP TABLE ##SharedOrders;
```

Global temporary tables should be used carefully because other sessions can see and interact with them.

6. Temporary Table vs Permanent Table

Permanent table

```
CREATE TABLE Employees
(
    EmployeeId INT,
    EmployeeName VARCHAR(100)
);
```

The table persists until you explicitly remove it.

Temporary table

```
CREATE TABLE #Employees
(
    EmployeeId INT,
    EmployeeName VARCHAR(100)
);
```

The table is temporary and has a limited lifetime.

Feature	Permanent Table	Temporary Table
Lifetime	Persistent	Temporary
Storage	Database	Temporary storage

Feature	Permanent Table	Temporary Table
# prefix	✗	✓ SQL Server
Automatically removed	✗	Usually
Good for intermediate data	Sometimes	✓
Can have indexes	✓	✓
Can have constraints	✓	✓

7. Temporary Table with Index

Temporary tables can have indexes.

```
CREATE TABLE #Employees
(
    EmployeeId INT,
    EmployeeName VARCHAR(100),
    Salary DECIMAL(10,2)
);

CREATE INDEX IX_Employees_Salary
ON #Employees(Salary);
```

This can improve performance when the temporary table contains a significant amount of data and is queried repeatedly.

However, indexes themselves have creation and maintenance costs, so don't add them blindly.

8. Temporary Tables in Multiple Steps

One of the biggest benefits is breaking complex processing into stages.

```
SELECT *
INTO #Sales
FROM Orders
WHERE OrderDate >= '2026-01-01';

SELECT
    CustomerId,
    SUM(Amount) AS TotalSales
INTO #CustomerSales
FROM #Sales
GROUP BY CustomerId;

SELECT *
```

```
FROM #CustomerSales
WHERE TotalSales > 100000;
```

Conceptually:

```
Orders
  ↓
#Sales
  ↓
#CustomerSales
  ↓
Final result
```

This can make complicated SQL easier to understand and debug.

9. Temporary Tables Inside Stored Procedures

Temporary tables are commonly used in stored procedures.

```
CREATE PROCEDURE GetHighSalaryEmployees
AS
BEGIN

    CREATE TABLE #Employees
    (
        EmployeeId INT,
        EmployeeName VARCHAR(100),
        Salary DECIMAL(10,2)
    );

    INSERT INTO #Employees
    SELECT
        EmployeeId,
        EmployeeName,
        Salary
    FROM Employees
    WHERE Salary > 50000;

    SELECT *
    FROM #Employees;

END;
```

10. Temporary Table vs Table Variable

This is a **very important interview topic**.

Temporary table

```
CREATE TABLE #Employees
(
    Id INT,
    Name VARCHAR(100)
);
```

Table variable

```
DECLARE @Employees TABLE
(
    Id INT,
    Name VARCHAR(100)
);
```

Both can store temporary/intermediate data, but they have different characteristics.

Feature	#Temp Table	@Table Variable
Syntax	#Table	@Table
Lifetime	Session/scope	Batch/procedure/function scope
Indexes	Yes	Supports indexes/constraints in specific ways
Statistics	Generally available	More limited, depending on SQL Server version
Large datasets	Often better choice	Often better for smaller/simple datasets
Transactions	Participates in transactions	Transaction behavior differs
Can be altered	Generally yes	More restricted
Complex operations	Better suited	Less flexible

A common practical rule is:

Use a temporary table when you have a larger or more complex intermediate dataset; use a table variable for smaller, simpler temporary data.

But don't treat that as an absolute rule—test performance for your workload and SQL Server version.

11. Temporary Table vs CTE

Another important comparison:

CTE

```
WITH EmployeeData AS
(
    SELECT *
    FROM Employees
    WHERE Salary > 50000
)
SELECT *
FROM EmployeeData;
```

A CTE is mainly a **query expression**.

Temporary table

```
SELECT *
INTO #EmployeeData
FROM Employees
WHERE Salary > 50000;

SELECT *
FROM #EmployeeData;
```

A temporary table actually stores the intermediate result and can be reused across multiple statements.

CTE	Temporary Table
Query expression	Temporary table
Usually used for a single statement	Can be used across multiple statements
Good for readability	Good for multi-step processing
Recursive queries supported	Not inherently recursive
No explicit temporary table needed	Creates temporary object
Useful for short transformations	Useful for larger workflows

12. Temporary Tables and Transactions

Temporary tables can participate in transactions.

```
BEGIN TRANSACTION;

CREATE TABLE #Orders
(
    OrderId INT,
    Amount DECIMAL(10,2)
);
```

```
INSERT INTO #Orders
VALUES (1, 1000);

ROLLBACK TRANSACTION;
```

The exact behavior of temporary objects and their contents can depend on the operation and SQL Server semantics, so transaction handling should be tested when correctness is important.

13. Common Real-World Uses

Reporting

```
Raw data
  ↓
#FilteredData
  ↓
#AggregatedData
  ↓
Report
```

ETL

```
Source
  ↓
Temporary staging
  ↓
Transformation
  ↓
Destination
```

Complex stored procedures

```
Step 1 → #Temp1
Step 2 → #Temp2
Step 3 → Final result
```

Performance optimization

A temporary table can sometimes allow you to materialize an intermediate result, index it, and then query it efficiently.

But using a temporary table **does not automatically make a query faster**.

14. Important Mistakes to Avoid

Mistake 1: Creating unnecessary temp tables

Don't create temporary tables when a simple query or CTE is sufficient.

Mistake 2: Ignoring indexes

If a large temporary table is repeatedly joined or filtered, an appropriate index may help.

Mistake 3: Assuming temp tables always improve performance

They can sometimes make performance worse because of:

- Additional writes
- TempDB usage
- Index creation
- Additional query steps

Mistake 4: Using global temporary tables unnecessarily

Prefer local temporary tables unless sharing between sessions is specifically required.

15. Interview Questions

Q1. What is a temporary table?

A temporary table is a temporary database object used to store intermediate data during a session or scope.

Q2. What is #TempTable?

A local temporary table in SQL Server.

Q3. What is ##TempTable?

A global temporary table that can be accessed by multiple sessions while it exists.

Q4. Temporary table vs table variable?

Temporary tables generally provide more flexibility and are often preferable for larger or more complex intermediate datasets, while table variables are convenient for smaller and simpler temporary data.

Q5. Temporary table vs CTE?

A CTE is a query expression primarily used within a statement, whereas a temporary table materializes intermediate data and can be reused across multiple statements.

Q6. Can temporary tables have indexes?

Yes. SQL Server temporary tables can have indexes.

Quick Memory

Temporary Tables

- ├─ #TempTable
 - └─ Local temporary table
- ├─ ##TempTable
 - └─ Global temporary table
- ├─ SELECT INTO
 - └─ Create + populate
- ├─ CREATE TABLE
 - └─ Define + populate separately
- ├─ Indexes
 - └─ Useful for appropriate workloads
- └─ Compare with
 - ├─ Table Variables
 - └─ CTEs

Key idea: Temporary table = materialized intermediate data that you can reuse, query, index, and process before producing the final result.

- Materialized Views (where supported)

Materialized Views

A **Materialized View (MV)** is a database object that **stores the result of a query physically**.

Unlike a normal view, which runs its underlying query when you access it, a materialized view stores the computed result and can therefore make repeated expensive queries much faster.

Important: Materialized views are database-specific. PostgreSQL, Oracle, and some other systems support them directly. SQL Server does not have an object literally called a materialized view; its closest equivalent is an **indexed view**.

1. Normal View vs Materialized View

Normal View

SELECT query
↓

```
View
  ↓
Query executes
  ↓
Result
```

The underlying query is generally executed when the view is queried.

Materialized View

```
SELECT query
  ↓
Materialized View
  ↓
Result physically stored
  ↓
Read stored result
```

The stored result must be **refreshed** when the underlying data changes.

2. Example

Suppose you have millions of sales records:

```
Sales
-----
OrderId
CustomerId
OrderDate
Amount
```

You frequently run:

```
SELECT
    CustomerId,
    SUM(Amount) AS TotalSales
FROM Sales
GROUP BY CustomerId;
```

This aggregation could be expensive if **Sales** contains millions of rows.

A materialized view can store the aggregated result.

Conceptually:

```
Sales
  ↓
GROUP BY CustomerId
  ↓
Materialized View
  ↓
CustomerId | TotalSales
```

Then applications can query the precomputed result instead of repeatedly calculating the aggregation.

3. PostgreSQL Example

PostgreSQL supports materialized views directly.

```
CREATE MATERIALIZED VIEW customer_sales AS
SELECT
    CustomerId,
    SUM(Amount) AS TotalSales
FROM Sales
GROUP BY CustomerId;
```

Now query it:

```
SELECT *
FROM customer_sales;
```

The result is stored physically.

4. Refreshing a Materialized View

The stored data does **not automatically represent every change** to the underlying tables.

In PostgreSQL, you can refresh it:

```
REFRESH MATERIALIZED VIEW customer_sales;
```

Conceptually:

```
Underlying Tables Change
  ↓
Materialized View becomes stale
  ↓
REFRESH
```



Materialized View updated

5. Concurrent Refresh

PostgreSQL also supports:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY customer_sales;
```

This can allow users to continue reading the existing materialized view while the refresh is being performed, subject to PostgreSQL's requirements for concurrent refresh.

A unique index is generally required.

For example:

```
CREATE UNIQUE INDEX  
    idx_customer_sales  
ON customer_sales(CustomerId);
```

Then:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY customer_sales;
```

6. Materialized View with Aggregation

This is one of the most common use cases.

```
CREATE MATERIALIZED VIEW monthly_sales AS  
SELECT  
    EXTRACT(YEAR FROM OrderDate) AS SalesYear,  
    EXTRACT(MONTH FROM OrderDate) AS SalesMonth,  
    SUM(Amount) AS TotalSales,  
    COUNT(*) AS OrderCount  
FROM Sales  
GROUP BY  
    EXTRACT(YEAR FROM OrderDate),  
    EXTRACT(MONTH FROM OrderDate);
```

Now instead of repeatedly processing millions of sales records:

```
SELECT *
FROM monthly_sales;
```

The application can read the precomputed summary.

7. Materialized View with Joins

Materialized views can also store results from expensive joins.

```
CREATE MATERIALIZED VIEW customer_order_summary AS
SELECT
    c.CustomerId,
    c.CustomerName,
    COUNT(o.OrderId) AS OrderCount,
    SUM(o.Amount) AS TotalAmount
FROM Customers c
JOIN Orders o
    ON c.CustomerId = o.CustomerId
GROUP BY
    c.CustomerId,
    c.CustomerName;
```

This is useful when the same expensive calculation is requested frequently.

8. Materialized View vs Normal View

Feature	View	Materialized View
Stores query definition	✓	✓
Stores query result	✗	✓
Physically stores data	✗	✓
Always reflects base-table changes immediately	Usually	✗
Needs refresh	✗	Usually
Faster for expensive repeated queries	Sometimes	Often
Uses additional storage	Little	✓
Good for reporting	✓	✓
Good for expensive aggregations	Sometimes	✓

9. Materialized View vs Temporary Table

These are very different concepts.

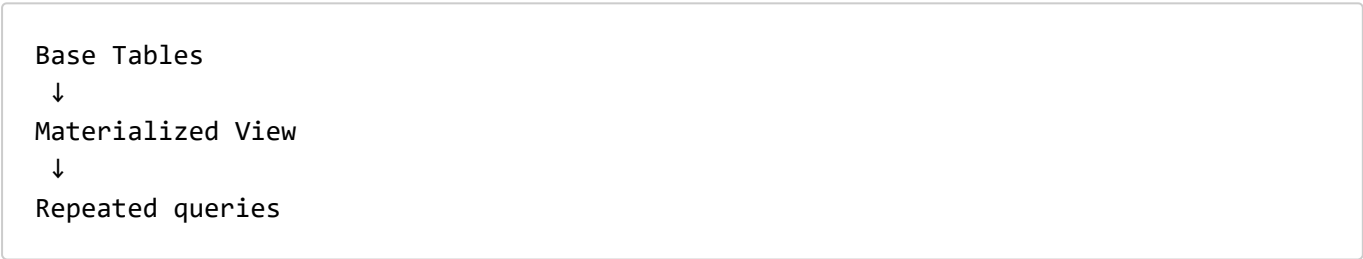
Temporary Table

Used for **temporary intermediate processing**:



Materialized View

Used for **persistently storing a reusable query result**:



Feature	Temporary Table	Materialized View
Purpose	Temporary processing	Reusable precomputed result
Lifetime	Limited	Persistent
Automatically refreshed	N/A	Usually explicitly/scheduled
Used across sessions	Depends on DB/type	Yes
Good for reporting	Sometimes	Excellent use case
Storage	Temporary storage	Persistent database storage

10. Materialized View vs Indexed View in SQL Server

If you're using **SQL Server**, you won't normally write:

CREATE MATERIALIZED VIEW ...

Instead, SQL Server provides **indexed views**.

Basic concept:

```
CREATE VIEW dbo.CustomerSales
WITH SCHEMABINDING
AS
SELECT
    CustomerId,
    COUNT_BIG(*) AS OrderCount,
    SUM(Amount) AS TotalAmount
FROM dbo.Orders
GROUP BY CustomerId;
```

Then an index can be created on the view:

```
CREATE UNIQUE CLUSTERED INDEX
    IX_CustomerSales
ON dbo.CustomerSales(CustomerId);
```

The indexed view stores the result in the database and SQL Server maintains it as the underlying data changes, subject to indexed-view restrictions and optimizer behavior.

So for interviews:

```
PostgreSQL / Oracle
    ↓
Materialized View

SQL Server
    ↓
Indexed View
```

They are conceptually related but **not identical features**.

11. When Should You Use a Materialized View?

Materialized views are particularly useful when:

Expensive aggregation

```
SUM()
COUNT()
AVG()
GROUP BY
```

Expensive joins

Customers
+
Orders
+
Products
+
Payments

Reporting

For example:

Daily Sales
Monthly Sales
Customer Revenue
Product Performance

Data warehouses

Large analytical queries often benefit from precomputed results.

12. When Should You NOT Use One?

Avoid unnecessary materialized views when:

- Data changes extremely frequently
- Queries are already cheap
- The result is rarely used
- Storage is limited
- Refreshing the view is more expensive than running the original query
- You require perfectly real-time results and cannot tolerate refresh lag

13. Main Trade-Off

The key trade-off is:

Faster Reads
↑
Materialized View
↓
More Storage + Refresh/Maintenance

You are essentially exchanging **computation at query time** for **storage and maintenance**.

14. Real-World Example

Imagine an e-commerce system with:

100 million Orders

Every dashboard request asks:

```
SELECT
  ProductId,
  SUM(Amount),
  COUNT(*)
FROM Orders
GROUP BY ProductId;
```

Doing this repeatedly can be expensive.

Instead:

```
100 Million Orders
    ↓
  Aggregation
    ↓
Materialized View
    ↓
ProductId | Sales | OrderCount
```

Dashboard:

```
SELECT *
FROM product_sales_summary;
```

Now the dashboard can read a much smaller precomputed dataset.

15. Important Interview Questions

What is a materialized view?

A materialized view physically stores the result of a query so that expensive queries can be answered faster.

Does a materialized view automatically stay up to date?

It depends on the database and implementation. In systems such as PostgreSQL, a materialized view generally needs to be refreshed. Other systems may support different refresh or maintenance mechanisms.

Why use a materialized view?

To improve read performance for expensive and frequently executed queries, especially aggregations and joins.

What is the disadvantage?

It consumes storage and requires maintenance or refreshing when underlying data changes.

Materialized View vs View?

A normal view stores the query definition, while a materialized view stores the query result physically.

Does SQL Server support materialized views?

SQL Server doesn't have a feature named materialized views. Its closest equivalent is an **indexed view**.

Quick Memory

```
VIEW
Query definition
  ↓
Execute underlying query
  ↓
Result
```

```
MATERIALIZED VIEW
Query
  ↓
Result physically stored
  ↓
Fast reads
  ↓
Refresh / maintenance required
```

Most important points to remember

Materialized View = Precomputed + Physically Stored Result

Advantages: Faster reads, excellent for reporting/analytics, expensive aggregations and joins.

Disadvantages: Extra storage, refresh/maintenance cost, potentially stale data.

PostgreSQL/Oracle: Materialized Views **SQL Server:** Indexed Views as the closest equivalent.

Topic Left add after

1. SQL Standards
2. SQL Statements & Keywords
3. SQL Comments
4. DROP DATABASE
5. CREATE SCHEMA
6. ALTER SCHEMA
7. DROP SCHEMA
8. Add/Modify/Drop Columns
9. Insert Multiple Rows
10. Insert from SELECT
11. Column Aliases
12. Table Aliases
13. NOT IN
14. NOT LIKE
15. Wildcards (% , _)
16. COUNT(DISTINCT)
17. Multiple-Column GROUP BY
18. Equi Join
19. Non-Equi Join
20. Semi Join
21. Anti Join
22. Multiple Joins
23. Cascading Constraints
24. Natural Key
25. Surrogate Key
26. Arithmetic Operators
27. ANY
28. ALL
29. EXISTS
30. NOT EXISTS
31. Multiple-Column Subqueries
32. CASE Expression
33. Materialized Views
34. Composite Index
35. Covering Index
36. Filtered/Partial Index
37. Included Columns
38. Index Seek
39. Index Scan
40. Index Fragmentation
41. OVER()
42. Window Frames

43. `FIRST_VALUE()`
44. `LAST_VALUE()`
45. Moving Average
46. Multiple CTEs
47. Temporary Tables
48. Table Variables
49. Stored Procedure Parameters
50. Output Parameters
51. Scalar UDF
52. Table-Valued Functions
53. DML Triggers
54. DDL Triggers
55. ACID Properties
56. Transaction Isolation Levels
57. Dirty Read
58. Non-Repeatable Read
59. Phantom Read
60. Locks
61. Blocking
62. Deadlocks
63. Optimistic Concurrency
64. Pessimistic Concurrency
65. Functional Dependency
66. 4NF
67. 5NF
68. ER Model
69. ER Diagram
70. One-to-One Relationship
71. One-to-Many Relationship
72. Many-to-Many Relationship
73. Junction/Bridge Tables
74. Roles & Permissions
75. Execution Plans
76. Table Scan
77. Key Lookup
78. Nested Loop Join
79. Hash Join
80. Merge Join
81. SARGability
82. Query Statistics
83. Parameter Sniffing
84. N+1 Query Problem
85. Pagination Optimization
86. Dynamic SQL
87. `PIVOT`
88. `UNPIVOT`

- 89. Hierarchical Queries
- 90. JSON Queries
- 91. XML Queries
- 92. Full-Text Search
- 93. Temporal Tables
- 94. Sequences
- 95. Identity/Auto Increment
- 96. Generated Columns
- 97. Variables
- 98. Cursors
- 99. Error Handling
- 100. Bulk Insert
- 101. Data Migration
- 102. ETL Basics
- 103. Backup & Restore
- 104. Point-in-Time Recovery
- 105. Replication
- 106. Read Replicas
- 107. Sharding
- 108. Partitioning
- 109. High Availability
- 110. Failover
- 111. Distributed Transactions
- 112. ADO.NET
- 113. Parameterized Queries
- 114. Connection Pooling
- 115. SQL Injection Prevention
- 116. Row-Level Security
- 117. Data Masking
- 118. Auditing
- 119. Encryption
- 120. SQL Interview Problem Patterns